MASSIMO PAPA

PYTHON PRIMI PASSI

VOLUME 2

ARGOMENTI AVANZATI



Per corsi Scuola Secondaria di 2° grado, Post-Diploma e appassionati!

CON CENNI GNU/LINUX



PUBLISHED COPY

PYTHON

P R I M I P A S S I

Volume 2

ARGOMENTI AVANZATI

WORD CLOUD


python
print
programma
metodo
funzione
lista
file
output
codice
classe
stringa
def
lanciamo
parametro
database
comando
import
oggetto
server
nome
dizionario
operatori
sistema
interprete
tuple
dataframe
client
console
table
numpy
home
massimo
figura
prova
versione
stampato
eseguire
oggetti
stringhe
errore
return
sequenza
linux
valori
modulo
vengono
stampa
uscita
istruzione
caso
visto
analisi
input
volta
ciclo
pip
interno
esempio
numero
parametri
funzioni
elementi
testo
stato
variabile
carattere
sequenze
utente
inserire
open
restituisce
installazione
path
chiave
directory
riga
valore
lettura
righe
cursor
html
comandi
video
tipo
files
metodi
float
int
str
bool
dati
tale
txt
tupla
diz
true
false
host
ftp
ssh
np
db
os
sql
web
ax
exec
axes
plot
array
nelle
tutto
bit
ele
plt
env
data
ls
script
utf
csv
sar
cfr
fun
sys
row
cat
sort
dizionari
generale
codifica
creazione
centro
gestire
fare
pop
classi
terzo
aver
usr
recent
locale
altezza
attributo
parentesi
cerchio
users
mysqldb
operazioni
permessi
porta
indice
rappresenta
andiamo
min
val
istanza
append
full
module
flow
errori
altre
shell
pem
serie
cosa
lavoro
serve
list
base
init
user
test
ssl
br
nei
area
id
gnu
zip
default
studente
lib
pandas
nulla
read
line
write
cwd
key
step
elemento
http
librerie
unix
grafico
install
acct
argomenti
operatore
map
loop
ip
nostro
stat
xls
flusso
numeri
record
alcun
ambiente
api
sintassi
apre
caratteri
liste
verr
suo
pd
vi
max
set
st
math
attributi
novas
corrente
connect
sep
size
sue
asse
tr
num
stdin
linuxmachine
contenuto
esso
call
rete
org
localhost
accetta
post
media
maniera
moduli
abs
avr
ftplib
close
elenco
query
linea
deve
nuovo
campo
algoritmo
remove
insert
presente
accedere
dello
istruzioni
formato
type

SOMMARIO

1. Premessa 16
2. Librerie del Python 18
2.1. Modulo math 18
2.1.1 Funzione esponenziale, logaritmo e potenza 19
2.1.2 Funzioni trigonometriche e loro inverse 19
2.1.3 Funzioni iperboliche 19
2.1.4 Costanti 19
2.2 Modulo numpy 19
2.4 Modulo pil 23
2.4.1 Creare un'immagine 24
2.4.2 Modificare un'immagine 24
2.4.3 Salvare un'immagine 24
3. Librerie per il Data Science 26
3.1 MatPlotLib 26
3.1.1 Un primo esempio 26
3.1.2 Oggetti di un grafico 27
3.1.3 Un esempio più complesso 29
3.1.4 Grafico cartesiano di punti e funzioni 30
3.2 Pandas 34
3.2.1 Creazione di oggetti 34
3.2.1.1 Serie 34
3.2.1.2 DataFrame 34
3.2.1.3 Visualizzazione dei dati 37
3.2.1.3.1 Righe superiori 37
3.2.1.3.2 Righe inferiori 37
3.2.1.3.3 Indice 37
3.2.1.3.4 Colonne 38
3.2.1.3.5 Riepilogo statistico 38
3.2.1.3.6 Trasposta 38
3.2.1.3.7 Ordinamento su un asse 38
3.2.1.3.8 Ordinamento per valori 39
3.2.1.4 Selezione dei dati 39
3.2.1.5 Operazioni sul DataFrame 40
3.2.1.5.1 Calcolo della media 40
3.2.1.6 Graficare i DataFrame 41
3.2.1.6.1 Grafico cartesiano di una serie 41
3.2.1.6.2 Grafico cartesiano di un DataFrame 42

3.2.1.7 Dati da file CSV 44
3.2.1.8 Dati da file XLS (o XLSX) 48
4. Gestione filesystem 49
4.1 I files 49
4.1.1 Lettura del contenuto di un file 49
4.1.1.1 Cicli di lettura riga per riga alternativo 50
4.1.1 La funzione open() al microscopio 51
4.1.1.1 Tabella delle modalità di apertura di un file 52
4.1.2 Scrittura di un file 53
4.1.3 Il metodo close() 55
4.1.4 Spostamento all'interno del file mediante i metodi seek() e tell() 55
4.1.5 Altri modi per leggere un file 56
4.1.5.1 L'oggetto file è un iterabile 58
4.1.6 Scrittura massiva di righe 59
4.1.7 L'istruzione with per la gestione dei file 59
4.1.7.1 Lettura del contenuto di un file con with 60
4.1.7.2 Scrittura su un file con with 61
4.1.7.3 Esempio combinato di lettura e scrittura con with 61
4.1.7.4 Utilizzo di with con operazioni complesse 62
4.1.7.5 Uso di più with annidati 63
4.2 Interazione con il file system - il modulo OS 64
4.2.1 Gestione della directory di lavoro corrente 64
4.2.1.1 Ottenere la directory di lavoro corrente 64
4.2.1.2 Modifica della directory di lavoro corrente 65
4.2.2 Creazione di una directory 65
4.2.2.1 os.mkdir() 66
4.2.2.2 os.makedirs() 67
4.2.3 Elencare files e directories 68
4.2.4 Eliminazione di files e directories 68
4.2.4.1 os.remove() 68
4.2.4.2 os.rmdir() 69
4.2.5 Funzioni di uso comune 70
4.2.5.1 os.name 71
4.2.5.2 os.popen() 71
4.2.5.3 os.rename() 72
4.2.5.4 os.stat() 72
4.2.5.4.1 Il modulo stat per l'interpretazione dei risultati di os.stat() 73
stat.S_ISDIR(mode) 73
stat.S_ISREG(mode) 73
stat.S_ISLNK(mode) 74

stat.S_ISSOCK(mode) 74
stat.S_IMODE(mode) 74
4.2.5.5 os.path.exists() 74
4.2.5.6 os.path.getsize() 75
5. Servizi di rete 76
5.1 Connessioni SSH 76
5.1.1 Libreria Paramiko 76
5.1.2 Installazione libreria Paramiko 76
5.1.3 La classe SSHClient 76
5.1.3.1 Connessione 76
5.1.3.1.1 La questione dei 'known host' 77
5.1.3.1.2 Il metodo connect() al microscopio 78
5.1.3.2 Lanciare dei comandi sul server remoto 79
5.1.3.3 Comandi che richiedono input dall'utente 80
5.1.3.3.1 Un piccolo esercizio esemplificativo 80
5.1.4 Trasferimenti di files - la classe SFTPClient 82
5.1.4.1 La classe SFTPClient 82
5.1.4.1.1 get() 82
5.1.4.1.2 put() 83
5.2 Connessioni FTP 84
5.2.1 Libreria ftplib 84
5.2.2 La classe FTP 84
5.2.2.1 La classe FTP al microscopio 85
5.2.2 I metodi della classe FTP 86
5.2.2.1 Listare il contenuto di una directory remota 86
5.2.2.2 Recupero files 87
6. Gestione Database 89
6.0 Introduzione 89
6.1 Modulo MySQLdb 89
6.1.1 Installazione MySQLdb 90
6.1.1.1 Ambiente di testing 90
6.1.2 Connessione al database 90
6.1.3 Creazione di una tabella 91
6.1.4 Operazione di inserimento 93
6.1.4.1 Preparazione di query dinamiche 94
6.1.5 Operazione di lettura 95
6.1.6 Operazione di aggiornamento 97
6.1.7 Operazione di cancellazione 99
6.1.8 Le transazioni: commit() e rollback() 100
6.1.9 La disconnessione 101

6.2 Modulo SQLite3 102
6.2.1 Installazione SQLite3 102
6.2.1.1 Ambiente di testing 102
6.2.2 Connessione al database 102
6.2.3 Creazione di una tabella 103
6.2.4 Operazione di inserimento 104
6.2.4.1 Preparazione di query dinamiche 106
6.2.5 Operazione di lettura 107
6.2.6 Operazione di aggiornamento 109
6.2.7 Operazione di cancellazione 110
6.2.8 Lettura di file SQL 112
7. Il WEB 115
7.1 WEB Server 115
7.1.1 Introduzione 115
7.1.2 Lanciare un'istanza del webserver 115
7.1.2.1 Il comando al microscopio 116
7.1.3 Supporto SSL 117
7.1.4 Le classi di http.server 119
7.1.5 Implementazione del metodo di risposta al GET 119
7.1.6 Implementazione del metodo di risposta al POST 121
7.2 Esempio notevole: semplice web application 122
7.2.1 Introduzione 122
7.2.2 Struttura del progetto 123
7.2.1 Codice dei templates HTML 123
7.2.1.1 form.html 124
7.2.1.1 showRecords.html 124
7.2.2 Codice per la gestione del database 125
7.2.1.1 database.py 125
7.2.2 Codice per il server http: serverWeb.py 126
7.2.3 Test della webapplication 127
7.2.4 Prospettive 129
7.3 Modulo requests 130
7.3.1 Installazione 130
7.3.2 Effettuare una richiesta GET 130
7.3.2.1 Parametri della richiesta 131
7.3.3 Richiesta POST 134
7.3.4 Aggiungere intestazioni (headers) 135
7.3.5 Gestione delle eccezioni 136
7.3.6 Timeout 137
7.3.7 Autenticazione 137

7.3.8 Gestione delle Sessioni 138
7.3.9 Una semplice applicazione: previsioni del tempo 139
7.3.9.1 Registrazione per un'API key 139
7.3.9.2 Codice dell'applicazione 139
7.3.9.3 Esecuzione del Programma 140
7.4 Modulo scrapy 141
7.4.1 Installazione 141
7.4.2 Creare un nuovo progetto scrapy 142
7.4.3 Creare il primo spider 145
7.4.4 Modifichiamo il metodo parse dello spider 147
7.4.5 Eseguire lo spider 148
7.4.6* Selettori CSS e XPath 149
7.4.6.1 Cosa è XPath? 150
7.4.6.2 Uso selettori CSS vs Selettori XPath 150
7.4.7 Items e Item Pipeline 150
7.4.7.1 Items 150
7.4.7.2 Item Pipeline 151
7.4.7.3 Esecuzione con le ultime modifiche 152
7.4.8* Gestione Avanzata delle Richieste 154
7.4.9* Gestione delle Sessioni e Autenticazione 155
7.4.10 Rispettare robots.txt e gestire la velocità di crawling 156
7.4.11 Una semplice applicazione: Analisi dei prezzi dei prodotti 156
7.4.11.1 Definizione degli Item 156
7.4.11.2 Creazione dello Spider 157
7.4.11.3 Pipeline per Salvare i Dati 158
7.4.11.4 Configurazione del Progetto 159
7.4.11.5 Esecuzione del Monitoraggio 159
7.4.11 Vantaggi rispetto alla libreria requests 160
8. Agenti AI: costruire sistemi intelligenti con Python e LLM 161
8.1 Che cosa è un agente AI? 161
8.2 Il mattone fondamentale: chiamare un LLM 161
8.2.1 Perché Groq: confronto tra i provider gratuiti 161
8.2.2 La funzione chiama_llm: tre implementazioni a confronto 162
8.3 Agenti in sequenza: il pattern Pipeline 164
8.3.1 Definizione degli agenti 165
8.3.2 L'orchestratore 166
8.3.3 Il codice completo 167
8.4 Agenti in parallelo: threading e comunicazione tramite bacheca 173
8.4.0* Digressione sul modulo threading 174
8.4.1 La bacheca condivisa 178

8.4.2 Gli agenti e la sincronizzazione 179
8.4.3 L'orchestratore parallelo 180
8.4.4 Il codice completo con threading 181
8.4.5* Threading vs asyncio: quale scegliere? 187
8.5 Agenti che utilizzano strumenti: il tool calling 190
8.5.1 Implementazione dei tool 190
8.5.2 Lo schema JSON dei tool 192
8.5.3 Eseguire il tool scelto dall'LLM 194
8.5.4 L'agentic loop 195
8.5.5 Il codice completo 197
8.6 Riepilogo dei pattern studiati 204
9. Le interfacce grafiche 205
9.1 La libreria tkinter 205
9.2 Finestre, frame e widget 205
9.2.1 Creare la prima finestra 205
9.2.2 Cosa sono i widget 206
9.2.3 Il frame: un contenitore di widget 209
9.2.4 Le finestre di dialogo 210
9.3 Posizionare i widget 211
9.3.1 Il metodo .pack() 212
9.3.2 Il metodo .grid() 213
9.3.3 Il metodo .place() 213
9.3.4 Riempimento ed espansione: fill ed expand 214
9.3.5 Ridimensionare i widget 215
9.4 La gestione degli eventi 216
9.4.1 Perché servono gli eventi 216
9.4.2 Il parametro command 217
9.4.3 Il metodo .bind() 218
9.4.4 Rimuovere un evento: .unbind() 219
9.5 Canvas: disegnare oggetti grafici 219
9.5.1 Cos'è il Canvas 219
9.5.2 Disegnare linee 220
9.5.3 Disegnare rettangoli e ovali 221
9.5.4 Inserire del testo 223
9.5.5 Modificare ed eliminare gli oggetti 224
9.5.6 Interagire con gli eventi sul Canvas 224
9.6 Creare e gestire i menu 226
9.6.1 La barra dei menu 226
9.6.2 Aggiungere voci con .add_cascade() e .add_command() 227
9.6.3 Scorciatoie da tastiera 229

9.6.4 I menu contestuali (popup menu) 230
9.6.5 Un esempio completo 231
9.7. Un tocco di modernità: la libreria customtkinter 232
9.7.1 Perché customtkinter 232
9.7.2 Installazione 233
9.7.3 La prima finestra moderna 233
9.7.4 I widget moderni 234
9.7.5 Personalizzare l'aspetto dei widget 236
9.7.6 Un esempio completo: una finestra di login 237
APPENDICE 240
1. Installazione Python su GNU/Linux 240
1.1 Installazione da pacchetto 240
1.2 Installazione da sorgenti 241
1.2.1 Percorsi e file relativi all'installazione 242
1.4 Installazione librerie 242
1.4.1. Installazione di PIP (Python Package Index) 242
1.4.1.1 Nota su PIP da Python 3.11 (PEP 668) 243
1.4.2 Numpy 244
1.4.3 Pillow 244
1.4.4 Matplotlib 244
1.4.5 Pandas 244
1.5. Installazione di un ambiente virtuale 244
1.5.1 Introduzione 245
1.5.2 Installazione 245
1.5.3 La creazione di ambienti virtuali 245
1.5.4. Gestire i pacchetti con pip (Python Package Index) 246
2. Installazione Python su Microsoft Windows 249
2.1 Installazione da pacchetto 249
2.2 Lanciare l'interprete python 251
2.3 Installazione librerie 253
2.3.1. Installazione di PIP (Python Package Index) 253
2.3.2 Numpy 254
2.3.3 Pillow 254
2.3.4 Matplotlib 254
2.3.5 Pandas 254
3. Digressione s.o. GNU/Linux 255
3.1 La console GNU/Linux 255
3.1.1 Come aprire una console GNU/Linux? 255
3.1.1.1 Ambiente Microsoft Winodws 255
3.1.1.2 Ambiente GNU/Linux e UNIX like 256

3.1.1.3 Anatomia del prompt 256
3.2 I comandi in linea forniti dalla shell 257
3.2.1 Comandi per la gestione del filesystem 257
3.2.2 Comandi per la manipolazione dei files 258
3.2.3 Comandi per la gestione degli utenti e dei processi 258
3.2.4 Comandi per la gestione dei permessi sui files 259
3.2.4.1 Digressione sui permessi sui files 259
3.3 I comandi di ridirezione dell'output e i comandi di pipe 260
3.3.1 Ridirezione dell'output 260
3.3.2 Comandi di pipe 261
3.4 Editare testi dalla console 262
3.4.1 Breve guida ai comandi di VI 262
3.4.1.1 Normal mode 263
3.4.1.2 Insert mode 263
3.4.1.3 Command mode 263

1. Premessa

Questo testo è una revisione completa degli appunti di laboratorio di matematica e statistica del corso triennale per conseguire il Diploma di Informatica presso gli Istituti Tecnici. La revisione si è resa necessaria in quanto in questa versione si vuole dare ampio spazio al linguaggio di programmazione Python versione 3.x e dare qualche cenno alle librerie più utilizzate per il data science, quali:

- numpy
- Pandas
- MatPlotLib
- Scikit-learn
- OpenCV (questa è specifica per la visione artificiale)

Si è dato anche spazio all'utilizzo del linguaggio per svolgere compiti sistemistici come:

- gestione del filesystem (file e modulo os)
- servizi di rete (ssh, ftp, http)
- database (mysql e sqlite3)

Lo scopo è quello di voler costruire un testo che possa essere utile a chi si vuole avvicinare al linguaggio Python avendo già una conoscenza della programmazione. Ci si rivolge agli studenti della scuola secondaria di secondo grado e dei corsi post-diploma.

L'approccio è sostanzialmente molto pratico, si forniranno esempi di programmi che introducono i concetti peculiari del linguaggio Python, di ogni codice viene fornito l'output e un'analisi approfondita di ciò che è avvenuto nell'esecuzione delle istruzioni.
Viene fornita alla fine di ogni capitolo una serie di esercizi utili a fissare la teoria illustrate nel capitolo.

In appendice è presente una breve digressione sul sistema operativo GNU/Linux, o meglio sul suo utilizzo attraverso la console dei comandi (*shell*). Si forniscono i cenni necessari per effettuare tutte le prove e gli esercizi che mano a mano si incontreranno nella lettura. Si apprenderà come utilizzare la console da utente. Lo scopo è quello di fornire le conoscenze per

- aprire una console Linux
- utilizzare i comandi in linea forniti dalla shell
- utilizzare i comandi di ridirezione dell'output e i comandi di pipe
- editare testi dalla console

Inoltre sono presenti due capitoli che forniscono indicazioni sull'installazione del Python rispettivamente su ambienti Linux e Windows.

Nell'appendice relativa all'installazione di Python per GNU/Linux vi è un capitolo sulla gestione degli ambienti virtuali che risulta essere molto utile quando si lavora con più

librerie e si vuole evitare di creare conflitti tra le diverse versioni di librerie che si utilizzano. Oppure quando si vuole creare un ambiente di lavoro con versione di Python differente dall'installazione di default presente sul sistema.

2. Librerie del Python

Esistono molti moduli sviluppati per risolvere i problemi più disparati: dalle librerie per la manipolazione dei files audio alle librerie grafiche, dalle librerie per la gestione dei database a quelle per i socket TCP.
Sono presenti molte librerie sotto licenza GPL, non sempre sviluppate per un utilizzo con il python. Per rendere utilizzabili tali librerie con il python è necessario sviluppare il cosiddetto "wrapper", cioè un'interfaccia tra le chiamate alle funzioni della libreria con l'interprete python. In questo modo il parco librerie a disposizione del linguaggio è praticamente illimitato.
Nel seguito studieremo 3 particolari librerie:

- ➔ math
- ➔ Numpy
- ➔ pil (Python Imagine Library)

2.1. Modulo math

Il modulo *math* è la libreria matematica standard del python. Standard nel senso che in tutte le versioni del python ed in tutte le implementazioni risulta essere sempre presente.

Cominciamo a conoscere alcune funzioni, scriviamo il seguente programma:

```
from math import *
print ceil(2.2)
print ceil(-2.5)
print floor(2.2)
print floor(-3)
print round(2.5)
```

Lanciamo il programma, l'output stampato a video è il seguente:

```
3.0
-2.0
2.0
-3.0
3.0
```

Dall'analisi del codice e dell'output osserviamo che:

- ➔ Ceil(x) restituisce l'intero più piccolo >= x
- ➔ Floor(x) restituisce l'intero più grande <= x
- ➔ Round arrotonda all'intero più vicino

2.1.1 Funzione esponenziale, logaritmo e potenza

math.exp(x)	⟶	e^x
math.log10(x)	⟶	log(x)
math.log(x)	⟶	ln(x)
math.pow(x,y)	⟶	x^y

2.1.2 Funzioni trigonometriche e loro inverse

math.cos(x)	⟶	cos(x)
math.sin(x)	⟶	sen(x)
math.tan(x)	⟶	tan(x)
math.acos(x)	⟶	arccos(x)
math.asin(x)	⟶	arcsen(x)
math.atan(x)	⟶	arctan(x)

Gli argomenti delle funzioni trigonometriche sono espressi in radianti.

2.1.3 Funzioni iperboliche

math.cosh(x)	⟶	cosh(x)
math.sinh(x)	⟶	senh(x)
math.tanh(x)	⟶	tanh(x)

2.1.4 Costanti

math.e	⟶	numero di Nepero
math.pi	⟶	pi greco

2.2 Modulo numpy

L'interprete python implementa nativamente 4 strutture informative:

- ➔ lista
- ➔ tupla
- ➔ dizionario
- ➔ insieme

Gli array non esistono. Per poterli utilizzare si deve importare un modulo che li implementi. Esistono diversi moduli con cui si possono definire gli array, in questa sede introdurremo il

modulo *numpy* che oltre ad implementare gli array ha tutta una serie di funzioni appositamente studiate per il calcolo numerico. Tutte le funzioni matematiche viste nel modulo *math* sono presenti in *numpy*, quindi risulta essere ridondante importare insieme i due moduli.

Scriviamo il seguente programma:

```
import numpy as np
a=np.array([[1,2,3],[4,5,6]])
print (a)
a[0][0]=10
print (a)
```

Lanciamo il programma, l'output stampato a video è il seguente:

```
[[1 2 3]
 [4 5 6]]
[[10  2  3]
 [ 4  5  6]]
```

Dall'analisi del codice e dell'output osserviamo che:

➔ Il file da importare si chiama nump, possiamo utilizzare un alias per il modulo usando la keyword *as*, in questo modo possiamo fare riferimento a numpy scrivendo solo np.

➔ *array* è la primitiva che ci permette di definire un array. Lo definiamo elencando gli elementi appartenenti all'array a partire dalla prima riga. Le parentesi quadre delimiteranno la riga.

➔ L'istruzione *print* riconosce che ha come argomento un'array e quindi stampa tutti gli elementi di quest'ultimo (ricordiamo che il python è un linguaggio ad oggetti). Non si ha la necessità di eseguire due loop annidati per stampare un'array bidimensionale.

➔ Se vogliamo accedere ad un elemento dell'array indichiamo tra parentesi quadre gli indici. Gli indici, al pari del linguaggio C , partono da 0.

Scriviamo, ora, il seguente programma:

```
import numpy as np
numeri=np.array((4,9,16),float)
print (numeri)
radici=np.sqrt(numeri)
print (radici)
```

Lanciamo il programma, l'output stampato a video è il seguente:

```
[  4.   9.  16.]
[ 2.  3.  4.]
```

Dall'analisi del codice e dell'output osserviamo che:

➔ Si può indicare esplicitamente all'interprete il tipo di elementi di un array con la direttiva *float*.

➔ Si possono usare al posto delle parentesi quadre le parentesi tonde per enumerare gli elementi di un'array.

➔ Gli operatori matematici riconoscono argomenti di tipo array in maniera tale da venire applicati a tutti gli elementi dell'array.

Può risultare particolarmente tedioso inserire gli elementi di un array in fase di inizializzazione, p.e. si consideri una matrice 100 righe x 1000 colonne. un modo alternativo di definire un array evitando di inserire tutti gli elementi è quello di usare alcune funzioni studiate per il calcolo matriciale.

A tal scopo si consideri il seguente programma:

```
import numpy as np
a=np.zeros([2,3])
b=np.ones([3,3])
print (a)
print (b)
```

Lanciamo il programma, l'output stampato a video è il seguente:

```
[[0. 0. 0.]
 [0. 0. 0.]]

[[1. 1. 1.]
 [1. 1. 1.]
 [1. 1. 1.]]
```

Dall'analisi del codice e dell'output osserviamo che:

- ➔ La funzione *zeros* dichiara un array andando ad indicare tra parentesi quadre le dimensioni dell'array stesso. Tutti gli elementi assumono il valore 0.
- ➔ La funzione *ones* esegue la stessa funzione di *zeros* con l'unica differenza che gli elementi assumono il valore 1.
- ➔ I valori se non altrimenti specificato sono in formato floating point.

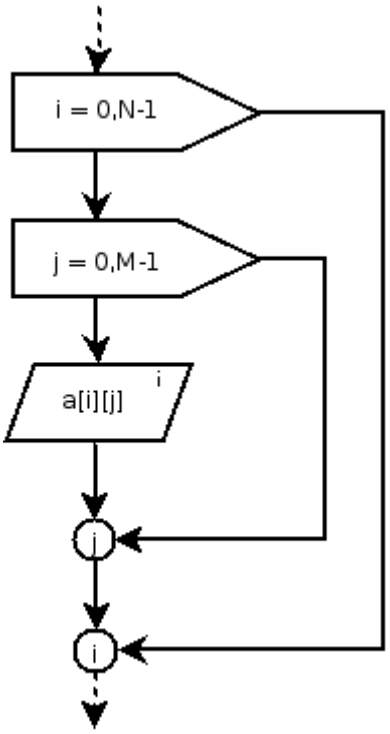
Se si vuole che gli elementi siano definiti come interi basta aggiungere la direttiva *int* come indicato nel seguente esempio:

```
import numpy as np
a=np.zeros([2,3],int)
print (a)
```

Lanciamo il programma, l'output stampato a video è il seguente:

```
[[0 0 0]
 [0 0 0]]
```

Supponiamo di voler far inserire all'utente gli elementi di un'array, abbiamo visto al capitolo 6 come far inserire dei valori dallo standard input mediante la funzione *input()*. Non possiamo utilizzare queste funzioni per inserire tutti gli elementi di un'array, dobbiamo necessariamente eseguire dei loop annidati e richiedere l'inserimento dei singoli elementi.
P.e. se si vuole inserire un array di N righe x M colonne si deve seguire il seguente diagramma di flusso:



Il caricamento degli elementi avviene per riga, da questo flow-chart si ottiene la seguente codifica:

```
import numpy as np
...
...
for i in range (0,N):
     for j in range (0,M):
          a[i][j]=int(input())
...
...
```

N.B. Gli estremi del ciclo for, indicati nella codifica sono differenti da quelli indicati nel

flow-chart perché la funzione range non comprende l'estremo superiore.

2.4 Modulo pil

Python Imaging Library è una libreria aggiuntiva gratuita e open source per il linguaggio di programmazione Python che aggiunge il supporto per l'apertura, la manipolazione e il salvataggio di diversi formati di file immagine .
Lo sviluppo del progetto originale, noto come PIL , è stato interrotto nel 2011. Successivamente, un progetto successivo chiamato Pillow ha biforcato il repository PIL e ha aggiunto il supporto per Python 3.x. Questo fork è stato adottato come sostituto del PIL originale nelle distribuzioni Linux tra cui Debian [5] e Ubuntu (dal 13.04). [1]

PIL è l'acronimo di Python Imaging Library, consiste in una collezione di moduli per gestire le immagini con l'interprete Python.
Sono presenti i seguenti moduli:

- Image
- ImageChops
- ImageCms
- ImageColor
- ImageDraw
- ImageEnhance
- ImageFile
- ImageFilter
- ImageFont
- ImageGrab
- ImageMath
- ImageMorph
- ImageOps
- ImagePalette
- ImagePath
- ImageQt
- ImageSequence
- ImageShow
- ImageStat
- ImageTk
- ImageWin
- ExifTags
- TiffTags
- JpegPresets
- PSDraw
- PixelAccess
- PyAccess
- features

Noi utilizzeremo il modulo Image che ci consentirà di creare un'immagine e di modificarla.

[1] https://en.wikipedia.org/wiki/Python_Imaging_Library

Per una panoramica dettagliata di tutti i moduli presenti si dia un'occhiata alla documentazione ufficiale (https://pillow.readthedocs.io/en/stable/)

Prima di tutto si importa il modulo:

```
from PIL import Image
```

2.4.1 Creare un'immagine

```
img=Image.new('RGB',(200,200),(255,255,255))
```

Dove:
il primo parametro è la modalità, si possono avere diverse modalità:

- '1': pixel a 1 bit, bianco e nero
- 'L': pixel a 8 bit, 256 tonalità di grigio
- 'P': pixel a 8 bit, con una palette di 256 colori
- 'RGB': 3 byte per pixel, modalità true color
- 'RGBA': 4 byte per pixel, modalità true color con banda alpha (trasparenza)
- 'I': pixel interi a 32 bit
- 'F': pixel a virgola mobile a 32 bit

Il secondo parametro è una coppia di numeri che indica la dimensione dell'immagine in pixel, il primo numero rappresenta la dimensione orizzontale ed il secondo quella verticale.

Il terzo parametro rappresenta il colore di sfondo, se omesso il colore sarà nero. Se mode='RGB' allora si dovrà fornire una terna di numeri, se mode = 'L' si fornirà solo un numero.

img rappresenta l'handle dell'immagine al quale ci riferiremo per qualsiasi operazione si voglia effettuare sull'immagine stessa.

2.4.2 Modificare un'immagine

```
img.putpixel((x,y),(r,g,b))
```

Dove:
(x,y) rappresenta la coordinata del punto dell'immagine sulla quale vogliamo agire
(r,g,b) rappresenta la terna del colore (in modalità 'RGB')

Con putpixel si va a colorare il pixel selezionato con il colore indicato.

2.4.3 Salvare un'immagine

```
img.save('nomeimmagine.ext')
```

Dove:

nomeimmagine è il pathname dell'immagine che si vuole salvare

ext è l'estensione del file, individua il tipo di file grafico che si vuole creare. p.e. se si indica 'bmp' allora si creerà un file bitmap, con 'jpg' un file jpeg, e così via.

Con queste tre primitive si riesce a creare tutti i grafici possibili.

3. Librerie per il Data Science

3.1 MatPlotLib

Dal sito ufficiale di matplotlib:[2] Matplotlib è una libreria completa per la creazione di visualizzazioni statiche, animate e interattive in Python. Matplotlib rende le cose facili facili e le cose difficili possibili.

- ➔ Creare grafici di qualità della pubblicazione .
- ➔ Crea figure interattive che si possono ingrandire, scorrere, aggiornare.
- ➔ Personalizza lo stile visivo e il layout .
- ➔ Esporta in molti formati di file .
- ➔ Incorpora in JupyterLab e interfacce utente grafiche .
- ➔ Usa una vasta gamma di pacchetti di terze parti costruiti su Matplotlib.

3.1.1 Un primo esempio

Direttamente dal sito ufficiale proviamo il seguente codice:
L'esempio utilizza i seguenti moduli:

```
import matplotlib as mpl
import matplotlib.pyplot as plt
import numpy as np
fig, ax = plt.subplots()  # Create a figure containing a single
axes.
ax.plot([1, 2, 3, 4], [1, 4, 2, 3]);  # Plot some data on the
axes.
plt.show()  # apre una finestra e vi visualizza il grafico
```

Lanciamo il programma, l'output stampato a video è il seguente:

```
[<matplotlib.lines.Line2D          object          at
0x7f6660c6cb50>]
```

Inoltre si apre una finestra e il prompt rimane in sospeso impedendoci di inserire altri comandi. La finestra che si apre è simile alla seguente:

[2] https://matplotlib.org/

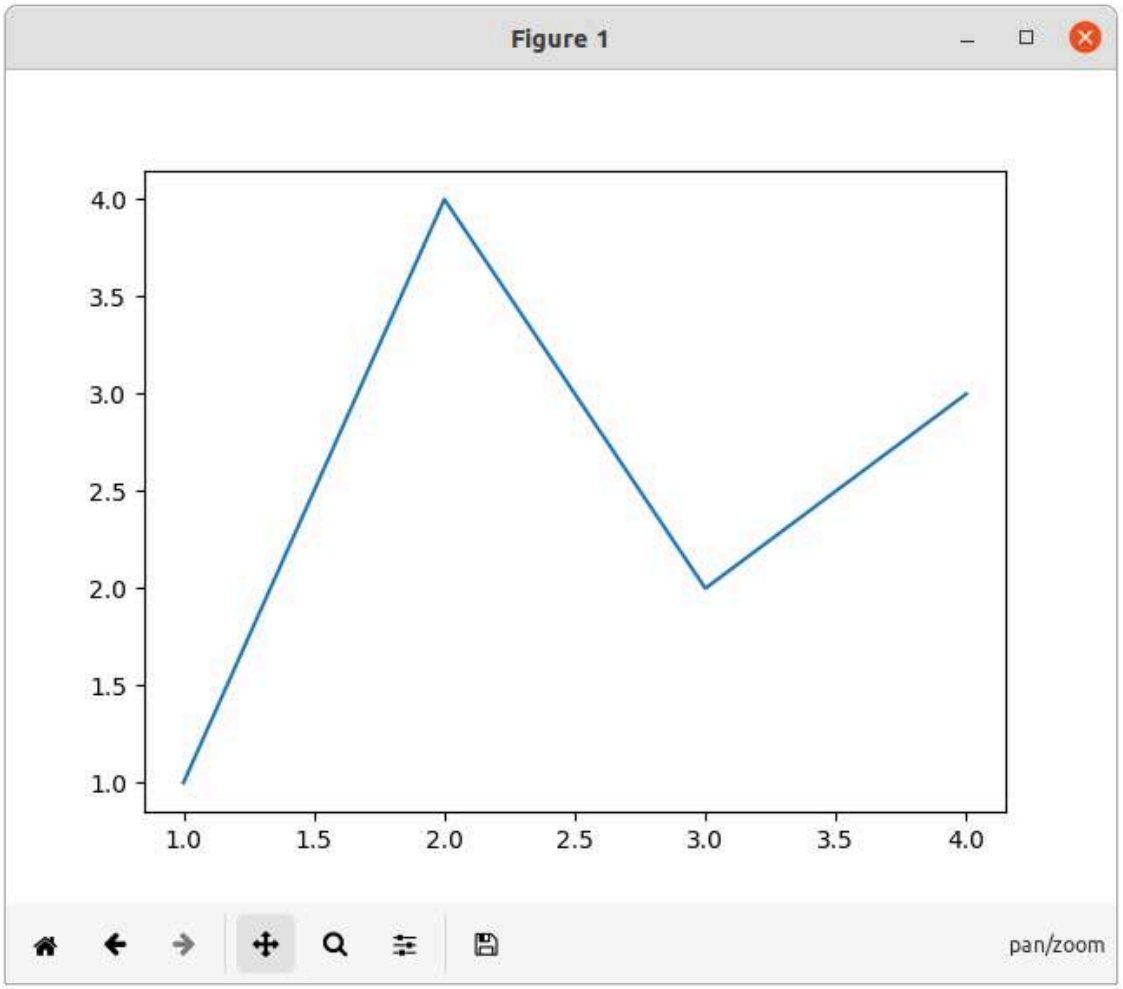


Figura 1

Dall'analisi del codice e dell'output osserviamo che:

➔ l'istruzione subplot genera un handle che identifica univocamente un grafico
➔ l'istruzione plot inserisce i punti che saranno graficati
➔ L'istruzione show mostra la finestra con il grafico

La finestra aperta contiene un applicativo che consente di "esplorare" il grafico effettuando zoom e traslazioni, inoltre permette di salvare l'immagine

3.1.2 Oggetti di un grafico

Di seguito gli oggetti presenti in un grafico.

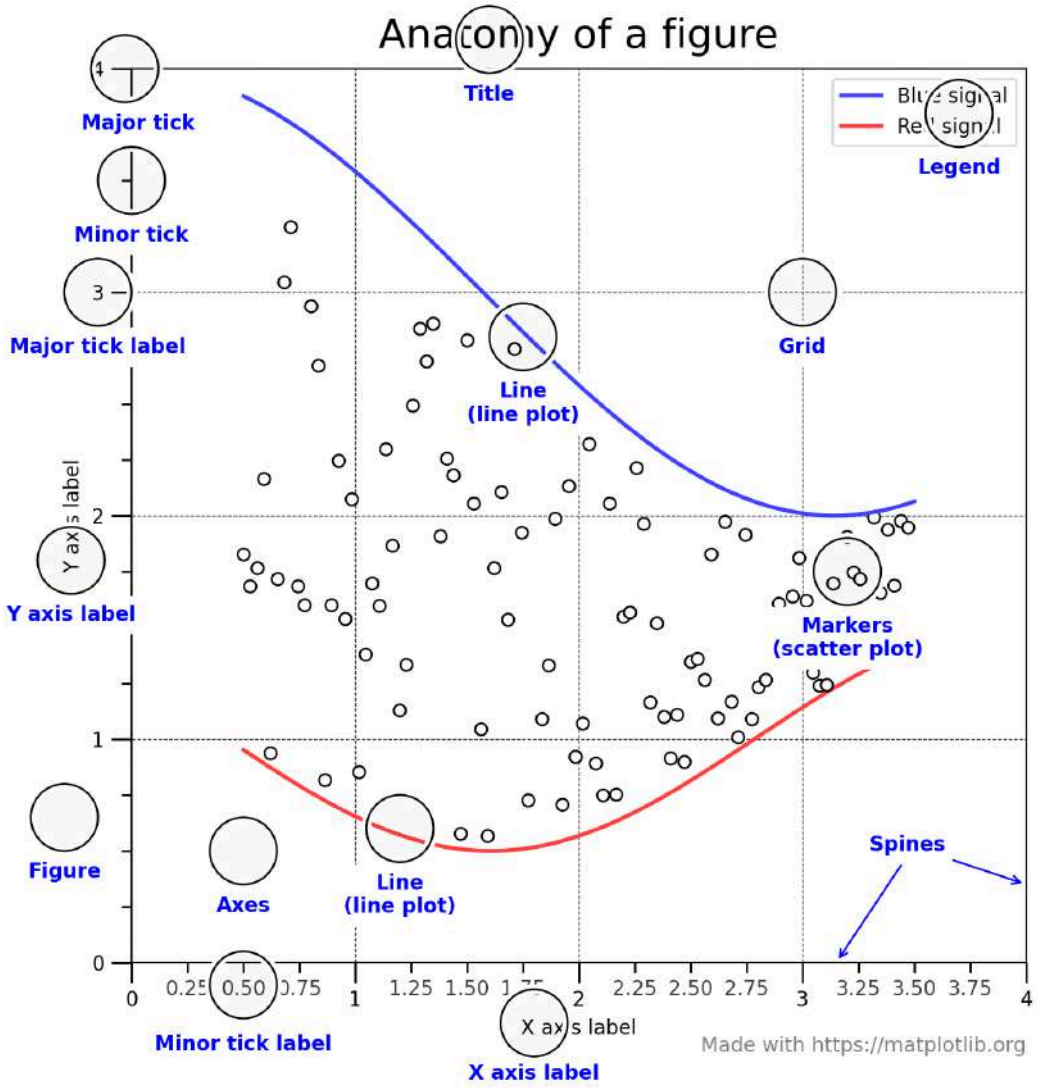


Figura 2

L' intera figura (figure) tiene traccia di tutti gli oggetti assi (Axes), di un gruppo di oggetti "speciali" (titoli, legende delle figure, barre dei colori, ecc.) e persino di sottofigure nidificate.

Il modo più semplice per creare una nuova figura è con pyplot:

```
import matplotlib as mpl
import matplotlib.pyplot as plt
import numpy as np
fig = plt.figure()  # an empty figure with no Axes
fig, ax = plt.subplots()  # a figure with a single Axes
fig, axs = plt.subplots(2, 2)  # a figure with a 2x2 grid of Axes
plt.show()
```

Questo codice crea tre grafici (figure) dalle caratteristiche indicate nei commenti presenti

nel codice. I grafici vengono visualizzati non appena viene eseguito il comando `plt.show()`

3.1.3 Un esempio più complesso

Proviamo il seguente programma:

```
import matplotlib as mpl
import matplotlib.pyplot as plt
import numpy as np

np.random.seed(19680801)  # seed the random number generator.
data = {'a': np.arange(50),
        'c': np.random.randint(0, 50, 50),
        'd': np.random.randn(50)}
data['b'] = data['a'] + 10 * np.random.randn(50)
data['d'] = np.abs(data['d']) * 100

fig, ax = plt.subplots(figsize=(5, 2.7), layout='constrained')
ax.scatter('a', 'b', c='c', s='d', data=data)
ax.set_xlabel('entry a')
ax.set_ylabel('entry b');

plt.show()
```

In output si ottiene il seguente grafico:

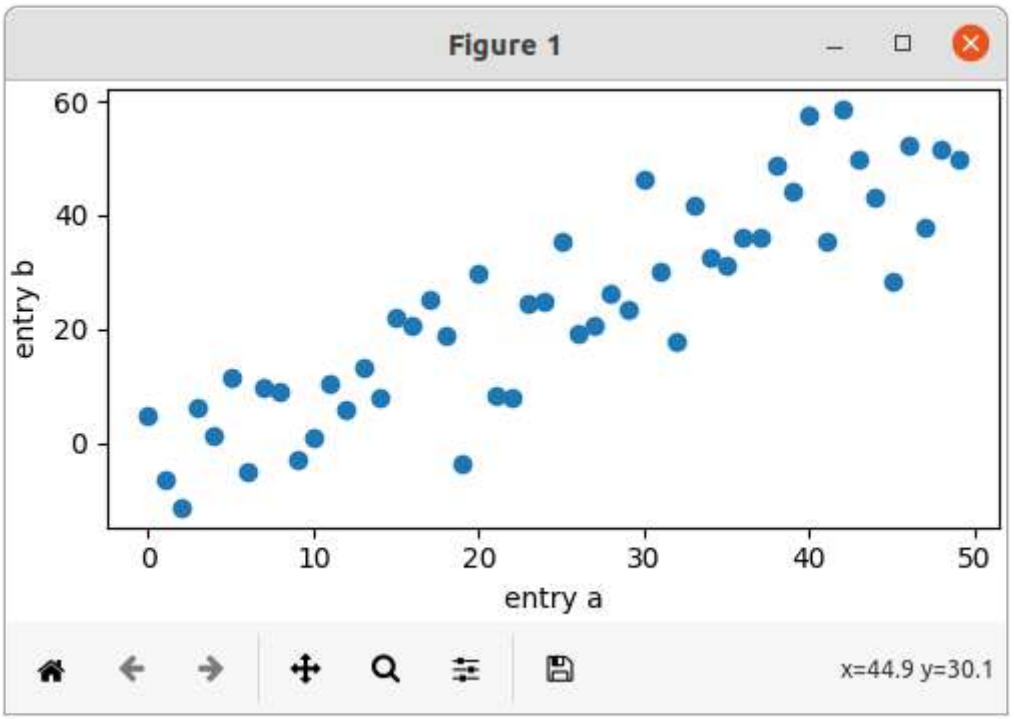


Figura 3

L'array data contiene 4 serie di dati marcate con le prime 4 lettere dell'alfabeto. La prima

serie rappresenta la variabile indipendente mentre le successive tre le grandezze che si vogliono rappresentare.

3.1.4 Grafico cartesiano di punti e funzioni

Utilizziamo l'insieme di primitive contenute in matplotlib.pyplot.

Proviamo il seguente programma:

```
import matplotlib.pyplot as plt
plt.plot([1, 2, 3, 4])
plt.ylabel('some numbers')
plt.show()
```

Il grafico che si ottiene è il seguente:

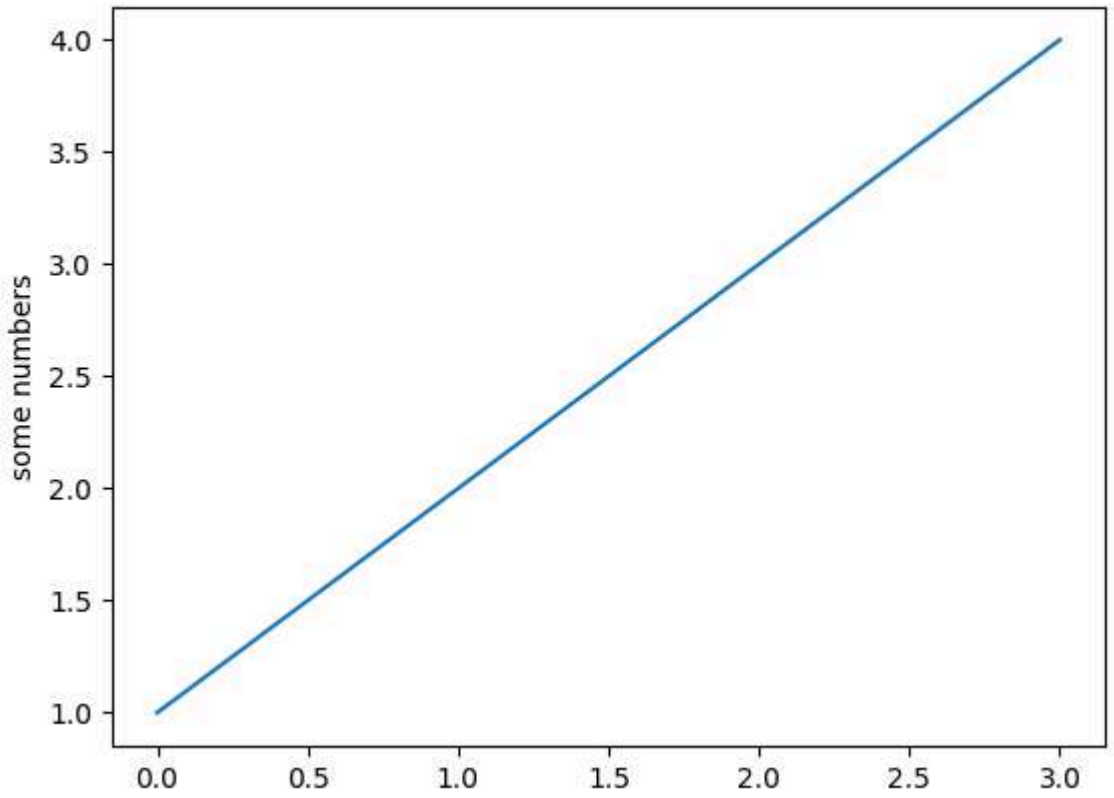


Figura 4

l'asse x va da 0-3 e l'asse y da 1-4. Se fornisci un singolo elenco o un array alla funzione plot, matplotlib presume che sia una sequenza di valori y e genera automaticamente i valori x. Poiché gli intervalli Python iniziano con 0, il vettore x predefinito ha la stessa lunghezza di y ma inizia con 0. Quindi i dati x sono : [0, 1, 2, 3]
Plot è una funzione versatile e accetta un numero arbitrario di argomenti. Ad esempio, per tracciare x rispetto a y, si può scrivere:

```
plt.plot([1, 2, 3, 4], [1, 4, 9, 16])
```

Se la seconda riga del precedente programma si sostituisse con questa nuova riga si otterrebbe il seguente grafico:

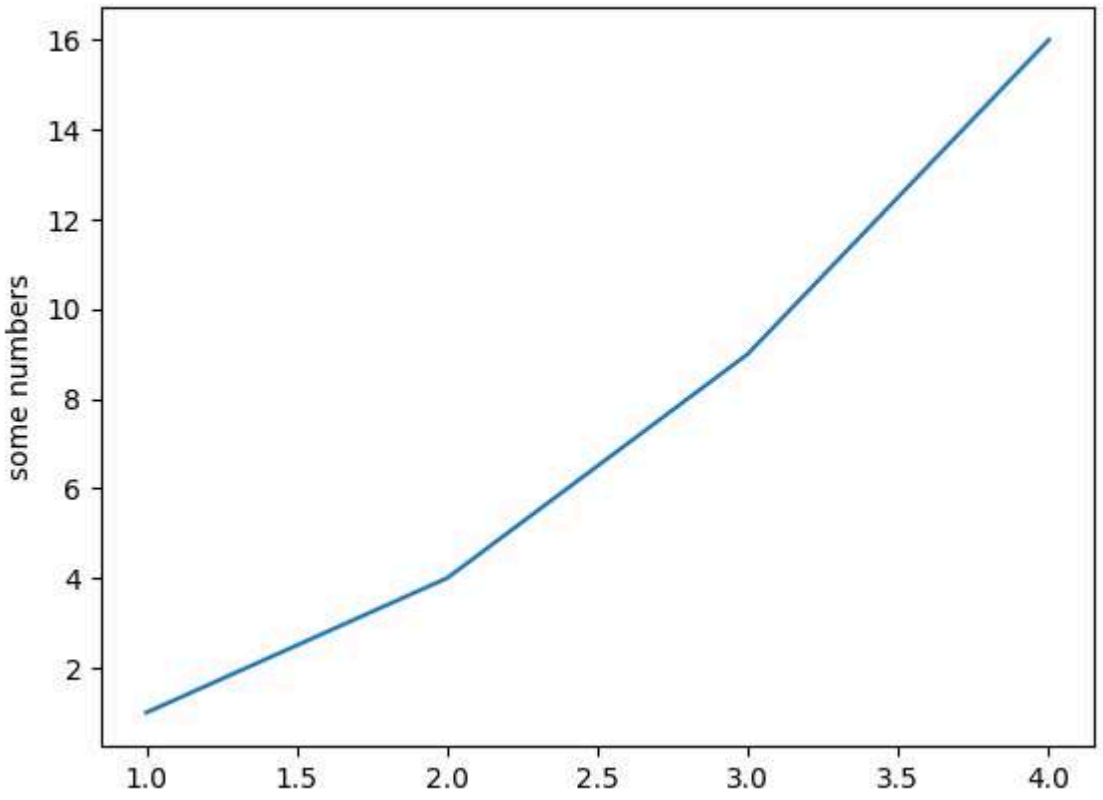


Figura 5

Per ogni coppia x, y di argomenti, c'è un terzo argomento opzionale che è la stringa di formato che indica il colore e il tipo di linea del grafico. Le lettere e i simboli della stringa di formato provengono da MATLAB, puoi concatenare una stringa di colori con una stringa di stile di linea. La stringa di formato predefinita è 'b-', che è una linea blu continua. Ad esempio, per tracciare quanto sopra con cerchi rossi, dovresti emettere:

```
plt.plot([1, 2, 3, 4], [1, 4, 9, 16], 'ro')
```

Proviamo il seguente programma:

```
import matplotlib.pyplot as plt
plt.plot([1, 2, 3, 4], [1, 4, 9, 16], 'ro')
plt.axis([0, 6, 0, 20])
plt.show()
```

Otteniamo il seguente grafico:

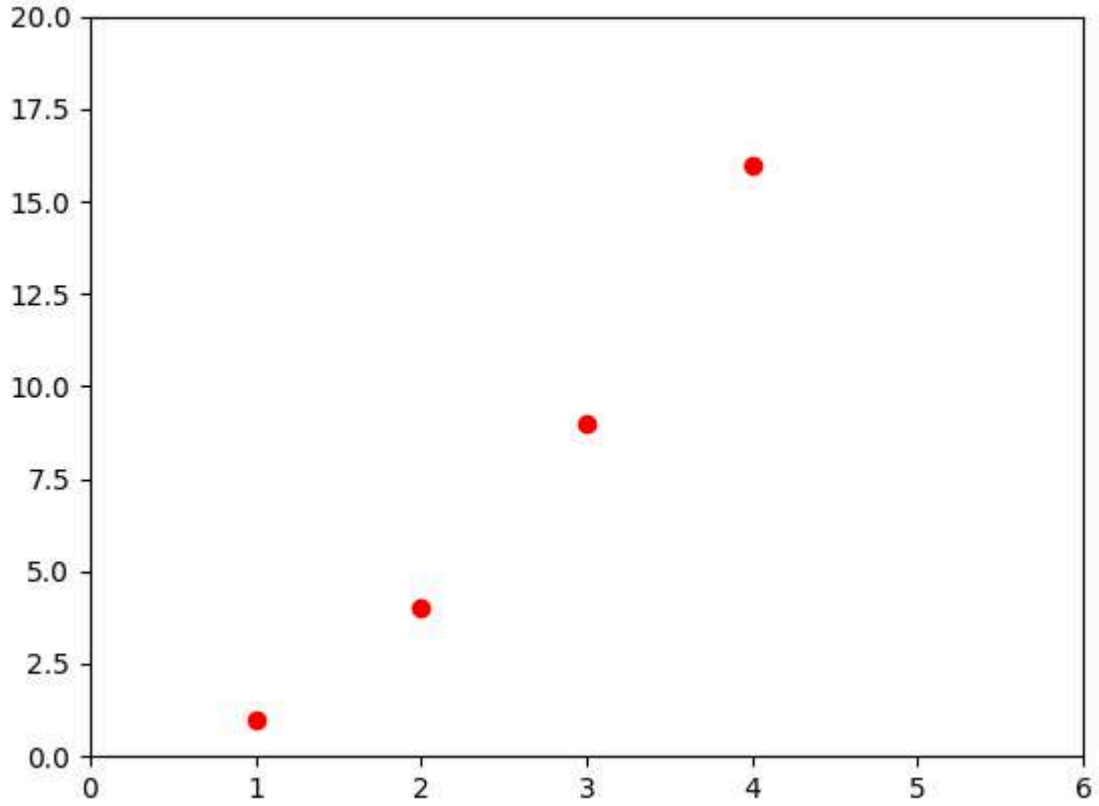


Figura 6

Consulta la documentazione della funzione plot[3] per avere un elenco completo degli stili di linea e delle stringhe di formato.
Nel programma precedente la funzione axis serve per specificare i valori min e max degli assi cartesiani nel seguente ordine [xmin, xmax, ymin, ymax]
Finalmente eseguiamo il seguente programma che ci permette di graficare due funzioni nello stesso piano cartesiano:

```
import numpy as np
import matplotlib.pyplot as plt
# evenly sampled time at 200ms intervals
t = np.arange(0., 5., 0.2)
# red dashes, blue squares and green triangles
plt.plot(t, t, 'r--', t, t**2, 'bs', t, t**3, 'g^')
plt.show()
```

Lanciando il programma si ottiene il seguente output

[3] https://matplotlib.org/stable/api/_as_gen/matplotlib.pyplot.plot.html#matplotlib.pyplot.plot

```
[<matplotlib.lines.Line2D object at 0x7fdd063f0fd0>, <matplotlib.lines.Line2D object at 0x7fdd063f0fa0>, <matplotlib.lines.Line2D object at 0x7fdd063861c0>]
```

e il seguente grafico:

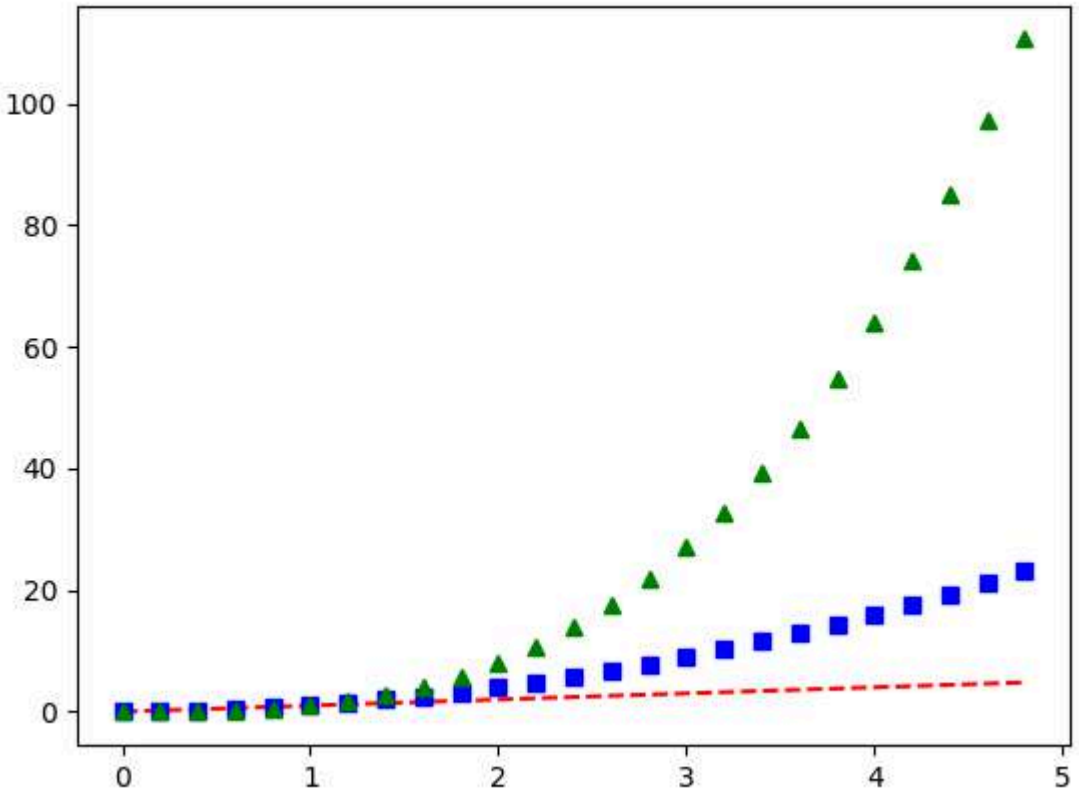


Figura 7

Dall'analisi del codice e dell'output osserviamo che:

➔ Abbiamo utilizzato un array numpy per i valori sull'asse delle ascisse. In effetti, tutte le sequenze vengono convertite internamente da pyplot in array numpy.

➔ Si è ottenuto il tracciamento di più curve con stili di formato diversi

➔ Al posto di passare a pyplot un array si è passato direttamente l'espressione di una funzione.

In generale matplotlib.pyplot è una raccolta di funzioni che fanno funzionare la libreria matplotlib come MATLAB. Ogni pyplot apporta alcune modifiche a una figura: ad esempio, crea una figura, crea un'area di stampa in una figura, traccia alcune linee in un'area di stampa, decora la stampa con etichette, ecc.

Per approfondimenti si consiglia di dare un'occhiata alla documentazione ufficiale da cui sono tratti gli esempi di questo paragrafo.[4]

[4] https://matplotlib.org/stable/tutorials/introductory/pyplot.html

3.2 Pandas

Pandas è una libreria software scritta per il linguaggio di programmazione Python per la manipolazione e l'analisi dei dati. In particolare, offre strutture dati e operazioni per manipolare tabelle numeriche e serie temporali. È un software libero rilasciato sotto la licenza BSD.
Il nome deriva dal termine "panel data", termine econometrico per set di dati che include osservazioni su più periodi di tempo per gli stessi individui.[5]

3.2.1 Creazione di oggetti

Questo paragrafo segue gli esempi di “10 minutes to pandas”[6]

3.2.1.1 Serie

Proviamo questo primo semplice programma:

```
import numpy as np
import pandas as pd
s = pd.Series([1, 3, 5, np.nan, 6, 8])
print(s)
```

Lanciando il programma si ottiene il seguente output:

```
0    1.0
1    3.0
2    5.0
3    NaN
4    6.0
5    8.0
dtype: Float64
```

Dall’analisi del codice e dell’output osserviamo che:
- ➔ Abbiamo creato una serie[7] da un elenco di valori
- ➔ La libreria Pandas ha creato un indice intero predefinito
- ➔ Abbiamo inserito un valore Nan tra valori numerici

3.2.1.2 DataFrame

Diamo un’occhiata al seguente programma:

```
import numpy as np
import pandas as pd
```

[5] https://it.wikipedia.org/wiki/Pandas_(software)
[6] https://pandas.pydata.org/docs/user_guide/10min.html
[7] https://pandas.pydata.org/docs/reference/api/pandas.Series.html#pandas.Series

```
dates = pd.date_range("20130101", periods=6)
print(dates)
df = pd.DataFrame(np.random.randn(6, 4), index=dates,
columns=list("ABCD"))
print(df)
```

Lanciando il programma si ottiene il seguente output:

```
DatetimeIndex(['2013-01-01','2013-01-02','2013-01-03','2013-01-04'
,'2013-01-05','2013-01-06'],dtype='datetime64[ns]', freq='D')
                   A         B         C         D
2013-01-01  0.549436 -0.927424  0.501841  0.785572
2013-01-02 -0.666015  1.220079  0.263829 -0.032252
2013-01-03  0.210596 -0.289914  0.571528 -0.037760
2013-01-04  0.107708  0.921978  0.129412  0.060242
2013-01-05 -0.660988  1.015658 -0.969598 -0.470753
2013-01-06  0.308218 -0.514407 -0.260273  0.013569
```

Dall'analisi del codice e dell'output osserviamo che:

➔ Abbiamo creato un DataFrame[8] da un array numpy

➔ Abbiamo creato un indice di date con la funzione *date_range* utilizzando il parametro *index* della funzione *DataFrame*

➔ Abbiamo creato delle colonne etichettate utilizzando il parametro *columns* della funzione *DataFrame*

Ora proviamo quest'altro programma più complesso:

```
import numpy as np
import pandas as pd
dizionario={
        "A": 1.0,
        "B": pd.Timestamp("20130102"),
        "C": pd.Series(1, index=list(range(4)), dtype="float32"),
        "D": np.array([3] * 4, dtype="int32"),
        "E": pd.Categorical(["test", "train", "test", "train"]),
        "F": "foo",
    }
df2 = pd.DataFrame(dizionario)
print(df2)
```

Lanciando il programma si ottiene il seguente output:

[8] https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.html#pandas.DataFrame

```
     A          B    C  D      E    F
0  1.0 2013-01-02  1.0  3   test  foo
1  1.0 2013-01-02  1.0  3  train  foo
2  1.0 2013-01-02  1.0  3   test  foo
3  1.0 2013-01-02  1.0  3  train  foo
```

Dall'analisi del codice e dell'output osserviamo che:

- ➔ Abbiamo creato un DataFrame da un dizionario python
- ➔ Le colonne del *DataFrame* hanno *dtype* differenti
- ➔ Anche in questo caso la libreria Pandas ha creato un indice intero predefinito

A riguardo dell'ultima osservazione, stampando `df2.dtypes` si ottiene:

```
A           float64
B    datetime64[ns]
C           float32
D             int32
E          category
F            object
dtype: object
```

Se vuoi vedere tutti i differenti *dtype* del dataframe *df2* puoi aprire l'interprete python e dopo aver importato le librerie e aver creato il dataframes puoi digitare dalla command line dell'interprete: `df2.<TAB>` (dopo il punto pigiare sul tasto tab della tastiera).
In questo modo verranno visualizzati tutti i dtype che l'oggetto puo' contenere. Ad esempio:

```
>>> df2.<TAB>
df2.A                  df2.bool
df2.abs                df2.boxplot
df2.add                df2.C
df2.add_prefix         df2.clip
df2.add_suffix         df2.columns
df2.align              df2.copy
df2.all                df2.count
df2.any                df2.combine
df2.append             df2.D
df2.apply              df2.describe
```

```
df2.applymap            df2.diff
df2.B                   df2.duplicated
```

Questi sono solo una parte degli attributi.
>>> è il prompt della console dell'interprete python

3.2.1.3 Visualizzazione dei dati

Consideriamo il dataframe prodotto dal seguente codice:

```
import numpy as np
import pandas as pd
dates = pd.date_range("20130101", periods=6)
df     =     pd.DataFrame(np.random.randn(6,     4),     index=dates,
columns=list("ABCD"))
```

Proviamo le istruzioni che ci permettono di visualizzare i dati contenuti nel dataframe:

3.2.1.3.1 Righe superiori

```
df.head()
```

```
                   A         B         C         D
2013-01-01  0.469112 -0.282863 -1.509059 -1.135632
2013-01-02  1.212112 -0.173215  0.119209 -1.044236
2013-01-03 -0.861849 -2.104569 -0.494929  1.071804
2013-01-04  0.721555 -0.706771 -1.039575  0.271860
2013-01-05 -0.424972  0.567020  0.276232 -1.087401
```

3.2.1.3.2 Righe inferiori

```
df.tail(3)
```

```
                   A         B         C         D
2013-01-04  0.721555 -0.706771 -1.039575  0.271860
2013-01-05 -0.424972  0.567020  0.276232 -1.087401
2013-01-06 -0.673690  0.113648 -1.478427  0.524988
```

3.2.1.3.3 Indice

```
df.index
```

```
DatetimeIndex(['2013-01-01','2013-01-02','2013-01-03',
'2013-01-04',
```

```
               '2013-01-05', '2013-01-06'],
              dtype='datetime64[ns]', freq='D')
```

3.2.1.3.4 Colonne

```
df.columns
```

```
Index(['A', 'B', 'C', 'D'], dtype='object')
```

3.2.1.3.5 Riepilogo statistico

```
df.describe()
```

```
              A         B         C         D
count  6.000000  6.000000  6.000000  6.000000
mean   0.073711 -0.431125 -0.687758 -0.233103
std    0.843157  0.922818  0.779887  0.973118
min   -0.861849 -2.104569 -1.509059 -1.135632
25%   -0.611510 -0.600794 -1.368714 -1.076610
50%    0.022070 -0.228039 -0.767252 -0.386188
75%    0.658444  0.041933 -0.034326  0.461706
max    1.212112  0.567020  0.276232  1.071804
```

3.2.1.3.6 Trasposta

```
df.T
```

```
     2013-01-01    2013-01-02    2013-01-03    2013-01-04    2013-01-05
2013-01-06
A       0.469112      1.212112     -0.861849      0.721555     -0.424972
-0.673690
B      -0.282863     -0.173215     -2.104569     -0.706771      0.567020
0.113648
C      -1.509059      0.119209     -0.494929     -1.039575      0.276232
-1.478427
D      -1.135632     -1.044236      1.071804      0.271860     -1.087401
0.524988
```

3.2.1.3.7 Ordinamento su un asse

```
df.sort_index(axis=1, ascending=False)
```

```
                   D         C         B         A
```

```
2013-01-01 -1.135632 -1.509059 -0.282863  0.469112
2013-01-02 -1.044236  0.119209 -0.173215  1.212112
2013-01-03  1.071804 -0.494929 -2.104569 -0.861849
2013-01-04  0.271860 -1.039575 -0.706771  0.721555
2013-01-05 -1.087401  0.276232  0.567020 -0.424972
2013-01-06  0.524988 -1.478427  0.113648 -0.673690
```

3.2.1.3.8 Ordinamento per valori

```
df.sort_values(by="B")
```

```
                   A         B         C         D
2013-01-03 -0.861849 -2.104569 -0.494929  1.071804
2013-01-04  0.721555 -0.706771 -1.039575  0.271860
2013-01-01  0.469112 -0.282863 -1.509059 -1.135632
2013-01-02  1.212112 -0.173215  0.119209 -1.044236
2013-01-06 -0.673690  0.113648 -1.478427  0.524988
2013-01-05 -0.424972  0.567020  0.276232 -1.087401
```

3.2.1.4 Selezione dei dati

Consideriamo il dataframe prodotto dal seguente codice:

```
import numpy as np
import pandas as pd
dates = pd.date_range("20130101", periods=6)
df    =    pd.DataFrame(np.random.randn(6,    4),    index=dates,
columns=list("ABCD"))
```

Proviamo le istruzioni che ci permettono di selezionare i dati contenuti nel dataframe:

Selezionare una singola colonna ci porta a ottenere una serie equivalente a `df.A`:

```
df["A"]
```

```
2013-01-01    0.469112
2013-01-02    1.212112
2013-01-03   -0.861849
2013-01-04    0.721555
2013-01-05   -0.424972
2013-01-06   -0.673690
Freq: D, Name: A, dtype: float64
```

Possiamo selezionare le righe:

```
df[0:3]
```

```
                   A         B         C         D
2013-01-01  0.469112 -0.282863 -1.509059 -1.135632
2013-01-02  1.212112 -0.173215  0.119209 -1.044236
2013-01-03 -0.861849 -2.104569 -0.494929  1.071804
```

o anche:

```
df["20130102":"20130104"]
```

```
                   A         B         C         D
2013-01-02  1.212112 -0.173215  0.119209 -1.044236
2013-01-03 -0.861849 -2.104569 -0.494929  1.071804
2013-01-04  0.721555 -0.706771 -1.039575  0.271860
```

Abbiamo utilizzato le espressioni standard di Python per selezionare i dati presenti in una collezione.
Pandas raccomanda di utilizzare, negli ambienti di produzione, i metodi di accesso ottimizzati:

- ➔ `.at`
- ➔ `.iat`
- ➔ `.loc`
- ➔ `.iloc`

Si rimanda alla documentazione ufficiale di Pandas per l'utilizzo di questi metodi.

3.2.1.5 Operazioni sul DataFrame

Consideriamo il dataframe prodotto dal seguente codice:

```
import numpy as np
import pandas as pd
dates = pd.date_range("20130101", periods=6)
df = pd.DataFrame(np.random.randn(6, 4), index=dates,
columns=list("ABCD"))
```

Proviamo le istruzioni che ci permettono di effettuare delle semplici operazioni sui dati contenuti nel dataframe:

3.2.1.5.1 Calcolo della media

```
df.mean()
```

```
A   -0.004474
```

```
B   -0.383981
C   -0.687758
D    5.000000
F    3.000000
dtype: float64
```

Stessa operazione sull'altro asse:

```
df.mean()
```

```
2013-01-01    0.872735
2013-01-02    1.431621
2013-01-03    0.707731
2013-01-04    1.395042
2013-01-05    1.883656
2013-01-06    1.592306
Freq: D, dtype: float64
```

Si rimanda alla documentazione ufficiale di Pandas per l'utilizzo delle varie operazioni che si possono eseguire sui DataFrame.

3.2.1.6 Graficare i DataFrame

3.2.1.6.1 Grafico cartesiano di una serie

Proviamo il seguente programma:

```
import matplotlib.pyplot as plt
import pandas as pd
import numpy as np

ts = pd.Series(np.random.randn(1000),
index=pd.date_range("1/1/2020", periods=1000))
ts = ts.cumsum()
ts.plot()

plt.show()
```

In output si ottiene il seguente grafico:

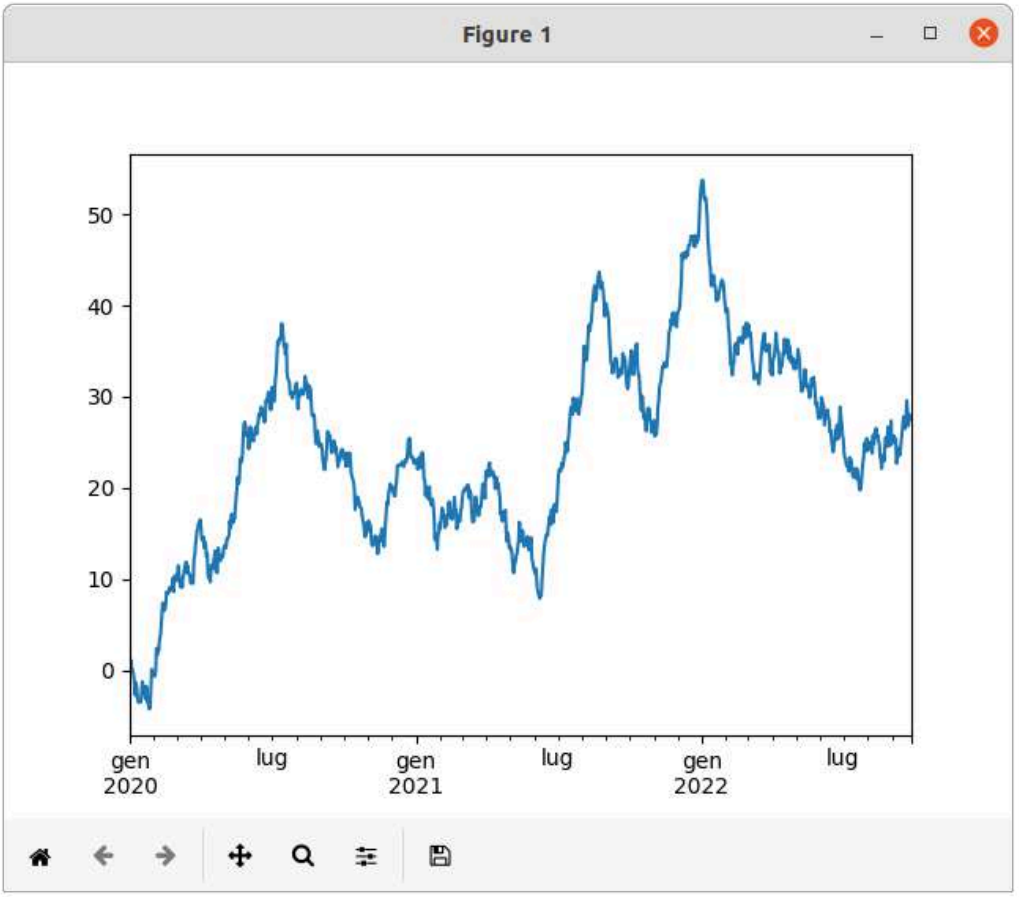


Figura 1

Dall'analisi del codice e dell'output osserviamo che:

- *ts* è una serie temporale formata da 1000 numeri pseudocasuali.
- L'indice di *ts* è rappresentato da un range di 1000 date a partire da 01/01/2020
- Il metodo `cumsum` effettua la somma cumulata dei valori
- `plot` è il metodo che ci permette di creare il grafico
- `plot.show` ci permette di mostrare sullo standard output il grafico creato con `plot`

3.2.1.6.2 Grafico cartesiano di un DataFrame

Proviamo il seguente programma:

```
import matplotlib.pyplot as plt
import pandas as pd
import numpy as np

df = pd.DataFrame(
    np.random.randn(1000, 4), index=pd.date_range("1/1/2020",
    periods=1000), columns=["A", "B", "C", "D"]
)
df = df.cumsum()
df.plot()
```

```
plt.show()
```

In output si ottiene il seguente output:

```
                   A         B           C           D
2020-01-01  -1.532794 -0.932718    0.243491  -1.023839
2020-01-02  -1.179775 -0.352232    0.306069  -1.479430
2020-01-03  -0.552050 -0.550845   -0.496691  -2.001847
2020-01-04  -0.960933  0.761976    1.558426  -3.191399
2020-01-05   0.518609  0.888329    2.079830  -3.581038
...               ...       ...         ...         ...
2022-09-22  24.185539  0.291500  -48.595703 -32.539170
2022-09-23  23.589834 -0.996163  -48.149875 -32.128592
2022-09-24  23.639240 -0.302085  -46.889850 -31.992047
2022-09-25  24.065064 -2.144455  -46.181928 -32.840093
2022-09-26  25.187588 -3.426459  -47.594617 -30.697849

[1000 rows x 4 columns]
```

e il seguente grafico:

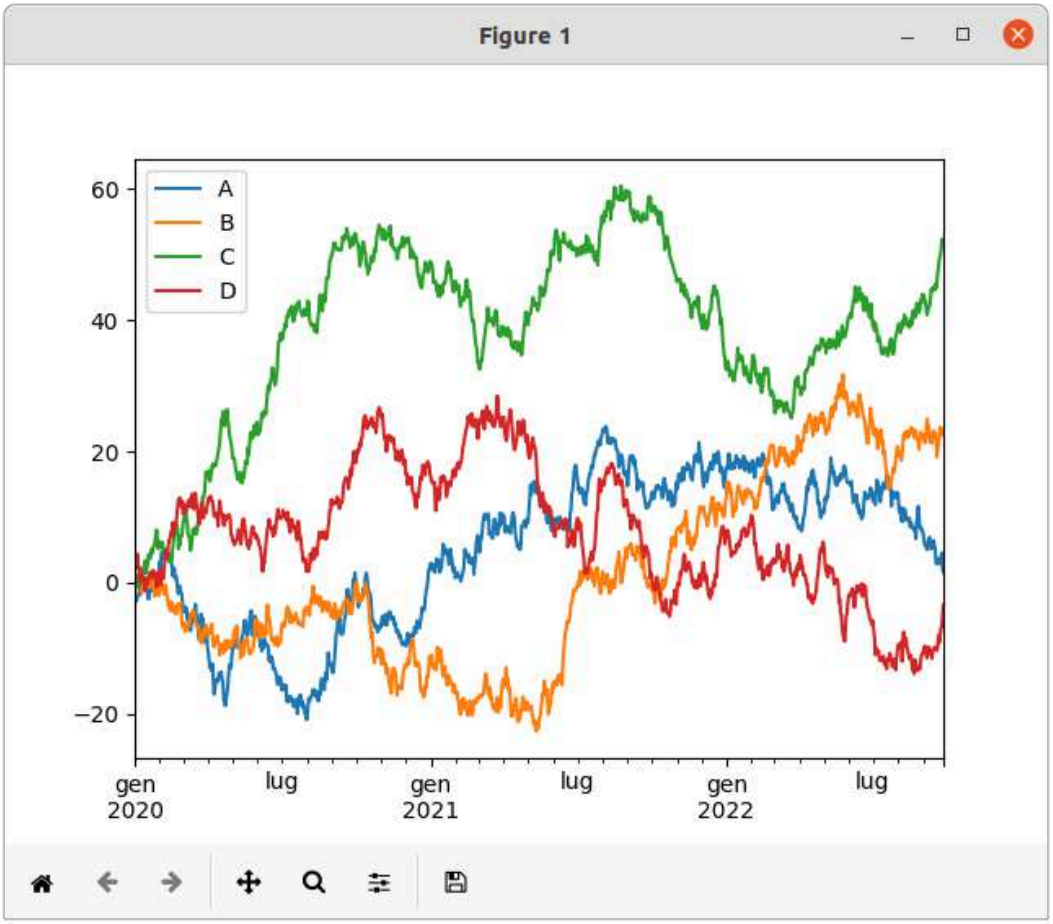


Figura 2

Dall'analisi del codice e dell'output osserviamo che:

- *df* è un DataFrame che rappresenta una matrice di 1000 righe e 5 colonne. La prima colonna contiene gli *indici* delle righe serie. Ogni colonna è una serie temporale formata da 1000 numeri pseudocasuali.
- Il metodo `cumsum` effettua la somma cumulata dei valori delle colonne. Analogamente a quanto fatto nell'esempio precedente, serve per 'addolcire' i valori e rendere meno sovrapponibile il grafico delle serie stesse. Si invita a provare lo stesso codice senza inserire il metodo `cumsum`.
- `plot` è il metodo che ci permette di creare il grafico, quindi anche l'oggetto *DataFrame* contiene questo metodo.
- `plot.show` ci permette di mostrare sullo standard output il grafico creato con `plot`
- la funzione print ci permette di stampare i valori del DataFrame per capire con che tipo di valori abbiamo a che fare. Naturalmente se riproducete questo codice otterrete dei valori differenti in quanto sono tutti numeri estratti casualmente.

3.2.1.7 Dati da file CSV

Proviamo il seguente programma:

```
import pandas as pd
import numpy as np
```

```
df = pd.DataFrame(
        np.random.randn(1000,  4),  index=pd.date_range("1/1/2020",
periods=1000), columns=["A", "B", "C", "D"]
)
df = df.cumsum()
print(df)
df.to_csv("miofile.csv")
```

Si ottiene il seguente output:

```
                   A         B          C          D
2020-01-01  -1.532794 -0.932718   0.243491  -1.023839
2020-01-02  -1.179775 -0.352232   0.306069  -1.479430
2020-01-03  -0.552050 -0.550845  -0.496691  -2.001847
2020-01-04  -0.960933  0.761976   1.558426  -3.191399
2020-01-05   0.518609  0.888329   2.079830  -3.581038
...               ...       ...        ...        ...
2022-09-22  24.185539  0.291500 -48.595703 -32.539170
2022-09-23  23.589834 -0.996163 -48.149875 -32.128592
2022-09-24  23.639240 -0.302085 -46.889850 -31.992047
2022-09-25  24.065064 -2.144455 -46.181928 -32.840093
2022-09-26  25.187588 -3.426459 -47.594617 -30.697849

[1000 rows x 4 columns]
```

Abbiamo riprodotto esattamente lo stesso *DataFrame* dell'esempio precedente (naturalmente con valori differenti).
Inoltre abbiamo creato sul Filesystem nella stessa directory da cui abbiamo lanciato il nostro script Python, un file di nome: `miofile.csv`. Questo file è stato creato dal metodo `to_csv` presente nell'oggetto *DataFrame*.
Eseguiamo dal prompt della shell questo comando:

```
$ ls miofile.csv
```

Si otterrà la seguente risposta:

```
miofile.csv
```

Quindi il file è effettivamente stato creato.
Esploriamone il contenuto visualizzando le prime righe e contando il numero totale delle righe del file stesso. Eseguiamo queste operazioni utilizzando due comandi GNU/Linux:

```
$ head miofile.csv
```

Questo comando visualizza le prime 10 righe del file indicato come argomento. Si ootiene il seguente output:

```
,A,B,C,D
2020-01-01,-0.26553883988890686,0.23340564688360357,0.9285786287435395,0.3836291152246073
2020-01-02,-1.2803238179859378,1.4277657179651781,0.5753853865597792,-0.13838317313087656
2020-01-03,-0.41054685585525097,0.7727412598920795,0.04834837264955183,0.359233663070411
2020-01-04,0.3286397037979193,0.2587987908990993,0.08924005746134442,1.326765432676096
2020-01-05,-0.7696130170464265,0.46996810603381906,1.31734798808734,2.159063311644289
2020-01-06,-0.8403692470864985,1.7178011417088994,1.3481533169712874,1.771168450314741
2020-01-07,-0.4878606279126995,2.266925842617381,2.0007153774873347,1.5034277123304995
2020-01-08,0.5245105390330209,2.1824004708082225,1.4038987638789937,2.374844095574561
2020-01-09,-0.4772940720107679,2.1790398906625024,2.4478993516208787,4.470582369487415
```

L'output è analogo al contenuto del DataFrame che avevamo creato, notiamo che tutti i valori sono separati dalla virgola come richiede il formato dei file CSV.

Ora proviamo a contare il numero di righe utilizzando il seguente comando:

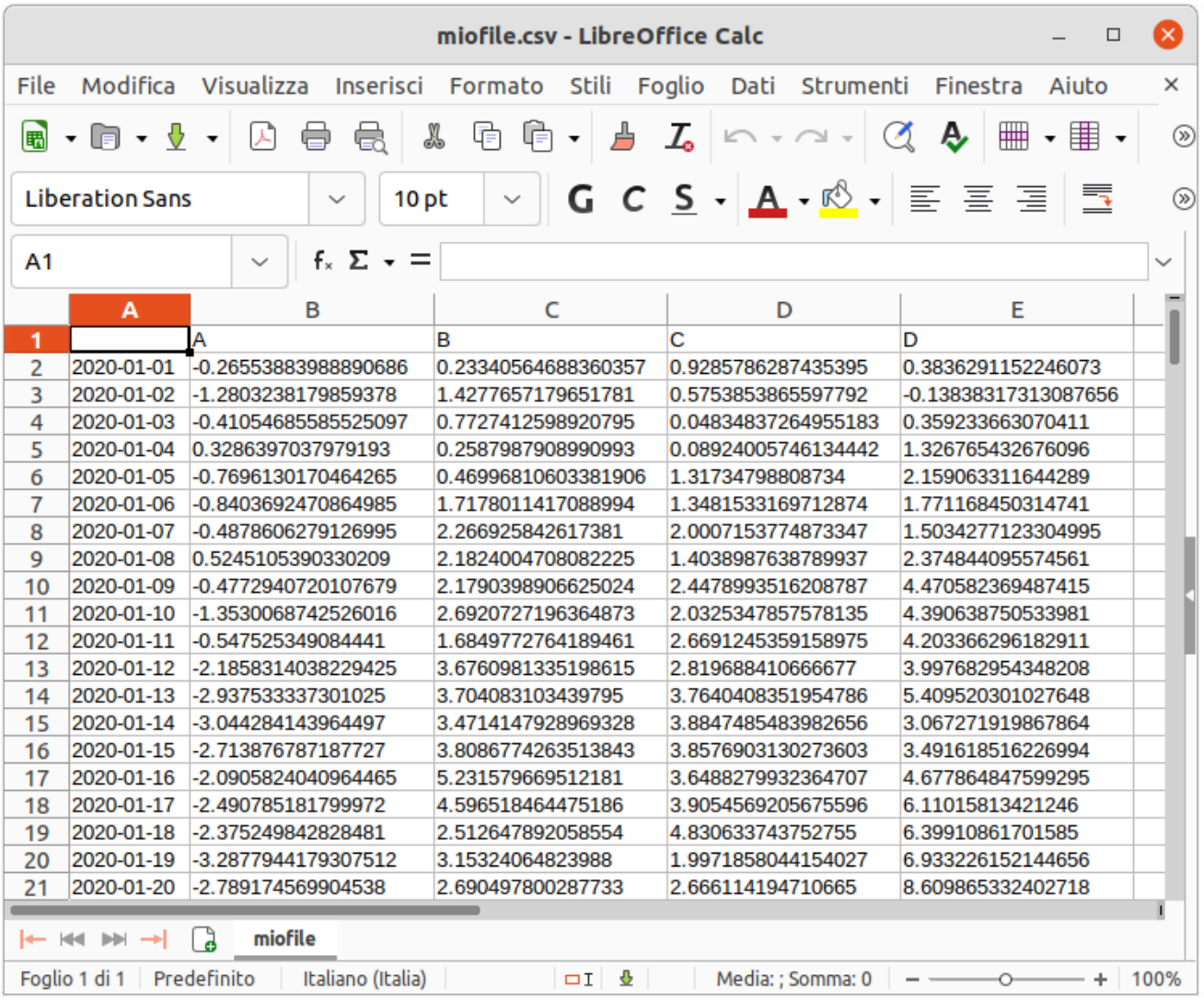
```
$ wc -l miofile.csv
```

`wc` è un comando che restituisce il conteggio dei caratteri di un file di testo dato come argomento. Se si utilizza il parametro `-l` visualizzerà il numero di righe contenute nel file. Quindi si otterrà il seguente output:

```
1001 miofile.csv
```

Il risultato è concorde con quanto ci aspettavamo in quanto le righe del nostro *DataFrame* sono 1000, una riga è riservata per l'intestazione delle colonne.

Proviamo ad aprire il nostro file con un programma applicativo che interpreta i files CSV, p.e. *LibreOffice Calc*. Otteniamo la seguente finestra:



	A	B	C	D	E
1		A	B	C	D
2	2020-01-01	-0.26553883988890686	0.23340564688360357	0.9285786287435395	0.3836291152246073
3	2020-01-02	-1.2803238179859378	1.4277657179651781	0.5753853865597792	-0.13838317313087656
4	2020-01-03	-0.41054685585525097	0.7727412598920795	0.04834837264955183	0.359233663070411
5	2020-01-04	0.3286397037979193	0.2587987908990993	0.08924005746134442	1.326765432676096
6	2020-01-05	-0.7696130170464265	0.46996810603381906	1.31734798808734	2.159063311644289
7	2020-01-06	-0.8403692470864985	1.7178011417088994	1.3481533169712874	1.771168450314741
8	2020-01-07	-0.4878606279126995	2.266925842617381	2.0007153774873347	1.5034277123304995
9	2020-01-08	0.5245105390330209	2.1824004708082225	1.4038987638789937	2.374844095574561
10	2020-01-09	-0.4772940720107679	2.1790398906625024	2.4478993516208787	4.470582369487415
11	2020-01-10	-1.3530068742526016	2.6920727196364873	2.0325347857578135	4.390638750533981
12	2020-01-11	-0.547525349084441	1.6849772764189461	2.6691245359158975	4.203366296182911
13	2020-01-12	-2.1858314038229425	3.6760981335198615	2.819688410666677	3.997682954348208
14	2020-01-13	-2.937533337301025	3.704083103439795	3.7640408351954786	5.409520301027648
15	2020-01-14	-3.044284143964497	3.4714147928969328	3.8847485483982656	3.067271919867864
16	2020-01-15	-2.713876787187727	3.8086774263513843	3.8576903130273603	3.491618516226994
17	2020-01-16	-2.0905824040964465	5.231579669512181	3.6488279932364707	4.677864847599295
18	2020-01-17	-2.490785181799972	4.596518464475186	3.9054569205675596	6.11015813421246
19	2020-01-18	-2.375249842828481	2.512647892058554	4.830633743752755	6.39910861701585
20	2020-01-19	-3.2877944179307512	3.15324064823988	1.9971858044154027	6.933226152144656
21	2020-01-20	-2.789174569904538	2.690497800287733	2.666114194710665	8.609865332402718

Figura 3

Quindi effettivamente abbiamo salvato il nostro DataFrame su un file utilizzando il formato CSV.

Ora proviamo a leggere un file CSV e a stampare il risultato.
Eseguiamo il seguente programma:

```
import pandas as pd

df= pd.read_csv("miofile.csv")
print(df)
print(type(df))
```

Otteniamo il seguente output:

```
     Unnamed: 0           A           B           C           D
0    2020-01-01   -0.265539    0.233406    0.928579    0.383629
1    2020-01-02   -1.280324    1.427766    0.575385   -0.138383
```

```
2     2020-01-03  -0.410547   0.772741   0.048348   0.359234
3     2020-01-04   0.328640   0.258799   0.089240   1.326765
4     2020-01-05  -0.769613   0.469968   1.317348   2.159063
..           ...         ...        ...        ...        ...
995   2022-09-22 -39.276224 -29.052450  22.999748   8.425073
996   2022-09-23 -37.812475 -28.315831  24.017133   9.500369
997   2022-09-24 -38.230467 -27.997949  24.234850   8.609038
998   2022-09-25 -38.476028 -26.815262  23.824295   8.948532
999   2022-09-26 -37.742205 -26.011061  26.098446  10.197984

[1000 rows x 5 columns]
<class 'pandas.core.frame.DataFrame'>
```

Dall'analisi del codice e dell'output osserviamo che:

- il metodo `read_csv` permette a Pandas di leggere un file CSV
- *df* è un *DataFrame*, lo si capisce dalla stampa del risultato della funzione `type` . Quindi il metodo `read_csv` restituisce un oggetto DataFrame
- il *DataFrame* risultante ha una colonna in più in quanto è presente la colonna indice delle righe del file CSV

3.2.1.8 Dati da file XLS (o XLSX)

Esistono dei metodi per la scrittura e lettura dei file XLS, prodotti dal programma Microsoft Excel, del tutto analoghi a quelli visti nel paragrafo precedente per i files CSV.

Elenchiamo i suddetti metodi:

- `to_excel("miofile.xls", sheet_name="Sheet1")`
  Questo è il metodo del DataFrame che permette di salvare i dati nel formato XLS. Il primo parametro è il nome del file , il secondo è il nome del foglio su cui si vuole salvare.
- `read_excel("miofile.xls", "Sheet1", index_col=None, na_values=["NA"])`
  Questo è il metodo di Pandas che permette di leggere uno sheet di un file in formato XLS e di salvare i valori all'interno di un oggetto DataFrame.

Si lascia al lettore di provare questi metodi per testarne il funzionamento.

4. Gestione filesystem

In questo capitolo affronteremo come manipolare i files e come reperire e modificare le informazioni sul filesystem.

4.1 I files

4.1.1 Lettura del contenuto di un file

Prendiamo in considerazione un programma che apre un file in sola lettura, legge ogni riga del file riga per riga e stampa ogni riga sullo standard output.

```
fh = open("file.txt","r")
print(fh)
riga=fh.readline()
while(riga!=''):
   print(riga,end='')
   riga=fh.readline()
```

Il file di testo che si vuole aprire è file.txt e contiene il seguente testo:

```
questo è un
file di testo
che serve per
effettuare una
prova di lettura file
```

Eseguendo il programma otteniamo il seguente output:

```
<_io.TextIOWrapper name='file.txt' mode='r' encoding='UTF-8'>
questo è un
file di testo
che serve per
effettuare una
prova di lettura file

```

Dall'analisi del codice e dell'output osserviamo che:

- La funzione `open()` permette di aprire il file di nome "`file.txt`", il nome del flusso è il primo parametro della funzione.
- La funzione `open()` restituisce un handler al file che contiene tutte le informazioni caratteristiche del file e delle modalità di apertura dello stesso. Come il nome del file, il modo di apertura e l'encoding.

- ➔ Come secondo parametro della funzione `open()` indichiamo il modo in cui si apre il flusso. In questo caso il valore "`r`" indica apertura in sola lettura.
- ➔ `readline()` è un metodo dell'handler del file. La funzione che esplica questo metodo è quella di leggere una singola riga del file. La riga del file termina con una newline e/o con un carriage return a seconda che si stia lavorando con un s.o. Microsoft Windows o Unix-like
- ➔ `readline()` restituisce la riga letta o, nel caso in cui si sia arrivati a fine file, restituisce una stringa vuota.
- ➔ Nella funzione print inizializziamo il parametro opzionale `end` per eliminare il *newline* che la funzione stessa va a inserire a fine righe. Questo in quanto la riga del file contiene già un `'\n'`

Il metodo `readline()` accetta un parametro facoltativo che indica il numero di byte (caratteri) da leggere.

Consideriamo a questo proposito il seguente programma:

```
fh = open("file.txt","r")
riga=fh.readline(10)
print(riga)
```

Eseguendo il programma otteniamo il seguente output:

```
questo è u
```

Dall'analisi del codice e dell'output osserviamo che:

- ➔ In output otteniamo la stampa di soli 10 caratteri (10 byte)

4.1.1.1 Cicli di lettura riga per riga alternativo

Un algoritmo alternativo per costruire un ciclo che legga tutte le linee senza effettuare una lettura fuori ciclo è il seguente:

```
fh = open("file.txt")
eof = False
while not eof :
    riga = fh.readline()
    if riga:
        print(riga,end='')
    else:
        eof = True
fh.close()
```

Consideriamo come file di input il nostro solito file di prova:

```
questo è un
file di testo
```

```
che serve per
effettuare una
prova di lettura file
```

Eseguendo il programma otteniamo il seguente output:

```
questo è un
file di testo
che serve per
effettuare una
prova di lettura file

```

Dall'analisi del codice e dell'output osserviamo che:

- ➔ Confrontando questo programma con il precedente notiamo che ha la classica struttura del loop while, ma a differenza di quello qui non effettuiamo una lettura fuori ciclo. Questo comporta l'utilizzo di un flag (la variabile `eof`) che viene settata quando si araggiunge la fine del file e che porta all'uscit del ciclo while di lettura.

Per ultimo si fa notare un'altra notazione pù concisa con lettura fuori ciclo:

```
fh = open("file.txt")
riga = fh.readline()
while riga:
     print(riga,end='')
     riga = fh.readline()
fh.close()
```

Consideriamo il solito file di esempio utilizzato nei precedenti casi. Quindi seguendo il programma otteniamo il seguente output:

```
questo è un
file di testo
che serve per
effettuare una
prova di lettura file

```

Dall'analisi del codice e dell'output osserviamo che:

- ➔ L'output è identico a quello degli altri esempi
- ➔ Si sfrutta il fatto che se la variabile è vuota viene valutata come un valore logico `False`

4.1.1 La funzione open() al microscopio

Un flusso di dati affinché possa essere gestito deve essere aperto, praticamente il sottosistema del sistema operativo che si occupa di gestire i file ha bisogno di un handler

a un file che riassume tutte le sue caratteristiche.
Tale handler viene fornito come valore di ritorno della funzione predefinita `open()`. Esaminamo in dettaglio la sintassi di questa importante funzione.

La funzione `open()` ha la seguente sintassi::

```
open( pathname , mode = 'r' , buffering = - 1 , encoding = None ,
errors = None , newline = None , closefd = True , opener = None )
```

dove :

`pathname`: è il nome del file, rappresenta il percorso del flusso che si vuole aprire

`modo`: indica la modalità di apertura del file. È un parametro facoltativo, il valore predefinito è `'rt'` che indica una modalità in sola lettura di file di testo.

`buffering`: è numero intero utilizzato per impostare la politica di buffering. 0 per disattivare il buffering (consentito solo in modalità binaria), 1 per selezionare il buffering di riga (utilizzabile solo in modalità testo) e > 1 per indicare la dimensione in byte di un buffer di blocchi di dimensioni fisse.

`encoding`: è il nome della codifica utilizzata per decodificare o codificare il file. (usare solo in modalità testo)

`errors` è una stringa che specifica come devono essere gestiti gli errori di codifica e decodifica. Non può essere utilizzata in modalità binaria. I valori che la stringa può assumere sono: `'strict'`, `'ignore'`, `'replace'`, `'surrogateescape'`, `'xmlcharrefreplace'`, `'backslashreplace'`, `'namereplace'`. Per una spiegazione dettagliata di ognuono di questi si rimanda alla documentazione ufficiale.

`newline`: controlla come funziona la modalità newline (si applica solo alla modalità testo). Può valere `None`, `''`, `'\n'`, `'\r'e '\r\n'` e serve per indicare come gestire il comando di newline visto che questo ha caratteristiche differenti da s.o. a s.o. Per una spiegazione dettagliata di ognuono di questi si rimanda alla documentazione ufficiale.

`closefd` e `opener`: si rimanda alla documentazione ufficiale.

Tranne il primo parametro tutti gli altri sono parametri facoltativi.

4.1.1.1 Tabella delle modalità di apertura di un file [9]

Modo	Significato
`'r'`	aperto per la lettura (predefinito)
`'w'`	aprire per la scrittura, tronca prima il file
`'x'`	aperto per la creazione esclusiva, errore se il file esiste già
`'a'`	aprire per la scrittura, aggiungendo alla fine del file se esiste

[9] https://docs.python.org/3/library/functions.html#open

`'b'`	modalità binaria
`'t'`	modalità testo (predefinita)
`'+'`	aperto per l'aggiornamento (lettura e scrittura)

La modalità predefinita è `'rt'` (aperta per la lettura del testo).
Modalità `'w+'` e `'w+b'` apri e tronca il file.
Modalità `'r+'` e `'r+b'` apri il file senza troncamento.
Python distingue tra I/O binario e di testo. I file aperti in modalità binaria (incluso `'b'` nell'argomento mode) restituiscono contenuti come bytes senza alcuna decodifica. In modalità testo, i contenuti del file vengono restituiti come stringhe, i byte sono stati prima decodificati utilizzando una codifica dipendente dalla piattaforma o utilizzando una codifica eventualmente specificata.

4.1.2 Scrittura di un file

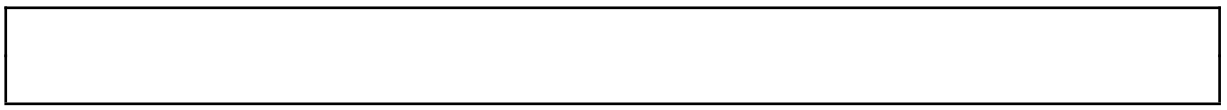
Andiamo a esaminare un programma che scriva delle righe su un file di testo.
Le righe sono memorizzate in una lista che viene inizializzata a inizio programma, le righe vengono scritte su un file che non esiste e che si chiama `fileout.txt`.

Il programma è il seguente:

```
righe=list()
righe.append("Prima riga di prova\n")
righe.append("seconda riga \n")
righe.append("terza riga ")
righe.append("-----------\n")
righe.append("Ultima riga\n")

fh = open("fileout.txt","w")
for riga in righe:
   fh.write(riga)
```

Eseguendo il programma otteniamo il seguente output:



Dall'analisi del codice e dell'output osserviamo che:

- ➔ L'output stampato sullo standard output è nullo, questo perchè non si sono utilizzate delle funzioni `print()` che stampano a monitor
- ➔ L'unica funzione di output è il metodo .write() che è stato richiamato passandogli gli elementi della lista.
- ➔ È stato creato sul filesystem un file di nome: "`fileout.txt`" che contiene le seguenti righe:

```
Prima riga di prova
```

```
seconda riga
terza riga ----------
Ultima riga
```

- ➔ Il metodo write, quindi, ci permette di scrivere dei dati su un file aperto in scrittura.
- ➔ Notiamo che ogni elemento della lista rappresenta una rispettiva riga nel file tranne per il terzo e quarto elemento. Questo perchè nel terzo elemento della lista non è stato aggiunto un `'\n'` (`newline`). Porre attenzione al diverso comportamento del newline a seconda del sistema operativo su cui gira lo script.

Attenzione al `mode`:

Se si provasse a rilanciare lo script si riotterrebbe esattamente lo stesso identico file, questo comportamento è dovuto alla modalità di apertura che abbiamo scelto, cioè:`"w"`
Se avessimo scelto la modalità:`"x"` avremmo ottenuto il seguente errore:

```
FileExistsError: [Errno 17] File exists: 'fileout.txt'
```

in quanto la modalità permette solo la creazione di file non esistenti, non la scrittura su file già esistenti.
Se avessimo voluto scrivere in coda al file già esistente avremmo dovuto usare la modalità:`"a"` che indica all'interprete di entrare nella modalità `append` di aggiornamento del file.

Valore di ritorno:

Se volessimo stampare il valore di ritorno del metodo `write()` otterremmo un interessante risultato. Quindi andiamo a modificare il programma in questo modo:

```
righe=list()
righe.append("Prima riga di prova\n")
righe.append("seconda riga \n")
righe.append("terza riga ")
righe.append("----------\n")
righe.append("Ultima riga\n")

fh = open("fileout.txt","w")
for riga in righe:
   print(fh.write(riga))
```

Eseguendo il programma otteniamo il seguente output:

```
20
14
11
12
12
```

Dall'analisi del codice e dell'output osserviamo che:

- ➔ Abbiamo ottenuto lo stesso identico file di output.
- ➔ Sullo standard output otteniamo la stampa di 5 numeri tanti quante sono le volte che richiamiamo il metodo `write()`, da una veloce verifica comprendiamo che quei numeri rappresentano il numero di byte che vengono scritti sul file ogni volta che utilizziamo `write()`!
- ➔ L'osservazione precedente ci fornisce l'informazione che il metodo `write()` retituisce il numero di byte scritti se l'operazione è andata a buon fine, altrimenti siamo in presenza di un errore.

4.1.3 Il metodo close()

Il metodo `close()` non lo abbiamo ancora mai visto nei nostri script. In realtà è buona norma richiamare questo metodo ogni qualvolta che non serve più la risorsa file che si è aperta con la funzione `open()`.
Quindi in generale le procedure che utilizzano i files dovrebbero avere la seguente struttura:

```
fh = open("file")
elaborazione sul file
...
...
fh.close()
...
...
```

Questo perchè i files consumano risorse andando a impattare sulle performance del sistema. Quindi si potrebbero rischiare dei malfunzionamenti e delle instabilità nei processi se manteniamoo aperti diversi files contemporaneamente senza che ce ne sia l'effettiva necessità.

I programmi che abbiamo lanciato prima non hanno rischiato di darci problemi perchè appena un programma termina, tutti i suoi flussi vengono chiusi. Ma anche in questi casi è sempre buona norma usare la `close()` in quanto ci aiuta ad avere la mentalità di occupare una risorsa critica solo quando veramente necessario[10].

4.1.4 Spostamento all'interno del file mediante i metodi `seek()` e `tell()`

Con il metodo `seek()` abbiamo la possibilità di spostarci attraverso i record di un file rompendo la struttura sequenziale che abbiamo visto sino a ora.
Usando il metodo `tell()` possiamo interrogare il file su dove sia attualmente il cursore che indica in che posizione verrà eseguita la prossima operazione di scrittura o lettura.

[10] Una gestione ottimizzata delle risorse viene effettuata con l'istruzione `with` che esamineremo nel paragrafo 4.1.7.

Combinando i metodi `seek()` e `tell()` possiamo spostarci a nostro piacimento tra le informazioni memorizzate all'interno di un file.

Analizziamo il seguente programma:

```
fh = open("filets.txt","w+")
print("scrivo: \t",fh.write("abcde\n")," bytes")
print("pos: \t\t",fh.tell())
print("scrivo: \t",fh.write("fghi\n")," bytes")
print("pos: \t\t",fh.tell())
print("Nuova pos: \t",fh.seek(2))
print("scrivo: \t",fh.write("X")," bytes")
print("pos: \t\t",fh.tell())
print("leggo: \t\t",fh.readline(1))
```

Eseguendo il programma otteniamo il seguente output:

```
scrivo:          6  bytes
pos:             6
scrivo:          5  bytes
pos:             11
Nuova pos:       2
scrivo:          1  bytes
pos:             3
leggo:           d
```

Dall'analisi del codice e dell'output osserviamo che:

- ➔ Il metodo `tell()` ci informa della posizione del cursore del file in unità di misura di byte(caratteri)
- ➔ Il metodo `seek()` ci permette di spostarci attraverso i dati del file. Il parametro che `seek()` accetta è espresso in unità di misura di byte (caratteri)
- ➔ il metodo `write()` scrive nella posizione in cui è posizionato il cursore del file risultante dall'ultima operazione effettuata.
- ➔ il metodo `readline()` legge i dati a partire dalla posizione in cui è posizionato il cursore del file risultante dall'ultima operazione effettuata.

4.1.5 Altri modi per leggere un file

Possiamo utilizzare i metodi `read()` o `readlines()` per leggere le righe di un file.

Esaminiamo il seguente codice:

```
fh = open("file.txt")
righe = fh.read()
print(righe)
```

```
fh.close()
```

Consideriamo come file di input il nostro solito file di prova:

```
questo è un
file di testo
che serve per
effettuare una
prova di lettura file
```

Eseguendo il programma otteniamo il seguente output:

```
questo è un
file di testo
che serve per
effettuare una
prova di lettura file

```

Dall'analisi del codice e dell'output osserviamo che:

- ➔ Il metodo `read` legge tutte le righe del file (cioè sino a incontrare la fine del file)

Esaminiamo un altro potente metodo per leggere un file con una singola istruzione.
Il metodo `readlines()` legge tutte le righe di un file non appena viene invocato.

```
fh = open("file.txt")
righe = fh.readlines()
print(righe)
fh.close()
```

Eseguendo il programma otteniamo il seguente output:

```
['questo è un\n', 'file di testo\n', 'che serve per\n',
'effettuare una\n', 'prova di lettura file']

```

Dall'analisi del codice e dell'output osserviamo che:

- ➔ Il metodo `readlines()` legge tutte le righe di un file andandole a memorizzare in una lista

Questi due metodi devono essere utilizzati con cautela in quanto se il file è molto pesante si potrebbe incorrere a problematiche di consumo eccessivo di risorse a scapito delle prestazioni dei processi.

Tutti e due questi metodi hanno un parametro opzionale per limitare la lettura dei bytes del file.

Nel caso del metodo `read()` è assolutamente analogo al parametro del metodo radline(). Nel caso del metodo `readlines()` il parametro suggerisce il numero di bytes superati i quali non si leggono più linee del file rispetto a quelle già lette. Il default è -1 che indica che non ci sono limiti di bytes e che quindi si possono leggere tutte le lineee del file.

Si esamini il seguente codice che chiarisce il funzionamento del parametro opzionale in `readlines()`:

```
fh = open("file.txt")
righe = fh.readlines(14)
print(righe)
fh.close()
```

Consideriamo il solito file di esempio utilizzato nei precedenti casi. Quindi seguendo il programma otteniamo il seguente output:

```
['questo è un\n', 'file di testo\n']
```

Dall'analisi del codice e dell'output osserviamo che:

- ➔ Abbiamo aggiunto il paramero opzionale che indica un massimo di 14 bytes
- ➔ Il programma legge oltre 14 caratteri, ma blocca il numero di righe lette appena supera il 14^{esimo} carattere, in altre parole l'ultima riga letta è quella che contiene il 14^{esimo} carattere.

4.1.5.1 L'oggetto file è un iterabile

Un altro modo molto interessante per leggere le righe di un file è osservare che l'handle del file (l'oggetto restitutito dalla funzione `open()`) è un oggetto *iterabile*.
Questa osservazione ci permette di utilizzare l'istruzione for per andare a leggere tutte le righe del file.

Esaminiamo il seguente esempio:

```
fh = open("file.txt")
for riga in fh:
    print(riga, end='')
fh.close()
```

Eseguendo il programma otteniamo il seguente output:

```
questo è un
file di testo
che serve per
effettuare una
prova di lettura file
```

Dall'analisi del codice e dell'output osserviamo che:

- ➔ L'istruzione `for` risulta molto utile e compatta per leggere le righe di un file
- ➔ In questa modalità si evita di verificare la condizione di fine file

4.1.6 Scrittura massiva di righe

Possiamo scrivere con una sola istruzione più righe su un file.
Riferendosi al programma di esempio del paragrafo 4.1.2 sulla scrittura di un file, modifichiamo il codice nel seguente modo:

```
righe=list()
righe.append("Prima riga di prova\n")
righe.append("seconda riga \n")
righe.append("terza riga ")
righe.append("-----------\n")
righe.append("Ultima riga\n")

fh = open("fileout.txt","w")
print(fh.writelines(righe))
fh.close()
```

Eseguendo il programma otteniamo il seguente output:

```
None
```

Dall'analisi del codice e dell'output osserviamo che:

- ➔ È stato creato sul filesystem un file di nome: "`fileout.txt`" che contiene le seguenti righe:

```
Prima riga di prova
seconda riga
terza riga -----------
Ultima riga
```

- ➔ Il metodo `writelines()` ha scritto su file tutto il contenuto della lista passatagli come parametro.
- ➔ VALORE RITORNO WRITELINES()

4.1.7 L'istruzione `with` per la gestione dei file

L'istruzione `with` è una funzionalità molto utile di Python che semplifica la gestione delle

risorse, è specificata nella PEP 343[11].
Noi la illustreremo soprattutto nell'uso della risorsa file, in quanto garantisce che questi siano correttamente chiusi dopo essere state utilizzati.
Utilizzare `with` per aprire un file è considerata una buona pratica, poiché assicura che il metodo `close()` venga chiamato automaticamente al termine del blocco, anche in presenza di errori.

La sintassi generale dell'istruzione with per aprire un file è la seguente:

```
with open(nome_file, modo) as variabile:
    operazioni_sul_file
```

Dove:
`nome_file` è il nome del file da aprire.
`modo` indica la modalità di apertura del file (es. 'r', 'w', 'a', etc.).
`variabile` è la variabile a cui farà riferimento l'oggetto file all'interno del blocco `with`.

Nei prossimi paragrafi rivedremo gli esempi di lettura e scrittura file secondo l'ottica di applicazione del `with`.

4.1.7.1 Lettura del contenuto di un file con `with`

Esaminiamo un esempio in cui leggiamo un file riga per riga utilizzando `with`:

```
with open("file.txt", "r") as fh:
    for riga in fh:
        print(riga, end='')
```

Consideriamo il nostro solito file di esempio:

```
questo è un
file di testo
che serve per
effettuare una
prova di lettura file
```

Eseguendo il programma otteniamo il seguente output:

```
questo è un
file di testo
che serve per
effettuare una
prova di lettura file

```

[11] https://peps.python.org/pep-0343/

Dall'analisi del codice e dell'output osserviamo che:

- L'istruzione `with` apre il file `file.txt` in modalità di sola lettura (`'r'`) e assegna l'oggetto file alla variabile `fh`.
- L'istruzione `for` itera su ogni riga del file, stampandola senza aggiungere ulteriori newline grazie al parametro `end=''` della funzione `print()`.
- Il file viene chiuso automaticamente al termine del blocco `with`, garantendo una gestione sicura della risorsa.

4.1.7.2 Scrittura su un file con `with`

Vediamo ora un esempio di scrittura su un file utilizzando l'istruzione `with`:

```
righe = [
    "Prima riga di prova\n",
    "seconda riga \n",
    "terza riga \n",
    "-----------\n",
    "Ultima riga\n"
]

with open("fileout.txt", "w") as fh:
    fh.writelines(righe)
```

Eseguendo il programma, creiamo il seguente file `fileout.txt`:

```
Prima riga di prova
seconda riga
terza riga
-----------
Ultima riga
```

Dall'analisi del codice e dell'output osserviamo che:

- In standard output non si ha nulla in quanto non si è stampato nulla su `stdout` (non si sono eseguite delle istruzioni `print()`)
- L'istruzione `with` apre il file `fileout.txt` in modalità di sola scrittura (`'w'`) e assegna l'oggetto file alla variabile `fh`.
- Il metodo `writelines()` scrive tutte le righe della lista `righe` nel file.
- Il file viene chiuso automaticamente al termine del blocco `with`, garantendo che tutte le operazioni di scrittura siano completate correttamente.

4.1.7.3 Esempio combinato di lettura e scrittura con `with`

Consideriamo un esempio in cui leggiamo da un file e scriviamo il contenuto in un altro file, utilizzando l'istruzione `with` per gestire entrambi i file:

```
with open("file.txt", "r") as file_in:
    contenuto = file_in.read()

with open("fileout.txt", "w") as file_out:
    file_out.write(contenuto)
```

Il `file.txt` contiene il (solito) seguente testo:

```
questo è un
file di testo
che serve per
effettuare una
prova di lettura file
```

Lanciando il programma otterremo un altro file: `fileout.txt` conterrà lo stesso testo:

```
questo è un
file di testo
che serve per
effettuare una
prova di lettura file
```

Dall'analisi del codice e dell'output osserviamo che:

- ➔ Anche in questo caso non essendoci delle istruzioni `print()` non si ottiene alcuna stampa su `stdout`.
- ➔ Il primo `with` apre `file.txt` in modalità lettura e legge tutto il contenuto del file.
- ➔ Il secondo `with` apre `fileout.txt` in modalità scrittura e vi scrive il contenuto letto dal primo file.
- ➔ Entrambi i files vengono automaticamente chiusi al termine dei rispettivi blocchi `with`, garantendo la corretta gestione delle risorse.

4.1.7.4 Utilizzo di `with` con operazioni complesse

L'istruzione `with` è particolarmente utile quando si effettuano operazioni complesse sui file, le quali potrebbero causare errori.
Ad esempio, vediamo come gestire una copia sicura di un file, includendo la gestione degli errori:

```
try:
    with open("file.txt", "r") as file_in:
        contenuto = file_in.read()

    with open("fileout.txt", "w") as file_out:
        file_out.write(contenuto)
```

```
except IOError as err:
    print(f"Errore nella gestione dei file: {err}")
```
Eseguendo il programma, nel caso di un errore, che può essere simulato modificando il nome del file che si apre in sola lettura da `file.txt` a `file1.txt`, otteniamo il seguente output:

```
Errore nella gestione dei file: [Errno 2] No such file or
directory: 'file1.txt'
```

Dall'analisi del codice e dell'output osserviamo che:

- Nel caso di errore verificatosi durante l'apertura, la lettura o la scrittura dei file, l'eccezione `IOError` viene catturata e gestita, stampando un messaggio di errore.
- Nell'esempio abbiamo simulato un errore in fase di apertura del file visto che il file non esiste.
- Anche in questo caso l'uso di `with` assicura che i file siano correttamente chiusi anche in caso di errore, prevenendo perdite di risorse o corruzioni di file.
- Naturalmente nel caso in cui non si verificano errori il programma non stamperà nulla a video.

4.1.7.5 Uso di più `with` annidati

Per operazioni più complesse, è possibile annidare più istruzioni `with`.
Esaminiamo il seguente esempio che apre un file ed esegue la copia del suo contenuto su un altro file.

```
with open("file.txt", "r") as file_in, open("fileout.txt", "w") as
file_out:
    for riga in file_in:
        file_out.write(riga)
```

Dall'analisi del codice osserviamo che:

- Utilizza una singola istruzione `with` per aprire contemporaneamente due file.
- Legge ogni riga da `file.txt` e la scrive immediatamente in `fileout.txt`.
- Garantisce la chiusura di entrambi i file al termine del blocco `with`.

Quindi possiamo dire che:

- La sintassi di `with` consente di aprire più file in modo chiaro e conciso.
- L'istruzione `with` garantisce che entrambe le risorse siano gestite correttamente e chiuse al termine delle operazioni.

Questi ultimi esempi ci fanno capire come l'istruzione `with` possa semplificare e rendere più sicura la gestione dei file in Python, migliorando la leggibilità del codice e prevenendo errori comuni legati alla gestione manuale delle risorse.

4.2 Interazione con il file system - il modulo OS[12]

Il modulo OS in Python fornisce funzioni per interagire con il sistema operativo. Ci fornisce un livello di astrazione tra noi e il sistema operativo.
Tale modulo rientra nei moduli di utilità standard di Python, quindi non necessita di installazione esplicita. I moduli `os` e `os.path` includono molte funzioni per interagire con il file system.

4.2.1 Gestione della directory di lavoro corrente

Considera la directory di lavoro corrente (*CWD Current Working Directory*) come una cartella in cui sta operando Python. Ogni volta che i files vengono chiamati solo con il loro nome, Python presume che il path corrisponda alla *CWD*, il che significa che il riferimento solo al nome avrà esito positivo solo se il file si trova nel *CWD* di Python.

Nota: la cartella in cui è in esecuzione lo script Python è nota come Directory corrente. Questo, generalmente, non è il percorso in cui si trova lo script Python.

4.2.1.1 Ottenere la directory di lavoro corrente

Per ottenere la posizione della directory di lavoro corrente viene utilizzato

```
os.getcwd()
```

Si esamini il seguente programma:

```
# Python program to explain os.getcwd() method
# importing os module
import os

# Get the current working
# directory (CWD)
cwd = os.getcwd()

# Print the current working
# directory (CWD)
print("Current working directory:", cwd)
```

Eseguendo il programma otteniamo il seguente output:

```
Current working directory: /home/massimo/PythonProjects
```

[12] https://www.geeksforgeeks.org/os-module-python-examples/amp/ (traduzione con qualche modifica)

4.2.1.2 Modifica della directory di lavoro corrente

Per modificare la directory di lavoro corrente (CWD) viene utilizzato il metodo

```
os.chdir(path)
```

Questo metodo cambia il *CWD* in un percorso specificato. Richiede solo un singolo argomento come nuovo percorso di directory.

Nota: la directory di lavoro corrente è la cartella in cui sta operando lo script Python.

Si esamini il seguente programma:

```
# Python program to change the
# current working directory
import os

# Function to Get the current
# working directory
def current_path():
    print("Directory di lavoro corrente")
    print(os.getcwd())
    print()

# Driver's code
# Printing CWD before
current_path()

# Changing the CWD
os.chdir('../')

# Printing CWD after
current_path()
```

Eseguendo il programma otteniamo il seguente output:

```
Directory di lavoro corrente
/home/massimo/PythonProjects

Directory di lavoro corrente
/home/massimo
```

4.2.2 Creazione di una directory

Sono disponibili diversi metodi nel modulo `os` per creare una directory.

Questi sono i seguenti:

- ➔ `os.mkdir()`
- ➔ `os.makedirs()`

4.2.2.1 `os.mkdir()`

Il metodo `os.mkdir()` in Python viene utilizzato per creare una directory denominata percorso con la modalità numerica specificata. Questo metodo genera `FileExistsError` se la directory da creare esiste già.

Si esamini il seguente programma:

```
# Python program to explain os.mkdir() method
# importing os module
import os

# Directory
directory = "prova1"

# Parent Directory path
parent_dir = "/home/massimo"

# Path
path = os.path.join(parent_dir, directory)

# Create the directory
# 'prova' in
# '/home / User '
os.mkdir(path)
print("Directory '% s' created" % directory)

# Directory
directory = "prova2"

# Parent Directory path
parent_dir = "/home/massimo/PythonProjects"

# mode
mode = 0o666

# Path
path = os.path.join(parent_dir, directory)

# Create the directory
# 'prova2' in
# '/home / User / PythonProjects"'
# with mode 0o666
os.mkdir(path, mode)
```

```
print("Directory '% s' created" % directory)
```

Eseguendo il programma otteniamo il seguente output:

```
Directory 'prova1' created
Directory 'prova2' created
```

4.2.2.2 `os.makedirs()`

Il metodo `os.makedirs()` in Python viene utilizzato per creare una directory in modo ricorsivo. Ciò significa che mentre si crea una directory foglia se manca una directory di livello intermedio, il metodo `os.makedirs()` le creerà tutte.

Si esamini il seguente programma:

```
# Python program to explain os.makedirs() method
# importing os module
import os

# Leaf directory
directory = "prova3"

# Parent Directories
parent_dir = "/home/massimo/pippo/pluto/"

# Path
path = os.path.join(parent_dir, directory)

# Create the directory
# 'prova3'
os.makedirs(path)
print("Directory '% s' created" % directory)

# Directory 'pippo' and 'pluto' will
# be created too
# if it does not exists

# os.makedirs() method can be
# used to create a directory tree
```

Eseguendo il programma otteniamo il seguente output:

```
Directory 'prova3' created
```

4.2.3 Elencare files e directories

Il metodo `os.listdir()` viene utilizzato per ottenere un elenco di tutti i files e le directories nella directory specificata. Se non specifichiamo alcuna directory, verrà restituito l'elenco di files e directories nella directory di lavoro corrente.

Si esamini il seguente programma:

```
# Python program to explain os.listdir() method
# importing os module
import os

# Get the list of all files and directories
# in the root directory
path = "/"
dir_list = os.listdir(path)
print("Files and directories in '", path, "' :")

# print the list
print(dir_list)
```

Eseguendo il programma otteniamo il seguente output:

```
Files and directories in ' / ' :
['lost+found', 'lib', 'tmp', 'proc', 'usr', 'cdrom', 'sbin',
'media', 'opt', 'snap', 'libx32', 'sys', 'mnt', 'swapfile',
'etc', 'home', 'var', 'dev', 'bin', 'boot', 'root', 'run',
'lib64', 'lib32', 'srv']
```

4.2.4 Eliminazione di files e directories

Il modulo OS dimostra diversi metodi per rimuovere directories e files.

Questi sono:

- `os.remove()`
- `os.rmdir()`

4.2.4.1 `os.remove()`

Il metodo `os.remove()` viene utilizzato per rimuovere o eliminare un percorso di file. Questo metodo non può rimuovere o eliminare una directory.
Se il percorso specificato è una directory, `OSError` verrà generato dal metodo.

Supponiamo che i files contenuti nella cartella siano:

```
$ ls dir1/

file1 file2
```

Ora si esegua il seguente codice:

```
# Python program to explain os.remove() method
# importing os module
import os

# File name
file = 'file1'

# File location
location = "/home/massimo/dir1/"

# Path
path = os.path.join(location, file)
print("path: ",path)

# Remove the file
# 'file1'
os.remove(path)
```

Eseguendo il programma otteniamo il seguente output:

```
path:  /home/massimo/dir1/file1
```

Verifichiamo l'azione sul file system:

```
$ ls dir1/

file2
```

4.2.4.2 os.rmdir()

Il metodo os.rmdir() in viene utilizzato per rimuovere o eliminare una directory vuota. OSError verrà generato se il percorso specificato non è una directory vuota.

Esempio: supponiamo che le directory siano:

```
$ ls dir1/

dirA dirB dirC
```

Ora si esegua il seguente codice:

```
# Python program to explain os.rmdir() method
# importing os module
import os

# Directory name
directory = "dirA"

# Parent Directory
parent = "/home/massimo/dir1/"

# Path
path = os.path.join(parent, directory)

# Remove the Directory
# "dirA"
print(os.rmdir(path))
```

Eseguendo il programma otteniamo il seguente output:

```
None
```

Verifichiamo l'azione sul file system:

```
$ ls dir1/

dirB dirC
```

4.2.5 Funzioni di uso comune

Esamineremo i seguenti metodi:

- ➔ os.name
- ➔ os.popen()
- ➔ os.rename()
- ➔ os.stat()
- ➔ os.path.exists()
- ➔ os.path.getsize()

4.2.5.1 os.name

Fornisce il nome del sistema operativo su cui sta girando l'interprete Python. Attualmente sono stati registrati i seguenti nomi: `'posix'`, `'nt'` e `'java'`.

Proviamo il seguente semplice programma:

```
import os
print(os.name)
```

Eseguendo il programma otteniamo il seguente output:

```
Posix
```

Questo è il risultato che otteniamo se eseguiamo il programma su un sistema Unix-like

4.2.5.2 os.popen()

Il metodo Python `popen()` apre una pipe da o verso il comando. Il valore restituito è un oggetto file aperto collegato alla pipe, che può essere letto o scritto a seconda che la modalità sia `'r'` (predefinita) o `'w'`. L'argomento `bufsize` ha lo stesso significato della funzione open() . Il flusso deve essere chiuso con il metodo `.close()` .

Sintassi:

```
os.popen(comando[, modalità[, bufsize]])
```

Parametri:

- `comando` - Questo è il comando utilizzato.
- `mode` – Questo è il Mode può essere `'r'`(predefinito) o `'w'`.
- `bufsize` - Se il valore di buffering è impostato su 0, non si verificherà alcun buffering. Se il valore di buffering è 1, il buffering di riga verrà eseguito durante l'accesso a un file. Se si specifica il valore di buffering come numero intero maggiore di 1, l'azione di buffering verrà eseguita con la dimensione del buffer indicata. Se negativo, la dimensione del buffer è l'impostazione predefinita del sistema (comportamento predefinito).

Valore di ritorno:

Questo metodo restituisce un oggetto file aperto connesso alla pipe.

Esaminiamo il seguente programma:

```
import os

os.chdir('/home/massimo/dir1/')
creaDir=os.popen('mkdir directory')
creaDir.close()
```

```
fd=os.popen('ls')
str=fd.read()
fd.close()
print(str)
```

Eseguendo il programma otteniamo il seguente output:

```
dirB
dirC
directory
```

4.2.5.3 `os.rename()`

Serve per rinominare un file. Il nome del file cambia solo se il file esiste e l'utente dispone di autorizzazioni sufficienti per modificare il file.
Il metodo `os.rename()` accetta come parametro il pathname sorgente e il pathname di destinazione.

Esaminiamo il seguente programma:

```
import os

file = "/home/massimo/dir1/file2"
os.rename(file,'/home/massimo/dir1/altronome')
```

Eseguendo il programma il file di nome `file2` viene rinominato in `altronome`.

Attenzione: se non avessimo indicato il pathname nel secondo parametro del metodo `rename` avremmo spostato il file all'interno della CWD dello script python.

4.2.5.4 `os.stat()`

Permette di ottenere lo stato di un file o un descrittore di file.
Esegue l'equivalente di una chiamata alla funzione di sistema *stat()* sul percorso specificato. Il percorso può essere specificato come stringa o come descrittore di file aperto.
Restituisce un oggetto di tipo `stat_result`.
Questa funzione normalmente segue i collegamenti simbolici.

Esaminiamo il seguente programma:

```
import os

file = "/home/massimo/dir1/file1"
statinfo = os.stat(file)
```

```
print(statinfo)
print(type(statinfo))
print(dir(statinfo))
print(statinfo.st_size)
```

Eseguendo il programma otteniamo il seguente output:

```
os.stat_result(st_mode=33204,     st_ino=1319778,     st_dev=66309,
st_nlink=1,       st_uid=1000,       st_gid=1000,       st_size=14,
st_atime=1652478941, st_mtime=1652478941, st_ctime=1652478941)
<class 'os.stat_result'>
['__add__',   '__class__',   '__class_getitem__',   '__contains__',
'__delattr__',   '__dir__',   '__doc__',   '__eq__',   '__format__',
'__ge__',   '__getattribute__',   '__getitem__',   '__getnewargs__',
'__gt__', '__hash__', '__init__', '__init_subclass__', '__iter__',
'__le__', '__len__', '__lt__', '__module__', '__mul__', '__ne__',
'__new__', '__reduce__', '__reduce_ex__', '__repr__', '__rmul__',
'__setattr__',    '__sizeof__',    '__str__',    '__subclasshook__',
'count',       'index',       'n_fields',       'n_sequence_fields',
'n_unnamed_fields',    'st_atime',    'st_atime_ns',    'st_blksize',
'st_blocks',    'st_ctime',    'st_ctime_ns',    'st_dev',    'st_gid',
'st_ino',    'st_mode',    'st_mtime',    'st_mtime_ns',    'st_nlink',
'st_rdev', 'st_size', 'st_uid']
14
```

4.2.5.4.1 Il modulo `stat` per l'interpretazione dei risultati di `os.stat()`[13]

Il modulo `stat` definisce costanti e funzioni per interpretare i risultati di `os.stat()`. Per i dettagli completi sulle chiamate alla funzione `stat()` si consiglia di consultare la documentazione del proprio sistema operativo.

Il modulo `stat` definisce diverse funzioni (ogni funzione accetta come parametro l'attributo `st_mode` dell'oggetto `os.stat_result`). Qui diamo una veloce panoramica di quelli più significativi, per la lista completa si consulti la documentazione ufficiale riportata nella bibliografia.

stat.S_ISDIR(mode)

Restituisce un valore diverso da zero se la modalità proviene da una directory.

stat.S_ISREG(mode)

Restituisce un valore diverso da zero se la modalità proviene da un file normale.

[13] https://docs.python.org/3/library/stat.html

stat.S_ISLNK(mode)

Restituisce un valore diverso da zero se la modalità proviene da un collegamento simbolico.

stat.S_ISSOCK(mode)

Restituisce un valore diverso da zero se la modalità proviene da un socket.

stat.S_IMODE(mode)

Restituisce la parte della modalità del file che può essere impostata da `os.chmod()`[14], ovvero i bit di autorizzazione del file, più i bit *sticky bit*, *set-group-id* e *set-user-id* (sui sistemi che li supportano).

Di norma si usano i metodi di os.path[15] per testare i vari tipi di file, p.e. per testare se un file è una directory si può utilizzare: `os.path.isdir(path)`.
L'utilizzo delle funzioni sopra elencate risulta utile quando si eseguono più test dello stesso file e si desidera evitare il sovraccarico della chiamata di sistema stat() per ogni test.

Nota: Naturalmente è necessario importare il modulo stat prima di utilizzare le funzioni sopra elencate.

Esempio di utilizzo:

```
import os,stat

file = "/home/massimo/dir1/file1"
statinfo = os.stat(file)
mode = statinfo.st_mode
print(stat.S_ISREG( mode ))
print(stat.S_IMODE( mode ))
```

Eseguendo il programma otteniamo il seguente output:

```
True
436
```

4.2.5.5 `os.path.exists()`

Questo metodo permette di verificare se un file esiste o meno passando il pathname del file come parametro. Il metodo è presente nel sottomodulo di `os` che si chiama `path`.

[14] https://docs.python.org/3.9/library/os.html#os.chmod
[15] https://docs.python.org/3/library/os.path.html

Esaminiamo il seguente programma:

```
import os

file = "/home/massimo/dir1/"
result = os.path.exists(file)
print(result)
file = "/home/massimo/dir1/filechenonesiste"
result = os.path.exists(file)
print(result)
```

Eseguendo il programma otteniamo il seguente output:

```
True
False
```

4.2.5.6 `os.path.getsize()`

Questo metodo ci fornisce la dimensione del file in byte. Accetta come parametro formale il pathname del file da verificare.
Esaminiamo il seguente programma:

```
import os

file = "/home/massimo/dir1/"
size = os.path.getsize(file)
print("Size of the file is", size," bytes.")
```

Eseguendo il programma otteniamo il seguente output:

```
Size of the file is 4096  bytes.
```

Naturalmente possiamo indicare nel pathname anche le directories come fatto in quest'ultimo esempio.

5. Servizi di rete

5.1 Connessioni SSH

SSH è il metodo generalmente utilizzato per accedere a una macchina remota ed eseguire comandi, recuperare file o caricare file.
È possibile trasferire file dalla macchina remota a quella locale o viceversa utilizzando SFTP (Secure File Transfer Protocol) e SCP (Secure Copy Protocol).

5.1.1 Libreria Paramiko

`Paramiko`[16] è un'implementazione in puro Python (3.6+) del protocollo SSHv2 , che fornisce funzionalità sia client che server.
Fornisce le basi per la libreria SSH di alto livello `Fabric`[17] , che è da utilizzare per casi d'uso client comuni come l'esecuzione di comandi shell remoti o il trasferimento di file.

Utilizzando paramiko per accesso client, puoi autenticarti utilizzando una password o una chiave e come accesso server puoi decidere a quali utenti sono consentiti gli accessi e i canali che consenti.

Utilizzeremo tale libreria come client.

5.1.2 Installazione libreria Paramiko

Installiamo Paramiko utilizzando il comando `pip`

```
$ pip install paramiko
```

Il comando andrà a installare la libreria e eventuali dipendenze non installate (come p.e. librerie per utilizzare gli algoritmi di criptazione).

5.1.3 La classe SSHClient

Il client principale di Paramiko è `Paramiko.SSHClient`.
Un' istanza di `Paramiko.SSHClient` può essere utilizzata per effettuare connessioni al server remoto e trasferire file.

5.1.3.1 Connessione

Proviamo subito il seguente codice che effettua una connessione a un server SSH

```
import paramiko
ssh_client=paramiko.SSHClient()
```

[16] https://www.paramiko.org/
[17] https://www.fabfile.org/

```
ssh_client.connect(hostname='localhost',username='utente',password
='password')
```

Eseguendo il programma otteniamo il seguente output:

```
Traceback (most recent call last):
   File "/home/massimo/PythonProjects/prova.py", line 3, in
<module>

ssh_client.connect(hostname='localhost',username='utente',passw
ord='password')
    File "/usr/lib/python3/dist-packages/paramiko/client.py",
line 415, in connect
   self._policy.missing_host_key(
    File "/usr/lib/python3/dist-packages/paramiko/client.py",
line 823, in missing_host_key
   raise SSHException(
paramiko.ssh_exception.SSHException: Server 'localhost' not
found in known_hosts
```

Notiamo che viene visualizzato il seguente errore:

```
missing_host_key
    raise SSHException(
paramiko.ssh_exception.SSHException: Server 'localhost' not found
in known_hosts
```

Questo errore è stato generato perché non hai informato la tua macchina che il server remoto si "fida" del server a cui stai tentando di accedere.

5.1.3.1.1 La questione dei 'known host'

Proviamo ad accedere a un server SSH attraverso un client da linea di comando, lanciamo il seguente comando:

```
$ ssh localhost
```

Otteniamo sullo standard output:

```
The authenticity of host 'localhost (127.0.0.1)' can't be
established.
ECDSA key fingerprint is
SHA256:d2iHpmr8Fgb5NHFmFPEPyIQdl2EdyluQjUn9NxZ54Tc.
```

```
Are you sure you want to continue connecting
(yes/no/[fingerprint])?
```

Se si seleziona: `'yes'` si dichiara che la macchina da cui si sta effettuando la connessione può essere considerata affidabile. Quindi sino a che la chiave non verrà modificata si potrà accedere al server SSH inserendo solo le credenziali utente.

Alla stessa maniera, Paramiko richiede di convalidare la fiducia con la macchina. Questa convalida viene gestita chiamando il metodo `set_missing_host_key_policy()` della classe `SSHClient` e passando la *policy* che si desidera implementare quando si accede a una nuova macchina remota.

Per impostazione predefinita, paramiko.SSHclient imposta la politica su *`RejectPolicy`*. La politica rifiuta la connessione senza effettuare la convalida come visto sopra. Paramiko tuttavia fornisce un modo per una sorta di policy chiave "Trust All", l' *`AutoAddPolicy`*. L'analisi di un'istanza di *`AutoAddPolicy`* in `set_missing_host_key_policy()` modifica la policy stessa per consentire qualsiasi connessione a qualsiasi host.

Dopo queste osservazioni proviamo il seguente programma

```
import paramiko
ssh_client =paramiko.SSHClient()
ssh_client.set_missing_host_key_policy(paramiko.AutoAddPolicy())
ssh_client.connect(hostname='localhost',username='utente',password
='password')
```

Notiamo che non viene stampato nulla in output, la connessione SSH è stata stabilita!

5.1.3.1.2 Il metodo `connect()` al microscopio

Il metodo `.connect()` della classe `SSHClient()` ha più parametri formali di quelli visti nei precedenti esempi. Per i dettagli si invita il lettore a esaminare la documentazione ufficiale[18] in cui, tra l'altro, vengono descritte le policy di connessione e le priorità di autenticazione.

In questo paragrafo elencheremo i parametri di tale metodo. La chiamata al metodo avviene nel seguente modo:

```
connect(hostname,port=22,username=None,password=None,pkey=None,key
_filename=None, timeout=None, allow_agent=True,
look_for_keys=True, compress=False, sock=None,
gss_auth=False,gss_kex=False,gss_deleg_creds=True,gss_host=None,ba
nner_timeout=None,auth_timeout=None,gss_trust_dns=True,
passphrase=None, disabled_algorithms=None)
```

18 https://docs.paramiko.org/en/stable/api/client.html?highlight=connect#paramiko.client.SSHClient.connect

L'unico parametro obbligatorio è l'`hostname`, come ci si potrebbe aspettare. Tutti gli altri sono predefiniti.

Ecco un' elenco di alcuni di questi:

- ➔ `port`: indica la porta del server SSH, il default è la porta standard 22
- ➔ `username`: l'utenza con cui ci si vuole connettere
- ➔ `password`: la password associato all'utenza
- ➔ `timeout`: un timeout (in secondi) per la connessione TCP
- ➔ `auth_timeout`: un timeout (in secondi) per attendere una risposta di autenticazione.

5.1.3.2 Lanciare dei comandi sul server remoto

Per eseguire un comando viene utilizzato il metodo `.exec_command(`*`comando`*`)` della classe `SSHClient`.
`comando` è il parametro che si passa al metodo e rappresenta il comando che si vuole eseguire nella shell remota.
La risposta viene restituita come una tupla *`(stdin,stdout,stderr)`*

Come esempio vogliamo costruire uno script che elenchi tutti i files in una directory che risiede sulla macchina remota:
La prima parte dello script prevede l'istanziazione della classe SSHClient, quindi una volta richiamato il metodo `.connect()` si va ad eseguire il seguente codice:

```
stdin,stdout,stderr=ssh_client.exec_command("ls")
print(type(stdin))
print(type(stdout))
print(type(stderr))
print(stdout.read())
print(stderr.read())
```

Eseguendo il programma otteniamo il seguente output:

```
<class 'paramiko.channel.ChannelStdinFile'>
<class 'paramiko.channel.ChannelFile'>
<class 'paramiko.channel.ChannelStderrFile'>
b"a.out\ndir1\ndir2\nfile1.txt\nfile2.py\nfile3.jpg\n"
b''
```

Dall'analisi del codice e dell'output osserviamo che:

- ➔ Il metodo `exec_command()` esegue un comando nella shell remota
- ➔ Tale metodo restituisce una tupla di 3 oggetti che rappresentano 3 canali aperti nell'ordine sullo standard input, standard output e standard error
- ➔ Visto che il comando restituisce una stampa sullo standard output, andiamo a leggere il flusso aperto con il metodo `.read()` oppure `.readlines()`. Naturalmente possiamo leggere i dati del flusso anche riga per riga utilizzando un

metodo qualsiasi di quelli visti nel capitolo in cui abbiamo trattato i files, cfr. cap. 4.1

➔ Leggendo il flusso `stderr` si ottiene una stringa vuota, questo indica che il comando non ha prodotto errori

5.1.3.3 Comandi che richiedono input dall'utente

A volte è necessario fornire una password o un input aggiuntivo per eseguire un comando. Quando questo accade si utilizza, naturalmente, il flusso *stdin*.
Simuliamo questa azione andando a eseguire il comando ls mediante `sudo`.

Il codice visto nel precedente paragrafo diventa (eliminiamo le stampe dei type):

```
stdin,stdout,stderr=ssh_client.exec_command("sudo -S ls")
stdin.write('password\n')
print (stdout.read())
print (stderr.read())
```

Eseguendo il programma otteniamo il seguente output:

```
b"a.out\ndir1\ndir2\nfile1.txt\nfile2.py\nfile3.jpg\n"
b'[sudo] password di utente: '
```

Dall'analisi del codice e dell'output osserviamo che:

➔ Abbiamo inserito l'opzione del comando `sudo -S` o `-stdin` che ci serve per indicare a tale comando che deve leggere la password dallo standard input e non dal terminal device (si consulti man sudo per ulteriori dettagli)

➔ Andiamo a scrivere la password utilizzando il metodo `.write()` sul canale `stdin`. Alla fine della password inseriremo un newline necessario per completare l'input.

➔ L'output del comando lo preleviamo dal canale `stdout` mediante il metodo `.read()`

5.1.3.3.1 Un piccolo esercizio esemplificativo

Per capire come funziona il meccanismo dello scambio di informazioni utilizzando lo standard input e lo standard output attraverso il metodo `.exec_command()` proviamo questo semplice esercizio.

Problema: Si vuole costruire un programma che esegua la somma di tre numeri interi forniti in input dall'utente. In output viene fornita la somma dei tre numeri inseriti. Tale programma viene eseguito su una macchina remota attraverso un collegamento SSH.

Soluzione: seguiamo i seguenti passi per risolvere il problema

1. Codifichiamo in linguaggio C++ il programma che richieda in input 3 numeri interi e

ne esegua la somma (possiamo utilizzare anche uno script Python ovviamente).
2. Scriviamo lo script Python che ci permetta crerae una connessione SSH e di lanciare il programma creato al punto 1
3. Lanciamo lo script e osserviamo i risultati

Implementazione:

1. Il sorgente in lunguaggio C++ che andremo a compilare è il seguente:

```
#include <iostream>

using namespace std;

int main(){

   int i;
   int numero;
   int somma;
   int n=3;

   somma = 0;
   for(i=0;i<n;i++) {
       cout << "\nInserire "<< i <<" numero:";
       cin >> numero;
       somma += numero;
   }

   cout << "\nSomma = "<< somma << endl;
}
```

   L'eseguibile creato con la compilazione avrà nome `somma3`

2. Lo script python completo è il seguente:

```
import paramiko

ssh_client =paramiko.SSHClient()
ssh_client.set_missing_host_key_policy(paramiko.AutoAddPolicy
())
ssh_client.connect(hostname='localhost',username='utente',pas
sword='password')
commandPathname="/home/massimo/somma3"
stdin,stdout,stderr=ssh_client.exec_command(commandPathname)
stdin.write('1\n')
stdin.write('2\n')
stdin.write('3\n')
print (stdout.read())
print (stderr.read())
```

3. Eseguendo il programma otteniamo il seguente output:

```
b'\nInserire 0 numero:\nInserire 1 numero:\nInserire 2
numero:\nSomma = 6\n'
```

5.1.4 Trasferimenti di files - la classe SFTPClient

I trasferimenti dei files sono gestiti dalla classe `paramiko.SFTPClient`.

Questa classe è utilizzata per aprire una sessione SFTP, su un flusso SSH (precedentemente aperto) ed eseguire operazioni su files remoti.

Non andremo a istanziare direttamente la classe ma utilizzeremo un metodo della classe SSHClient che eseguirà questa operazione per noi.

5.1.4.1 La classe SFTPClient

La classe `paramiko.SFTPClient` viene istanziata dalla chiamata al metodo `.open_sftp()` di un' istanza di `Paramiko.SSHClient`.

Esamineremo i seguenti metodi tra i tanti messi a disposizione dalla classe SFTPClient:

- ➔ `get()`
- ➔ `put()`

5.1.4.1.1 get()

Questo metodo ci permette di scaricare un file da una macchina remota.

Consideriamo i files contenuti nella directory di test `dir1`, sul server remoto, siano:

```
$ ls dir1/

fileremoto
```

Esaminiamo le seguenti linee di codice:

```
import paramiko

ssh_client =paramiko.SSHClient()
ssh_client.set_missing_host_key_policy(paramiko.AutoAddPolicy())
ssh_client.connect(hostname='localhost',username='utente',password
='password')
remotePathname = '/home/massimo/dir1/fileremoto'
localPathname = '/home/massimo/dir1/filelocale'
ftpClient=ssh_client.open_sftp()
```

```
ftpClient.get(remotePathname,localPathname)
ftpClient.close()
```

Su sfondo grigio sono riportate le righe di codice necessarie per effettuare la connessione alla macchina server (cfr. par. 5.1.3.1). Eseguendo il programma notiamo che il file `remotepathname` è stato uploadato sulla macchina in locale, infatti nella directory di prova ora abbiamo:

```
$ ls dir1/

fileremoto filelocale
```

ATTENZIONE: In questo esempio la macchina locale e il server remoto coincidono.

5.1.4.1.2 put()

Questo metodo ci permette di caricare un file dalla macchina locale sul server remoto.

Consideriamo i files contenuti nella directory di test `dir1`, sulla macchina locale, siano:

```
$ ls dir1/

filelocale
```

Esaminiamo le seguenti linee di codice:

```
import paramiko

ssh_client =paramiko.SSHClient()
ssh_client.set_missing_host_key_policy(paramiko.AutoAddPolicy())
ssh_client.connect(hostname='localhost',username='utente',password
='password')
remotePathname = '/home/massimo/dir1/fileremoto'
localPathname = '/home/massimo/dir1/filelocale'
ftpClient=ssh_client.open_sftp()
ftpClient.put(localPathname,remotePathname)
ftpClient.close()
```

Su sfondo grigio sono riportate le righe di codice necessarie per effettuare la connessione alla macchina server (cfr. par. 5.1.3.1). Eseguendo il programma notiamo che il file remotepathname è stato uploadato sulla macchina in locale, infatti nella directory di prova ora abbiamo:

```
$ ls dir1/
```

```
filelocale fileremoto
```

ATTENZIONE: In questo esempio la macchina locale e il server remoto coincidono.

5.2 Connessioni FTP

FTP o File Transfer Protocol è un noto protocollo di rete utilizzato per trasferire file tra computer in una rete. È basato sull'architettura client-server e si utilizza generalmente insieme all'autenticazione dell'utente. Può essere utilizzato anche senza autenticazione, ma sarà meno sicuro.

Le connessioni FTP gestiscono una directory di lavoro corrente e a ogni trasferimento richiedono una connessione secondaria attraverso le quali vengono trasferiti i dati.

I browser Web più comuni possono recuperare file ospitati su server FTP.

5.2.1 Libreria `ftplib`[19]

Analizzermo la libreria ftplib presente di default nell'installazione standard del Python.
Questo modulo definisce la classe FTP e alcuni elementi correlati.
La classe FTP implementa il lato client del protocollo FTP. Puoi usarlo per scrivere programmi Python che eseguono una varietà di lavori FTP automatizzati, come il mirroring di altri server FTP.
Viene anche utilizzato dal modulo *`urllib.request`* per gestire gli URL che utilizzano FTP.
Per ulteriori informazioni su FTP (File Transfer Protocol), vedere RFC 959[20] di Internet .

5.2.2 La classe FTP

È la classe che utilizzeremo per tutte le operazioni.
Restituisce una nuova istanza della classe FTP.

Proviamo il seguente programma che ci consente di loggarci a un server FTP:

```
import ftplib

# connect to host, default port
ftp = ftplib.FTP('ftp.us.debian.org')
# user anonymous, passwd anonymous@
print(ftp.login())
ftp.quit()
```

[19] https://docs.python.org/3/library/ftplib.html
[20] https://www.rfc-editor.org/rfc/rfc959.html

Eseguendo il programma otteniamo il seguente output:

```
230 Login successful.
```

Dall'analisi del codice e dell'output osserviamo che:

- ➔ Abbiamo istanziato la classe FTP del modulo ftplib passando come parametro il nome a dominio del server ftp a cui ci vogliamo connettere
- ➔ Il metodo `login()` dell'oggetto FTP ci permette di autenticarci al server remoto. In questo caso non fornendo alcun parametro abbiamo chiesto un'autenticazione con utente `anonymous` . Se l'autenticazione avviene correttamente il metodo restituisce un messaggio di successo.
- ➔ Il metodo `quit()` dell'oggetto FTP permette di uscire dal servere remoto

5.2.2.1 La classe FTP al microscopio

Esaminiamo più in dettaglio i parametri utili per istanziare la classe FTP.

```
ftplib.FTP( host = '' , utente = '' , passwd = '' , acct = '' ,
timeout = None , source_address = None , codifica = 'utf-8' )
```

Quando istanziamo la classe FTP e viene passato un host (nome a dominio o indirizzo IP), viene effettuata la chiamata al metodo `connect(host)`.
Quando viene fornito anche l'utente, viene eseguita anche la chiamata al metodo `login(user, passwd, acct)` dove `passwd` e `acct` sono impostati per default a stringa vuota (`acct` sta per: *"accounting information"*, è un parametro che pochi server FTP utilizzano)
Il parametro facoltativo `timeout` specifica un timeout in secondi per le operazioni di blocco come il tentativo di connessione (se non è specificato, verrà utilizzata l'impostazione di timeout predefinita globale).
`source_address` è una tupla di 2 elementi `(host,port)` per il socket a cui associarsi come indirizzo di origine prima della connessione.
Il parametro `encoding` specifica la codifica per directory e nomi di file.

Si ha la possibilità di connettersi utilizzando una connessione criptata utilizzando la classe FTP_SSL.

Il codice di connessione visto nel precedente paragrafo si potrebbe riscrivere istanziando un oggetto FTP senza passare alcun parametro e poi successivamente, andando a richiamare nell'ordine il metodo `connect()` e il metodo `login()`. Come il seguente esempio:

```
import ftplib
# connect to host, default port
ftp = ftplib.FTP()
print(ftp.connect('ftp.us.debian.org',21))
# user anonymous, passwd anonymous@
```

```
print(ftp.login())
ftp.quit()
```

5.2.2 I metodi della classe FTP

Abbiamo già esaminato i seguenti metodi della classe FTP che qui riportiamo:

- `connect(host='', port=0, timeout=None, source_address=None)`
  Attenzione a inserire come porta la numero 21 (il default) o comunque la corretta risposta a cui risponde il server FTP a cui ci si vuole connettere
- `login(user='anonymous', passwd='', acct='')`
  Se non si passano parametri ci si collega come `anonymous`
- `quit()`

Esaminiamo anche una lista dei metodi più significativi che potremmo utilizzare per gestire i files:

- `pwd()`:
  Directory di lavoro corrente.
- `cwd()`:
  Cambia la directory di lavoro corrente nel percorso.
- `dir([path[,...[,cb]])`:
  Visualizza l'elenco delle directory del percorso path. Opzionalmente viene passata la funzione di callback cb al metodo che preleva le righe fetchate (`retrlines()`).
- `delete(path)`:
  Elimina il file remoto che si trova in path.
- `mkd(directory)`:
  Crea directory remota.
- `retrbinary(cmd,cb,blocksize=8192, rest=None)`
  cmd è un comando "`RETR` " e cb è la funzione di callback
- `storbinary(cmd, fp, blocksize=8192, cb=None, rest=None)`
  cmd è un comando "`STOR` " , fp è un oggetto file aperto in lettura binario e cb è la funzione di callback

Nei prossimi paragrafi andremo a usare alcuni di questi metodi.
Per una lista completa si consulti la documentazione ufficiale.[21]

5.2.2.1 Listare il contenuto di una directory remota

Proviamo a collegarci mediante accesso anonimo al server ftp `ftp.nluug.nl` , ci spostiamo in una directory e ne elenchiamo il contenuto.

Esaminiamo il seguente programma:

```
import ftplib
```

[21] https://docs.python.org/3/library/ftplib.html#ftp-objects

```
data = list()
ftp = ftplib.FTP('ftp.nluug.nl')
print(ftp.login())
print(ftp.pwd())
ftp.cwd('/pub/')
ftp.dir(data.append)
ftp.quit()
for line in data:
   print ("-", line)
```

Eseguendo il programma otteniamo il seguente output:

```
230 Login successful.
/
/pub
- drwxrwsr-x  2 504   450    4096 Mar 03  2011 office
- drwxrwsr-x  7 0     450    4096 Oct 16  2020 os
- drwxrwsr-x  2 504   450    4096 Jan 30  2013 programming
- lrwxrwxrwx  1 504   450       1 Nov 02  2007 pub -> .
- -rw-r--r--  1 0     450     237 May 24  2016 robots.txt
                  - - - OUTPUT TRONCATO - - -
```

Dall'analisi del codice e dell'output osserviamo che:

- ➔ Il metodo `pwd()` restituisce la directory corrente.
- ➔ Il metodo `cwd()` si sposta nella directory indicata nel suo argomento.
- ➔ Il metodo `dir()` lista il contenuto della directory corrente in quanto non è stato passato alcun directory specifica. Ha come argomento una funzione di callback: il metodo `append()` di una lista che serve per andare a memorizzare ogni riga restituita da `dir()` nella lista di supporto `data`.

5.2.2.2 Recupero files

Dopo aver ottenuto l'elenco dei files come mostrato nel precedente paragrafo, possiamo recuperare un file specifico utilizzando il metodo `retrbinary()` attraverso la funzione di supporto `getfile()`. Questa funzione effettua una copia del file dal sistema remoto al sistema locale da cui è stata avviata la connessione ftp.

Esaminiamo il seguente programma:

```
import ftplib

def getFile(ftp, filename):
   try:
     ftp.retrbinary("RETR " + filename, open(filename,'wb').write)
```

```
    except:
      print("Errore!")

ftp = ftplib.FTP("ftp.nluug.nl")
ftp.login()
ftp.cwd('/pub/')
getFile(ftp,'README.nluug.txt')
ftp.quit()
```

Eseguendo il programma ed esaminando la current working directory, dovremmo vedere il file `README.nluug.txt` correttamente scaricato sulla nostra macchina locale.

Se volessimo andare a uploadare un file sul server dovremmo usare il metodo `storbinary()` ma, naturalmente, dovremmo autenticarci al server FTP con un utente che abbia i diritti di scrittura.

6. Gestione Database

6.0 Introduzione

Lo standard Python per le interfacce di database è Python DB-API 2.0, specifiche standard descritte in PEP 249[22]. La maggior parte delle interfacce di database Python seguono questo standard.
Python Database API supporta un'ampia gamma di server di database come, per citarne solo alcuni:

- MySQL
- PostgreSQL
- Microsoft SQL Server
- Oracle
- Sqlite3

Potete trovare un elenco completo dei database supportati al seguente indirizzo: Interfacce di database Python e API .

È necessario scaricare un modulo API DB separato per ogni database a cui si deve fare accesso.
Ad esempio, se fosse necessario accedere a un database Oracle sarebbe necessario scaricare il modulo del database Oracle. Se invece servisse connettersi a un database MySQL sarebbe necessario il modulo per MySQL.

L'API DB fornisce uno standard minimo per lavorare con i database utilizzando le strutture e la sintassi Python, ove possibile.
Questa API include :

- Importazione del modulo API.
- Acquisizione di una connessione con il database.
- Emissione di istruzioni SQL e stored procedure.
- Chiusura della connessione

In questo capitolo affronteremo questi concetti con il modulo *MySQLdb* e il modulo *sqlite3* rispettivamente per gestire DBMS MySQL e per gestire sqlite3 che è un database leggero basato su file e che non richiede processi server separati.

6.1 Modulo MySQLdb

MySQLdb è un'interfaccia per la connessione a un server di database MySQL da Python. Implementa l'API del database Python v2.0 ed è basato sull'API C di MySQL.

[22] https://peps.python.org/pep-0249/

6.1.1 Installazione MySQLdb

Il modulo MySQLdb non è installato di default, quindi è necessario provvedere alla sua installazione.

Installiamo MySQLdb su una macchina GNU/Linux con distribuzione Ubuntu utilizzando il comando `apt`

```
# apt install -y python3-mysqldb
```

L'installazione deve essere effettuata da utente *root*.

Il comando andrà a installare la libreria e eventuali dipendenze non installate necessarie al pacchetto.

Per verificare la corretta installazione della libreria eseguire la seguente riga di codice:

```
import MySQLdb
```

Se si riceve il seguente errore:

```
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
ModuleNotFoundError: No module named 'MySQLdb'
```

Allora il modulo non è stato installato correttamente.

6.1.1.1 Ambiente di testing

Per effettuare le prove del codice è necessario avere un server MySQL installato in locale o avere l'accesso a un DMBS MySQL remoto con credenziali che ci permettano di avere diritti CRUD su un database.

Noi supporremo di avere accesso a un DBMS server MySQL sulla macchina locale e di accedere a un database di nome `PYTHONDB` cui può accedere l'utente *'utente'* con password *'password'*.

6.1.2 Connessione al database

Il seguente programma ci consente di effettuare la connessione al nostro DBMS in cui è stato creato il database PYTHONDB:

```
import MySQLdb

# Open database connection
db = MySQLdb.connect("localhost","utente","password","PYTHONDB" )
```

```
print(db)

# disconnect from server
db.close()
```

Eseguendo il programma otteniamo il seguente output:

```
<_mysql.connection open to 'localhost' at 0x1e1b640>
```

Dall'analisi del codice e dell'output osserviamo che:

- La classe connect di MySQldb permette di istanziare un oggetto connessione che permette di stabilire, appunto, una connessione con il DBMS target.
- I parametri che servono per istanziare l'oggetto connessione sono l'host del DBMS, l'utente e la password e il database a cui si vuole connettere.
- Il metodo `.close()` della classe `connection` permette di chiudere la connessione al DBMS e assicura che le risorse allocate vengano rilasciate.

6.1.3 Creazione di una tabella

Una volta stabilita una connessione al database, siamo pronti per creare tabelle o record nelle tabelle del database. Per fare questo si utilizza una classe che possiede i metodi per effettuare l'interazione necessaria con il database: la classe *cursor*.

Esaminiamo il seguente codice:

```
import MySQLdb

# Open database connection
db = MySQLdb.connect("localhost","utente","password","PYTHONDB" )

# prepare a cursor object using cursor() method
cursor = db.cursor()
print(cursor)

# Drop table if it already exist using execute() method.
print(cursor.execute("DROP TABLE IF EXISTS IMPIEGATO"))

# Create table
sql = """CREATE TABLE IMPIEGATO (
         ID INT(3) NOT NULL,
         NOME  CHAR(20) NOT NULL,
         COGNOME  CHAR(20),
         ETA INT,
         SESSO CHAR(1),
         STIPENDIO FLOAT )"""

print(cursor.execute(sql))
```

```
# Alter table
sql = """ALTER TABLE IMPIEGATO
         ADD PRIMARY KEY (ID)
    """
print(cursor.execute(sql))

# disconnect from server
db.close()
```

Eseguendo il programma otteniamo il seguente output:

```
<MySQLdb.cursors.Cursor object at 0x7f0fedd4ab80>
0
0
0
```

Dall'analisi del codice e dell'output osserviamo che:

- L'oggetto `cursor` è l'istanza della classe *`cursor`* del modulo MySQLdb
- `execute()` è il metodo della classe *`cursor`* che permette di eseguire del codice SQL. Nell'esempio si sono eseguite delle istruzioni di `DROP TABLE`, `CREATE TABLE` e `ALTER TABLE`.
- Il metodo execute restituisce il numero di righe coinvolte nella query. In questi due casi, trattasi di operazioni di *DDL* (*Data Definition Language*) che non influenzano alcuna riga.

La tabella creata ha la seguente struttura come visualizzato dal client *phpMyAdmin*:



Fig. 6.1.1

6.1.4 Operazione di inserimento

Inseriamo delle righe nella tabella creata nel precedente paragrafo.

Esaminiamo il seguente codice, su sfondo grigio è evidenziata la parte dell'inserimento:

```
import MySQLdb

# Open database connection
db = MySQLdb.connect("localhost","utente","password","PYTHONDB" )

# prepare a cursor object using cursor() method
cursor = db.cursor()

# Prepare SQL query to INSERT a record into the database.
sql = """INSERT INTO IMPIEGATO(ID, NOME,
       COGNOME, ETA, SESSO, STIPENDIO)
       VALUES (1,'Paolino', 'Paperino', 20, 'M', 2000)"""
try:
  # Execute the SQL command
  print(cursor.execute(sql))
  # Commit your changes in the database
  db.commit()
except:
  # Rollback in case there is any error
  print("ERRORE, effettuo il ROLLBACK!")

  db.rollback()
```

```
# disconnect from server
db.close()
```

Eseguendo il programma otteniamo il seguente output:

```
1
```

Dall'analisi del codice e dell'output osserviamo che:

- Effettuiamo l'inserimento di una riga in una tabella utilizzando il metodo `execute()` della classe `cursor`.
- Tale metodo restituisce 1 in quanto abbiamo agito su una sola riga della tabella.
- L'effettivo inserimento della riga nella tabella PYTHONDB avviene solo dopo aver richiamato il metodo `commit()` della classe cursor.
- Inseriamo l'esecuzione del metodo `execute()` all'interno di un `try - except` in quanto trattasi di una *sezione critica*. Se non avviene alcun errore allora la riga viene inserita, se si rileva un errore allora si effettua un'operazione di *rollback* e si riporta il database nello stato iniziale (cioè prima di effettuare l'inserimento). Il metodo `rollback()` permette di eseguire questa azione ed è un metodo della classe `cursor`.

Come riportato dal client web phpMyAdmin, la riga inserita nella tabella è la seguente:

ID	NOME	COGNOME	ETA	SESSO	STIPENDIO
1	Paolino	Paperino	20	M	2000

Fig. 6.1.2

NOTA BENE: Se rilanci il programma una seconda volta, la presenza della chiave primaria ID nella tabella fa generare un errore di inserimento. Tale errore viene rilevato nella sezione critica e quindi eseguito la parte di codice nel ramo dell'`except` portando alla stampa a monitor dell'avviso di errore e dell'esecuzione del metodo di `rollback()`.

6.1.4.1 Preparazione di query dinamiche

La maggior parte delle volte che eseguiamo delle query all'interno di un programma, conosciamo i valori che devono assumere i parametri solo in runtime, perchè, per esempio, sono inseriti da un utente o perchè vengono letti da un flusso dati esterno.
Per gestire queste situazioni possiamo andare a creare dinamicamente la *query string* utilizzando la concatenazione delle stringhe o la funzione di stampa utilizzando i formattatori % (cfr. 5.5.4 Vol. 1) o, ancora meglio, le f-string (cfr. 5.5.5 Vol. 1).

Per esempio supponiamo di comporre la query per inserire i campi del record della tabella IMPIEGATO, abbiamo la seguente parametrizzazione dei valori da inserire:

```
id = 1
```

```
nome = 'Paolino'
cognome = 'Paperino'
eta = 20
sesso = 'M'
stipendio = 2000.0
```

si potrebbe allora scrivere la seguente linea di codice:

```
sql=f"INSERT INTO IMPIEGATO(ID, NOME, COGNOME,  ETA,  SESSO"+ \
f", STIPENDIO) VALUES ( '{id}','{nome}', '{cognome}', '{eta}',”+\
f”'{sesso}', '{stipendio}')"
```

Se utilizzassimo il formattatore % (cfr. 5.5.4 Vol. 1), otterremmo:

```
sql="INSERT INTO IMPIEGATO(ID, NOME, COGNOME,  ETA,  SESSO"+ \
", STIPENDIO) VALUES ( '%d','%s', '%s', '%d', '%c', '%f')" % \
( id,nome, cognome, eta, sesso, stipendio)
```

Oppure potremo passare direttamente i parametri al metodo `execute()`. Vediamo quest'ultimo caso analizzando l'inserimento dell'utente e della password per la connessione al database:

```
userID = “utente”
password = “password”
cursore.execute(“INSERT INTO LOGIN VALUES(‘{userID}’,’{password}’)”
```

oppure con il formattatore %:

```
cursore.execute(“INSERT INTO LOGIN VALUES(“%s”, “%s”)” % \
(userID,password))
```

Oppure molto meglio usando:

```
cursore.execute(“INSERT INTO LOGIN VALUES(?, ?)” % \
(userID,password))
```

In questo ultimo caso utilizziamo **?** come segnaposto per poter permettere l'inserimento dei valori nelle nostre query dinamiche.

6.1.5 Operazione di lettura

Andare a recuperare le informazioni contenute nelle tabelle di un database è sinonimo di effettuare delle operazioni di lettura sul database stesso. Nell'acronimo *CRUD* la seconda lettera sta a indicare proprio questa operazione (*READ*).

Esistono dei metodi della classe *cursor* che ci permettono di gestire l'insieme di dati restituito da una query dopo aver richiamato il metodo `execute()`. Tale insieme è un oggetto che viene restituito quando un'istanza di `Cursor` viene utilizzata per interrogare una tabella tramite, appunto, il metodo `.execute()`.

Utilizziamo principalmente i seguenti metodi della classe *cursor* per reperire e gestire i dati letti da una query sul database:

- ➔ `fetchone()`: recupera la riga successiva di un set di risultati di query.
- ➔ `fetchall()`: recupera tutte le righe in un set di risultati. Se alcune righe sono già state estratte dal set di risultati, recupera le righe rimanenti dal set di risultati.
- ➔ `rowcount`: attributo di sola lettura, restituisce il numero di righe che sono state interessate da un metodo `execute()`.

Esaminiamo il seguente programma che reperisce le righe di una tabella utilizzando il metodo `fetchall()`, su sfondo grigio è evidenziata la parte dI interrogazione al database:

```
import MySQLdb

# Open database connection
db = MySQLdb.connect("localhost", "utente", "password", "PYTHONDB")

# prepare a cursor object using cursor() method
cursor = db.cursor()
```

```
# Prepare SQL query to query database
sql = "SELECT * FROM IMPIEGATO \
      WHERE STIPENDIO > '1000.50'"
try:
    # Execute the SQL command
    n = cursor.execute(sql)
    print("Righe: ", n, " ", cursor.rowcount)
    # Fetch all the rows in a list of lists.
    results = cursor.fetchall()
    print("results: ", results)
    for row in results:
        id = row[0]
        nome = row[1]
        cognome = row[2]
        eta = row[3]
        sesso = row[4]
        stipendio = row[5]
        # Now print fetched result
        print(
            f"id={id},
nome={nome},cognome={cognome},eta={eta},sesso={sesso},stipendio={s
tipendio}"
```

```
        )
except:
    print("Errore: inabile nel reperire i records")

# disconnect from server
db.close()
```

Supponiamo di avere nella tabella IMPIEGATO i seguenti record:

ID	NOME	COGNOME	ETA	SESSO	STIPENDIO
1	Paolino	Paperino	20	M	2000
2	Paperon	De Paperoni	60	M	1000000

Fig. 6.1.3

Eseguendo il programma otteniamo il seguente output:

```
Righe:  2   2
results:  ((1, 'Paolino', 'Paperino', 20, 'M', 2000.0), (2,
'Paperon', 'De Paperoni', 60, 'M', 1000000.0))
id=1,
nome=Paolino,cognome=Paperino,eta=20,sesso=M,stipendio=2000.000000
id=2, nome=Paperon,cognome=De
Paperoni,eta=60,sesso=M,stipendio=1000000.000000
```

Dall'analisi del codice e dell'output osserviamo che:

- ➔ Come visto nei precedenti esempi il metodo `execute()` restituisce il numero di righe coinvolte nell'istruzione SQL eseguita. In questo caso risulta 2 in quanto vengono restituite due righe.
- ➔ L'attributo `rowcount` della classe `cursor` risulta essere valorizzato a 2 in quanto il metodo `execute()` restituisce 2 righe.
- ➔ Il metodo `fetchall()` restituisce una tupla i cui elementi sono le righe restituite memorizzate a loro volta in tuple.

6.1.6 Operazione di aggiornamento

Andare a modificare le informazioni già presenti nelle tabelle di un database è sinonimo di effettuare delle operazioni di aggiornamento sul database stesso. Nell'acronimo *CRUD* la terza lettera sta a indicare proprio questa operazione (*UPDATE*).

Il metodo che andremo a utilizzare è stato già analizzato nei precedenti esempi ed è il metodo `execute()` della classe *cursor*.

Esaminiamo il seguente programma che aggiorna tutti i record della tabella IMPIEGATO andando a incrementare di uno il campo ETA a tutti i record che hanno nel campo SESSO il valore 'M'. Su sfondo grigio è evidenziata la parte di aggiornamento del database:

```
import MySQLdb

# Open database connection
db = MySQLdb.connect("localhost","utente","password","PYTHONDB" )

# prepare a cursor object using cursor() method
cursor = db.cursor()

sesso = 'M'
# Prepare SQL query to query database
sql = f"UPDATE IMPIEGATO SET ETA = ETA + 1 \
      WHERE SESSO = '{sesso}'"
try:
  # Execute the SQL command
  n = cursor.execute(sql)
  print("Righe: ",n," ",cursor.rowcount)
  db.commit()

except:
  print ("Errore UPDATE")
  db.rollback()

# disconnect from server
db.close()
```

Supponiamo di avere nella tabella IMPIEGATO i seguenti record:

ID	NOME	COGNOME	ETA	SESSO	STIPENDIO
1	Paolino	Paperino	20	M	2000
2	Paperon	De Paperoni	60	M	1000000

Fig. 6.1.4

Eseguendo il programma otteniamo il seguente output:

```
Righe:  2   2
```

e nella tabella IMPIEGATO otteniamo i seguenti record:

ID	NOME	COGNOME	ETA	SESSO	STIPENDIO
1	Paolino	Paperino	21	M	2000
2	Paperon	De Paperoni	61	M	1000000

Fig6.1.5

Dall'analisi del codice e dell'output osserviamo che:

- è necessario e sufficiente utilizzare unicamente il metodo `execute()` della classe `cursor` per effettuare una operazione di update su una tabella del database.
- Il metodo `execute()` restituisce 2 in quanto sono 2 le righe che vengono coinvolte nell'operazione di aggiornamento.
- L'attributo `rowcount` della classe cursor assume il valore 2 in quanto sono 2 le righe che vengono coinvolte nell'operazione di aggiornamento.
- Analogamente a quanto fatto per l'operazione di inserimento, si inserisce il codice che effettua l'aggiornamento in una sezione critica utilizzando `try - except`, facendo uso dei metodi `commit()` e `rollback()` .

6.1.7 Operazione di cancellazione

Andare a eliminare le informazioni presenti nelle tabelle di un database è sinonimo di effettuare delle operazioni di cancellazione sul database stesso. Nell'acronimo *CRUD* la quarta lettera sta a indicare proprio questa operazione (*DELETE*).

Il metodo che andremo a utilizzare è stato già analizzato nei precedenti esempi ed è il metodo `execute()` della classe `cursor`.

Esaminiamo il seguente programma che cancella tutti i record della tabella IMPIEGATO che hanno nel campo ETA un valore maggiore di 50. Su sfondo grigio è evidenziata la parte dI cancellazione nel database:

```
import MySQLdb

# Open database connection
db = MySQLdb.connect("localhost","utente","password","PYTHONDB" )

# prepare a cursor object using cursor() method
cursor = db.cursor()
```

```
eta = 50
# Prepare SQL query to query database
sql = f"DELETE FROM IMPIEGATO \
      WHERE ETA > '{eta}'"
try:
  # Execute the SQL command
  n = cursor.execute(sql)
  print("Righe: ",n," ",cursor.rowcount)
  db.commit()

except:
  print ("Errore DELETE")
  db.rollback()
```

```
# disconnect from server
db.close()
```

Supponiamo di avere nella tabella IMPIEGATO i seguenti record:

ID	NOME	COGNOME	ETA	SESSO	STIPENDIO
1	Paolino	Paperino	21	M	2000
2	Paperon	De Paperoni	61	M	1000000

Fig. 6.1.6

Eseguendo il programma otteniamo il seguente output:

```
Righe:  1   1
```

e nella tabella IMPIEGATO otteniamo i seguenti record:

ID	NOME	COGNOME	ETA	SESSO	STIPENDIO
1	Paolino	Paperino	21	M	2000

Fig6.1.7

Dall'analisi del codice e dell'output osserviamo che:

- è necessario e sufficiente utilizzare unicamente il metodo execute() della classe `cursor` per effettuare una operazione di `delete` su una tabella del database.
- Il metodo `execute()` restituisce 1 in quanto è coinvolta solo una riga nell'operazione di cancellazione.
- L'attributo `rowcount` della classe cursor assume il valore 1 in quanto è coinvolta solo una riga nell'operazione di cancellazione.
- Analogamente a quanto fatto per l'operazione di inserimento, si inserisce il codice che effettua la cancellazione in una sezione critica utilizzando `try - except`, facendo uso dei metodi `commit()` e `rollback()` .

6.1.8 Le transazioni: `commit()` e `rollback()`

Le transazioni sono un meccanismo che garantisce la coerenza dei dati.

Senza voler avere la pretesa di essere esaustivi, si elencano di seguito alcune caratteristiche delle transazioni, più che altro per comprendere il contesto.

Le transazioni hanno le seguenti quattro proprietà:

- *Atomicità*: una transazione deve essere completata oppure non accade nulla, cioè è come se non fosse mai avvenuta.
- *Coerenza*: una transazione deve iniziare in uno stato coerente e lasciare il sistema in uno stato coerente.

- ➔ *Isolamento*: i risultati intermedi di una transazione non sono visibili al di fuori della transazione corrente.
- ➔ *Durabilità*: una volta che una transazione è stata confermata, gli effetti sono persistenti, anche dopo un errore di sistema.

Python DB API 2.0 fornisce due metodi per eseguire una transazione: `commit()` e `rollback()`.

Abbiamo già incontrato questi metodi quando abbiamo affrontato le operazioni *CRUD*, sono metodi della classe `cursor`.

Riportiamo a titolo di esempio la parte di codice del precedente paragrafo che serve per cancellare delle righe nella tabella di esempio IMPIEGATO del database di test PYTHONDB:

```
eta = 50
# Prepare SQL query to query database
sql = f"DELETE FROM IMPIEGATO \
      WHERE ETA > '{eta}'"

try:
  # Execute the SQL command
  cursor.execute(sql)
  db.commit()

except:
  print ("Errore DELETE")
  db.rollback()
```

Il metodo di `commit()` viene richiamato solo se l'operazione è andata a buon fine. Consideriamo una *commit* come l'operazione che dà un segnale verde al database per finalizzare le modifiche. Dopo questa operazione, nessuna modifica può essere ripristinata.

Il metodo `rollback()` viene richiamato se nella *sezione critica* non tutte le operazioni sul database sono andate a buon fine. Allora, in questo caso, è preferibile ripristinare tutto il database a come era prima di iniziare le operazioni nella *sezione critica*. Il ripristino viene effettuato richiamando il metodo `rollback()`.

6.1.9 La disconnessione

Abbiamo già visto che la disconnessione dal DBMS si ottiene invocando il metodo `close()` dell'oggetto `connect` di MySQLdb.

Se la connessione a un database viene chiusa dall'utente con il metodo `close()`, tutte le transazioni in sospeso vengono annullate dal DB.
Detto questo si consiglia di progettare applicazioni che richiamano esplicitamente i metodi `commit()` e `rollback()`, questo in quanto si evita di dipendere da qualsiasi dettaglio di implementazione a basso livello del modulo e/o del database visto che

potrebbero modificarsi nel tempo e portare a criticità l'applicazione.

6.2 Modulo SQLite3

SQLite3 è un'interfaccia per la connessione a un database SQLite da Python. Implementa l'API del database Python v2.0 ed è basato sull'API di SQLite.

6.2.1 Installazione SQLite3

Il modulo SQLite3 è installato di default in Python, quindi non è necessario provvedere alla sua installazione.

Per verificare la corretta installazione della libreria eseguire la seguente riga di codice:

```
import sqlite3
```

Se si riceve il seguente errore:

```
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
ModuleNotFoundError: No module named 'sqlite3'
```

Allora il modulo non è stato installato correttamente.

6.2.1.1 Ambiente di testing

Per effettuare le prove del codice è necessario avere un file di database SQLite locale. Noi supporremo di avere accesso a un database di nome `python.db`.

6.2.2 Connessione al database

Il seguente programma ci consente di effettuare la connessione al nostro database SQLite:

```
import sqlite3

# Open database connection
db = sqlite3.connect("python.db")
print(db)

# disconnect from server
db.close()
```

Eseguendo il programma otteniamo il seguente output:

```
'sqlite3'<sqlite3.Connection object at 0x7da8b9f12540>
```

Dall'analisi del codice e dell'output osserviamo che:

- La classe `connect` di SQLite3 permette di istanziare un oggetto connessione che permette di stabilire una connessione con il database target.
- Il parametro che serve per istanziare l'oggetto connessione è il nome del file di database.
- Il metodo `.close()` della classe `connection` permette di chiudere la connessione al database e assicura che le risorse allocate vengano rilasciate.

6.2.3 Creazione di una tabella

Una volta stabilita una connessione al database, siamo pronti per creare tabelle o record nelle tabelle del database. Per fare questo si utilizza una classe che possiede i metodi per effettuare l'interazione necessaria con il database: la classe `cursor`.

Esaminiamo il seguente codice:

```
import sqlite3

# Open database connection
db = sqlite3.connect("python.db")

# prepare a cursor object using cursor() method
cursor = db.cursor()
print(cursor)

# Drop table if it already exists using execute() method.
print(cursor.execute("DROP TABLE IF EXISTS IMPIEGATO"))

# Create table
sql = """CREATE TABLE IMPIEGATO (
           ID INT PRIMARY KEY NOT NULL,
           NOME TEXT NOT NULL,
           COGNOME TEXT,
           ETA INT,
           SESSO CHAR(1),
           STIPENDIO REAL)"""
print(cursor.execute(sql))

# disconnect from server
db.close()
```

Eseguendo il programma otteniamo il seguente output:

```
<sqlite3.Cursor object at 0x715a274a6c40>
<sqlite3.Cursor object at 0x715a274a6c40>
<sqlite3.Cursor object at 0x715a274a6c40>
```

Dall'analisi del codice e dell'output osserviamo che:

- L'oggetto `cursor` è l'istanza della classe `Cursor` del modulo SQLite3.
- `.execute()` è il metodo della classe `Cursor` che permette di eseguire del codice SQL. Nell'esempio si sono eseguite delle istruzioni di `DROP TABLE` e `CREATE TABLE`.
- Il metodo .execute() non restituisce nulla, a differenza del comportamento dell'analogo metodo della libreria MysqlDB (cfr. 1.4.3). Se si stampa il valore di ritorno si ottiene l'istanza della classe `Cursor`.

La tabella creata ha la seguente struttura come visualizzato dal client *sqlitebrowser*:

IMPIEGATO		CREATE TABLE IMPIEGATO
ID	INT	"ID" INT NOT NULL
NOME	TEXT	"NOME" TEXT NOT NULL
COGNOME	TEXT	"COGNOME" TEXT
ETA	INT	"ETA" INT
SESSO	CHAR(1)	"SESSO" CHAR(1)
STIPENDIO	REAL	"STIPENDIO" REAL

Fig. 6..2.1

ATTENZIONE: Il metodo `.execute()` è stato dichiarato deprecato dalla versione di python 3.12 e verrà rimosso dalla versione 3.14. Al suo posto si dovrà utilizzare il metodo `.executescript()` [23]

6.2.4 Operazione di inserimento

Inseriamo delle righe nella tabella creata nel precedente paragrafo.

Esaminiamo il seguente codice, su sfondo grigio è evidenziata la parte dell'inserimento:

```
import sqlite3
# Open database connection
db = sqlite3.connect("python.db")

# prepare a cursor object using cursor() method
cursor = db.cursor()
```

[23] https://docs.python.org/3/library/sqlite3.html#sqlite3.Cursor.execute

```
# Prepare SQL query to INSERT a record into the database.
sql = """INSERT INTO IMPIEGATO(ID, NOME, COGNOME, ETA, SESSO,
STIPENDIO)
         VALUES (1, 'Paolino', 'Paperino', 20, 'M', 2000)"""
try:
    # Execute the SQL command
    print(cursor.execute(sql))
    print(cursor.rowcount)
    # Commit your changes in the database
    db.commit()
except:
    # Rollback in case there is any error
    print("ERRORE, effettuo il ROLLBACK!")
    db.rollback()

# disconnect from server
db.close()
```

Eseguendo il programma otteniamo il seguente output:

```
<sqlite3.Cursor object at 0x715a274a6c40>
1
```

Dall'analisi del codice osserviamo che:

- Effettuiamo l'inserimento di una riga in una tabella utilizzando il metodo `.execute()` della classe `Cursor`.
- L'effettivo inserimento della riga nella tabella `IMPIEGATO` avviene solo dopo aver richiamato il metodo `.commit()` della classe `Connection`.
- Inseriamo l'esecuzione del metodo `execute()` all'interno di un `try - except` in quanto trattasi di una sezione critica. Se non avviene alcun errore allora la riga viene inserita, se si rileva un errore allora si effettua un'operazione di *rollback* e si riporta il database nello stato iniziale (cioè prima di effettuare l'inserimento). Il metodo `rollback()` permette di eseguire questa azione ed è un metodo della classe `Connection`.
- Il metodo `.execute()` non restituisce il numero di righe inserite.
- Per conoscere il numero di righe inserite si può utilizzare l'attributo a sola lettura: `.rowcount`. Questo restituisce il numero di righe coinvolte nel caso di utilizzo delle istruzioni SQL: `INSERT, UPDATE, DELETE e REPLACE`[24].

Come riportato dal client sqlitebrowser, la riga inserita nella tabella è la seguente:

[24] https://docs.python.org/3/library/sqlite3.html#sqlite3.Cursor.execute (ricercare rowcount)



Fig. 6.2.2

NOTA BENE: Se rilanci il programma una seconda volta, la presenza della chiave primaria ID nella tabella fa generare un errore di inserimento. Tale errore viene rilevato nella sezione critica e quindi eseguito la parte di codice nel ramo dell'`except` portando alla stampa a monitor dell'avviso di errore e dell'esecuzione del metodo di `rollback()`.

6.2.4.1 Preparazione di query dinamiche

La maggior parte delle volte che eseguiamo delle query all'interno di un programma, conosciamo i valori che devono assumere i parametri solo in runtime, perchè, per esempio, sono inseriti da un utente o perchè vengono letti da un flusso dati esterno.
Per gestire queste situazioni possiamo andare a creare dinamicamente la *query string* utilizzando la concatenazione delle stringhe o la funzione di stampa utilizzando i formattatori % (cfr. 5.5.4 Vol. 1) o, ancora meglio, le f-string (cfr. 5.5.5 Vol. 1).

Per esempio supponiamo di comporre la query per inserire i campi del record della tabella IMPIEGATO, abbiamo la seguente parametrizzazione dei valori da inserire:

```
id = 1
nome = 'Paolino'
cognome = 'Paperino'
eta = 20
sesso = 'M'
stipendio = 2000.0
```

si potrebbe allora scrivere la seguente linea di codice:

```
sql=f"INSERT INTO IMPIEGATO(ID, NOME, COGNOME,  ETA,  SESSO"+ \
f", STIPENDIO) VALUES ( '{id}','{nome}', '{cognome}', '{eta}',”+\
f”'{sesso}', '{stipendio}')"
```

Se utilizzassimo il formattatore % (cfr. 5.5.4 Vol. 1), otterremmo:

```
sql="INSERT INTO IMPIEGATO(ID, NOME, COGNOME,  ETA,  SESSO"+ \
", STIPENDIO) VALUES ( '%d','%s', '%s', '%d', '%c', '%f')" % \
( id,nome, cognome, eta, sesso, stipendio)
```

Oppure potremo passare direttamente i parametri al metodo `execute()`. Vediamo quest'ultimo caso analizzando l'inserimento dell'utente e della password per la connessione al database:

```
userID = "utente"
password = "password"
cursore.execute("INSERT INTO LOGIN VALUES('{userID}','{password}')"
```

oppure con il formattatore %:

```
cursore.execute("INSERT INTO LOGIN VALUES("%s", "%s")" % \
(userID,password))
```

Oppure molto meglio usando:

```
cursore.execute("INSERT INTO LOGIN VALUES(?, ?)" % \
(userID,password))
```

In questo ultimo caso utilizziamo **?** come segnaposto per poter permettere l'inserimento dei valori nelle nostre query dinamiche.

6.2.5 Operazione di lettura

Andare a recuperare le informazioni contenute nelle tabelle di un database è sinonimo di effettuare delle operazioni di lettura sul database stesso. Nell'acronimo CRUD la seconda lettera sta a indicare proprio questa operazione (READ).

Esistono dei metodi della classe `Cursor` che ci permettono di gestire l'insieme di dati restituito da una query dopo aver richiamato il metodo `.execute()`. Tale insieme è un oggetto che viene restituito quando un'istanza di `Cursor` viene utilizzata per interrogare una tabella tramite, appunto, il metodo `.execute()`.

Utilizziamo principalmente i seguenti metodi della classe `Cursor` per reperire e gestire i dati letti da una query sul database:

- ➔ `.fetchone()`: recupera la riga successiva di un set di risultati di query.
- ➔ `.fetchall()`: recupera tutte le righe in un set di risultati. Se alcune righe sono già state estratte dal set di risultati, recupera le righe rimanenti dal set di risultati.

Esaminiamo il seguente programma che reperisce le righe di una tabella utilizzando il metodo `.fetchall()`, su sfondo grigio è evidenziata la parte di interrogazione al database:

```
import sqlite3

# Open database connection
db = sqlite3.connect("python.db")

# prepare a cursor object using cursor() method
```

```
cursor = db.cursor()
```

```
# Prepare SQL query to query database
sql = "SELECT * FROM IMPIEGATO WHERE STIPENDIO > 1000.50"

try:
    # Execute the SQL command
    cursor.execute(sql)
    # Fetch all the rows in a list of lists.
    results = cursor.fetchall()
    print(f"Numero di record: {len(results)}")
    for row in results:
        print(row)
except:
    print("Errore nella lettura dei dati")
    db.rollback()
```

```
# disconnect from server
db.close()
```

Supponiamo di avere nella tabella IMPIEGATO i seguenti record:



	ID	NOME	COGNOME	ETA	SESSO	STIPENDIO
	F...	Filtro	Filtro	Fi...	Filtro	Filtro
1	1	Paolino	Paperino	20	M	2000
2	3	Pape...	De ...	60	M	1000000

Fig. 6.2.3

Eseguendo il programma otteniamo il seguente output:

```
Numero di record: 2
(1, 'Paolino', 'Paperino', 20, 'M', 2000.0)
(3, 'Paperone', 'De Paperoni', 60, 'M', 1000000)
```

Dall'analisi del codice osserviamo che:

- Dopo aver richiamato il metodo `.execute()` l'oggetto `Cursor` è pronto a restituire il set di risultati.
- L'istruzione `results = cursor.fetchall()` recupera tutte le righe del set di risultati della query effettuata.
- La variabile `results` è un elenco di tuple, dove ogni tupla contiene una riga del set di risultati della query effettuata.

➔ Per conoscere il numero di righe estratte andiamo a utilizzare la funzione `len()` applicata alla tupla che viene utilizzata per memorizzare il results set.

6.2.6 Operazione di aggiornamento

Effettuare un aggiornamento su una tabella di un database è un' operazione che consente di modificare uno o più campi di una riga o più righe di una tabella. Nell'acronimo CRUD la terza lettera sta a indicare proprio questa operazione (UPDATE).

Esaminiamo il seguente programma che aggiorna tutti i record della tabella IMPIEGATO andando a incrementare di mille il campo STIPENDIO a tutti i record che hanno nel campo SESSO il valore 'M'. Su sfondo grigio è evidenziata la parte di aggiornamento del database:

```
import sqlite3

# Open database connection
db = sqlite3.connect("python.db")

# prepare a cursor object using cursor() method
cursor = db.cursor()

sesso = 'M'
# Prepare SQL query to update required records
sql = f"UPDATE IMPIEGATO SET STIPENDIO = STIPENDIO + 1000 WHERE
SESSO = '{sesso}'"
try:
    # Execute the SQL command
    cursor.execute(sql)
    # Commit your changes in the database
    db.commit()
    print("Righe aggiornate: ",cursor.rowcount)

except:
    # Rollback in case there is any error
    print("Errore nell'aggiornamento dei dati")
    db.rollback()

# disconnect from server
db.close()
```

Supponiamo di avere nella tabella IMPIEGATO i seguenti record:



Fig. 6.2.4

Eseguendo il programma otteniamo il seguente output:

```
Righe aggiornate:  2
```

e nella tabella IMPIEGATO otteniamo i seguenti record:



Fig6.2.5

Dall'analisi del codice osserviamo che:

- Effettuiamo l'aggiornamento di una o più righe di una tabella utilizzando il metodo `.execute()` della classe `Cursor`.
- L'effettivo aggiornamento delle righe nella tabella `IMPIEGATO` avviene solo dopo aver richiamato il metodo `.commit()` della classe `Connection`.
- L'attributo `rowcount` della classe cursor assume il valore 2 in quanto sono 2 le righe che vengono coinvolte nell'operazione di aggiornamento.
- Inseriamo l'esecuzione del metodo `execute()` all'interno di un `try - except` in quanto trattasi di una sezione critica. Se non avviene alcun errore allora la riga viene inserita, se si rileva un errore allora si effettua un'operazione di rollback e si riporta il database nello stato iniziale (cioè prima di effettuare l'aggiornamento). Il metodo `.rollback()` permette di eseguire questa azione ed è un metodo della classe `Connection`.

6.2.7 Operazione di cancellazione

La cancellazione di un record di una tabella è un'operazione che elimina uno o più record di una tabella di un database. Nell'acronimo CRUD l'ultima lettera sta a indicare proprio questa operazione (DELETE).

Esaminiamo il seguente programma che cancella tutti i record della tabella IMPIEGATO che hanno nel campo ETA un valore maggiore di 50. Su sfondo grigio è evidenziata la parte dI cancellazione nel database:

```
import sqlite3

# Open database connection
db = sqlite3.connect("python.db")

# prepare a cursor object using cursor() method
cursor = db.cursor()

eta = 20
# Prepare SQL query to DELETE required records
sql = f"DELETE FROM IMPIEGATO WHERE ETA > {eta}"
try:
    # Execute the SQL command
    cursor.execute(sql)
    # Commit your changes in the database
    db.commit()
    print("Righe cancellate: ",cursor.rowcount)
except:
    # Rollback in case there is any error
    print("Errore nella cancellazione dei dati")
    db.rollback()

# disconnect from server
db.close()
```

Supponiamo di avere nella tabella IMPIEGATO i seguenti record:



	ID	NOME	COGNOME	ETA	SESSO	STIPENDIO
	F...	Filtro	Filtro	Fi...	Filtro	Filtro
1	1	Paolino	Paperino	20	M	2000
2	3	Pape...	De ...	60	M	1000000

Fig. 6.2.6

Eseguendo il programma otteniamo il seguente output:

```
Righe cancellate:  1
```

e nella tabella IMPIEGATO otteniamo i seguenti record:


Tabella: IMPIEGATO

	ID	NOME	COGNOME	ETA	SESSO	STIPENDIO
	F...	Filtro	Filtro	Fi...	Filtro	Filtro
1	4	Paolino	Paperino	20	M	3000.0

Fig6.2.7

Dall'analisi del codice osserviamo che:

- Effettuiamo la cancellazione di una o più righe di una tabella utilizzando il metodo `.execute()` della classe `Cursor`.
- L'effettiva cancellazione delle righe nella tabella `IMPIEGATO` avviene solo dopo aver richiamato il metodo `.commit()` della classe `Connection`.
- L'attributo `rowcount` della classe cursor assume il valore 1 in quanto è coinvolta solo una riga nell'operazione di cancellazione.
- Inseriamo la chiamata del metodo `.execute()` all'interno di un `try - except` in quanto trattasi di una sezione critica. Se non avviene alcun errore allora la riga viene inserita, se si rileva un errore allora si effettua un'operazione di rollback e si riporta il database nello stato iniziale (cioè prima di effettuare l'aggiornamento). Il metodo `.rollback()` permette di eseguire questa azione ed è un metodo della classe `Connection`.

6.2.8 Lettura di file SQL

La lettura di file SQL con Python può essere effettuata con *SQLite3*. Il metodo `.executescript()` del modulo `sqlite3` permette di eseguire direttamente uno script SQL contenuto in un file di testo.

Vediamo un esempio di lettura di file SQL che inserisce tre record nella tabella IMPIEGATO. Su sfondo grigio chiaro è evidenziata la parte di lettura dello script SQL, mentre su sfondo grigio scuro è evidenziata la parte di esecuzione delle istruzioni SQL memorizzate nel file:

```
import sqlite3

# Open database connection
db = sqlite3.connect("python.db")

# prepare a cursor object using cursor() method
cursor = db.cursor()

# Read SQL script
with open('script.sql', 'r') as fh:
```

```
    sql_script = fh.read()

try:
    # Execute SQL script
    cursor.executescript(sql_script)
    # Commit your changes in the database
    db.commit()
    print("Righe inserite: ",cursor.rowcount)
except:
    # Rollback in case there is any error
    print("Errore nell'esecuzione  dello script")
    db.rollback()

# disconnect from server
db.close()
```

Il file di testo `script.sql` contiene 3 righe con 3 insert nella tabella `IMPIEGATO`:

```
INSERT INTO IMPIEGATO(ID, NOME, COGNOME, ETA, SESSO, STIPENDIO)
VALUES (11, 'Cognome1', 'Nome1', 30, 'M', 1500);
INSERT INTO IMPIEGATO(ID, NOME, COGNOME, ETA, SESSO, STIPENDIO)
VALUES (12, 'Cognome2', 'Nome2', 35, 'F', 2500);
INSERT INTO IMPIEGATO(ID, NOME, COGNOME, ETA, SESSO, STIPENDIO)
VALUES (13, 'Cognome3', 'Nome3', 50, 'M', 2800);
```

Supponiamo di avere nella tabella IMPIEGATO i seguenti record:



Fig. 6.2.8

Eseguendo il programma otteniamo il seguente output:

```
Righe inserite:  -1
```

e nella tabella IMPIEGATO otteniamo i seguenti record:



	ID	NOME	COGNOME	ETA	SESSO	STIPENDIO
	F...	Filtro	Filtro	Fi...	Filtro	Filtro
1	4	Paolino	Paperino	20	M	3000.0
2	11	Cogn...	Nome1	30	M	1500.0
3	12	Cogn...	Nome2	35	F	2500.0
4	13	Cogn...	Nome3	50	M	2800.0

Fig6.2.9

Dall'analisi del codice e dell'output osserviamo che:

- Utilizziamo la funzione `open` per aprire in lettura il file che contiene lo script SQL
- Leggiamo tutto il file e lo memorizziamo in una stringa.
- Richiamiamo il metodo `.executescript()` della classe `Cursor` per eseguire lo script SQL contenuto nel file.
- Lo script SQL deve essere formattato secondo la sintassi SQL (notare anche li punti e virgola per terminare un comando)
- L'attributo `.rowcount` è inizializzato a -1 in quanto funziona solo con i metodi `.execute()` e `.executemany()`

7. Il WEB

7.1 WEB Server

7.1.1 Introduzione

La libreria Python standard ha un modulo integrato che può essere utilizzato come server Web HTTP/HTTPS minimalista. Fornisce il supporto del protocollo e consente di estendere le capacità mediante sottoclassi.

Fornire file HTML/CSS statici al mondo esterno può essere molto utile in molte situazioni pratiche. Ad esempio, per mostrare delle pagine HTML, testare delle API od ottenere un modo veloce per visualizzare alcune informazioni su dei server remoti.

Utilizzare il webserver integrato nelle librerie standard di Python è più veloce e immediato che utilizzare un server web implementato con Flask (ma ovviamente le features sono molto minori di quest'ultima implementazione).

7.1.2 Lanciare un'istanza del webserver

Proviamo subito a creare un'istanza del webserver sulla nostra macchina locale.

Si esegua il seguente comando dal prompt:

```
$ python3 -m http.server 1970 --bind 127.0.0.1
```

Se il comando sta funzionando correttamente e non ha prodotto alcun errore dovreste osservare una situazione analoga alla seguente:

```
$ python3 -m http.server 1970 --bind 127.0.0.1
Serving HTTP on 127.0.0.1 port 1970 (http://127.0.0.1:1970/) ...
```

Questo indica che il server è partito correttamente e che è in attesa di rispondere a richieste HTTP.

Aprendo un qualsiasi browser internet e andando all'indirizzo:

`http://localhost:1970/`

dovreste visualizzare il contenuto della directory corrente da cui è stato lanciato il comando. Praticamente dovreste visualizzare un qualcosa di simile a questo:

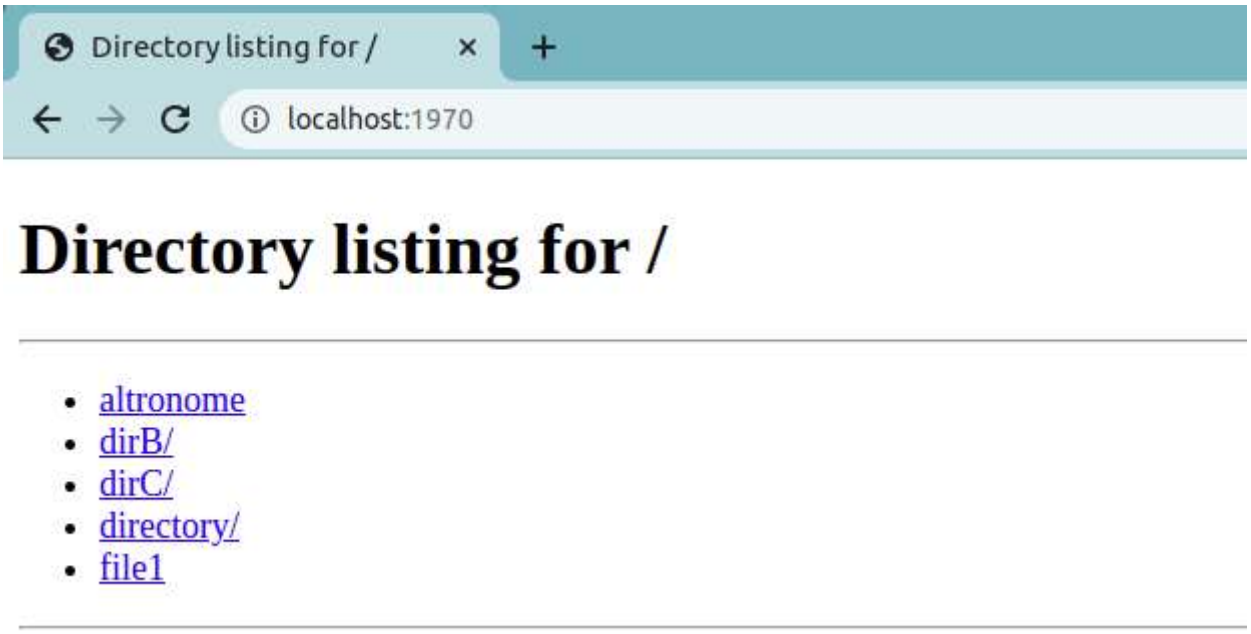


Fig. 7.1

Le directory sono navigabili e i files sono scaricabili.

Questo potrebbe essere un modo semplice per trasferire velocemente e facilmente dei file da una macchina remota a una macchina locale.

Attenzione, però, alla sicurezza visto che il protocollo HTTP non è criptato!

La documentazione ufficiale del modulo[25] specifica un'avvertenza che esplicita che questo modulo implementa solo una verifica di sicurezza base e quindi non è raccomandato in ambienti di produzione.

7.1.2.1 Il comando al microscopio

Abbiamo lanciato il seguente comando:

```
$ python3 -m http.server 1970 –bind 127.0.0.1
```

Analizziamo i vari parametri:

Il parametro -m dell'interprete dei comandi Python indica che stiamo lanciando la libreria specificata come uno script e i parametri che seguono rappresentano la lista delle opzioni del modulo stesso.

Quindi `1970` e `--bind 127.0.0.1` rappresentano le opzioni di `http.server`.

Il primo parametro rappresenta la porta su cui si mette in ascolto il webserver, è un parametro opzionale, se non si specifica la porta è 8000.

[25] https://docs.python.org/3/library/http.server.html

Il secondo parametro indica l'indirizzo IP su cui il webserver lavora. Anche questo è opzionale, se non specificato il webserver si lega a tutte le interfacce di rete attive.

Provare a lanciare il seguente comando:

```
$ python3 -m http.server
```

Si noterà che il server è attivo sulla porta `8000` sia su `localhost` che sugli altri indirizzi IP attivi sulle interfacce di rete della macchina host.

7.1.3 Supporto SSL

Per eseguire un server HTTP over SSL è necessario scrivere del codice per modificare il comportamento di default della classe `http.server`.

Si esamini il seguente codice:

```
from http.server import HTTPServer, BaseHTTPRequestHandler
import ssl

httpd = HTTPServer(("localhost", 1970), BaseHTTPRequestHandler)
ctx = ssl.create_default_context(ssl.Purpose.CLIENT_AUTH)
ctx.load_cert_chain(certfile="./cert.pem", keyfile="./key.pem")
httpd.socket = ctx.wrap_socket(httpd.socket, server_side=True)
httpd.serve_forever()
```

Eseguendo il programma otteniamo il seguente output:

```
Traceback (most recent call last):
  File "/home/massimo/PythonProjects/prova.py", line 7, in
<module>
    httpd.socket = ssl.wrap_socket (httpd.socket,
  File "/usr/lib/python3.9/ssl.py", line 1402, in wrap_socket
    ctx.load_cert_chain(certfile="./cert.pem",
keyfile="./key.pem")
FileNotFoundError: [Errno 2] No such file or directory
```

Si ottiene il seguente errore perchè si deve creare il file che contiene le chiavi di autorizzazione.
Per generare file di chiavi e certificati con OpenSSL, utilizzare il comando seguente:

```
$ openssl req -x509 -newkey rsa:2048 -keyout key.pem -out cert.pem -days 365
```

Si rimanda alla documentazione di openssl e di come si gestiscono i certificati per una comprensione del comando e dei suoi parametri. Quello che a noi interessa è che così facendo creiamo i files necessari per aprire una connessione over SSL. Dobbiamo

ricordare la passphrase che inseriamo in quanto ogni volta che lanciamo lo script Python questa ci verrà chiesta.

Ora se lanciamo il programma otteniamo la richiesta di inserire la pass phrase, dopodichè il server si mette in ascolto sulla porta 1970 utilizzando il protocollo https. Otteniamo sul prompt quanto segue:

```
$ python3 miohttpserver.py
Enter PEM pass phrase:
```

Dopo aver inserito la corretta pass phrase sul prompt non appare più nulla, questo indica che il server è partito correttamente e che è in attesa di rispondere a richieste HTTP.

Aprendo un qualsiasi browser internet e andando all'indirizzo:

`https://localhost:1970/`

dovreste visualizzare un qualcosa di simile a questo:



Fig. 7.2

Inanzitutto notate che il protocollo è `https`, se utilizzate `http` non otterrete alcuna risposta. Naturalmente la vostra certificazione non è riconosciuta da nessun ente accreditato quindi, per navigare il sito, dovete accettare un'eccezione di sicurezza.

La questione più importante è l'errore che si vede nella pagina. Questo indica che il server non ha nessuna implementazione del metodo GET, in altre parole quando il browser si collega al webserver esegue una richiesta GET, ma il webserver non sa come rispondere perchè il metodo che implementa la risposta non è implementato!

Comprendiamo ancora meglio le richieste del browser web se andiamo a dare un'occhiata alla CLI aperta per lanciare lo script Python che istanzia il webserver, verosimilmente avrete ottenuto una cosa del genere:

```
$ python3 miohttpserver.py
Enter PEM pass phrase:
```

```
127.0.0.1 - - [28/May/2022 20:32:06] code 501, message Unsupported method
('GET')
127.0.0.1 - - [28/May/2022 20:32:06] "GET / HTTP/1.1" 501 -
127.0.0.1 - - [28/May/2022 20:32:07] code 501, message Unsupported method
('GET')
127.0.0.1 - - [28/May/2022 20:32:07] "GET /favicon.ico HTTP/1.1" 501 -
```

Le righe di testo in cui appare:"GET / HTTP/1.1" sono le richieste effettuate dal browser internet.

Prima di risolvere questo comportamento cerchiamo di esaminare più approfonditamente il codice che abbiamo lanciato.

7.1.4 Le classi di http.server

Appaiono due classi contenute nel modulo http.server:

- ➔ `HTTPServer`
- ➔ `BaseHTTPRequestHandler`

- ➔ `HTTPServer( server_address , RequestHandlerClass )`

Questa classe è una sottoclasse di `socket.TCPServer` (della standard library del Python) che porta ad avere `server_address` come una tupla che memorizza l'indirizzo del server e la porta. Queste sono variabili di istanza rispettivamente `server_name` e `server_port`.
`RequestHandlerClass` è la classe che l'istanza di HTTPServer utilizza per gestire le richieste che gli arrivano.
Questa classe espone il metodo `serve_forever()` che permette di startare effettivamente il server e di porlo in ascolto sull'indirizzo e sulla porta indicata nella tupla `server_address`.

- ➔ `BaseHTTPRequestHandler(request, client_address, server)`

Questa classe viene utilizzata per gestire le richieste HTTP che arrivano al server. Di per sé, non può rispondere a nessuna richiesta HTTP effettiva; deve essere ereditata da un'altra classe per gestire ogni metodo di richiesta (es. GET o POST). Questa classe fornisce una serie di variabili di classe e di istanza e metodi che possono essere utilizzate dalle sue sottoclassi.

7.1.5 Implementazione del metodo di risposta al GET

Ora andremo a modificare il codice in maniera tale da implementare tale metodo seppur in maniera molto semplice e banale.

Per ovviare al problema del metodo che risponde alla richiesta GET del browser internet andremo a implementare la classe `MyHTTPRequestHandler` che è una sottoclasse della classe `BaseHTTPRequestHandler` in cui andiamo a effettuare un override del metodo `do_GET()`.

Esaminiamo il seguente codice:

```
from http.server import HTTPServer, BaseHTTPRequestHandler

class MyHTTPRequestHandler(BaseHTTPRequestHandler):

    def do_GET(self):
        self.send_response(200)
        self.end_headers()
        self.wfile.write(b'Hello, world!')

httpd = HTTPServer(('localhost', 1970), MyHTTPRequestHandler)

httpd.serve_forever()
```

Aprendo un qualsiasi browser internet e andando all'indirizzo:

```
http://localhost:1970/
```

dovreste visualizzare un qualcosa di simile a questo:

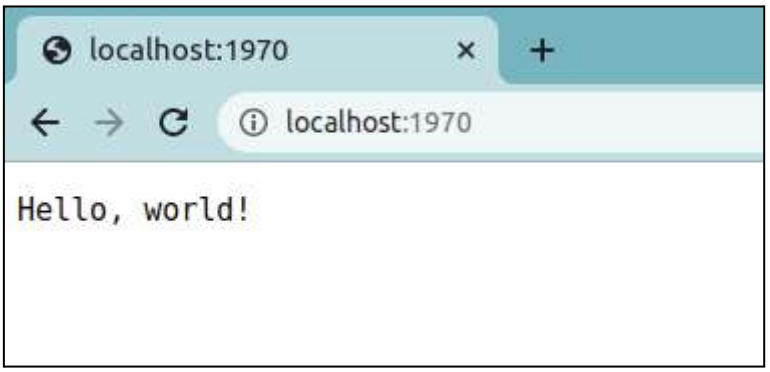


Fig. 7.3

L'output sulla CLI sarà qualcosa del genere:

```
127.0.0.1 - - [28/May/2022 21:19:48] "GET / HTTP/1.1" 200 -
```

Dall'analisi del codice e dell'output osserviamo che:

- Sull'output viene stampata la richiesta effettuata dal browser internet che esegue la sua richiesta standard di `GET`
- Nel browser viene visualizzato la stringa:"Hello, world!" che indica che il metodo `do_GET()` ha risposto correttamente e correttamente viene visualizzato dal browser internet.
- I metodi `send_response()` e `end_headers()` sono metodi che obbligatoriamente devono essere richiamati all'interno del metodo `do_GET()` affinchè la risposta del webserver verso il browser web possa funzionare correttamente. Si rimanda alla documentazione ufficiale l'approfondimento

sull'azione di questi metodi.

➔ L'oggetto `wfile` è un oggetto simile a un file, ci permette di scrivere la risposta, che vogliamo inviare al browser web, attraverso il suo metodo `write()`.

7.1.6 Implementazione del metodo di risposta al POST

Gestiamo le richieste POST del browser web.
Andremo a effettuare l'overriding del metodo `do_POST()` della classe `BaseHTTPRequestHandler`.

Esaminiamo il seguente codice, la parte a sfondo grigio è riferita all'implementazione del metodo do_POST( ):

```
from http.server import HTTPServer, BaseHTTPRequestHandler

class MyHTTPRequestHandler(BaseHTTPRequestHandler):

    def do_GET(self):
        self.send_response(200)
        self.end_headers()
        self.wfile.write(b'Hello, world!')

    def do_POST(self):
         content_length = int(self.headers['Content-Length'])
         body = self.rfile.read(content_length)
         self.send_response(200)
         self.end_headers()
         self.wfile.write(b"This is POST request.\n")
         self.wfile.write(f"Received: {content_length} bytes\n"
.encode())
         self.wfile.write(body+b"\n")

httpd = HTTPServer(('localhost', 1970), MyHTTPRequestHandler)

httpd.serve_forever()
```

Per testare una richiesta POST utilizziamo il client testuale CURL, tale client ci permette di verificare, tra l'altro, cosa restituisce un server web a seguito di una richiesta POST.

Il comando da dare è il seguente:

```
$ curl -i -d "param1=value1" http://localhost:1970
```

Il parametro `-i` include nell'output l'headers della risposta, non è necessario per i nostri scopi, ma in generale è utile visualizzarlo.
Il parametro `-d` consente di inserire l'HTTP POST data, naturalmente questo è un parametro necessario per testare il nostro metodo.

L'output sulla CLI sarà qualcosa del genere:

```
HTTP/1.0 200 OK
Server: BaseHTTP/0.6 Python/3.9.7
Date: Thu, 16 Jun 2022 19:34:48 GMT

This is POST request.
Received:
param1=value1
```

Le prime tre righe dell'output è l'header della pagina web che si ottiene come risposta grazie al parametro `-i` del comando `curl`. Le successive tre righe sono l'output effettivo fornito dal metodo `doPOST()`.

Dall'analisi del codice e dell'output osserviamo che:

- Innanzitutto andiamo a reperire la lunghezza dei dati del POST forniti al server.
- Utilizzando la lunghezza così ottenuta andiamo a leggere l'oggetto `file` che rappresenta il flusso di input al nostro server web
- Dopo aver letto l'oggetto `file` abbiamo in mano i dati del post che possiamo elaborare, prima di passare alla fase di elaborazione è necessario richiamare i metodi `send response()` e `send_headers()` necessari affinché il protocollo sia consistente.
- L'elaborazione dei dati POST consistono semplicemente nella loro stampa sul flusso `wfile`.
- `.encode()` è un metodo delle stringhe che converte una stringa (tipo `str`) in una sequenza di bytes (tipo `bytes`). Tanto per essere chiari il tipo `str` rappresenta testo Unicode (quindi caratteri leggibili) il tipo `bytes` rappresenta una sequenza di valori numerici da 0 a 255 (non leggibili)

7.2 Esempio notevole: semplice web application

7.2.1 Introduzione

Ora andremo a creare una semplice applicazione web utilizzando il Python senza installare nessuno dei framework più popolari per la creazione di web application come, p.e., Django [26] e Flask[27] .
Possiamo usare il modulo `http.server` visto subito sopra e il database SQLite3 incontrato nei precedenti capitoli (cfr. cap. 6.2).

Andremo a creare un semplice modulo HTML che accetti gli input dell'utente come

[26] https://www.djangoproject.com/
[27] https://flask.palletsprojects.com/

richieste POST e visualizza i record salvati attraverso richieste GET.
Per salvare gli input dell'utente utilizzeremo database SQLite3 che, come sappiamo, è supportato di default in Python.

7.2.2 Struttura del progetto

Innanzitutto creiamo la directory che conterrà tutto il nostro progetto. Chiamiamo questa directory `webApplication`.
Per creare tale directory e spostarcisi al suo interno utilizziamo i seguenti comandi (supponiamo di essere su una macchina con s.o. GNU/Linux)

```
$ mkdir webApplication
$ cd webApplication/
```

Il primo comando serve per creare la directory `webApplication` nella directory corrente e il secondo comando serve per spostarsi all'interno della directory appena creata.

Successivamente andiamo a creare una struttura di directories atta a contenere il codice HTML e gli script per il database. Quindi eseguiamo i seguenti comandi:

```
$ mkdir database
$ mkdir templates
```

Creiamo all'interno della cartella `database` i files: `database.py` e `userRecords.db` mentre all'interno della cartella `templates` creiamo i files: `form.html` e `showRecords.html`.
Infine creiamo nella cartella `webApplication` il file `webServer.py`.

Nei prossimi paragrafi vedremo cosa conterranno i files creati.
La struttura del filesystem del nostro progetto è riassunta dal seguente grafo ad albero:

```
webApplication
├── database
│   ├── database.py
│   └── userRecords.db
├── templates
│   ├── form.html
│   └── showRecords.html
└── webServer.py
```

7.2.1 Codice dei templates HTML

I files HTML all'interno della cartella templates devono essere i seguenti:

7.2.1.1 form.html

```
<!DOCTYPE html>
<html>
<body>
   <h2>Python HTTP Server Form</h2>
   <form method="POST" action='/success'>
       <label for="full_name">Full Name:</label><br>
       <input type="text" name="full_name" value="Pippo"><br>
       <label for="country">Country:</label><br>
       <input type="text" name="country" value="Paperopoli"><br><br>
       <input type="submit" value="Submit">
   </form>
</body>
</html>
```

Questo codice crea una pagina HTML che chiede all'utente di inserire due campi: nome e paese.

7.2.1.1 showRecords.html

```
<!DOCTYPE html>
<html>
<head>
   <style>
       table {
           font-family: arial, sans-serif;
           border-collapse: collapse;
           width: 100%;
       }
       td,
       th {
           border: 1px solid #dddddd;
           text-align: left;
           padding: 8px;
       }
       tr:nth-child(even) {
           background-color: #dddddd;
       }
   </style>
</head>
<body>
   <h2>Python HTTP Server (Records in sqlite3 database)</h2>
   <table>
       <tr>
           <th>Name</th>
           <th>Country</th>
       </tr>
```

```
        {{userRecords}}
    </table>
</body>
</html>
```

Questo codice ha lo scopo di visualizzare tutte le coppie nome e paese memorizzate nel database.

7.2.2 Codice per la gestione del database

Andiamo ora a creare lo script Python che ci servirà per gestire il database SQLite3. Ricordiamo che il modulo SQLite3 non ha bisogno di essere installato in quanto presente nella Standard Library di Python.

7.2.1.1 database.py

```
from sqlite3 import connect
from sqlite3.dbapi2 import Cursor

DB_NAME = "database/userRecords.db" # database name
# create database inside database folder if not exists
connection = connect(DB_NAME)
cursor = connection.cursor()

def create_table():
    """function to create table inside database"""
   # create table user inside database if not exists
    table_script = '''CREATE TABLE IF NOT EXISTS User(
                    full_name VARCHAR(255),
                    country VARCHAR(150)
                );
                '''
    cursor.executescript(table_script)
    connection.commit()

def insert_record(fullname, country):
    """function to insert record inside table"""
    cursor.execute("INSERT INTO User(full_name, country) VALUES(?, ?)",
                   (fullname, country))
    connection.commit()

def fetch_records():
    """function to fetch User records"""
    data = cursor.execute("SELECT * FROM User")
    return data
```

Questo codice effettua una connessione al database `SQLite3 userRecords` e implementa 3 funzioni che servono, rispettivamente, per :

- ➔ creare la tabella `User` con campi `full_name` e `country`,
- ➔ inserire un record nella tabella User e

➔ recuperare tutti i record memorizzati nella tabella `User`.

7.2.2 Codice per il server http: serverWeb.py

A questo punto andiamo a codificare il nostro server http come visto nel precedente capitolo. Andremo a gestire le richieste `GET` e `POST` utilizzando le funzioni definite nel modulo `database.py`.

Andiamo a scrivere il seguente codice nel modulo webServer.py sotto il folder webApplication

```
import sys
from urllib.parse import parse_qs
from http.server import HTTPServer, BaseHTTPRequestHandler
from database.database import create_table, insert_record,
fetch_records

HOST_NAME = "localhost"
PORT = 1970

def read_html_template(path):
  """function to read HTML file"""
  try:
      with open(path) as f:
          file = f.read()
  except Exception as e:
      file = e
  return file

def show_records(self):
  """function to show records in template"""
  file = read_html_template(self.path)
 # fetch records from database
  table_data = fetch_records()
  table_row = ""
  for data in table_data:
      table_row += "<tr>"
      for item in data:
          table_row += "<td>"
          table_row += item
          table_row += "</td>"
      table_row += "</tr>"
 # replace {{user_records}} in template by table_row
  file = file.replace("{{userRecords}}", table_row)
  self.send_response(200, "OK")
  self.end_headers()
  self.wfile.write(bytes(file, "utf-8"))

class webApplication(BaseHTTPRequestHandler):
  """Python HTTP Server that handles GET and POST requests"""
```

```
  def do_GET(self):
      if self.path == '/':
          self.path = './templates/form.html'
          file = read_html_template(self.path)
          self.send_response(200, "OK")
          self.end_headers()
          self.wfile.write(bytes(file, encoding="utf-8"))
      if self.path == '/showRecords':
          self.path = './templates/showRecords.html'
         # call show_records function to show users entered
          show_records(self)
   def do_POST(self):
        if self.path == "/success":
            # read body length from header, then decode the raw bytes
            content_length = int(self.headers.get("Content-Length", 0))
            body = self.rfile.read(content_length).decode("utf-8")
            # parse_qs handles application/x-www-form-urlencoded (HTML
default)
            fields = parse_qs(body)
            full_name = fields.get("full_name", [""])[0]
            country = fields.get("country", [""])[0]
            # create table User if it runs first time, else skip
            create_table()
            # insert record into User table
            insert_record(full_name, country)
            html = "<html><head></head><body><h1>Form data successfully
recorded!!!</h1></body></html>"
            self.send_response(200, "OK")
            self.end_headers()
            self.wfile.write(html.encode(encoding="utf-8"))

if __name__ == "__main__":
  server = HTTPServer((HOST_NAME, PORT), webApplication)
  print(f"Server started http://{HOST_NAME}:{PORT}")
  try:
      server.serve_forever()
  except KeyboardInterrupt:
      server.server_close()
      print("Server stopped successfully")
      sys.exit(0)
```

7.2.3 Test della webapplication

Proviamo a eseguire la webApplication lanciando lo script webServer.py come segue:

```
/webApplication$ python3 webServer.py
```

Una volta lanciato dovreste vedere il seguente messaggio visualizzato sul terminale:

```
Server started http://localhost:1970
```

E aprendo un qualsiasi browser internet e andando all'indirizzo:

`http://localhost:1970/`

dovreste visualizzare un qualcosa di simile a questo:



sulla CLI dovreste visualizzare la seguente richiesta:

```
Server started http://localhost:1970
127.0.0.1 - - [14/Jun/2022 22:55:06] "GET / HTTP/1.1" 200 -
```

Cliccato sul bottone di submit dovreste visualizzare la seguente schermata di successo:



Fig. 7.4

e nella console del terminale:

```
Server started http://localhost:1970
127.0.0.1 - - [14/Jun/2022 22:55:06] "GET / HTTP/1.1" 200 -
```

```
127.0.0.1 - - [14/Jun/2022 23:00:13] "POST /success HTTP/1.1" 200 -
```

Proviamo adesso, a inserire altre coppie di dati, una volta fatto possiamo visualizzare i dati memorizzati utilizzando la seguente URL:

`http://localhost:1970/showRecords`

appena dato invio dovreste visualizzare un qualcosa di simile a questo:



Fig. 7.5

Questi sono i due records che sono stati inseriti con le precedenti operazioni effettuate con il nostro applicativo web.

7.2.4 Prospettive

Abbiamo visto come creare una semplice webApplication utilizzando i moduli della Standard Library di Python: http.server e SQLite3.
Tale applicazione gestisce l'inserimento e la visualizzazione di semplici informazioni memorizzate in una tabella.
Per interagire con il database abbiamo utilizzato le richieste HTTP GET e POST.

Potrebbe essere interessante pensare a un'espansione delle funzionalità dell'applicazione, p.e. si potrebbe implementare la cancellazione di un record e la ricerca di un particolare utente o paese. Si lascia tali modifiche al lettore come utile esercizio.

ATTENZIONE: riportiamo quanto già scritto nel capitolo della trattazione di http.server e cioè che non è consigliato per i casi d'uso di produzione poiché implementa solo controlli di sicurezza di base.

NOTA BENE: Questa è un'applicazione di esempio didattica. Normalmente tali applicazioni vengono implementate utilizzando dei framework come quelli indicati all'inizio del capitolo.

7.3 Modulo requests

La libreria `requests` di Python è una delle più popolari per effettuare richieste HTTP. È semplice, intuitiva e permette di interagire con i server web in modo efficace.

Potete trovare su https://requests.readthedocs.io/en/latest/[28] una documentazione esaustiva.

7.3.1 Installazione

Prima di iniziare a utilizzare `requests`, è necessario installarla. Puoi farlo facilmente utilizzando pip, il gestore dei pacchetti di Python:

```
$ pip3 install requests
```

L'installazione non deve necessariamente essere effettuata da utente *root*.

7.3.2 Effettuare una richiesta GET

Una delle operazioni più comuni in HTTP è la richiesta GET, che viene utilizzata per recuperare dati da un server.
Nel nostro esempio vogliamo utilizzare le API che ci vengono fornite dal famoso sistema di version control basato su *git*[29] : *github*[30]. Le API di github ci permettono di automatizzare alcune operazioni e di costruire applicazioni complesse inerenti lo sviluppo di applicazioni, per un'approfondimento esaminare la documentazione ufficiale delle API di github[31].
Quindi il server a cui ci riferiamo e': `api.github.com`.

Si esamini il seguente programma per comprendere come utilizzeremo `requests` per reperire dati dal server di github:

```
import requests

response = requests.get("https://api.github.com")
print(type(response))
print(response.status_code)
print(response.text)
```

[28] https://requests.readthedocs.io/en/latest/
[29] https://git-scm.com/
[30] https://github.com/
[31] https://docs.github.com/en/rest

Lanciando il programma otteniamo in output:

```
<class 'requests.models.Response'>
200
{
  "current_user_url": "https://api.github.com/user",
  ...
}
```

L'output e' stato troncato per comodita'.

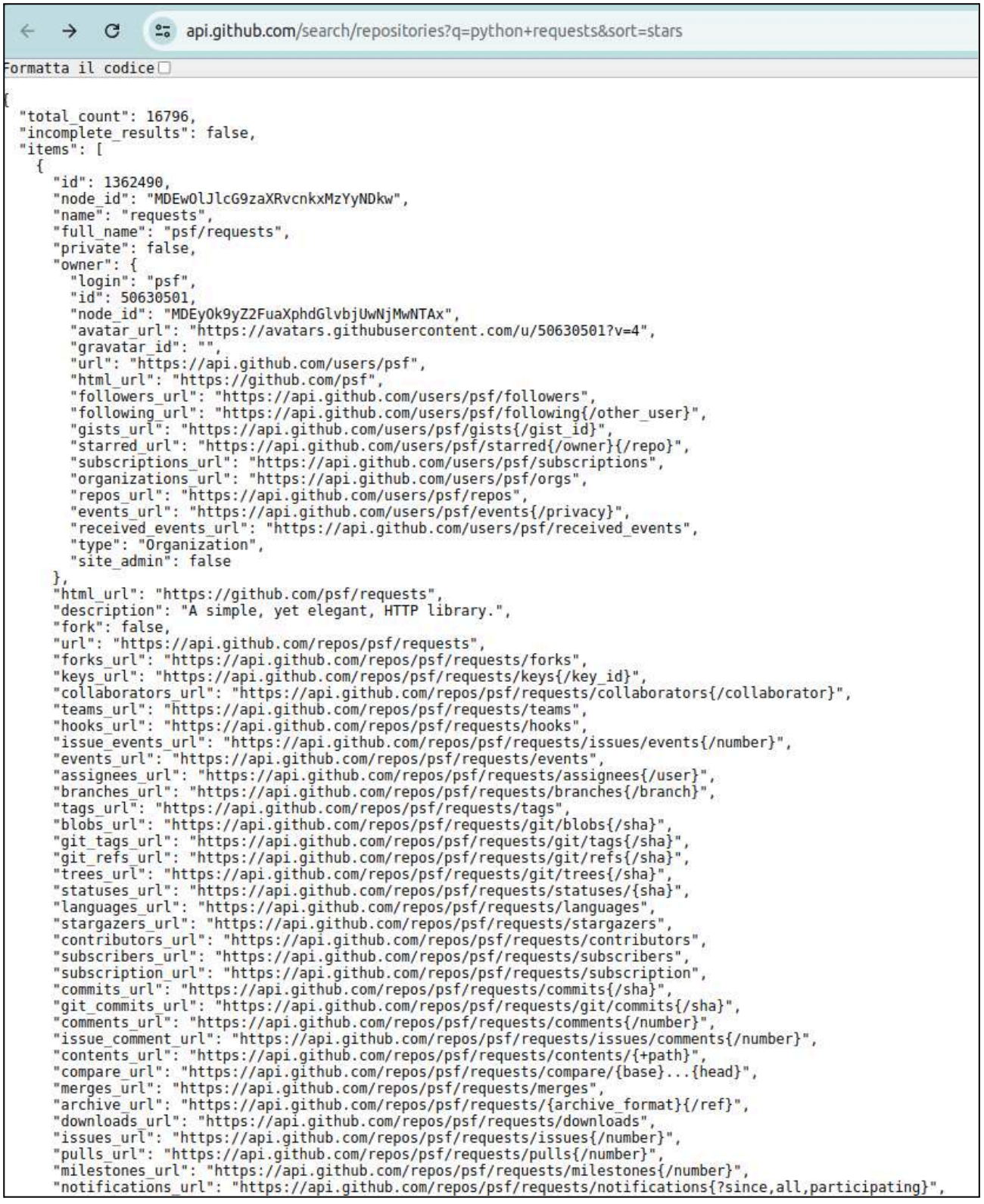
Dall'analisi del codice e dell'output osserviamo che:

- `requests.get()` invia una richiesta GET all'URL specificato.
- `requests.get()` restituisce un'istanza della classe `Response` che contiene i dati ottenuti dalla richiesta GET al sito web.
- `response.status_code` contiene il codice di stato HTTP della risposta. Un codice 200 indica che la richiesta è stata eseguita correttamente.
- `response.text` contiene il corpo della risposta in formato testo.

7.3.2.1 Parametri della richiesta

A volte è necessario passare dei parametri alla richiesta GET. Questo può essere fatto utilizzando il parametro `params del metodo get() di requests`.
Supponiamo di voler richiedere questa URL :
https://api.github.com/search/repositories?q=python+requests&sort=stars

Utilizzando un browser web otterremo:




```
{
  "total_count": 16796,
  "incomplete_results": false,
  "items": [
    {
      "id": 1362490,
      "node_id": "MDEwOlJlcG9zaXRvcnkxMzYyNDkw",
      "name": "requests",
      "full_name": "psf/requests",
      "private": false,
      "owner": {
        "login": "psf",
        "id": 50630501,
        "node_id": "MDEyOk9yZ2FuaXphdGlvbjUwNjMwNTAx",
        "avatar_url": "https://avatars.githubusercontent.com/u/50630501?v=4",
        "gravatar_id": "",
        "url": "https://api.github.com/users/psf",
        "html_url": "https://github.com/psf",
        "followers_url": "https://api.github.com/users/psf/followers",
        "following_url": "https://api.github.com/users/psf/following{/other_user}",
        "gists_url": "https://api.github.com/users/psf/gists{/gist_id}",
        "starred_url": "https://api.github.com/users/psf/starred{/owner}{/repo}",
        "subscriptions_url": "https://api.github.com/users/psf/subscriptions",
        "organizations_url": "https://api.github.com/users/psf/orgs",
        "repos_url": "https://api.github.com/users/psf/repos",
        "events_url": "https://api.github.com/users/psf/events{/privacy}",
        "received_events_url": "https://api.github.com/users/psf/received_events",
        "type": "Organization",
        "site_admin": false
      },
      "html_url": "https://github.com/psf/requests",
      "description": "A simple, yet elegant, HTTP library.",
      "fork": false,
      "url": "https://api.github.com/repos/psf/requests",
      "forks_url": "https://api.github.com/repos/psf/requests/forks",
      "keys_url": "https://api.github.com/repos/psf/requests/keys{/key_id}",
      "collaborators_url": "https://api.github.com/repos/psf/requests/collaborators{/collaborator}",
      "teams_url": "https://api.github.com/repos/psf/requests/teams",
      "hooks_url": "https://api.github.com/repos/psf/requests/hooks",
      "issue_events_url": "https://api.github.com/repos/psf/requests/issues/events{/number}",
      "events_url": "https://api.github.com/repos/psf/requests/events",
      "assignees_url": "https://api.github.com/repos/psf/requests/assignees{/user}",
      "branches_url": "https://api.github.com/repos/psf/requests/branches{/branch}",
      "tags_url": "https://api.github.com/repos/psf/requests/tags",
      "blobs_url": "https://api.github.com/repos/psf/requests/git/blobs{/sha}",
      "git_tags_url": "https://api.github.com/repos/psf/requests/git/tags{/sha}",
      "git_refs_url": "https://api.github.com/repos/psf/requests/git/refs{/sha}",
      "trees_url": "https://api.github.com/repos/psf/requests/git/trees{/sha}",
      "statuses_url": "https://api.github.com/repos/psf/requests/statuses/{sha}",
      "languages_url": "https://api.github.com/repos/psf/requests/languages",
      "stargazers_url": "https://api.github.com/repos/psf/requests/stargazers",
      "contributors_url": "https://api.github.com/repos/psf/requests/contributors",
      "subscribers_url": "https://api.github.com/repos/psf/requests/subscribers",
      "subscription_url": "https://api.github.com/repos/psf/requests/subscription",
      "commits_url": "https://api.github.com/repos/psf/requests/commits{/sha}",
      "git_commits_url": "https://api.github.com/repos/psf/requests/git/commits{/sha}",
      "comments_url": "https://api.github.com/repos/psf/requests/comments{/number}",
      "issue_comment_url": "https://api.github.com/repos/psf/requests/issues/comments{/number}",
      "contents_url": "https://api.github.com/repos/psf/requests/contents/{+path}",
      "compare_url": "https://api.github.com/repos/psf/requests/compare/{base}...{head}",
      "merges_url": "https://api.github.com/repos/psf/requests/merges",
      "archive_url": "https://api.github.com/repos/psf/requests/{archive_format}{/ref}",
      "downloads_url": "https://api.github.com/repos/psf/requests/downloads",
      "issues_url": "https://api.github.com/repos/psf/requests/issues{/number}",
      "pulls_url": "https://api.github.com/repos/psf/requests/pulls{/number}",
      "milestones_url": "https://api.github.com/repos/psf/requests/milestones{/number}",
      "notifications_url": "https://api.github.com/repos/psf/requests/notifications{?since,all,participating}",
```


Fig. 7.6

Questo e' il contenuto del risultato della query effettuata sui repositories github, abbiamo richiesto il primo progetto che appare con una ricerca su parole chiavi: "python" e "requests" ordinati per numero di stelle.

Per effettuare una query del genere dobbiamo costruire un dizionario in cui come chiavi inseriamo i parametri di ricerca e come valori i valori della ricerca stessa.

Quindi params sara':

```
params = {
    'q': 'python requests',
    'sort': 'stars',
    'per_page': 1
}
```

L'url che andremo a interrogare sara':

```
https://api.github.com/search/repositories
```

A questo punto il nostro programma diverra':

```
import requests

params = {
    'q': 'python requests',
    'sort': 'stars',
    'per_page': 1
}
response = requests.get("https://api.github.com/search/repositories",
params=params)
print(response.url)
print(response.json())
```

Eseguendo questo programma, otteniamo il seguente output:

```
https://api.github.com/search/repositories?q=python+requests&sort=s
tars&per_page=1
{'total_count': 16796, 'incomplete_results': False, 'items':
[{'id': 1362490, 'node_id': 'MDEwOlJlcG9zaXRvcnkxMzYyNDkw', 'name':
'requests', 'full_name': 'psf/requests', 'private': False, 'owner':
...
}
```

L'output e' stato troncato per comodita'.

Dall'analisi del codice e dell'output osserviamo che:

- `params` è un dizionario che contiene i parametri della richiesta.
- `response.url` mostra l'URL completo con i parametri della query.
- `response.json()` restituisce il corpo della risposta in formato JSON. Questo metodo è utile quando si lavora con API che restituiscono dati in formato JSON.

(Solitamente le risposte delle API forniscono sempre dati formattati JSON)

➔ Naturalmente, come si puo' vedere dall'output, cio' che restituisce il metodo `json()` e' un *dizionario* python. Nel primo esempio `response.text` forniva una tipo *stringa*.

7.3.3 Richiesta POST

Le richieste POST vengono utilizzate per inviare dati al server.

Per testare le richieste POST useremo il sito `httpbin.org`[32] che permette agli sviluppatori di provare le chiamate a delle API standard. E' stato scritto da uno sviluppatore *Kenneth Reitz* e reso disponibile gratuitamente dallo stesso.

Diamo un'occhiata al seguente programma:

```
import requests

data = {
    'username': 'testuser',
    'password': 'password123'
}
response = requests.post("https://httpbin.org/post", data=data)
print(response.status_code)
print(response.text)
```

Eseguendo questo programma, otteniamo il seguente output:

```
200
{
  "args": {},
  "data": "",
  "files": {},
  "form": {
    "password": "password123",
    "username": "testuser"
  },
  "headers": {
    "Accept": "*/*",
    "Accept-Encoding": "gzip, deflate, br",
    "Content-Length": "38",
    "Content-Type": "application/x-www-form-urlencoded",
    "Host": "httpbin.org",
    "User-Agent": "python-requests/2.32.3",
```

[32] https://httpbin.org/

```
    "X-Amzn-Trace-Id": "Root=1-67f134e6-5c7d6b1e4433873d6bd7f5b5"
  },
  "json": null,
  "origin": "93.43.240.159",
  "url": "https://httpbin.org/post"
}
```

Dall'analisi del codice e dell'output osserviamo che:

- ➔ `data` è un dizionario che contiene i dati da inviare.
- ➔ `requests.post()` invia una richiesta POST all'URL specificato con i dati presenti nel dizionario `data`.
- ➔ `response.json()` mostra il corpo della risposta in formato JSON, in cui possiamo vedere i dati che abbiamo inviato e la risposta alla richiesta che non contiene dati in quanto stiamo interrogando un servizio di prova delle API (tale risposta la leggete nelle ultime 3 righe della response) .

7.3.4 Aggiungere intestazioni (headers)

Le intestazioni HTTP sono utili per fornire informazioni aggiuntive con la richiesta.
Si possono aggiungere intestazioni personalizzate usando il parametro `headers`.

Si provi il seguente programma:

```
import requests

headers = {
    'User-Agent': 'my-app/0.0.1',
    'Accept': 'application/json'
}
response = requests.get("https://api.github.com", headers=headers)
print(response.status_code)
print(response.headers)
```

Eseguendo questo programma, otteniamo il seguente output:

```
200
{'Date': 'Sat, 05 Apr 2025 13:55:07 GMT', 'Content-Type':
'application/json; charset=utf-8', 'Cache-Control': 'public,
max-age=60, s-maxage=60', 'Vary': 'Accept,Accept-Encoding, Accept,
X-Requested-With', 'ETag':
'W/"e1173261b2ef116792e39d54ab6d90e5b6e1054a049fd5730b84c7d9cd028aed"',
'X-GitHub-Media-Type': 'github.v3', 'x-github-api-version-selected':
'2022-11-28', 'Access-Control-Expose-Headers': 'ETag, Link, Location,
Retry-After, X-GitHub-OTP, X-RateLimit-Limit, X-RateLimit-Remaining,
```

```
X-RateLimit-Used, X-RateLimit-Resource, X-RateLimit-Reset,
X-OAuth-Scopes, X-Accepted-OAuth-Scopes, X-Poll-Interval,
X-GitHub-Media-Type, X-GitHub-SSO, X-GitHub-Request-Id, Deprecation,
Sunset', 'Access-Control-Allow-Origin': '*',
'Strict-Transport-Security': 'max-age=31536000; includeSubdomains;
preload', 'X-Frame-Options': 'deny', 'X-Content-Type-Options':
'nosniff', 'X-XSS-Protection': '0', 'Referrer-Policy':
'origin-when-cross-origin, strict-origin-when-cross-origin',
'Content-Security-Policy': "default-src 'none'", 'Content-Encoding':
'gzip', 'Server': 'github.com', 'Accept-Ranges': 'bytes',
'X-RateLimit-Limit': '60', 'X-RateLimit-Remaining': '57',
'X-RateLimit-Reset': '1743863091', 'X-RateLimit-Resource': 'core',
'X-RateLimit-Used': '3', 'Content-Length': '510',
'X-GitHub-Request-Id': 'AC56:12B3:4A4CC46:4CED6A6:67F1363B'}
```

Dall'analisi del codice e dell'output osserviamo che:

- ➔ `headers` è un dizionario che contiene le intestazioni HTTP personalizzate.
- ➔ `requests.get()` invia una richiesta GET con le intestazioni specificate.
- ➔ `response.headers` restituisce un dizionario delle intestazioni della risposta.

7.3.5 Gestione delle eccezioni

Quando si lavora con le richieste HTTP è importante gestire correttamente le eccezioni. La libreria `requests` fornisce diversi tipi di eccezioni per gestire vari errori:

Si provi il seguente codice:

```
import requests

try:
    response = requests.get("https://api.github.com/nonexistent")
    response.raise_for_status()
except requests.exceptions.HTTPError as err:
    print(f"HTTP error occurred: {err}")
except requests.exceptions.ConnectionError as err:
    print(f"Connection error occurred: {err}")
except requests.exceptions.Timeout as err:
    print(f"Timeout error occurred: {err}")
except requests.exceptions.RequestException as err:
    print(f"An error occurred: {err}")
```

Eseguendo questo programma, otteniamo il seguente output:

```
HTTP error occurred: 404 Client Error: Not Found for url:
```

```
https://api.github.com/nonexistentHTTP error occurred: 404 Client
Error: Not Found for url: https://api.github.com/nonexistent
```

Dall'analisi del codice e dell'output osserviamo che:

- `response.raise_for_status()` solleva un'eccezione se la richiesta HTTP non è andata a buon fine (ad esempio, se il server restituisce un codice di stato 4xx o 5xx).
- `requests.exceptions.HTTPError` gestisce gli errori HTTP.
- `requests.exceptions.ConnectionError` gestisce i problemi di connessione.
- `requests.exceptions.Timeout` gestisce i timeout.
- `requests.exceptions.RequestException` è una classe base per tutte le eccezioni relative alle richieste.

7.3.6 Timeout

Per evitare che il programma rimanga bloccato in attesa di una risposta, è possibile impostare un timeout:

Si provi il seguente codice:

```
import requests

try:
    response = requests.get("https://api.github.com", timeout=1)
    print(response.status_code)
except requests.exceptions.Timeout:
    print("The request timed out")
```

Eseguendo questo programma, otteniamo il seguente output:

```
200
```

Dall'analisi del codice e dell'output osserviamo che:

- `timeout` è il tempo massimo (in secondi) che la richiesta può impiegare prima di sollevare un'eccezione `Timeout`.
- Se la richiesta non riesce a ottenere una risposta entro il tempo specificato, viene sollevata un'eccezione `Timeout`.

7.3.7 Autenticazione

La libreria `requests` supporta vari schemi di autenticazione, incluso l'autenticazione di base:

```
import requests
from requests.auth import HTTPBasicAuth

response = requests.get("https://httpbin.org",
           auth=HTTPBasicAuth("testuser", "password123"))
print(response.status_code)
print(response.text)
```

Eseguendo questo programma, otteniamo il seguente output (se le credenziali sono corrette):

```
200
<!DOCTYPE html>
<html lang="en">

<head>
    <meta charset="UTF-8">
    <title>httpbin.org</title>
...

</body>

</html>
```

L'output e' stato troncato per comodita'.

Dall'analisi del codice e dell'output osserviamo che:

- ➔ `HTTPBasicAuth` è una classe utilizzata per l'autenticazione di base.
- ➔ `requests.get()` invia una richiesta GET con le credenziali specificate.
- ➔ **ATTENZIONE:** nel codice si importa due volte la libreria `requests` solo per evidenziare la classe `HTTPBasicAuth`

Utilizzate questo codice modificando il sito e le credenziali di accesso per provare a utilizzare `requests` per autenticarvi ad un sito di cui siete in possesso delle credenziali.

Si invita il lettore ad approfondire gli altri schemi di autenticazione che il modulo `requests` supporta, esaminando la documentazione ufficiale.

7.3.8 Gestione delle Sessioni

Utilizzando una sessione, è possibile rendere persistenti i parametri e le intestazioni tra le varie richieste.

Si esamini il seguente codice:

```
import requests

session = requests.Session()
session.headers.update({'User-Agent': 'my-app/0.0.1'})

response = session.get("https://api.github.com")
print(response.status_code)

response = session.get("https://api.github.com/user")
print(response.status_code)
```

Eseguendo questo programma, otteniamo il seguente output:

```
200
401
```

Dall'analisi del codice e dell'output osserviamo che:

- `requests.Session()` crea una sessione che rende persistenti i parametri e le intestazioni tra le richieste.
- `session.headers.update()` aggiorna le intestazioni della sessione.
- Le richieste successive utilizzano le stesse intestazioni della sessione, evitando di doverle specificare ogni volta.

7.3.9 Una semplice applicazione: previsioni del tempo

Mostreremo ora come utilizzare la libreria `requests` per creare una piccola applicazione.

Andremo a creare una semplice webapp che interagisce con un' API di previsioni del tempo per ottenere le previsioni meteo di una città specifica.

7.3.9.1 Registrazione per un'API key

Per utilizzare un servizio di previsioni del tempo, dobbiamo prima ottenere un'API key. Per questo esempio, useremo l'API di *OpenWeatherMap*. Puoi registrarti gratuitamente su *OpenWeatherMap* e ottenere la tua API key.

7.3.9.2 Codice dell'applicazione

Ecco un esempio di come creare un'applicazione che richiede le previsioni meteo per una città specifica:

```
import requests
```

```
def get_weather(city):
    api_key = 'la_tua_api_key'  # Sostituisci con la tua API key
    base_url = 'http://api.openweathermap.org/data/2.5/weather'
    params = {
        'q': city,
        'appid': api_key,
        'units': 'metric'
    }

    response = requests.get(base_url, params=params)

    if response.status_code == 200:
        data = response.json()
        main = data['main']
        weather = data['weather'][0]

        print(f"Previsioni del tempo per {city}:")
        print(f"Temperatura: {main['temp']}°C")
        print(f"Umidità: {main['humidity']}%")
        print(f"Condizioni: {weather['description']}")
    else:
        print("Città non trovata. Verifica il nome della città e
riprova.")

if __name__ == "__main__":
    city = input("Inserisci il nome della città: ")
    get_weather(city)
```

7.3.9.3 Esecuzione del Programma

Quando esegui il programma, ti verrà chiesto di inserire il nome di una città. Il programma invierà una richiesta all'API di *OpenWeatherMap* e visualizzerà le previsioni meteo per la città specificata.
Provando il programma e inserendo Turin,IT o Turin o Torino si otterrà un output simile al seguente:

```
Inserisci il nome della città: Torino
Previsioni del tempo per Torino:
Temperatura: 18.84°C
Umidità: 48%
Condizioni: clear sky
```

ATTENZIONE: alla stesura di questo testo le chiamate alle API gratuite sono 60 al minuto arrivando ad una massimo di un milione al giorno.

7.4 Modulo scrapy

Scrapy è un framework Python per l'estrazione di dati da siti web[33].

Rispetto a strumenti più semplici come `requests`, `scrapy` offre funzionalità avanzate per lo scraping su larga scala, con gestione asincrona delle richieste, parsing HTML integrato e una struttura ben organizzata per progetti complessi.

7.4.1 Installazione

Installiamo Scrapy utilizzando il comando *pip*

```
$ pip3 install scrapy
```

L'installazione non deve necessariamente essere effettuata da utente *root*.

Il comando andrà a installare la libreria e eventuali dipendenze non installate necessarie al pacchetto com, p.e., `parsel`, `twisted`, `pyOpenSSL`, `cryptography` e `zope.interface` per citarne alcuni. Visto che la lista delle dipendenze è cospicua si consiglia di installare scrapy in un virtual-env (cfr. Appendice 1.5)

Per verificare la corretta installazione della libreria eseguire la seguente riga di codice:

```
import scrapy
```

Se si riceve il seguente errore:

```
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
ModuleNotFoundError: No module named 'scrapy'
```

Allora il modulo non è stato installato correttamente.

Il comando di installazione ha installato anche un comando utilizzabile da console che si chiama *scrapy*. Lanciando l'help di questo comando:

```
$ scrapy --help
```

si ottiene il seguente output:

```
Scrapy 2.12.0 - active project: progettoScrapy
```

[33] https://scrapy.org/

```
Usage:
  scrapy <command> [options] [args]

Available commands:
  bench         Run quick benchmark test
  check         Check spider contracts
  crawl         Run a spider
  edit          Edit spider
  fetch         Fetch a URL using the Scrapy downloader
  genspider     Generate new spider using pre-defined templates
  list          List available spiders
  parse         Parse URL (using its spider) and print the
results
  runspider     Run a self-contained spider (without creating a
project)
  settings      Get settings values
  shell         Interactive scraping console
  startproject  Create new project
  version       Print Scrapy version
  view          Open URL in browser, as seen by Scrapy

Use "scrapy <command> -h" to see more info about a command
```

7.4.2 Creare un nuovo progetto `scrapy`

A differenza di `requests` che funziona come una semplice libreria, `scrapy` è un framework che incoraggia una struttura di progetto ben organizzata.

Per cominciare a lavorare con `scrapy` quindi è necessario creare un progetto.
Abbiamo l'intenzione di reperire le informazioni della pagina:
`https://quotes.toscrape.com/`
che ha il seguente aspetto:



Fig.1

Questo è un sito per testare i programmi di webscraping, si trova all'indirizzo: https://toscrape.com/ (scraping sandbox), al suo interno troviamo due siti su cui fare prove. Noi stiamo considerando il secondo di questi. Questo sito è mantenuto dall'azienda *Zyte* che è il principale contributor di `scrapy`.

Iniziamo creando un nuovo progetto::

```
$ scrapy startproject progettoScrapy
```

si ottiene il seguente output:

```
New Scrapy project 'progettoScrapy', using template directory
'/home/massimo/.local/lib/python3.9/site-packages/scrapy/templat
es/project', created in:
    /home/massimo/PythonProjects/progettoScrapy
```

```
You can start your first spider with:
    cd progettoScrapy
    scrapy genspider example example.com
```

Indica che è stato creato un progetto di nome progettoScrapy nel percorso: `/home/massimo/PythonProjects/progettoScrapy`

Sotto tale percorso si ha il seguente filesystem:



Fig. 2

Il file `scrapy.cfg` è il file di configurazione, il cui contenuto è qualcosa di simile al seguente:

```
# Automatically created by: scrapy startproject
#
# For more information about the [deploy] section see:
# https://scrapyd.readthedocs.io/en/latest/deploy.html

[settings]
default = progettoScrapy.settings

[deploy]
#url = http://localhost:6800/
project = progettoScrapy
```

All'interno della cartella `progettoScrapy` saranno contenuti i seguenti file:

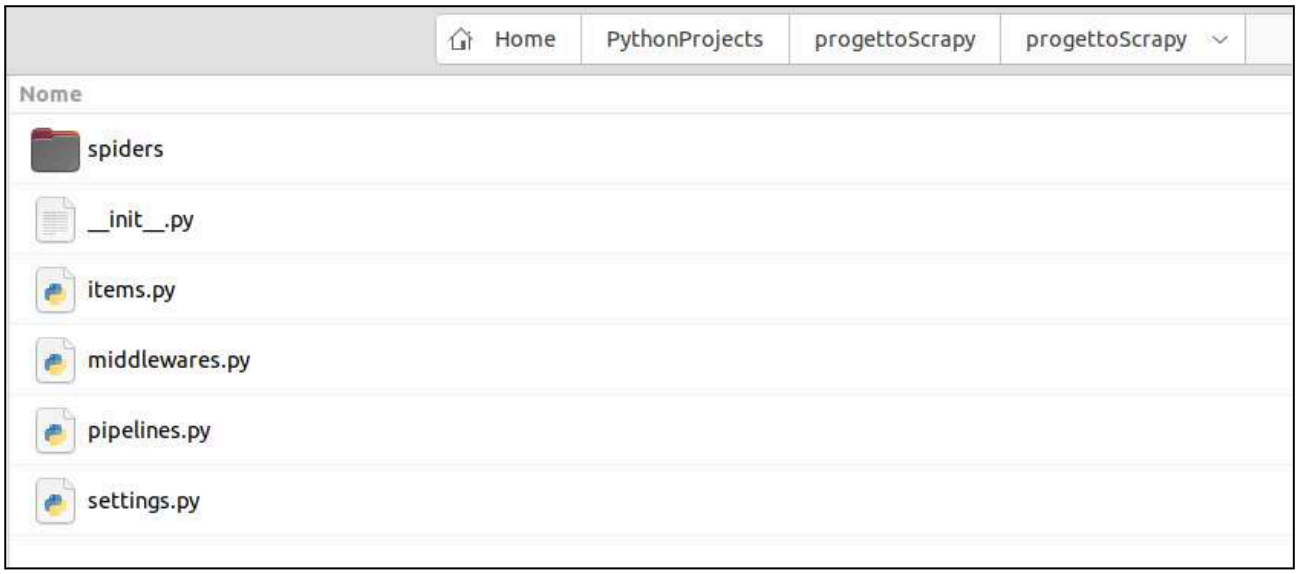


Fig. 3

In breve la struttura creata è la seguente:

```
progettoScrapy/
    scrapy.cfg            # file di configurazione del progetto
    progettoScrapy/       # codice sorgente del progetto
        __init__.py
        items.py          # definizione degli item
        middlewares.py    # middleware personalizzati
        pipelines.py      # pipeline per l'elaborazione degli item
        settings.py       # impostazioni del progetto
        spiders/          # cartella per i nostri spider
            __init__.py
```

La cartella `spiders` conterrà gli *spider* che andranno a "scandagliare" il web per ricercare i termini che rappresentano meglio i contenuti del sito su cui si vuole investigare.

Questa struttura aiuta a mantenere il codice pulito e modulare, specialmente per progetti di scraping complessi.

7.4.3 Creare il primo spider

Gli *spider* sono classi che definiscono come un sito web viene analizzato e come vengono estratte le informazioni.

Creiamo un semplice *spider* per estrarre i titoli e gli autori delle citazioni dal sito *quotes.toscrape.com*.

Ci spostiamo nella directory del progetto:
`/home/massimo/PythonProjects/progettoScrapy/progettoScrapy`
per generare lo spider insieme all'URL del sito Web (Fig.1) di cui si desidera eseguire la

scansione. Visto che la pagina che desideriamo esaminare contiene le:"quotes" allora chiamiamo lo spider quotes.
Quindi lanciamo il seguente comando:

```
$ scrapy genspider quotes toscrape.com
```

si ottiene il seguente output:

```
Created spider 'quotes' using template 'basic' in module:
  progettoScrapy.spiders.quotes
```

Nella cartella spiders sarà presente il file `quotes.py`.

Possiamo creare più spider all'interno dello stesso progetto.

Ora apriamo il file `quotes.py`, otteniamo il seguente listato:

```
import scrapy

class QuotesSpider(scrapy.Spider):
    name = 'quotes'
    allowed_domains = ['toscrape.com']
    start_urls = ['http://toscrape.com/']

    def parse(self, response):
        pass
```

Esaminando il codice notiamo che:

- `QuotesSpider` è sottoclasse di `scrapy.Spider` (infatti abbiamo generato uno spider di nome `quotes` quando abbiamo lanciato il comando `scrapy genspider`)
- `allowed_domains` è il dominio accessibile da questo spider.
- `start_urls` è l'URL da cui verrà avviata la scansione web, oppure in altre parole è l'URL iniziale da cui inizia la scansione web.
- È presente l'override del metodo `parse` che analizza il contenuto della pagina.

Per eseguire la scansione della pagina secondaria del nostro URL (**quotes.**toscrape.com), dobbiamo modificare il links nella proprietà `start_urls`:

```
start_urls = ['http://quotes.toscrape.com']
```

NOTA:
Se si volesse eseguire la scansione di più di una pagina si deve aggiungere un altro collegamento alla lista start_urls p.e. nel seguente modo:

```
start_urls = ['http://quotes.toscrape.com/',
              'http://quotes.toscrape.com/altrapagina']
```

Inoltre è consigliabile far ereditare la classe spider dalla classe `CrawlSpider` invece della classe `scrapy.spider`. Per questo si deve importare il seguente modulo:

```
from scrapy.spiders import CrawlSpider
```

Quindi la nostra classe diventerebbe:

```
class JobsSpider(CrawlSpider):
     ...
     ...
```

7.4.4 Modifichiamo il metodo `parse` dello spider

Ora andiamo a dire allo spider come deve analizzare la pagina web. Per farlo dobbiamo sovrascrivere il metodo `parse` della classe `QuotesSpider`.

A tal scopo esaminiamo il seguente codice:

```
import scrapy

class QuotesSpider(scrapy.Spider):
    name = "quotes"  # nome identificativo per lo spider
    allowed_domains = ['toscrape.com']
    start_urls = [
        'http://quotes.toscrape.com/page/1/',
    ]

    def parse(self, response):
        # Estrae tutte le citazioni dalla pagina
        for quote in response.css('div.quote'):
            yield {
                'text': quote.css('span.text::text').get(),
                'author': quote.css('small.author::text').get(),
                'tags': quote.css('div.tags a.tag::text').getall(),
            }

        # Segue il link alla pagina successiva
        next_page = response.css('li.next a::attr(href)').get()
        if next_page is not None:
            yield response.follow(next_page, callback=self.parse)
```

Dall'analisi del codice osserviamo che:

- ➔ `name` definisce il nome dello spider, che useremo per avviarlo
- ➔ `start_urls` contiene un elenco di URL da cui lo spider inizierà a eseguire lo scraping

- ➔ `parse` è il metodo che scrapy chiama per elaborare la risposta e estrarre dati
- ➔ `response.css()` utilizza selettori CSS per estrarre dati dalla pagina
- ➔ `yield` restituisce i dati estratti
- ➔ `response.follow()` permette di seguire i link ad altre pagine. Questo metodo viene richiamato se è presente un link a un'altra pagina.
- ➔ Si fa notare che la funzione di *callback* `response.follow()` è lo stesso metodo chiamante `self.parse()`, in questo modo andiamo a utilizzare ricorsivamente il metodo `self.parse()`.

ATTENZIONE: l'istruzione `yield`[34] crea un metodo *generatore* . Un *generatore* è una funzione/metodo che restituisce un *iteratore*. Non affronteremo in questo testo tale argomento, si rimanda il lettore ad un approfondimento su altre fonti. Da quanto appena affermato ne deriva che il metodo `self.parse()` è un metodo *generatore*. che restituisce l'*iteratore* delle informazioni ricercate sulla pagina web che si sta analizzando.

7.4.5 Eseguire lo spider

A questo punto possiamo eseguire lo spider.
È necessario spostarsi nella cartella principale del progetto e digitare nella CLI il seguente comando:

```
$ scrapy crawl quotes -o quotes.json
```

si otterrà un output analogo al seguente:

```
2025-04-05 19:37:17 [scrapy.utils.log] INFO: Scrapy 2.12.0 started (bot: progettoScrapy)
2025-04-05 19:37:17 [scrapy.utils.log] INFO: Versions: lxml 5.2.2.0, libxml2 2.12.6,
cssselect 1.1.0, parsel 1.6.0, w3lib 1.22.0, Twisted 22.4.0, Python 3.10.12 (main, Feb  4
2025, 14:57:36) [GCC 11.4.0], pyOpenSSL 25.0.0 (OpenSSL 3.4.1 11 Feb 2025), cryptography
44.0.2, Platform Linux-6.5.0-28-generic-x86_64-with-glibc2.35
2025-04-05 19:37:17 [scrapy.addons] INFO: Enabled addons:
[]
2025-04-05 19:37:17 [asyncio] DEBUG: Using selector: EpollSelector
2025-04-05 19:37:17 [scrapy.utils.log] DEBUG: Using reactor:
twisted.internet.asyncioreactor.AsyncioSelectorReactor
2025-04-05 19:37:17 [scrapy.utils.log] DEBUG: Using asyncio event loop:
asyncio.unix_events._UnixSelectorEventLoop
2025-04-05 19:37:17 [scrapy.utils.log] DEBUG: Using reactor:
twisted.internet.asyncioreactor.AsyncioSelectorReactor
2025-04-05 19:37:17 [scrapy.utils.log] DEBUG: Using asyncio event loop:
asyncio.unix_events._UnixSelectorEventLoop
2025-04-05 19:37:17 [scrapy.extensions.telnet] INFO: Telnet Password: 98bde1dd0724e7ab
2025-04-05 19:37:17 [scrapy.middleware] INFO: Enabled extensions:
['scrapy.extensions.corestats.CoreStats',
 'scrapy.extensions.telnet.TelnetConsole',
 'scrapy.extensions.memusage.MemoryUsage',
 'scrapy.extensions.feedexport.FeedExporter',
 'scrapy.extensions.logstats.LogStats']
2025-04-05 19:37:17 [scrapy.crawler] INFO: Overridden settings:
{'BOT_NAME': 'progettoScrapy',
 'FEED_EXPORT_ENCODING': 'utf-8',
```

[34] https://docs.python.it/html/ref/yield.html

```
 'NEWSPIDER_MODULE': 'progettoScrapy.spiders',
 'ROBOTSTXT_OBEY': True,
 'SPIDER_MODULES': ['progettoScrapy.spiders'],
 'TWISTED_REACTOR': 'twisted.internet.asyncioreactor.AsyncioSelectorReactor'}

...

 'start_time': datetime.datetime(2025, 4, 5, 17, 37, 17, 751089,
tzinfo=datetime.timezone.utc)}
2025-04-05 19:37:21 [scrapy.core.engine] INFO: Spider closed (finished)
```

L'output e' stato troncato per comodita'.

Eseguendo questo comando, otteniamo il file `quotes.json` contenente i seguenti dati estratti dalla pagina `http://quotes.toscrape.com/page/1/`

```
[
    {
         "text": "“The world as we have created it is a process of our
thinking. It cannot be changed without changing our thinking.”",
        "author": "Albert Einstein",
        "tags": [
            "change",
            "deep-thoughts",
            "thinking",
            "world"
        ]
    },
    {
          "text": "“It is our choices, Harry, that show what we truly
are, far more than our abilities.”",
        "author": "J.K. Rowling",
        "tags": [
            "abilities",
            "choices"
        ]
    },
    ...
    ...
    ...

]
```

7.4.6* Selettori CSS e XPath

Per estrarre dati dalle pagine web, `scrapy` supporta sia i selettori CSS che `XPath`.

7.4.6.1 Cosa è XPath?

XPath è un linguaggio, parte della famiglia XML, che permette di individuare i nodi all'interno di un documento XML.

Le espressioni XPath, a differenza delle espressioni XML, non servono a identificare la struttura di un documento, bensì a localizzarne con precisione i nodi.

7.4.6.2 Uso selettori CSS vs Selettori XPath

Nell'esempio precedente abbiamo utilizzato selettori CSS, ma possiamo anche usare XPath:

```
# Esempio con CSS
text = response.css('div.quote span.text::text').get()

# Equivalente con XPath
text =
response.xpath('//div[@class="quote"]/span[@class="text"]/text()')
.get()
```

Entrambi i metodi sono potenti, ma XPath offre maggiore flessibilità per query complesse.

7.4.7 Items e Item Pipeline

Per progetti più strutturati, è consigliabile utilizzare gli Items di scrapy per definire i dati da estrarre.

Scrapy usa i dizionari dell'interprete Python per restituire i dati estratti.

7.4.7.1 Items

A tale scopo scrapy fornisce la classe Item, la quale fornisce oggetti item. Possiamo utilizzare questi oggetti come contenitori per i dati reperiti da scrapy.
Gli Items mettono a disposizione una semplice sintassi per dichiarare i campi.
La sintassi è simile alla seguente:

```
import scrapy
class Quote(scrapy.Item):
     text = scrapy.Field()
     author = scrapy.Field()
     tags = scrapy.Field()
```

L'oggetto Field specifica i metadati per ogni campo.
Potresti notare quando abbiamo creato il progetto, che scrapy ha creato il file items.py nella nostra directory di progetto. Ora modifichiamo questo file per aggiungere i nostri items come segue:

```
# progettoScrapy/items.py
import scrapy

class Quote(scrapy.Item):
    text = scrapy.Field()
    author = scrapy.Field()
    tags = scrapy.Field()
```

E aggiorniamo il nostro spider per utilizzare questo Item:

```
# progettoScrapy/spiders/quotes.py
import scrapy
from progettoScrapy.items import Quote

class QuotesSpider(scrapy.Spider):
    name = "quotes"
    start_urls = [
        'http://quotes.toscrape.com/page/1/',
    ]

    def parse(self, response):
        for quote_div in response.css('div.quote'):
            quote = Quote()
            quote['text'] = quote_div.css('span.text::text').get()
            quote['author'] =
quote_div.css('small.author::text').get()
            quote['tags'] = quote_div.css('div.tags
a.tag::text').getall()
            yield quote

        next_page = response.css('li.next a::attr(href)').get()
        if next_page is not None:
            yield response.follow(next_page, callback=self.parse)
```

Su sfondo grigio sono indicate gli aggiornamenti da apportare al file: `progettoScrapy/spiders/quotes.py`

7.4.7.2 Item Pipeline

Le `Item Pipeline` consentono di elaborare gli `item` estratti.

Ad esempio, possiamo creare una pipeline per pulire i dati o salvarli in un database.

Nel seguente codice andiamo a salvare i dati in un database Sqlite3 (cfr. Cap. 6.2)

```
# progettoScrapy/pipelines.py
import sqlite3
```

```
class SqlitePipeline:
    def __init__(self):
        self.conn = sqlite3.connect('quotes.db')
        self.cur = self.conn.cursor()
        self.cur.execute('''
            CREATE TABLE IF NOT EXISTS quotes
            (text TEXT, author TEXT)
        ''')

    def process_item(self, item, spider):
        self.cur.execute("INSERT INTO quotes VALUES (?, ?)",
                         (item['text'], item['author']))
        self.conn.commit()
        return item

    def close_spider(self, spider):
        self.conn.close()
```

Per attivare questa pipeline, dobbiamo modificare il file `settings.py`:

```
# ProgettoScrapy/settings.py
ITEM_PIPELINES = {
    'progettoScrapy.pipelines.SqlitePipeline': 300,
}
```

Il valore 300 rappresenta la priorità della pipeline. Pipeline con valori più bassi vengono eseguite prima.

7.4.7.3 Esecuzione con le ultime modifiche

Ora abbiamo modificato tutti i files del progetto per poter eseguire lo scraping delle citazioni andando a memorizzarle in un database `Sqlite3`.

Quindi per lanciare lo spider ci spostiamo nella cartella principale del progetto e digitiamo nella CLI il seguente comando:

```
$ scrapy crawl quotes
```

si otterrà un output analogo al seguente:

```
2025-04-12 17:03:22 [scrapy.utils.log] INFO: Scrapy 2.12.0 started (bot: progettoScrapy)
2025-04-12 17:03:22 [scrapy.utils.log] INFO: Versions: lxml 5.2.2.0, libxml2 2.12.6,
cssselect 1.1.0, parsel 1.6.0, w3lib 1.22.0, Twisted 22.4.0, Python 3.10.12 (main, Feb  4
2025, 14:57:36) [GCC 11.4.0], pyOpenSSL 25.0.0 (OpenSSL 3.4.1 11 Feb 2025), cryptography
44.0.2, Platform Linux-6.5.0-28-generic-x86_64-with-glibc2.35
2025-04-12 17:03:22 [scrapy.addons] INFO: Enabled addons:
[]
2025-04-12 17:03:22 [asyncio] DEBUG: Using selector: EpollSelector
```

```
2025-04-12 17:03:22 [scrapy.utils.log] DEBUG: Using reactor:
twisted.internet.asyncioreactor.AsyncioSelectorReactor
2025-04-12 17:03:22 [scrapy.utils.log] DEBUG: Using asyncio event loop:
asyncio.unix_events._UnixSelectorEventLoop
2025-04-12 17:03:22 [scrapy.utils.log] DEBUG: Using reactor:
twisted.internet.asyncioreactor.AsyncioSelectorReactor
2025-04-12 17:03:22 [scrapy.utils.log] DEBUG: Using asyncio event loop:
asyncio.unix_events._UnixSelectorEventLoop
2025-04-12 17:03:22 [scrapy.extensions.telnet] INFO: Telnet Password: a542e5364bf753bc
2025-04-12 17:03:22 [scrapy.middleware] INFO: Enabled extensions:
['scrapy.extensions.corestats.CoreStats',
 'scrapy.extensions.telnet.TelnetConsole',
 'scrapy.extensions.memusage.MemoryUsage',
 'scrapy.extensions.logstats.LogStats']
2025-04-12 17:03:22 [scrapy.crawler] INFO: Overridden settings:
{'BOT_NAME': 'progettoScrapy',


...

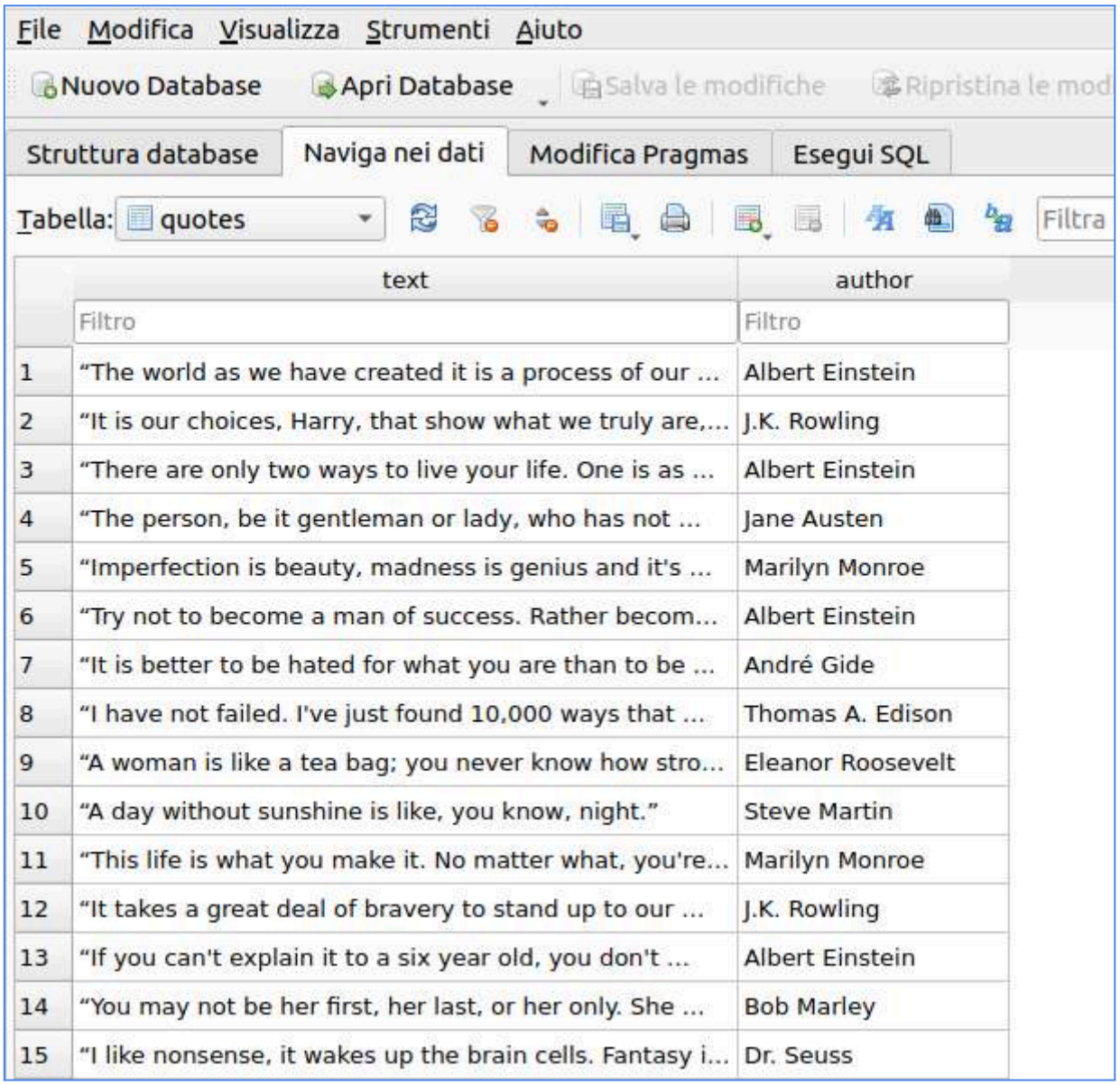
 'start_time': datetime.datetime(2025, 4, 5, 17, 37, 17, 751089,
tzinfo=datetime.timezone.utc)}
2025-04-05 19:37:21 [scrapy.core.engine] INFO: Spider closed (finished)
```

L'output e' stato troncato per comodita'.

Nella cartella principale del progetto, quella da cui è stato lanciato lo spider, abbiamo un nuovo file: `quotes.db`.

Questo file contiene il database `Sqlite3` che abbiamo creato. Il DB ha un'unica tabella, di nome `quotes`, che contiene i campi: citazione e autore che sono stati prelevati dal sito di prova.

In Fig.1 si puo' esaminare uno screenshot del contenuto delle prime 15 righe della tabella `quotes`.



	text	author
	Filtro	Filtro
1	“The world as we have created it is a process of our ...	Albert Einstein
2	“It is our choices, Harry, that show what we truly are,...	J.K. Rowling
3	“There are only two ways to live your life. One is as ...	Albert Einstein
4	“The person, be it gentleman or lady, who has not ...	Jane Austen
5	“Imperfection is beauty, madness is genius and it's ...	Marilyn Monroe
6	“Try not to become a man of success. Rather becom...	Albert Einstein
7	“It is better to be hated for what you are than to be ...	André Gide
8	“I have not failed. I've just found 10,000 ways that ...	Thomas A. Edison
9	“A woman is like a tea bag; you never know how stro...	Eleanor Roosevelt
10	“A day without sunshine is like, you know, night.”	Steve Martin
11	“This life is what you make it. No matter what, you're...	Marilyn Monroe
12	“It takes a great deal of bravery to stand up to our ...	J.K. Rowling
13	“If you can't explain it to a six year old, you don't ...	Albert Einstein
14	“You may not be her first, her last, or her only. She ...	Bob Marley
15	“I like nonsense, it wakes up the brain cells. Fantasy i...	Dr. Seuss

Figura 1

7.4.8* Gestione Avanzata delle Richieste

`scrapy` offre molte opzioni per gestire le richieste HTTP in modo avanzato:

```
# Richiesta GET con parametri
yield scrapy.Request(
    url='https://example.com/search',
    callback=self.parse_results,
    cb_kwargs={'category': 'electronics'},
    meta={'page': 1},
    headers={'User-Agent': 'Mozilla/5.0'},
    cookies={'session_id': '12345'},
)
```

```
# Richiesta POST
yield scrapy.FormRequest(
    url='https://example.com/login',
    formdata={'username': 'user', 'password': 'pass'},
    callback=self.parse_login
)
```

Riportiamo una rapida descrizione dei parametri meno comprensibili:

- ➔ `meta` è un'oggetto particolarmente utile per passare dati tra callback diversi,
- ➔ `cb_kwargs` permette di passare parametri aggiuntivi alla funzione di callback.

Gli altri parametri sono facilmente comprensibili.

7.4.9* Gestione delle Sessioni e Autenticazione

`scrapy` gestisce automaticamente i cookie tra le richieste. Ecco un esempio di login:

```
class LoginSpider(scrapy.Spider):
    name = 'login_example'
    start_urls = ['https://example.com/login']

    def parse(self, response):
        # Estrae il token CSRF, se presente
        token =
response.css('input[name="csrf_token"]::attr(value)').get()

        return scrapy.FormRequest.from_response(
            response,
            formdata={
                'username': 'myusername',
                'password': 'mypassword',
                'csrf_token': token  # include il token CSRF se
necessario
            },
            callback=self.after_login
        )

    def after_login(self, response):
        # Verifica se il login è avvenuto con successo
        if 'Benvenuto' in response.text:
            self.logger.info("Login completato con successo!")
            # Continua lo scraping delle pagine protette
            yield scrapy.Request('https://example.com/area_protetta',
callback=self.parse_protected)
        else:
            self.logger.error("Login fallito")
```

```
    def parse_protected(self, response):
        # Estrai dati dalle pagine protette
        yield {
            'title': response.css('h1::text').get(),
            'content': response.css('div.content::text').get()
        }
```

Un veloce commento:

- ➔ Il token CSRF è un valore casuale e univoco che viene utilizzato per proteggere le applicazioni web dagli attacchi CSRF. Il *Cross-Site Request Forgery* (CSRF) è un attacco web invisibile che sfrutta le sessioni autenticate per eseguire azioni non autorizzate[35].
- ➔ In questo esempio, `FormRequest.from_response()` mantiene automaticamente i cookie e altri dati necessari per la sessione.

7.4.10 Rispettare robots.txt e gestire la velocità di crawling

`scrapy` è configurato per rispettare le direttive `robots.txt` per default. Questo comportamento può essere modificato nel file `settings.py`:

```
# Rispettare robots.txt
ROBOTSTXT_OBEY = True

# Impostare un ritardo tra le richieste per non sovraccaricare il
server
DOWNLOAD_DELAY = 1  # attesa di 1 secondo tra le richieste

# Impostare il numero massimo di richieste simultanee
CONCURRENT_REQUESTS = 16

# Limitare le richieste per dominio
CONCURRENT_REQUESTS_PER_DOMAIN = 8
```

Queste impostazioni sono fondamentali per uno *scraping etico* che non sovraccarichi i server target.

7.4.11 Una semplice applicazione: Analisi dei prezzi dei prodotti

Come esempio pratico, creiamo un'applicazione che monitora i prezzi dei prodotti su un sito di e-commerce fittizio, presente all'indirizzo: `https://fakeshop.example.com`

7.4.11.1 Definizione degli Item

```
# tutorial/items.py
import scrapy
```

35 https://www.ictsecuritymagazine.com/articoli/limportanza-di-effettuare-il-logout-cose-il-csrf-e-come-proteggersi/

```
class Product(scrapy.Item):
    name = scrapy.Field()
    price = scrapy.Field()
    currency = scrapy.Field()
    url = scrapy.Field()
    date = scrapy.Field()
```

7.4.11.2 Creazione dello Spider

```
# tutorial/spiders/price_monitor.py
import scrapy
from datetime import datetime
from tutorial.items import Product

class PriceMonitorSpider(scrapy.Spider):
    name = "price_monitor"
    allowed_domains = ["fakeshop.example.com"]
    start_urls = ["https://fakeshop.example.com/products/"]

    def parse(self, response):
        # Estrae i link a tutti i prodotti
        product_links = response.css('div.product
a::attr(href)').getall()
        for link in product_links:
            yield response.follow(link, callback=self.parse_product)

        # Segue il link alla pagina successiva
        next_page = response.css('a.next-page::attr(href)').get()
        if next_page:
            yield response.follow(next_page, callback=self.parse)

    def parse_product(self, response):
        product = Product()
        product['name'] =
response.css('h1.product-title::text').get().strip()

        # Estrae il prezzo e la valuta
        price_text = response.css('span.price::text').get().strip()
        if '€' in price_text:
            product['currency'] = '€'
            product['price'] = float(price_text.replace('€',
'').strip().replace(',', '.'))
        elif '$' in price_text:
            product['currency'] = '$'
            product['price'] = float(price_text.replace('$',
'').strip())

        product['url'] = response.url
        product['date'] = datetime.now().strftime('%Y-%m-%d')
```

```
        yield product
```

7.4.11.3 Pipeline per Salvare i Dati

```
# tutorial/pipelines.py
import json
from datetime import datetime

class PriceHistoryPipeline:
    def open_spider(self, spider):
        self.file = open('price_history.json', 'w')
        self.file.write('[\n')
        self.first_item = True

        # Carica la cronologia precedente, se esiste
        try:
            with open('price_history_archive.json', 'r') as f:
                self.history = json.load(f)
        except FileNotFoundError:
            self.history = {}

    def process_item(self, item, spider):
        # Formatta l'item come JSON
        line = json.dumps(dict(item))

        # Aggiungi una virgola tra gli item, ma non per il primo
        if self.first_item:
            self.first_item = False
        else:
            self.file.write(',\n')

        self.file.write(line)

        # Aggiorna la cronologia dei prezzi
        product_url = item['url']
        if product_url not in self.history:
            self.history[product_url] = []

        self.history[product_url].append({
            'date': item['date'],
            'price': item['price'],
            'currency': item['currency']
        })

        return item

    def close_spider(self, spider):
        self.file.write('\n]')
        self.file.close()

        # Salva la cronologia aggiornata
```

```
        with open('price_history_archive.json', 'w') as f:
            json.dump(self.history, f)

        # Genera un report
        self.generate_report()

    def generate_report(self):
        report = "REPORT DI MONITORAGGIO PREZZI\n"
        report += "Data: " + datetime.now().strftime('%Y-%m-%d
%H:%M:%S') + "\n\n"

        for url, prices in self.history.items():
            if len(prices) < 2:
                continue  # Salta i prodotti con un solo rilevamento

            latest = prices[-1]
            previous = prices[-2]

            product_name = url.split('/')[-1]  # Versione semplificata
per l'esempio
            report += f"Prodotto: {product_name}\n"
            report += f"Prezzo attuale: {latest['price']}
{latest['currency']}\n"

            price_diff = latest['price'] - previous['price']
            if price_diff > 0:
                report += f"Variazione: +{price_diff}
{latest['currency']} (aumento)\n"
            elif price_diff < 0:
                report += f"Variazione: {price_diff}
{latest['currency']} (diminuzione)\n"
            else:
                report += "Variazione: nessuna\n"

            report += "\n"

        with open('price_report.txt', 'w') as f:
            f.write(report)
```

7.4.11.4 Configurazione del Progetto

```
# tutorial/settings.py
ITEM_PIPELINES = {
    'tutorial.pipelines.PriceHistoryPipeline': 300,
}

# Rispetta i limiti del server
DOWNLOAD_DELAY = 2
ROBOTSTXT_OBEY = True
```

7.4.11.5 Esecuzione del Monitoraggio

Per eseguire il monitoraggio, utilizziamo il comando:

```
scrapy crawl price_monitor
```

Questo spider analizza il sito web, estrae i prezzi dei prodotti e genera un report che mostra come i prezzi sono cambiati nel tempo.

Il report viene salvato in un file `price_report.txt`, mentre la cronologia completa viene mantenuta in `price_history_archive.json`.

7.4.11 Vantaggi rispetto alla libreria `requests`

`Scrapy` è un framework che fornisce potenza e versatilità per lo scraping di dati dal web. Rispetto a soluzioni più semplici come `requests`, offre numerosi vantaggi:

- ➔ Architettura asincrona che consente richieste parallele, rendendo lo scraping molto più veloce
- ➔ Struttura organizzata per progetti complessi
- ➔ Gestione integrata dei *cookie* e delle *sessioni*
- ➔ Pipeline per l'elaborazione automatica dei dati estratti
- ➔ Supporto per selettori CSS e XPath
- ➔ Gestione della concorrenza e dei ritardi per il crawling
- ➔ Estensibilità attraverso middleware e pipeline personalizzati

Tuttavia, per semplici progetti, potrebbe risultare troppo laborioso rispetto alla libreria `requests`. La scelta dipende dalla complessità del progetto e dalle performance richieste.

8. Agenti AI: costruire sistemi intelligenti con Python e LLM

8.1 Che cosa è un agente AI?

Negli ultimi anni il termine agente è entrato prepotentemente nel vocabolario della programmazione moderna. Ma cosa significa, concretamente, quando parliamo di agente AI in Python?

Un agente AI è un programma che percepisce un input, ragiona su di esso e produce un'azione o una risposta in modo autonomo. Ciò che lo distingue da una semplice funzione è la capacità di prendere decisioni: un agente può scegliere quale strumento usare, in quale ordine eseguire operazioni, se chiedere ulteriori informazioni o rispondere direttamente.

Il motore di ragionamento di un agente moderno è tipicamente un Large Language Model (LLM): un modello linguistico pre-addestrato su enormi quantità di testo, capace di comprendere istruzioni in linguaggio naturale e di generare risposte coerenti e contestualizzate.

8.2 Il mattone fondamentale: chiamare un LLM

Prima di costruire sistemi complessi, è necessario capire come si comunica con un LLM via API. Il formato adottato da Groq — lo stesso di OpenAI, diventato de facto lo standard del settore — si basa su una lista di messaggi, ciascuno con un ruolo.

Esistono tre ruoli principali:

system: definisce la persona e le istruzioni generali dell'agente (chi è, cosa deve fare, in che lingua rispondere)

user: contiene il messaggio dell'utente, ovvero il compito da svolgere

assistant: contiene la risposta generata dall'LLM; viene incluso nella storia della conversazione nei dialoghi multi-turno

8.2.1 Perché Groq: confronto tra i provider gratuiti

Per gli esempi di questo capitolo abbiamo scelto Groq come provider LLM. Non è l'unica opzione gratuita disponibile: esistono anche Google Gemini e Ollama, ciascuno con caratteristiche diverse. Vediamo perché Groq è la scelta più adatta a un contesto didattico.

Groq (https://console.groq.com) è una piattaforma cloud che offre accesso a modelli open-source di alto livello (tra cui Llama 3 di Meta e Mixtral di Mistral AI) tramite un'API compatibile con il formato OpenAI. I vantaggi principali per chi inizia sono:

Registrazione immediata con account Google o GitHub, senza inserire dati di pagamento

Free tier permanente e generoso: circa 14.400 richieste al giorno e 6.000 token al minuto sul modello llama-3.3-70b-versatile — più che sufficienti per un corso

Dashboard dei consumi chiara e in tempo reale: si vede immediatamente quanti token sono stati utilizzati, il limite giornaliero residuo e lo storico delle chiamate API

Velocità di inferenza straordinaria: Groq utilizza hardware proprietario (LPU - Language

Processing Unit) che rende le risposte sensibilmente più rapide rispetto ad altri provider, rendendo l'esperienza interattiva più fluida durante le lezioni
Formato API identico a OpenAI: imparare a usare Groq significa imparare lo standard de facto del settore, valido anche per Azure OpenAI, Together AI, Ollama e molti altri
Setup puramente cloud: nessuna installazione locale, funziona su qualsiasi computer con una connessione internet, ideale per laboratori didattici con macchine eterogenee

Vediamo ora un confronto strutturato tra le tre principali opzioni gratuite:

Caratteristica	Groq	Google Gemini	Ollama
Registrazione	Account Google/GitHub, no carta	Account Google, no carta	Nessuna, installazione locale
Free tier	~14.400 req/giorno	1.500 req/giorno	Illimitato (locale)
Velocità risposta	Molto veloce (LPU)	Veloce	Dipende dall'hardware
Dashboard consumi	Chiara e real-time	Disponibile	Locale, no dashboard
Setup	Solo API key	Solo API key	Installazione app + download modello
Funziona offline	Richiede internet	Richiede internet	Completamente offline
Privacy dati	Dati elaborati su cloud	Dati elaborati su cloud	Tutto in locale
Formato API	OpenAI compatibile	Proprietario Google	OpenAI-compatibile
Libreria Python	groq	google-generativeai	ollama

La scelta del provider non è definitiva e non è esclusiva. Una volta compreso il pattern degli agenti con Groq, passare a Gemini o Ollama richiede di modificare soltanto la funzione `chiama_llm`, lasciando invariata tutta la logica di orchestrazione. Questo è un concetto particolarmente importante: la separazione tra logica di business e infrastruttura.

8.2.2 La funzione `chiama_llm`: tre implementazioni a confronto

La funzione `chiama_llm` è il punto di contatto tra il codice Python e il modello LLM. Le tre implementazioni che seguono sono funzionalmente identiche — stessa firma, stesso comportamento — ma si rivolgono a provider diversi. Confrontarle è un ottimo esercizio per comprendere come le API differiscano nei dettagli pur condividendo la stessa struttura

concettuale.

Implementazione con Groq (quella usata in tutto il capitolo):

```
# pip install groq
import os
from groq import Groq

client = Groq(api_key=os.environ.get("GROQ_API_KEY"))
MODELLO = "llama-3.3-70b-versatile"

def chiama_llm(system_prompt: str, user_message: str) -> str:
    risposta = client.chat.completions.create(
        model=MODELLO,
        max_tokens=1024,
        messages=[
            {"role": "system", "content": system_prompt},
            {"role": "user",   "content": user_message}
        ]
    )
    # Formato OpenAI: choices[0].message.content
    return risposta.choices[0].message.content
```

Implementazione con Google Gemini:

```
# pip install google-generativeai
import os
import google.generativeai as genai

genai.configure(api_key=os.environ.get("GEMINI_API_KEY"))
# API key gratuita su https://aistudio.google.com → 'Get API key'
MODELLO = "gemini-2.0-flash"

def chiama_llm(system_prompt: str, user_message: str) -> str:
      # In Gemini il system_prompt si passa al costruttore del
modello
    modello = genai.GenerativeModel(
        model_name=MODELLO,
        system_instruction=system_prompt   # ← differenza chiave
    )
    risposta = modello.generate_content(user_message)
    # Accesso diretto: risposta.text  ← il più semplice dei tre
    return risposta.text
```

Implementazione con Ollama (LLM completamente locale, nessuna API key):

```
# pip install ollama
# Prima scaricare il modello: ollama pull llama3.1
```

```
import ollama

MODELLO = "llama3.1"    # oppure: mistral, phi3, gemma2

def chiama_llm(system_prompt: str, user_message: str) -> str:
    # Stesso formato messaggi di Groq/OpenAI
    risposta = ollama.chat(
        model=MODELLO,
        messages=[
            {"role": "system", "content": system_prompt},
            {"role": "user",   "content": user_message}
        ]
    )
    # Il modello gira localmente: i dati non lasciano il computer
    return risposta["message"]["content"]
```

Dall'analisi delle tre implementazioni osserviamo che:

- La firma della funzione è identica in tutti e tre i casi: `chiama_llm(system_prompt, user_message) -> str`. Questo è il principio di intercambiabilità: tutta la logica degli agenti nei paragrafi successivi funziona con qualsiasi delle tre implementazioni
- Groq e Ollama usano lo stesso formato di messaggi OpenAI (lista di dizionari con role e content); Gemini invece accetta il `system_prompt` come parametro separato nel costruttore del modello
- L'accesso alla risposta varia: `risposta.choices[0].message.content` per Groq (standard OpenAI), `risposta.text` per Gemini (il più conciso), `risposta["message"]["content"]` per Ollama
- Ollama non richiede alcuna variabile d'ambiente o chiave: il server LLM gira in locale sulla porta 11434 e viene raggiunto dalla libreria automaticamente

ATTENZIONE: Per sostituire Groq con Gemini o Ollama in qualsiasi esempio del capitolo, è sufficiente incollare la funzione `chiama_llm` corrispondente all'inizio del file, adattare l'import e impostare la variabile `MODELLO`. Nessun'altra riga di codice va modificata.

8.3 Agenti in sequenza: il pattern Pipeline

Il modo più semplice di far cooperare più agenti è organizzarli in una pipeline sequenziale: il primo agente elabora l'input e produce un output, il secondo riceve quell'output come input e produce il proprio, e così via. Ogni agente è specializzato in un compito preciso.

Nel nostro esempio, tre agenti collaborano per scrivere e migliorare un articolo su un argomento dato:

- Agente Ricercatore: identifica i punti chiave sull'argomento
- Agente Scrittore: usa i punti chiave per scrivere una bozza
- Agente Revisore: migliora la bozza e produce la versione finale

8.3.1 Definizione degli agenti

Ogni agente è implementato come una funzione Python. La differenza tra un agente e l'altro risiede esclusivamente nel system_prompt: cambiando le istruzioni, cambia la personalità e il compito dell'agente.

Esaminiamo il seguente codice:

```
# pip install google-generativeai
import os
import google.generativeai as genai

genai.configure(api_key=os.environ.get("GEMINI_API_KEY"))
# API key gratuita su https://aistudio.google.com → 'Get API key'
MODELLO = "gemini-2.0-flash"

def chiama_llm(system_prompt: str, user_message: str) -> str:
    # In Gemini il system_prompt si passa al costruttore del modello
    modello = genai.GenerativeModel(
        model_name=MODELLO,
        system_instruction=system_prompt   # ← differenza chiave
    )
    risposta = modello.generate_content(user_message)
    # Accesso diretto: risposta.text  ← il più semplice dei tre
    return risposta.text

def agente_ricercatore(argomento: str) -> str:
    print("\n [AGENTE 1 - RICERCATORE] sta lavorando...")
    system = (
        "Sei un ricercatore esperto. Il tuo compito è identificare "
        "i punti chiave, i fatti più importanti e gli aspetti
interessanti "
        "su un argomento dato. Rispondi SEMPRE in italiano. "
        "Fornisci un elenco puntato conciso di 5-7 punti."
    )
    task = f"Identifica i punti chiave sull'argomento: '{argomento}'"
    risultato = chiama_llm(system, task)
    print(risultato)
    return risultato


def agente_scrittore(argomento: str, punti_chiave: str) -> str:
    print("\n [AGENTE 2 - SCRITTORE] sta lavorando...")
    system = (
        "Sei un giornalista scientifico che scrive articoli chiari e
coinvolgenti. "
        "Usa un linguaggio accessibile ma preciso. Rispondi SEMPRE in
italiano. "
        "Scrivi articoli di circa 200 parole con introduzione, corpo e
conclusione."
```

```
    )
    task = (
        f"Scrivi un articolo sull'argomento: '{argomento}'.\n\n"
        f"Usa questi punti chiave forniti dal
ricercatore:\n{punti_chiave}"
    )
    risultato = chiama_llm(system, task)
    print(risultato)
    return risultato


def agente_revisore(articolo_bozza: str) -> str:
    print("\n [AGENTE 3 - REVISORE] sta lavorando...")
    system = (
        "Sei un editor esperto. Il tuo lavoro è migliorare testi già
scritti: "
        "correggi errori, migliora la fluidità, rendi il testo più
coinvolgente. "
        "Rispondi SEMPRE in italiano. "
        "Fornisci: 1) il testo migliorato, 2) una breve nota con le
modifiche fatte."
    )
    task = (
        f"Revisiona e migliora questo
articolo:\n\n{articolo_bozza}\n\n"
        "Fornisci prima il testo finale migliorato, poi una sezione "
        "'NOTE DI REVISIONE:' con le modifiche apportate."
    )
    risultato = chiama_llm(system, task)
    print(risultato)
    return risultato
```

Dall'analisi del codice osserviamo che:

- ➔ La prima parte richiama le librerie per la gestione della chiamata al large language model che abbiamo già discusso nei capitoli precedenti.
- ➔ Ogni agente ha un *system_prompt* dedicato che ne definisce la specializzazione
- ➔ `agente_scrittore` riceve due parametri: l'argomento originale e i punti_chiave prodotti dall'agente precedente, è questo il meccanismo con cui le informazioni fluiscono da un agente all'altro
- ➔ Il testo prodotto da ciascun agente viene sia stampato a schermo (per monitorare l'avanzamento) che restituito come valore di ritorno

8.3.2 L'orchestratore

L'orchestratore è la funzione che coordina gli agenti, passando l'output di ciascuno come input al successivo.
Diamo un'occhiata al seguente codice:

```
def pipeline_multi_agente(argomento: str) -> dict:
```

```
    print("=" * 60)
    print(f"  AVVIO PIPELINE MULTI-AGENTE  [modello: {MODELLO}]")
    print(f"  Argomento: {argomento}")
    print("=" * 60)

    punti_chiave   = agente_ricercatore(argomento)
    bozza          = agente_scrittore(argomento, punti_chiave)
    versione_finale = agente_revisore(bozza)

    return {
        "argomento":       argomento,
        "punti_chiave":    punti_chiave,
        "bozza":           bozza,
        "versione_finale": versione_finale,
    }

if __name__ == "__main__":
    ARGOMENTO = "L'intelligenza artificiale nella medicina moderna"
    risultati = pipeline_multi_agente(ARGOMENTO)
```

Dall'analisi del codice osserviamo che:

- ➔ La pipeline segue un flusso lineare: `Agente1 -> output1 -> Agente2(output1) -> output2 -> Agente3(output2)`
- ➔ L'orchestratore restituisce un dizionario con tutti gli output intermedi, utile per il debug e l'analisi dei passaggi
- ➔ Cambiare la variabile `ARGOMENTO` è sufficiente per applicare l'intera pipeline a qualsiasi argomento

NOTA: Il pattern pipeline è il più semplice da implementare e da comprendere. È adatto quando ogni passo dipende necessariamente dal precedente e non è possibile parallelizzare il lavoro. Man mano che il numero di agenti cresce, il tempo di esecuzione aumenta linearmente: se ogni agente impiega T secondi, il totale sarà N×T.

8.3.3 Il codice completo

Ecco il codice completo recante anche tutta la parte di autodocumentazione del codice stesso:

```
"""
============================================================
  ESEMPIO: SISTEMA MULTI-AGENTE CON LLM GRATUITO (GROQ)
============================================================

SCENARIO:
  Un utente vuole che venga scritto e migliorato un breve articolo.
  Tre agenti cooperano in sequenza (pipeline):

    1. AGENTE RICERCATORE  → raccoglie i punti chiave sull'argomento
    2. AGENTE SCRITTORE    → scrive una bozza dell'articolo
```

```
    3. AGENTE REVISORE     → critica e migliora il testo finale

LLM GRATUITO USATO: GROQ
  https://console.groq.com  ← registrazione GRATUITA, no carta di credito
  - Free tier generoso: ~14.400 richieste/giorno su Llama 3
  - Velocissimo grazie all'hardware LPU dedicato di Groq
  - Compatibile con la libreria OpenAI (stesso formato API!)

MODELLI GRATUITI DISPONIBILI SU GROQ (maggio 2025):
  "llama-3.3-70b-versatile"   ← potente, consigliato
  "llama-3.1-8b-instant"      ← più veloce e leggero
  "mixtral-8x7b-32768"        ← contesto lungo (32k token)
  "gemma2-9b-it"              ← Google Gemma 2

INSTALLAZIONE:
  pip install groq

CONFIGURAZIONE:
  1. Vai su https://console.groq.com e crea un account gratuito
  2. Genera una API key dalla sezione "API Keys"
  3. Imposta la variabile d'ambiente:
       Windows:   set GROQ_API_KEY=gsk_...
       Mac/Linux: export GROQ_API_KEY=gsk_...

CONFRONTO CON VERSIONE ANTHROPIC:
  Le uniche differenze sono:
    - pip install groq          (invece di anthropic)
    - import groq               (invece di anthropic)
    - groq.Groq()               (invece di anthropic.Anthropic())
    - risposta.choices[0].message.content  (formato OpenAI)
  La logica degli agenti è IDENTICA! → stesso pattern, LLM intercambiabile
============================================================
"""

import os
from groq import Groq

# ------------------------------------------------------------
# CONFIGURAZIONE
# ------------------------------------------------------------
# Il client legge automaticamente GROQ_API_KEY dall'ambiente
# client = Groq(api_key=os.environ.get("GROQ_API_KEY"))

# Prova a cambiare modello per vedere le differenze di stile/velocità!
MODELLO = "llama-3.3-70b-versatile"   # potente e completamente gratuito


# ------------------------------------------------------------
# FUNZIONE BASE: chiama l'LLM con un prompt
# ------------------------------------------------------------
def chiama_llm(system_prompt: str, user_message: str) -> str:
    """
    Invia un messaggio all'LLM e restituisce la risposta testuale.

    NOTA DIDATTICA: Groq usa il formato API OpenAI (chat completions).
    È lo stesso formato usato da OpenAI, Azure OpenAI, Ollama, ecc.
    Args:
```

```
        system_prompt:  ruolo/persona dell'agente (es. "Sei un esperto...")
        user_message:   il compito specifico da svolgere

    Returns:
        La risposta dell'LLM come stringa
    """
    risposta = client.chat.completions.create(
        model=MODELLO,
        max_tokens=1024,
        messages=[
            {"role": "system", "content": system_prompt},
            {"role": "user",   "content": user_message}
        ]
    )
    # Formato risposta OpenAI: choices[0].message.content
    return risposta.choices[0].message.content


# ------------------------------------------------------------
# AGENTE 1 - RICERCATORE
# ------------------------------------------------------------
def agente_ricercatore(argomento: str) -> str:
    """
    Primo agente: dato l'argomento, genera i punti chiave da trattare.
    Output: una lista strutturata di spunti/fatti.
    """
    print("\n [AGENTE 1 - RICERCATORE] sta lavorando...")

    system = (
        "Sei un ricercatore esperto. Il tuo compito è identificare "
        "i punti chiave, i fatti più importanti e gli aspetti interessanti "
        "su un argomento dato. Rispondi SEMPRE in italiano. "
        "Fornisci un elenco puntato conciso di 5-7 punti."
    )

    task = f"Identifica i punti chiave sull'argomento: '{argomento}'"

    risultato = chiama_llm(system, task)
    print(risultato)
    return risultato


# ------------------------------------------------------------
# AGENTE 2 - SCRITTORE
# ------------------------------------------------------------
def agente_scrittore(argomento: str, punti_chiave: str) -> str:
    """
    Secondo agente: usa i punti del ricercatore per scrivere una bozza.
    Riceve l'output dell'Agente 1 come contesto.
    Output: un articolo di ~200 parole.
    """
    print("\n  [AGENTE 2 - SCRITTORE] sta lavorando...")

    system = (
        "Sei un giornalista scientifico che scrive articoli chiari e coinvolgenti.
"
        "Usa un linguaggio accessibile ma preciso. Rispondi SEMPRE in italiano. "
```

```
        "Scrivi articoli di circa 200 parole con introduzione, corpo e
conclusione."
    )

    task = (
        f"Scrivi un articolo sull'argomento: '{argomento}'.\n\n"
        f"Usa questi punti chiave forniti dal ricercatore:\n{punti_chiave}"
    )

    risultato = chiama_llm(system, task)
    print(risultato)
    return risultato


# ------------------------------------------------------------
# AGENTE 3 - REVISORE
# ------------------------------------------------------------
def agente_revisore(articolo_bozza: str) -> str:
    """
    Terzo agente: revisiona la bozza, suggerisce miglioramenti
    e produce la versione finale corretta.
    Riceve l'output dell'Agente 2 come input.
    Output: versione finale migliorata + breve nota di revisione.
    """
    print("\n [AGENTE 3 - REVISORE] sta lavorando...")

    system = (
        "Sei un editor esperto. Il tuo lavoro è migliorare testi già scritti: "
        "correggi errori, migliora la fluidità, rendi il testo più coinvolgente. "
        "Rispondi SEMPRE in italiano. "
        "Fornisci: 1) il testo migliorato, 2) una breve nota con le modifiche
fatte."
    )

    task = (
        f"Revisiona e migliora questo articolo:\n\n{articolo_bozza}\n\n"
        "Fornisci prima il testo finale migliorato, poi una sezione "
        "'NOTE DI REVISIONE:' con le modifiche apportate."
    )

    risultato = chiama_llm(system, task)
    print(risultato)
    return risultato


# ------------------------------------------------------------
# PIPELINE PRINCIPALE: coordina i 3 agenti
# ------------------------------------------------------------
def pipeline_multi_agente(argomento: str) -> dict:
    """
    Orchestratore: lancia gli agenti in sequenza e passa
    l'output di ciascuno come input al successivo.

    Pattern: PIPELINE (catena sequenziale)
    Agente1 → output1 → Agente2(output1) → output2 → Agente3(output2)
    """
    print("=" * 60)
```

```
    print(f"  AVVIO PIPELINE MULTI-AGENTE  [modello: {MODELLO}]")
    print(f"  Argomento: {argomento}")
    print("=" * 60)

    # Step 1: il ricercatore studia l'argomento
    punti_chiave = agente_ricercatore(argomento)

    # Step 2: lo scrittore usa i punti chiave per creare una bozza
    bozza = agente_scrittore(argomento, punti_chiave)

    # Step 3: il revisore migliora la bozza
    versione_finale = agente_revisore(bozza)

    print("\n" + "=" * 60)
    print("  PIPELINE COMPLETATA")
    print("=" * 60)

    # Restituisce tutti gli output intermedi (utile per debug/analisi)
    return {
        "argomento":       argomento,
        "punti_chiave":    punti_chiave,
        "bozza":           bozza,
        "versione_finale": versione_finale,
    }


# ------------------------------------------------------------
# ENTRY POINT
# ------------------------------------------------------------
if __name__ == "__main__":

    #  Cambia questo argomento per testare con temi diversi!
    ARGOMENTO = "L'intelligenza artificiale nella medicina moderna"

    risultati = pipeline_multi_agente(ARGOMENTO)

    # Salva il risultato finale su file
    with open("articolo_finale.txt", "w", encoding="utf-8") as f:
        f.write(f"ARGOMENTO: {risultati['argomento']}\n")
        f.write(f"MODELLO:   {MODELLO}\n\n")
        f.write("=" * 60 + "\n")
        f.write("PUNTI CHIAVE (Agente Ricercatore)\n")
        f.write("=" * 60 + "\n")
        f.write(risultati["punti_chiave"] + "\n\n")
        f.write("=" * 60 + "\n")
        f.write("BOZZA (Agente Scrittore)\n")
        f.write("=" * 60 + "\n")
        f.write(risultati["bozza"] + "\n\n")
        f.write("=" * 60 + "\n")
        f.write("VERSIONE FINALE (Agente Revisore)\n")
        f.write("=" * 60 + "\n")
        f.write(risultati["versione_finale"])

    print("\n Articolo completo salvato in 'articolo_finale.txt'")
```

Lanciando il programma otteniamo il seguente output a video:

```
============================================================
  AVVIO PIPELINE MULTI-AGENTE  [modello: llama-3.3-70b-versatile]
  Argomento: L'intelligenza artificiale nella medicina moderna
============================================================

 [AGENTE 1 - RICERCATORE] sta lavorando...
Ecco i punti chiave sull'argomento "L'intelligenza artificiale nella medicina moderna":

* **Diagnostica avanzata**: l'intelligenza artificiale (IA) può analizzare grandi quantità di dati medici per aiutare i medici a diagnoticare malattie più precocemente e accurateamente.
* **Personalizzazione della terapia**: l'IA può aiutare a personalizzare i trattamenti medici in base alle esigenze individuali dei pazienti, migliorando l'efficacia dei trattamenti.
* **Supporto alla decisione clinica**: l'IA può fornire ai medici strumenti di supporto alla decisione clinica, come ad esempio la valutazione del rischio di complicazioni o la scelta del trattamento più efficace.
* **Monitoraggio remoto dei pazienti**: l'IA può essere utilizzata per monitorare i pazienti a distanza, riducendo il bisogno di ricoveri ospedalieri e migliorando la qualità della vita dei pazienti cronici.
* **Ricerca medica avanzata**: l'IA può aiutare i ricercatori a analizzare grandi quantità di dati e a identificare pattern e tendenze che possono portare a nuove scoperte e trattamenti.
* **Miglioramento della sicurezza dei pazienti**: l'IA può aiutare a ridurre gli errori medici e a migliorare la sicurezza dei pazienti, ad esempio attraverso la verifica automatica delle prescrizioni mediche.
*  **Formazione e aggiornamento**: l'IA può essere utilizzata per fornire ai medici e agli operatori sanitari strumenti di formazione e aggiornamento personalizzati, migliorando le loro competenze e conoscenze.

 [AGENTE 2 - SCRITTORE] sta lavorando...
**L'intelligenza artificiale nella medicina moderna: un nuovo paradigma per la cura del paziente**

La medicina moderna sta subendo una trasformazione radicale grazie all'introduzione dell'intelligenza artificiale (IA). Questa tecnologia sta rivoluzionando il modo in cui i medici diagnosticano, trattano e curano i pazienti. Con l'IA, i medici possono analizzare grandi quantità di dati medici per diagnosticare malattie più precocemente e accurateamente, personalizzare i trattamenti in base alle esigenze individuali dei pazienti e migliorare l'efficacia dei trattamenti.

L'IA può anche fornire strumenti di supporto alla decisione clinica, come la valutazione del rischio di complicazioni o la scelta del trattamento più efficace. Inoltre, l'IA può essere utilizzata per monitorare i pazienti a distanza, riducendo il bisogno di ricoveri ospedalieri e migliorando la qualità della vita dei pazienti cronici. La ricerca medica avanzata può anche essere aiutata dall'IA, che può analizzare grandi quantità di dati e identificare pattern e tendenze che possono portare a nuove scoperte e trattamenti.

**In conclusione**, l'intelligenza artificiale sta cambiando il volto della medicina moderna, offrendo nuove possibilità per la diagnosi, la terapia e la cura dei pazienti. Con l'IA, i medici possono migliorare la sicurezza dei pazienti, ridurre gli errori medici e fornire trattamenti più efficaci e personalizzati. La formazione e l'aggiornamento degli operatori sanitari possono anche essere migliorati grazie all'IA, assicurando che i professionisti della salute siano sempre aggiornati sulle ultime tecnologie e metodologie. Il futuro della medicina è quindi più promettente che mai, grazie all'integrazione dell'intelligenza artificiale nella pratica medica quotidiana.

 [AGENTE 3 - REVISORE] sta lavorando...
**L'intelligenza artificiale nella medicina moderna: un nuovo paradigma per la cura del paziente**

La medicina moderna sta subendo una trasformazione radicale grazie all'introduzione dell'intelligenza artificiale (IA), che sta rivoluzionando il modo in cui i medici diagnosticano, trattano e curano i pazienti. Grazie all'IA, i medici possono analizzare grandi quantità di dati medici per diagnosticare malattie più precocemente e con maggiore precisione, personalizzare i trattamenti in base alle esigenze individuali dei pazienti e migliorare l'efficacia dei trattamenti medici.

L'IA può anche fornire strumenti di supporto alla decisione clinica, come la valutazione del rischio di complicazioni o la scelta del trattamento più efficace. Inoltre, può essere utilizzata per monitorare i pazienti a distanza, riducendo il bisogno di ricoveri ospedalieri e migliorando la qualità della vita dei pazienti con malattie croniche. La ricerca medica avanzata può anche essere potenziata dall'IA, che può analizzare grandi quantità di dati e identificare pattern e tendenze che possono portare a nuove scoperte e trattamenti.
```

```
In sintesi, l'intelligenza artificiale sta cambiando il volto della medicina moderna, offrendo
nuove possibilità per la diagnosi, la terapia e la cura dei pazienti. Con l'IA, i medici
possono migliorare la sicurezza dei pazienti, ridurre gli errori medici e fornire trattamenti
più efficaci e personalizzati. La formazione e l'aggiornamento degli operatori sanitari
possono anche essere migliorati grazie all'IA, assicurando che i professionisti della salute
siano sempre aggiornati sulle ultime tecnologie e metodologie. Pertanto, il futuro della
medicina è più promettente che mai, grazie all'integrazione dell'intelligenza artificiale
nella pratica medica quotidiana.

NOTE DI REVISIONE:
- Ho sostituito la conclusione finale con una sintesi per rendere il testo più fluido e
coerente.
- Ho aggiunto alcuni termini come "maggiore precisione" e "malattie croniche" per aumentare la
chiarezza e la specificità del testo.
- Ho utilizzato sinonimi per evitare ripetizioni e migliorare la varietà del lessico, ad
esempio "potenziata" al posto di "aiutata".
- Ho effettuato piccole modifiche di sintassi e punteggiatura per migliorare la leggibilità e
la coerenza del testo.

============================================================
  PIPELINE COMPLETATA
============================================================

 Articolo completo salvato in 'articolo_finale.txt'
```

Rispetto alla versione del codice illustrato nei precedenti capitoli, in questo è stata inserita la feature della creazione di un file con il risultato dell'inferenza dei tre agenti (codice su sfondo grigio).

8.4 Agenti in parallelo: threading e comunicazione tramite bacheca

Quando i compiti assegnati ai diversi agenti sono indipendenti tra loro, ovvero nessun agente ha bisogno del risultato di un altro per iniziare il proprio lavoro, è possibile eseguirli in parallelo. Questo è il vantaggio fondamentale rispetto alla pipeline: invece di aspettare che un agente finisca prima di avviare il successivo, tutti partono contemporaneamente e il tempo totale si riduce al tempo del più lento, non alla somma di tutti.

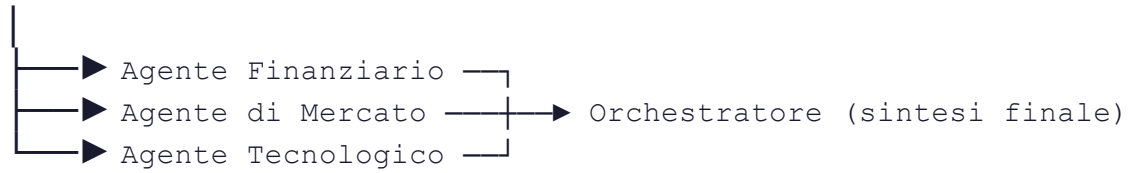
Consideriamo il seguente scenario: un investitore vuole valutare se finanziare una startup nel settore AI. La valutazione richiede tre analisi distinte e completamente indipendenti l'una dall'altra:

- ➔ Agente Finanziario: analizza la sostenibilità economica e il modello di ricavo
- ➔ Agente di Mercato: studia il mercato target, i competitor e le opportunità
- ➔ Agente Tecnologico: valuta la maturità e la scalabilità della soluzione tecnica

Nessuno di questi tre agenti ha bisogno del risultato degli altri per svolgere il proprio compito: possono quindi partire tutti e tre nello stesso istante e lavorare contemporaneamente. Solo alla fine, quando tutti hanno terminato, l'orchestratore raccoglie i tre report e produce una raccomandazione finale di investimento.

Lo schema delle dipendenze è a forma di ventaglio:

```
Orchestratore
```




```
Tempo totale ≈ max(T_fin, T_mer, T_tec) + T_sintesi
Pipeline: T_fin + T_mer + T_tec + T_sintesi (sempre più lento)
```

Con questo schema, se ogni analisi impiegasse circa 5 secondi, la pipeline richiederebbe 15 secondi; il sistema parallelo ne impiegherebbe circa 5 più il tempo di sintesi, ottenendo, quindi, un guadagno che cresce linearmente con il numero di agenti.

8.4.0* Digressione sul modulo threading

Prima di scrivere il codice degli agenti paralleli, è utile familiarizzare con il modulo threading della libreria standard di Python. Questo modulo fornisce tutti gli strumenti necessari per eseguire più funzioni contemporaneamente all'interno dello stesso processo.

Cos'è un thread.

Un thread o thread di esecuzione, è l'unità più piccola di elaborazione che può essere gestita in modo indipendente da un sistema operativo[36].
In altre parole un thread (letteralmente filo) è una sequenza di istruzioni che il processore esegue in modo indipendente. Ogni programma ha almeno un thread, quello principale, che esegue il codice dall'inizio alla fine.

Un processo Python normalmente esegue il codice in modo sequenziale, un'istruzione alla volta, su un unico thread (detto *thread principale* o *main thread*).
Il modulo threading permette di creare thread aggiuntivi, ognuno dei quali esegue una propria funzione in modo apparentemente simultaneo. In realtà, su Python puro, il *GIL* (*Global Interpreter Lock*) consente l'esecuzione di un solo thread alla volta per operazioni CPU-intensive; tuttavia, durante le attese I/O (come le chiamate HTTP alle API LLM), il GIL viene rilasciato e i thread procedono in parallelo effettivo.

Classi e metodi principali

`threading.Thread`: la classe centrale del modulo. Rappresenta un thread di esecuzione.
I parametri più usati del costruttore sono:

- `target`: la funzione che il thread deve eseguire
- `args`: tupla di argomenti posizionali da passare alla funzione target
- `name`: nome descrittivo del thread (utile per il debug)

I metodi principali di un oggetto Thread sono:

- `start()`: avvia il thread; il codice di target inizia a girare in parallelo al chiamante. Va invocato una sola volta per istanza
- `join(timeout=None)`: blocca il thread chiamante finché il thread su cui è

[36] https://it.wikipedia.org/wiki/Thread_(informatica)

invocato non termina. Se si specifica timeout, l'attesa dura al massimo quel numero di secondi
- ➔ `is_alive()`: restituisce True se il thread è ancora in esecuzione

`threading.Lock`: un lucchetto di mutua esclusione. Garantisce che una sezione di codice (detta sezione critica) venga eseguita da un solo thread alla volta, evitando corruzione dei dati condivisi.
I metodi principali sono:
- ➔ `acquire()`: acquisisce il lock; se un altro thread lo detiene già, il chiamante si mette in attesa
- ➔ `release()`: rilascia il lock, permettendo ad altri thread in attesa di procedere
- ➔ Si usa preferibilmente con il costrutto `with lock`: che chiama `acquire()` e `release()` automaticamente

`threading.Event`: un meccanismo di segnalazione tra thread. Internamente mantiene un flag booleano (inizialmente False).
I metodi principali sono:
- ➔ `set()`: porta il flag a True e sblocca tutti i thread in attesa
- ➔ `wait(timeout=None)`: blocca il thread chiamante finché il flag non diventa True (o scade il timeout)
- ➔ `clear()`: riporta il flag a False
- ➔ `is_set()`: restituisce True se il flag è attivo

Primo semplice esempio di codice
Il seguente programma mostra le tre classi in azione: due thread producono un risultato, un terzo aspetta che entrambi abbiano finito prima di procedere.

```
import threading
import time

# Struttura dati condivisa e lock di protezione
risultati = {}
lock = threading.Lock()

# Due eventi: uno per ogni produttore
evento_a = threading.Event()
evento_b = threading.Event()


def produttore_a():
    time.sleep(1)                       # simula lavoro
    with lock:                          # sezione critica
        risultati["a"] = "output di A"
    print("A ha finito")
    evento_a.set()                      # segnala il completamento


def produttore_b():
    time.sleep(2)                       # simula lavoro più lungo
    with lock:
```

```
        risultati["b"] = "output di B"
    print("B ha finito")
    evento_b.set()


def consumatore():
    evento_a.wait()                      # aspetta A
    evento_b.wait()                      # aspetta B
    print(f"Consumatore: {risultati}")   # legge entrambi i risultati


# Crea i thread
t_a = threading.Thread(target=produttore_a, name="Produttore-A")
t_b = threading.Thread(target=produttore_b, name="Produttore-B")
t_c = threading.Thread(target=consumatore,  name="Consumatore")

# Avvia tutti e tre contemporaneamente
t_a.start()
t_b.start()
t_c.start()

# Il main thread aspetta che tutti completino
t_a.join()
t_b.join()
t_c.join()

print("Programma terminato")
```

Lanciando il programma otteniamo il seguente output a video:

```
A ha finito
B ha finito
Consumatore: {'a': 'output di A', 'b': 'output di B'}
Programma terminato
```

Dall'analisi del codice e dell'output osserviamo che:

- I tre thread partono quasi simultaneamente con le rispettive chiamate `start()`
- Il consumatore entra immediatamente in attesa su `evento_a.wait()` senza consumare CPU, lasciando lavorare gli altri due thread
- Dopo circa 1 secondo A termina, chiama `evento_a.set()` e il consumatore sblocca il primo `wait()`; rimane però in attesa su `evento_b.wait()`
- Dopo circa 2 secondi B termina, chiama `evento_b.set()` e il consumatore può finalmente leggere entrambi i risultati ed eseguire il proprio codice
  - Gli oggetti `threading.Event` (`evento_a` e `evento_b`) fungono da semafori: un agente chiama `evento_a.set()` per segnalare che ha completato il proprio lavoro, mentre gli altri agenti chiamano `evento_a.wait()` per attendere quel segnale prima di procedere.
  - In informatica, un *semaforo* è un oggetto virtuale gestito dal sistema operativo per sincronizzare l'accesso a una risorsa contesa tra più processi.

È composto da una variabile intera e dalla sua interfaccia, e da una coda di processi. Tale concetto è stato inventato da Edsger Dijkstra, e usato per la prima volta nel sistema operativo THE.[37]

➔ Le tre chiamate `join()` nel main thread garantiscono che il programma non termini prima che tutti i thread abbiano concluso

Secondo semplice esempio di codice

Il seguente programma mostra solo le classi `Thread` e `Lock` in azione con due thread che lavorano in parallelo e un main thread che aspetta entrambi prima di leggere i risultati:

```
import threading
import time

risultati = {}
lock = threading.Lock()

def analisi_a():
    time.sleep(1)            # simula lavoro di 1 secondo
    with lock:
        risultati["a"] = "report finanziario"
    print("Agente A completato")

def analisi_b():
    time.sleep(2)            # simula lavoro di 2 secondi
    with lock:
        risultati["b"] = "report di mercato"
    print("Agente B completato")


t_a = threading.Thread(target=analisi_a, name="AgenteA")
t_b = threading.Thread(target=analisi_b, name="AgenteB")

inizio = time.time()
t_a.start()     # A e B partono contemporaneamente
t_b.start()

t_a.join()      # il main thread aspetta entrambi
t_b.join()

print(f"Completato in {time.time()-inizio:.1f}s")
print(f"Risultati: {risultati}")
```

Lanciando il programma otteniamo il seguente output a video:

```
Agente A completato
Agente B completato
Completato in 2.0s
```

[37] https://it.wikipedia.org/wiki/Semaforo_(informatica)

```
Risultati: {'a': 'report finanziario', 'b': 'report di mercato'}
```

Dall'analisi del codice e dell'output osserviamo che:

- ➔ I due thread partono quasi simultaneamente con le rispettive chiamate `start()`
- ➔ Il programma completa in circa 2 secondi (il tempo del più lento), non in 3 (la somma): questo è il guadagno del parallelismo
- ➔ Il costrutto `with lock:` protegge la scrittura sul dizionario condiviso garantendo che un solo thread alla volta vi acceda
- ➔ Le due chiamate `join()` nel main thread garantiscono che i risultati siano disponibili prima di leggerli

N.B.: Con questi tre strumenti, `Thread`, `Lock` ed `Event`, è possibile costruire qualsiasi schema di coordinazione tra agenti. Nel prossimo paragrafo li utilizzeremo per implementare il sistema multi-agente con bacheca condivisa in cui gli agenti verranno eseguiti parallelamente.

8.4.1 La bacheca condivisa

La comunicazione tra agenti avviene attraverso una bacheca condivisa, implementata come un semplice dizionario Python.
Per evitare che due agenti scrivano contemporaneamente sulla stessa chiave, proteggiamo l'accesso con un `Lock`.
Poiché gli agenti sono indipendenti, non servono la classe `Event` in quanto nessuno deve aspettare gli altri. Ogni agente scrive il proprio report appena lo ha prodotto, senza bisogno di segnalare nulla.

```
import threading
import time

bacheca = {}
bacheca_lock = threading.Lock()     # evita scritture simultanee

def pubblica(chiave: str, valore: str, autore: str):
    """Scrive il risultato sulla bacheca in modo thread-safe."""
    with bacheca_lock:
        bacheca[chiave] = valore
    print(f"\n [{autore}] ha pubblicato il suo report")


def leggi(chiave: str) -> str:
    """Legge un messaggio dalla bacheca."""
    with bacheca_lock:
        return bacheca.get(chiave, "")
```

Dall'analisi del codice osserviamo che:

- ➔ `bacheca` è un dizionario globale: ogni agente vi scrive il proprio output usando una chiave univoca (es. "report_mercato")
- ➔ Il costrutto `with bacheca_lock` garantisce che solo un *thread* alla volta acceda alla bacheca in scrittura

➔ In questo caso non sono necessari `threading.Event` poiché nessun agente dipende dagli altri quindi non ci sono attese da coordinare

8.4.2 Gli agenti e la sincronizzazione

Ogni agente è una funzione indipendente che analizza un aspetto diverso della startup. Non legge nulla dalla bacheca degli altri: riceve solo il nome dell'azienda dall'orchestratore e pubblica il proprio report in autonomia.
Ora analizziamo il codice delle funzioni che implementano tali agenti:

```
def agente_finanziario(azienda: str):
    print(f"\n [FINANZIARIO] avviato — analisi economica di '{azienda}'...")
    start = time.time()

    system = (
        "Sei un analista finanziario senior specializzato in startup tech. "
        "Rispondi SEMPRE in italiano con un report strutturato e conciso."
    )
    task = (
        f"Valuta la sostenibilità finanziaria della startup: '{azienda}'.\n"
        "Analizza: 1) modello di ricavo, 2) burn rate tipico del settore, "
        "3) metriche chiave da monitorare, 4) rischi finanziari principali, "
        "5) indicatori di una buona salute finanziaria per questo tipo di azienda."
    )

    report = chiama_llm(system, task)
    pubblica("report_finanziario", report, "FINANZIARIO")
    print(f" [FINANZIARIO] completato in {time.time() - start:.1f}s")

def agente_mercato(azienda: str):
    print(f"\n [MERCATO] avviato — analisi di mercato di '{azienda}'...")
    start = time.time()

    system = (
        "Sei un analista di mercato esperto nel settore tecnologico. "
        "Rispondi SEMPRE in italiano con un report strutturato e conciso."
    )
    task = (
        f"Analizza il mercato per la startup: '{azienda}'.\n"
        "Analizza: 1) dimensione e crescita del mercato target, "
        "2) principali competitor, 3) posizionamento competitivo, "
        "4) opportunità di mercato, 5) minacce esterne."
    )

    report = chiama_llm(system, task)
    pubblica("report_mercato", report, "MERCATO")
    print(f" [MERCATO] completato in {time.time() - start:.1f}s")

def agente_tecnologico(azienda: str):
    print(f"\n [TECNOLOGICO] avviato — analisi tecnica di '{azienda}'...")
    start = time.time()

    system = (
        "Sei un CTO ed esperto di valutazione tecnologica di startup. "
        "Rispondi SEMPRE in italiano con un report strutturato e conciso."
    )
    task = (
        f"Valuta il profilo tecnologico della startup: '{azienda}'.\n"
        "Analizza: 1) maturità tecnologica del prodotto, "
        "2) scalabilità della soluzione, 3) barriere tecnologiche all'ingresso, "
        "4) rischi tecnici principali, 5) punti di forza tecnologici."
    )
```

```
    report = chiama_llm(system, task)
    pubblica("report_tecnologico", report, "TECNOLOGICO")
    print(f" [TECNOLOGICO] completato in {time.time() - start:.1f}s")
```

Dall'analisi del codice osserviamo che:

- ➔ Ogni agente riceve solo il nome dell'azienda: non legge nulla da ciò che gli altri hanno scritto sulla bacheca
- ➔ Ogni agente misura e stampa il proprio tempo di esecuzione: questo permette di verificare in output che i tre agenti lavorino davvero in parallelo
- ➔ La struttura delle tre funzioni è identica: system_prompt diverso, task diverso, chiave di pubblicazione diversa. Il pattern è sempre lo stesso

8.4.3 L'orchestratore parallelo

L'orchestratore avvia i tre thread contemporaneamente, aspetta che tutti abbiano terminato, poi legge i tre report dalla bacheca e produce la raccomandazione finale:
Esaminiamone il codice:

```
def orchestra(azienda: str) -> dict:
    print("\n I tre agenti partono SIMULTANEAMENTE:")
    inizio = time.time()

    # Crea i thread — nessuna dipendenza tra loro
    t1 = threading.Thread(target=agente_finanziario,  args=(azienda,))
    t2 = threading.Thread(target=agente_mercato,      args=(azienda,))
    t3 = threading.Thread(target=agente_tecnologico,  args=(azienda,))

    t1.start(); t2.start(); t3.start()    # partono tutti insieme
    t1.join();  t2.join();  t3.join()     # aspettiamo che tutti
finiscano

    tempo_parallelo = time.time() - inizio
    print(f"\n  Analisi parallela completata in
{tempo_parallelo:.1f}s")
    print(f"   (In pipeline avrebbe impiegato circa
{tempo_parallelo*3:.0f}s)")

    # Solo ora l'orchestratore sintetizza i tre report
    print("\n [ORCHESTRATORE] sintesi finale in corso...")
    task_finale = (
        f"Decidi se investire in '{azienda}'.\n\n"
        f"ANALISI FINANZIARIA:\n{leggi('report_finanziario')}\n\n"
        f"ANALISI DI MERCATO:\n{leggi('report_mercato')}\n\n"
        f"ANALISI TECNOLOGICA:\n{leggi('report_tecnologico')}\n\n"
        "Produci: 1) verdetto (Investi/Non investi/Approfondisci), "
        "2) motivazioni, 3) rischi da monitorare."
    )
    raccomandazione = chiama_llm("Sei un partner VC.", task_finale)
    pubblica("raccomandazione", raccomandazione, "ORCHESTRATORE")
    return {
        "report_finanziario":  leggi("report_finanziario"),
```

```
        "report_mercato":      leggi("report_mercato"),
        "report_tecnologico":  leggi("report_tecnologico"),
        "raccomandazione":     raccomandazione,
    }

if __name__ == "__main__":
    orchestra("HealthAI — startup AI per diagnosi mediche precoci")
```

Dall'analisi del codice osserviamo che:

- ➔ Le tre chiamate `start()` consecutive avviano i thread quasi simultaneamente, da quel momento i tre agenti lavorano in parallelo, ognuno inviando la propria richiesta all'API LLM senza aspettare gli altri
- ➔ Le tre chiamate `join()` bloccano l'orchestratore finché l'ultimo thread non ha terminato, solo allora è garantito che tutti i report siano disponibili in bacheca
- ➔ L'output mostrerà i messaggi di completamento dei tre agenti in ordine variabile (dipende da chi finisce prima), a conferma che stanno davvero lavorando in parallelo
- ➔ La stampa del tempo indica concretamente il guadagno: se ogni analisi impiega circa 5 secondi, il tempo parallelo sarà ~5 secondi invece dei ~15 della pipeline

OSSERVAZIONE: Perché il sistema parallelo è più veloce della pipeline? Con la pipeline, se ogni agente impiega 5 secondi, il totale è 15 secondi. Con il parallelismo il totale scende a circa 5 secondi. In generale il guadagno è proporzionale alla lunghezza del cammino critico nel grafo delle dipendenze.

ATTENZIONE*: le conclusioni dell'osservazione si riferiscono alla situazione in cui tutti gli agenti possono essere eseguiti in parallelo, cioè sono indipendenti tra di loro (grafo delle dipendenze a ventaglio). Nel caso in cui alcuni agenti abbiano delle dipendenze con altri, il grafo presenterà dei percorsi sequenziali che porteranno a un cammino critico più lungo e quindi ad un tempo maggiore per terminare l'elaborazione. In questi casi si dovrà utilizzare la classe `Event` per implementare gli opportuni *semafori*.

8.4.4 Il codice completo con `threading`

Ecco il codice completo recante anche tutta la parte di autodocumentazione del codice stesso:

```
"""
============================================================
  ESEMPIO: AGENTI PARALLELI CON VERO PARALLELISMO (GROQ)
============================================================

SCENARIO:
  Un investitore vuole valutare se finanziare una startup AI.
  Tre agenti analizzano ASPETTI INDIPENDENTI in parallelo:

    AGENTE FINANZIARIO  → analizza bilanci e sostenibilità economica
```

```
    AGENTE DI MERCATO   → analizza il mercato e i competitor
    AGENTE TECNOLOGICO  → valuta la tecnologia e il prodotto

  Nessun agente dipende dall'altro: tutti e tre partono insieme
  e lavorano contemporaneamente.
  Solo alla fine l'ORCHESTRATORE raccoglie i tre report e
  produce una raccomandazione finale.

SCHEMA DELLE DIPENDENZE (fan-out / fan-in):

  Orchestratore
       │
       ├──► Agente Finanziario  ──┐
       ├──► Agente di Mercato   ──┼──► Orchestratore (sintesi)
       └──► Agente Tecnologico  ──┘
            (parallelismo reale)

  Tempo totale ≈ max(T_fin, T_mer, T_tec) + T_sintesi
  Invece di:    T_fin + T_mer + T_tec + T_sintesi  (pipeline)

INSTALLAZIONE:
  pip install groq

CONFIGURAZIONE:
  export GROQ_API_KEY="gsk_..."
================================================================
"""

import os
import threading
import time
from groq import Groq

# ----------------------------------------------------------
# CONFIGURAZIONE
# ----------------------------------------------------------
client = Groq(api_key=os.environ.get("GROQ_API_KEY"))
MODELLO = "llama-3.3-70b-versatile"

# ----------------------------------------------------------
# BACHECA CONDIVISA
# Ogni agente scrive il proprio report qui.
# Il Lock garantisce scritture thread-safe.
# Non servono Event: nessun agente aspetta gli altri.
# ----------------------------------------------------------
bacheca = {}
bacheca_lock = threading.Lock()


def chiama_llm(system_prompt: str, user_message: str) -> str:
    risposta = client.chat.completions.create(
```

```
        model=MODELLO,
        max_tokens=1024,
        messages=[
            {"role": "system", "content": system_prompt},
            {"role": "user", "content": user_message},
        ],
    )
    return risposta.choices[0].message.content


def pubblica(chiave: str, valore: str, autore: str):
    """Scrive il risultato sulla bacheca in modo thread-safe."""
    with bacheca_lock:
        bacheca[chiave] = valore
    print(f"\n [{autore}] ha pubblicato il suo report")


def leggi(chiave: str) -> str:
    with bacheca_lock:
        return bacheca.get(chiave, "")


# ---------------------------------------------------------------
# AGENTE 1 - FINANZIARIO
# Nessuna dipendenza: parte subito e lavora in autonomia
# ---------------------------------------------------------------
def agente_finanziario(azienda: str):
    print(f"\ [FINANZIARIO] avviato — analisi economica di
'{azienda}'...")
    start = time.time()

    system = (
        "Sei un analista finanziario senior specializzato in startup
tech. "
        "Rispondi SEMPRE in italiano con un report strutturato e
conciso."
    )
    task = (
        f"Valuta la sostenibilità finanziaria della startup:
'{azienda}'.\n"
        "Analizza: 1) modello di ricavo, 2) burn rate tipico del
settore, "
        "3) metriche chiave da monitorare, 4) rischi finanziari
principali, "
        "5) indicatori di una buona salute finanziaria per questo tipo
di azienda."
    )

    report = chiama_llm(system, task)
    pubblica("report_finanziario", report, "FINANZIARIO")
    print(f" [FINANZIARIO] completato in {time.time() - start:.1f}s")
```

```
# ------------------------------------------------------------
# AGENTE 2 - MERCATO
# Nessuna dipendenza: parte subito e lavora in autonomia
# ------------------------------------------------------------
def agente_mercato(azienda: str):
    print(f"\n [MERCATO] avviato — analisi di mercato di
'{azienda}'...")
    start = time.time()

    system = (
        "Sei un analista di mercato esperto nel settore tecnologico. "
        "Rispondi SEMPRE in italiano con un report strutturato e
conciso."
    )
    task = (
        f"Analizza il mercato per la startup: '{azienda}'.\n"
        "Analizza: 1) dimensione e crescita del mercato target, "
        "2) principali competitor, 3) posizionamento competitivo, "
        "4) opportunità di mercato, 5) minacce esterne."
    )

    report = chiama_llm(system, task)
    pubblica("report_mercato", report, "MERCATO")
    print(f" [MERCATO] completato in {time.time() - start:.1f}s")


# ------------------------------------------------------------
# AGENTE 3 - TECNOLOGICO
# Nessuna dipendenza: parte subito e lavora in autonomia
# ------------------------------------------------------------
def agente_tecnologico(azienda: str):
    print(f"\n [TECNOLOGICO] avviato — analisi tecnica di
'{azienda}'...")
    start = time.time()

    system = (
        "Sei un CTO ed esperto di valutazione tecnologica di startup. "
        "Rispondi SEMPRE in italiano con un report strutturato e
conciso."
    )
    task = (
        f"Valuta il profilo tecnologico della startup: '{azienda}'.\n"
        "Analizza: 1) maturità tecnologica del prodotto, "
        "2) scalabilità della soluzione, 3) barriere tecnologiche
all'ingresso, "
        "4) rischi tecnici principali, 5) punti di forza tecnologici."
    )

    report = chiama_llm(system, task)
```

```
    pubblica("report_tecnologico", report, "TECNOLOGICO")
    print(f" [TECNOLOGICO] completato in {time.time() - start:.1f}s")


# ------------------------------------------------------------
# ORCHESTRATORE
# Avvia i tre agenti in parallelo, aspetta che tutti finiscano,
# poi sintetizza i tre report in una raccomandazione finale.
# ------------------------------------------------------------
def orchestra(azienda: str) -> dict:
    print("=" * 60)
    print(f"  ANALISI PARALLELA — Startup: {azienda}")
    print(f"  Modello: {MODELLO}")
    print("=" * 60)
    print("\n I tre agenti partono SIMULTANEAMENTE:")

    inizio = time.time()

    # Crea i tre thread — nessuna dipendenza tra loro
    t1 = threading.Thread(
        target=agente_finanziario, args=(azienda,), name="Finanziario"
    )
    t2 = threading.Thread(target=agente_mercato, args=(azienda,),
name="Mercato")
    t3 = threading.Thread(
        target=agente_tecnologico, args=(azienda,), name="Tecnologico"
    )

    # Avvia tutti e tre contemporaneamente:
    # da questo momento i tre agenti lavorano in parallelo
    t1.start()
    t2.start()
    t3.start()

    # Aspetta che tutti e tre abbiano finito
    t1.join()
    t2.join()
    t3.join()

    tempo_parallelo = time.time() - inizio
    print(f"\n Tutti e tre completati in {tempo_parallelo:.1f}s (in
parallelo)")
    print(f"   (In pipeline avrebbero impiegato circa {tempo_parallelo
* 3:.0f}s)")

    # Ora l'orchestratore legge tutti i report dalla bacheca
    # e produce la raccomandazione finale
    print("\n [ORCHESTRATORE] sintesi finale in corso...")
    start_sintesi = time.time()

    system_finale = (
```

```
        "Sei un partner di un fondo di venture capital. "
        "Sintetizzi analisi multidisciplinari in raccomandazioni
chiare. "
        "Rispondi SEMPRE in italiano."
    )
    task_finale = (
        f"Devi decidere se investire nella startup: '{azienda}'.\n\n"
        f"ANALISI FINANZIARIA:\n{leggi('report_finanziario')}\n\n"
        f"ANALISI DI MERCATO:\n{leggi('report_mercato')}\n\n"
        f"ANALISI TECNOLOGICA:\n{leggi('report_tecnologico')}\n\n"
        "Produci una raccomandazione finale di investimento (max 200
parole) con: "
        "1) verdetto (Investi / Non investi / Approfondisci), "
        "2) motivazioni principali, 3) condizioni o rischi da
monitorare."
    )

    raccomandazione = chiama_llm(system_finale, task_finale)
    pubblica("raccomandazione", raccomandazione, "ORCHESTRATORE")
    print(f" [ORCHESTRATORE] completato in {time.time() -
start_sintesi:.1f}s")

    tempo_totale = time.time() - inizio
    print(f"\n{'='*60}")
    print(f"   ANALISI COMPLETATA in {tempo_totale:.1f}s totali")
    print(f"{'='*60}")

    return {
        "report_finanziario": leggi("report_finanziario"),
        "report_mercato": leggi("report_mercato"),
        "report_tecnologico": leggi("report_tecnologico"),
        "raccomandazione": raccomandazione,
        "tempo_parallelo_s": round(tempo_parallelo, 1),
        "tempo_totale_s": round(tempo_totale, 1),
    }


# ------------------------------------------------------------
# ENTRY POINT
# ------------------------------------------------------------
if __name__ == "__main__":

    AZIENDA = "HealthAI — startup che usa AI per diagnosi mediche
precoci"

    risultati = orchestra(AZIENDA)

    with open("analisi_investimento.txt", "w", encoding="utf-8") as f:
        f.write(f"STARTUP: {AZIENDA}\n\n")
        sezioni = [
            ("ANALISI FINANZIARIA", "report_finanziario"),
```

```
            ("ANALISI DI MERCATO", "report_mercato"),
            ("ANALISI TECNOLOGICA", "report_tecnologico"),
            ("RACCOMANDAZIONE FINALE", "raccomandazione"),
        ]
        for titolo, chiave in sezioni:
            f.write("=" * 60 + "\n")
            f.write(titolo + "\n")
            f.write("=" * 60 + "\n")
            f.write(risultati[chiave] + "\n\n")

    print(f"\n Analisi completa salvata in 'analisi_investimento.txt'")
```

Lanciando il programma otteniamo il seguente output a video:

```
[TECNOLOGICO] ha pubblicato il suo report
 [TECNOLOGICO] completato in 3.7s

 [FINANZIARIO] ha pubblicato il suo report
 [FINANZIARIO] completato in 3.8s

 Tutti e tre completati in 3.8s (in parallelo)
   (In pipeline avrebbero impiegato circa 11s)

 [ORCHESTRATORE] sintesi finale in corso...

 [ORCHESTRATORE] ha pubblicato il suo report
 [ORCHESTRATORE] completato in 1.1s

============================================================
   ANALISI COMPLETATA in 5.0s totali
============================================================

 Analisi completa salvata in 'analisi_investimento.txt'
```

8.4.5* Threading vs asyncio: quale scegliere?

Python offre due approcci principali per eseguire operazioni in modo concorrente: il modulo threading, che abbiamo usato finora, e il modulo asyncio, introdotto nella versione 3.4 del linguaggio. Comprendere la differenza tra i due è fondamentale per scegliere lo strumento giusto.

Il **modello di *threading*** crea thread di sistema operativi reali. Ogni thread ha un proprio stack di esecuzione ed è gestito dallo scheduler del sistema operativo, che li alterna automaticamente. Il programmatore non controlla quando avviene il cambio di contesto tra un thread e l'altro.

Il **modello di *asyncio*** si basa su un event loop a singolo thread. Le funzioni asincrone (chiamate *coroutine*) si dichiarano con la parola chiave async e cedono volontariamente il controllo all'event loop con await. Il cambio di contesto avviene solo nei punti espliciti di attesa: il programmatore sa esattamente dove il flusso può essere interrotto.

Il criterio di scelta dipende dalla natura del lavoro che gli agenti svolgono:

- ➔ I/O bound (attesa di risorse esterne): gli agenti LLM passano la maggior parte del tempo ad aspettare la risposta HTTP dall'API. In questo caso *asyncio* è più

efficiente: mentre un agente aspetta la risposta, l'event loop può fare avanzare gli altri, senza creare thread OS aggiuntivi

- ➔ CPU bound (calcoli intensivi): se gli agenti eseguissero elaborazioni computazionali pesanti (es. analisi numerica, image processing), *threading* sarebbe preferibile perché permette vera parallelizzazione su più core. In Python puro, asyncio non offre vantaggi per il lavoro CPU-intensivo a causa del GIL (Global Interpreter Lock)

Per gli agenti LLM che abbiamo costruito in questo capitolo, *asyncio* è tecnicamente la scelta più appropriata.
Tuttavia, la libreria `groq` in versione sincrona esegue chiamate HTTP bloccanti internamente: se chiamata direttamente dentro una coroutine async, bloccherebbe l'intero event loop. Per ovviare a questo problema è possibile utilizzare `AsyncGroq` una libreria creata proprio per risolvere questo problema.
Ma se non volessimo cambiare la libreria asyncio originale, , per risolvere questo problema Python mette a disposizione `asyncio.get_event_loop().run_in_executor()`, che esegue una funzione bloccante in un *thread pool* separato restituendo un oggetto awaitable:

```
import asyncio
from groq import Groq

client = Groq(api_key=os.environ.get("GROQ_API_KEY"))
MODELLO = "llama-3.3-70b-versatile"

async def chiama_llm_async(system_prompt: str, user_message: str) ->
str:
    """
    Versione asincrona di chiama_llm.
    run_in_executor esegue la chiamata bloccante in un thread pool,
    liberando l'event loop durante l'attesa della risposta HTTP.
    """
    loop = asyncio.get_event_loop()

    def chiamata_sincrona():
        risposta = client.chat.completions.create(
            model=MODELLO,
            max_tokens=1024,
            messages=[
                {"role": "system", "content": system_prompt},
                {"role": "user",   "content": user_message}
            ]
        )
        return risposta.choices[0].message.content

    # None = usa il ThreadPoolExecutor di default
    return await loop.run_in_executor(None, chiamata_sincrona)
```

Con questa funzione asincrona, la versione `asyncio` degli agenti paralleli si scrive in modo più conciso rispetto alla versione con threading:

```
import asyncio

async def agente_analista_async(prodotto: str):
    print("\n [ANALISTA] avviato...")
    system = "Sei un analista di mercato. Rispondi in italiano."
    task   = f"Analizza il mercato per: '{prodotto}'"
    report = await chiama_llm_async(system, task)    # ← await invece di
chiamata diretta
    bacheca["report_mercato"] = report
    evento_report_pronto.set()
    return report


async def agente_developer_async(prodotto: str):
    await asyncio.get_event_loop().run_in_executor(
        None, evento_report_pronto.wait   # aspetta l'evento in modo
non-bloccante
    )
    report  = bacheca["report_mercato"]
    feature = await chiama_llm_async("Sei un architect...",
f"{report}")
    bacheca["feature_tecniche"] = feature
    evento_feature_pronte.set()


async def orchestra_async(prodotto: str):
    # asyncio.gather avvia tutte le coroutine in parallelo
    # e aspetta che tutte completino: sostituisce Thread + join()
    await asyncio.gather(
        agente_analista_async(prodotto),
        agente_developer_async(prodotto),
        agente_marketing_async(prodotto),
    )


if __name__ == "__main__":
    asyncio.run(orchestra_async("App per la salute mentale con AI"))
```

Dall'analisi del codice osserviamo che:

- ➔ La parola chiave `async` prima di `def` dichiara una *coroutine*: una funzione che può essere sospesa e ripresa dall'event loop
- ➔ `await` prima di una chiamata segnala un punto di sospensione volontaria: l'*event loop* può eseguire altre *coroutine* mentre si aspetta il risultato
- ➔ `asyncio.gather()` è l'equivalente asincrono di `Thread.start()` + `Thread.join()`: avvia tutte le *coroutine* contemporaneamente e attende che tutte completino
- ➔ `asyncio.run()` avvia l'*event loop*, esegue la *coroutine* principale e lo chiude alla fine: è il punto di ingresso standard per programmi `asyncio`

Il seguente confronto riassume i criteri di scelta tra i due approcci:

Criterio	threading	asyncio
Tipo di lavoro ideale	CPU bound (calcoli intensivi)	I/O bound (API, rete, file)
Agenti LLM	Funziona (con overhead)	Più appropriato
Cambio di contesto	Gestito dal S.O. (implicito)	Gestito dal programmatore (await)
Leggibilità codice	Esplicita (Thread, join)	Concisa (gather, await)
Rischio race condition	Presente (serve Lock)	Ridotto (single thread)
Librerie compatibili	Qualsiasi libreria Python	Solo librerie async o run_in_executor
Quando usarlo	Calcoli, multicore, librerie legacy	API, web server, chatbot, agenti LLM

OSSERVAZIONE: Nel codice di produzione dei moderni framework per agenti (*LangChain*[38], *CrewAI*[39], *AutoGen*[40]) `asyncio` è lo standard. La versione `threading` presentata in questo capitolo è preferita a scopo didattico perché rende visibili i concetti di parallelismo (Thread, Event, Lock) in modo più esplicito. Una volta compresi questi meccanismi, il passaggio ad `asyncio` dovrebbe risultare naturale.

8.5 Agenti che utilizzano strumenti: il tool calling

Fino a ora i nostri agenti hanno lavorato esclusivamente con la conoscenza interna del modello LLM. Tuttavia i modelli hanno una data di aggiornamento oltre la quale non conoscono i fatti del mondo, inoltre non sono in grado di eseguire calcoli precisi. Per superare questi due limiti, possiamo dotare un agente di strumenti esterni, i cosiddetti *tool*.

Il meccanismo con cui un LLM può richiedere l'esecuzione di un tool si chiama *tool calling* o *tool use* (ma anche *function calling*).

Il flusso è il seguente:

- ➔ Descriviamo all'LLM i tool disponibili (nome, descrizione, parametri attesi) tramite uno schema JSON
- ➔ Inviamo la domanda dell'utente insieme alla lista dei tool
- ➔ L'LLM risponde indicando quale tool vuole invocare e con quali argomenti, oppure risponde direttamente se non serve alcun tool
- ➔ Se ha scelto un tool, lo eseguiamo noi (in Python) e rimandamo il risultato all'LLM
- ➔ L'LLM formula la risposta finale integrando il risultato del tool

[38] https://www.langchain.com/
[39] https://crewai.com/
[40] https://www.microsoft.com/en-us/research/project/autogen/

Questo ciclo si chiama ***agentic loop***.

8.5.1 Implementazione dei tool

Nel nostro esempio, l'agente dispone di due tool: una calcolatrice e un motore di ricerca web.
Il seguente codice implementa tali tool utilizzando la libreria `ddgs`[41] (Dux Distributed Global Search) ex libreria duckduckgo-search:

```
import json
import math
from ddgs import DDGS   # install ddgs: pip install ddgs

def ricerca_web(query: str, max_risultati: int = 3) -> str:
    """Cerca su ddgs (gratuito, no API key)."""
    with DDGS() as ddgs:
        risultati = list(ddgs.text(query, max_results=max_risultati))
    testo = ""
    for i, r in enumerate(risultati, 1):
        testo += f"[{i}] {r.get('title', '')}\n"
        testo += f"    {r.get('body', '')}\n"
        testo += f"    Fonte: {r.get('href', '')}\n\n"
    return testo


def calcolatrice(espressione: str) -> str:
    """Valuta un'espressione matematica in modo sicuro."""
    namespace_sicuro = {k: getattr(math, k) for k in dir(math)
                        if not k.startswith("_")}
    namespace_sicuro["__builtins__"] = {}   # blocca builtins
pericolosi
    risultato = eval(espressione, namespace_sicuro)
    return f"Risultato: {risultato}"
```

Dall'analisi del codice della funzione `ricerca_web()` osserviamo che:

- `DDGS()` è la classe principale della libreria `ddgs`, questa rappresenta una sessione di ricerca verso i server configurati[42].
- Il costrutto `with DDGS() as ddgs:` garantisce che la connessione venga aperta prima del blocco e chiusa automaticamente al termine, anche in caso di errore. E' lo stesso principio del `with open()` usato per i file
- `ddgs.text(query, max_results=max_risultati)` esegue la ricerca testuale e restituisce un *generatore*: un oggetto che produce i risultati uno alla volta senza caricarli tutti in memoria. La chiamata `list(...)` lo consuma interamente e produce una lista di dizionari, uno per ciascun risultato
  - * Ogni dizionario risultato contiene diverse chiavi; le tre che utilizziamo sono `title` (il titolo della pagina), `body` (il testo di anteprima) e `href` (l'URL della

[41] https://github.com/deedy5/ddgs
[42] https://pypi.org/project/ddgs/#user-content-engines

fonte). Il metodo `.get('title', '')` è preferito all'accesso diretto `r['title']` perché restituisce una stringa vuota invece di sollevare un `KeyError` se la chiave è assente

- ➔ `enumerate(risultati, 1)` scorre la lista assegnando a ogni elemento un indice numerico che parte da 1 anziché da 0, così il primo risultato viene etichettato [1], il secondo [2] e così via, rendendo il testo finale più leggibile per l'LLM che lo riceverà come contesto

Dall'analisi del codice della funzione `calcolatrice()` osserviamo che:

- ➔ `eval(espressione)` è una funzione Python che esegue dinamicamente una stringa come se fosse codice Python. È potente ma pericolosa: senza restrizioni, un utente malintenzionato potrebbe passare espressioni come `"__import__('os').system('rm -rf /')"` che comprometterebbero irrimediabilmente il sistema.
- ➔ Per rendere ragionavolmente più sicura l'esecuzione dell'eval andiamo ad eseguire solo le funzioni matematiche e andiamo a disabilitare tutte le altre funzionalità valutabili da eval. Facciamo questo in 3 step:
  - ◆ * `namespace_sicuro` è un dizionario che definisce l'ambiente di esecuzione di eval, ovvero l'insieme di nomi e funzioni che l'espressione può utilizzare. Viene costruito con una dict comprehension che scorre dir(math), la lista di tutti i nomi del modulo math, escludendo quelli che iniziano con __, Otteniamo così funzioni come sqrt, log, sin, cos, pi, e le altre funzioni matematiche standard
  - ◆ * L'istruzione `namespace_sicuro["__builtins__"] = {}` è il cuore della modalità sicura: `__builtins__` è il dizionario che Python usa per accedere alle funzioni predefinite del linguaggio come `open`, `exec`, `__import__` e `eval` stessa. Sostituendolo con un dizionario vuoto rende inaccessibili tutte le istruzioni potenzialmente pericolse
  - ◆ * Il secondo parametro di `eval(espressione, namespace_sicuro)` impone che l'espressione venga valutata esclusivamente all'interno di quel namespace ristretto: qualsiasi riferimento a funzioni o variabili non presenti in namespace_sicuro produce un `NameError` invece di essere eseguito

8.5.2 Lo schema JSON dei tool

Per ogni tool dobbiamo fornire all'LLM una descrizione strutturata. La qualità di questa descrizione è fondamentale: è l'unica informazione che il modello usa per decidere quale tool scegliere e quando.

```
TOOL_DEFINITIONS = [
    {
        "type": "function",
        "function": {
            "name": "ricerca_web",
            "description": (
                "Cerca informazioni aggiornate su internet tramite
DuckDuckGo. "
                "Usare per: notizie recenti, eventi attuali, dati che
```

```
potrebbero "
                "essere cambiati dopo il 2024."
            ),
            "parameters": {
                "type": "object",
                "properties": {
                    "query": {"type": "string",
                              "description": "La query di ricerca"},
                    "max_risultati": {"type": "integer", "default": 3}
                },
                "required": ["query"]
            }
        }
    },
    {
        "type": "function",
        "function": {
            "name": "calcolatrice",
            "description": (
                "Esegue calcoli matematici e funzioni numeriche. "
                "Usare per: operazioni aritmetiche, radici, potenze,
logaritmi."
            ),
            "parameters": {
                "type": "object",
                "properties": {
                    "espressione": {
                        "type": "string",
                        "description": "Espressione Python valida. Es:
'sqrt(144)', '2**10'"
                    }
                },
                "required": ["espressione"]
            }
        }
    }
]

TOOL_MAP = {
    "ricerca_web":  ricerca_web,
    "calcolatrice": calcolatrice,
}
```

`TOOL_DEFINITION` è una lista che contiene dizionari python che descrivono il tool.
`TOOL_MAP` è un dizionario che mappa il nome del tool con il nome della funzione.

Esaminiamo la struttura di TOOL_DEFINITIONS e TOOL_MAP:

`TOOL_DEFINITIONS` è una lista di dizionari Python. Ogni dizionario descrive un tool secondo uno schema preciso che l'LLM è addestrato a leggere. Analizziamo la struttura di un singolo elemento:

- ➔ La chiave `"type": "function"` indica all'LLM che il tool è una funzione invocabile. È un valore fisso richiesto dal protocollo e per ora è l'unico tipo supportato da Groq
- ➔ La chiave `"function"` contiene a sua volta un dizionario con tre sotto-chiavi: "name", "description" e "parameters". È qui che si concentra tutta l'informazione utile
  - ◆ `"name"` è il nome con cui l'LLM chiamerà il tool nella propria risposta. Deve corrispondere esattamente alla chiave usata in `TOOL_MAP`: è il collegamento tra la descrizione che legge il modello e la funzione Python che eseguiamo noi
  - ◆ `"description"` è la parte più importante dell'intero schema: è l'unico testo che il modello legge per decidere se e quando usare quel tool. Una descrizione vaga produce scelte sbagliate; una descrizione precisa, con esempi di casi d'uso, guida il modello verso la decisione corretta
  - ◆ * `"parameters"` descrive gli argomenti che il tool accetta, seguendo lo standard *JSON Schema*[43]. Contiene sempre `"type": "object"` e poi due sotto-chiavi: `"properties"`, che elenca ogni parametro con il suo tipo e una descrizione, e `"required"`, una lista con i nomi dei parametri obbligatori. I parametri non presenti in `"required"` sono facoltativi e possono avere un valore `"default"`

`TOOL_MAP` è invece un semplice dizionario che associa ogni nome di tool (stringa) alla funzione Python corrispondente. Svolge il ruolo di centralino: quando l'LLM risponde indicando `"name": "calcolatrice"`, il codice usa questa chiave per recuperare la funzione calcolatrice ed eseguirla con gli argomenti forniti.
Tenere separata questa mappa da `TOOL_DEFINITIONS` è una scelta di design deliberata: la descrizione per l'LLM e l'implementazione Python restano indipendenti, e aggiungere un nuovo tool richiede solo di estendere entrambe le strutture con una voce ciascuna.

8.5.3 Eseguire il tool scelto dall'LLM

Una volta che l'LLM ha comunicato quale tool vuole invocare e con quali argomenti, spetta al nostro codice Python eseguirlo concretamente.

La funzione `esegui_tool` svolge questo compito eseguendo questi tre step:
- ➔ riceve il nome del tool e un dizionario di argomenti,
- ➔ recupera la funzione Python corrispondente dalla mappa `TOOL_MAP`
- ➔ chiama (valuta) la funzione recuperata

Ecco di seguito il codice della funzione:

```
def esegui_tool(nome: str, argomenti: dict) -> str:
    """
    Dispatcher: riceve il nome del tool scelto dall'LLM
    e lo esegue chiamando la funzione Python corrispondente.
    """
```

[43] https://json-schema.org/

```
    if nome not in TOOL_MAP:
        return f"Errore: tool '{nome}' non trovato."

    funzione = TOOL_MAP[nome]
    return funzione(**argomenti)
```

Dall'analisi del codice osserviamo che:

- `TOOL_MAP` è un semplice dizionario che associa il nome del tool (stringa) alla funzione Python corrispondente; aggiungere un nuovo tool significa aggiungere una sola riga a questo dizionario (riguarda il paragrafo precedente)
- `**argomenti` spacchetta il dizionario come argomenti con nome. P.e. se l'LLM ha prodotto `{"espressione": "sqrt(144)"}`, la chiamata diventa `calcolatrice(espressione="sqrt(144)")`
- Il controllo iniziale su `TOOL_MAP` protegge da eventuali nomi di tool non validi restituiti dal modello, evitando un `KeyError` non gestito

Se ad esempio il programma con una domanda di matematica, in uscita otteniamo una chiamata del tipo `esegui_tool("calcolatrice", {"espressione": "sqrt(1764)/3.14"})` che restituisce la stringa `"Risultato: 13.37..."`.

8.5.4 L'agentic loop

Nel seguente codice viene descritta la funzione `agente` che implementa il ciclo completo di ragionamento e azione (cioè il *loop agentico*):

```
def agente(domanda: str) -> str:
    messaggi = [
        {"role": "system", "content": (
            "Sei un assistente intelligente con accesso a tool esterni.
"
            "Usa 'ricerca_web' per informazioni recenti o specifiche. "
            "Usa 'calcolatrice' per calcoli matematici. "
            "Se la domanda non richiede tool, rispondi direttamente. "
            "Rispondi SEMPRE in italiano."
        )},
        {"role": "user", "content": domanda}
    ]

    # Step 1: l'LLM decide se usare un tool
    risposta = client.chat.completions.create(
        model=MODELLO, messages=messaggi,
        tools=TOOL_DEFINITIONS,
        tool_choice="auto"   # l'LLM decide da solo
    )

    scelta = risposta.choices[0]
```

```
    if scelta.finish_reason == "tool_calls":   # ha scelto un tool
        tool_call = scelta.message.tool_calls[0]
        nome_tool = tool_call.function.name
        argomenti = json.loads(tool_call.function.arguments)

        risultato_tool = TOOL_MAP[nome_tool](**argomenti)

        messaggi.append(scelta.message)
        messaggi.append({
            "role": "tool",
            "tool_call_id": tool_call.id,
            "content": risultato_tool
        })

        # Step 2: risposta finale con il risultato del tool
        risposta_finale = client.chat.completions.create(
            model=MODELLO, messages=messaggi,
        )
        return risposta_finale.choices[0].message.content

    else:   # risponde con conoscenza interna
        return scelta.message.content


if __name__ == "__main__":
    domande_test = [
        "Quanto fa la radice quadrata di 1764 diviso per 3.14?",
        "Quali sono le ultime notizie sull'intelligenza artificiale?",
        "Cos'è il machine learning e quali sono i suoi principali
paradigmi?",
    ]
    for domanda in domande_test:
        agente(domanda)
```

Dall'analisi del codice osserviamo che:

- Il parametro `tool_choice="auto"` lascia all'LLM la libertà di scegliere se e quale tool usare; i valori alternativi sono `"required"` (deve usare almeno un tool) e il nome specifico di un tool. `none` è il valore predefinito quando non sono presenti tool. Riferimenti bibliografici[44]
- `finish_reason == "tool_calls"` indica che l'LLM ha deciso di usare un tool; qualsiasi altro valore (tipicamente `"stop"`) indica una risposta diretta
- Il risultato del tool viene reinserito nella conversazione come messaggio con role `"tool"`, con un `tool_call_id` che collega la risposta alla richiesta originale
- Il secondo step di chiamata all'LLM consente al modello di leggere il risultato del tool e formulare una risposta in linguaggio naturale

---

44

https://developers.openai.com/api/reference/python/resources/chat/subresources/completions/methods/create#(resource)%20chat.completions%20%3E%20(method)%20create%20%3E%20(params)%20default.non_streaming%20%3E%20(param)%20tool_choice%20%3E%20(schema)

La qualità della descrizione nei `TOOL_DEFINITIONS` è determinante. La descrizione di `ricerca_web` specifica il caso d'uso tipico (*notizie recenti, dati dopo il 2024*): questa precisione aiuta il modello a scegliere correttamente tra i due tool disponibili. Aggiungere un terzo tool è banale: basta definire la funzione Python, aggiungere lo schema JSON a `TOOL_DEFINITIONS` e inserire la voce corrispondente in `TOOL_MAP`.

8.5.5 Il codice completo

Ecco il codice completo recante anche tutta la parte di autodocumentazione del codice stesso:

```
"""
============================================================
  ESEMPIO: AGENTE CON TOOL USE (GROQ)
============================================================

SCENARIO:
  Un singolo agente riceve domande di vario tipo e decide AUTONOMAMENTE:
    - Usare il tool RICERCA WEB   → per domande su fatti recenti
    - Usare il tool CALCOLATRICE  → per domande matematiche
    - Rispondere da solo          → per domande di cultura generale

  L'agente NON sa a priori quale tool usare: lo decide il modello LLM
  in base al contenuto della domanda. Questo si chiama "tool use" o
  "function calling".

COME FUNZIONA IL TOOL USE (concetto chiave):
  1. Descrivi i tool disponibili all'LLM (nome, descrizione, parametri)
  2. Mandi la domanda dell'utente
  3. L'LLM risponde con:
       a) tool_use → "chiama questo tool con questi argomenti"
       b) text     → risponde direttamente senza tool
  4. Se ha scelto un tool: lo esegui, rimandi il risultato all'LLM
  5. L'LLM formula la risposta finale usando il risultato del tool

  Questo ciclo si chiama "agentic loop".

TOOL DISPONIBILI:
  ricerca_web(query)      -> DuckDuckGo (gratuito, no API key)
  calcolatrice(espressione) → esegue calcoli Python in sicurezza

INSTALLAZIONE:
  pip install groq requests duckduckgo-search

CONFIGURAZIONE:
  export GROQ_API_KEY="gsk_..."   # da https://console.groq.com
============================================================
"""

import os
import json
import math
from groq import Groq
```

```
# ddgs: pip install ddgs
try:
    from ddgs import DDGS
    DDGS_DISPONIBILE = True
except ImportError:
    DDGS_DISPONIBILE = False
    print("  ddgs non installato. Esegui: pip install ddgs")

# ------------------------------------------------------------
# CONFIGURAZIONE
# ------------------------------------------------------------
client = Groq(api_key=os.environ.get("GROQ_API_KEY"))
MODELLO = "llama-3.3-70b-versatile"

# ------------------------------------------------------------
# IMPLEMENTAZIONE DEI TOOL
# (queste sono le funzioni Python reali che verranno chiamate)
# ------------------------------------------------------------

def ricerca_web(query: str, max_risultati: int = 3) -> str:
    """
    Esegue una ricerca su DuckDuckGo e restituisce i risultati.
    Gratuito, nessuna API key necessaria.
    """
    if not DDGS_DISPONIBILE:
        return "Errore: libreria duckduckgo-search non installata."

    print(f"   Ricerca web: '{query}'")
    try:
        with DDGS() as ddgs:
            risultati = list(ddgs.text(query, max_results=max_risultati))

        if not risultati:
            return "Nessun risultato trovato."

        # Formatta i risultati come testo leggibile dall'LLM
        testo = ""
        for i, r in enumerate(risultati, 1):
            testo += f"[{i}] {r.get('title', '')}\n"
            testo += f"    {r.get('body', '')}\n"
            testo += f"    Fonte: {r.get('href', '')}\n\n"
        return testo

    except Exception as e:
        return f"Errore nella ricerca: {str(e)}"


def calcolatrice(espressione: str) -> str:
    """
    Valuta un'espressione matematica in modo sicuro.
    Usa solo funzioni math standard — non esegue codice arbitrario.
    """
    print(f"   Calcolo: '{espressione}'")
    try:
        # Namespace sicuro: solo funzioni matematiche consentite
        namespace_sicuro = {k: getattr(math, k) for k in dir(math) if not
```

```
k.startswith("_")}
        namespace_sicuro["__builtins__"] = {}  # blocca builtins pericolosi

        risultato = eval(espressione, namespace_sicuro)
        return f"Risultato: {risultato}"

    except ZeroDivisionError:
        return "Errore: divisione per zero."
    except Exception as e:
        return f"Errore nel calcolo: {str(e)}"


# ------------------------------------------------------------
# DEFINIZIONE DEI TOOL PER L'LLM
# (schema JSON che descrive i tool al modello)
# ------------------------------------------------------------
# NOTA DIDATTICA: questa è la "firma" che l'LLM legge per capire
# quando e come usare ogni tool. La qualità della descrizione
# influenza direttamente la bontà delle decisioni dell'agente.

TOOL_DEFINITIONS = [
    {
        "type": "function",
        "function": {
            "name": "ricerca_web",
            "description": (
                "Cerca informazioni aggiornate su internet tramite DuckDuckGo. "
                "Usare per: notizie recenti, eventi attuali, dati che potrebbero "
                "essere cambiati dopo il 2024, informazioni su
persone/aziende/prodotti specifici."
            ),
            "parameters": {
                "type": "object",
                "properties": {
                    "query": {
                        "type": "string",
                        "description": "La query di ricerca in linguaggio
naturale"
                    },
                    "max_risultati": {
                        "type": "integer",
                        "description": "Numero massimo di risultati (default 3)",
                        "default": 3
                    }
                },
                "required": ["query"]
            }
        }
    },
    {
        "type": "function",
        "function": {
            "name": "calcolatrice",
            "description": (
                "Esegue calcoli matematici, espressioni numeriche e funzioni
matematiche. "
                "Usare per: operazioni aritmetiche, calcoli con
```

```
radici/potenze/logaritmi, "
                "conversioni numeriche, qualsiasi espressione che richieda
precisione numerica."
            ),
            "parameters": {
                "type": "object",
                "properties": {
                    "espressione": {
                        "type": "string",
                        "description": (
                            "Espressione matematica Python valida. "
                            "Es: '2**10', 'sqrt(144)', 'log(1000, 10)', '(15 * 8)
/ 3.5'"
                        )
                    }
                },
                "required": ["espressione"]
            }
        }
    }
]

# Mappa nome tool → funzione Python corrispondente
TOOL_MAP = {
    "ricerca_web":  ricerca_web,
    "calcolatrice": calcolatrice,
}


# ------------------------------------------------------------
# ESECUTORE DEI TOOL
# ------------------------------------------------------------
def esegui_tool(nome: str, argomenti: dict) -> str:
    """
    Riceve il nome del tool scelto dall'LLM e i suoi argomenti,
    chiama la funzione Python corrispondente e restituisce il risultato.
    """
    if nome not in TOOL_MAP:
        return f"Errore: tool '{nome}' non trovato."

    funzione = TOOL_MAP[nome]
    return funzione(**argomenti)


# ------------------------------------------------------------
# AGENTIC LOOP
# ------------------------------------------------------------
def agente(domanda: str) -> str:
    """
    Implementa l'agentic loop completo:

    1. Manda la domanda all'LLM con la lista dei tool disponibili
    2. L'LLM decide:
         a) Chiamare un tool  → esegui il tool, rimanda il risultato
         b) Rispondere diretto → restituisci la risposta
    3. Risposta finale all'utente
```

```
    NOTA: in questo esempio il loop fa al massimo 1 chiamata tool
    (sufficiente per scopi didattici). Sistemi reali possono fare
    loop multipli (ReAct pattern).
    """

    print(f"\n{'='*55}")
    print(f"  Domanda: {domanda}")
    print(f"{'='*55}")

    messaggi = [
        {
            "role": "system",
            "content": (
                "Sei un assistente intelligente con accesso a tool esterni. "
                "Usa 'ricerca_web' per informazioni recenti o specifiche. "
                "Usa 'calcolatrice' per calcoli matematici. "
                "Se la domanda non richiede tool, rispondi direttamente "
                "con la tua conoscenza. Rispondi SEMPRE in italiano."
            )
        },
        {
            "role": "user",
            "content": domanda
        }
    ]

    # --- TURNO 1: l'LLM decide se usare un tool ---
    print("\n[AGENTE] Analizzo la domanda...")
    risposta = client.chat.completions.create(
        model=MODELLO,
        messages=messaggi,
        tools=TOOL_DEFINITIONS,
        tool_choice="auto"   # "auto" = l'LLM decide da solo
    )

    scelta = risposta.choices[0]
    motivo = scelta.finish_reason

    # --- CASO A: l'LLM ha scelto un tool ---
    if motivo == "tool_calls":
        tool_call = scelta.message.tool_calls[0]
        nome_tool = tool_call.function.name
        argomenti = json.loads(tool_call.function.arguments)

        print(f"[AGENTE] Ha scelto il tool: '{nome_tool}'")
        print(f"   Argomenti: {argomenti}")

        # Esegui il tool
        risultato_tool = esegui_tool(nome_tool, argomenti)
        print(f"   Tool eseguito, risultato ricevuto ({len(risultato_tool)}
chars)")

        # Aggiungi al contesto: risposta dell'LLM + risultato del tool
        messaggi.append(scelta.message)   # messaggio dell'LLM con tool_call
        messaggi.append({
            "role": "tool",
            "tool_call_id": tool_call.id,
```

```
            "content": risultato_tool
        })

        # --- TURNO 2: l'LLM formula la risposta finale con il risultato ---
        print("\n[AGENTE] Formulazione risposta finale...")
        risposta_finale = client.chat.completions.create(
            model=MODELLO,
            messages=messaggi,
        )
        testo_finale = risposta_finale.choices[0].message.content
        print(f"\nRISPOSTA (usando tool '{nome_tool}'):")

    # --- CASO B: l'LLM risponde con conoscenza interna ---
    else:
        testo_finale = scelta.message.content
        print(f"[AGENTE] Risponde con conoscenza interna (nessun tool
necessario)")
        print(f"\nRISPOSTA (conoscenza interna):")

    print(f"\n{testo_finale}")
    return testo_finale


# ------------------------------------------------------------
# ENTRY POINT — Test con domande di tipo diverso
# ------------------------------------------------------------
if __name__ == "__main__":

    domande_test = [
        # dovrebbe usare la CALCOLATRICE
        "Quanto fa la radice quadrata di 1764 diviso per 3.14?",

        # dovrebbe usare la RICERCA WEB
        "Quali sono le ultime notizie sull'intelligenza artificiale?",

        # dovrebbe rispondere con CONOSCENZA INTERNA
        "Cos'è il machine learning e quali sono i suoi principali paradigmi?",
    ]

    risposte = {}
    for domanda in domande_test:
        risposte[domanda] = agente(domanda)

    # Salva tutte le risposte
    with open("risposte_agente.txt", "w", encoding="utf-8") as f:
        for domanda, risposta in risposte.items():
            f.write(f"DOMANDA: {domanda}\n")
            f.write("-" * 50 + "\n")
            f.write(risposta + "\n\n")

    print("\n\nTutte le risposte salvate in 'risposte_agente.txt'")
```

Lanciando il programma otteniamo il seguente output a video:

```
======================================================
  Domanda: Quanto fa la radice quadrata di 1764 diviso per 3.14?
======================================================
```

```
[AGENTE] Analizzo la domanda...
[AGENTE] Ha scelto il tool: 'calcolatrice'
   Argomenti: {'espressione': 'sqrt(1764) / 3.14'}
   Calcolo: 'sqrt(1764) / 3.14'
   Tool eseguito, risultato ricevuto (29 chars)

[AGENTE] Formulazione risposta finale...

RISPOSTA (usando tool 'calcolatrice'):

Il risultato della radice quadrata di 1764 diviso per 3.14 è 13.375796178343949.

============================================================
  Domanda: Quali sono le ultime notizie sull'intelligenza artificiale?
============================================================

[AGENTE] Analizzo la domanda...
[AGENTE] Ha scelto il tool: 'ricerca_web'
   Argomenti: {'max_risultati': 5, 'query': 'ultime notizie intelligenza artificiale'}
   Ricerca web: 'ultime notizie intelligenza artificiale'
   Tool eseguito, risultato ricevuto (1669 chars)

[AGENTE] Formulazione risposta finale...

RISPOSTA (usando tool 'ricerca_web'):

Le ultime notizie sull'intelligenza artificiale sono disponibili su vari siti web e portali di
notizie. Alcune delle fonti più importanti includono:

*  Corriere.it: fornisce notizie e approfondimenti sull'intelligenza artificiale, inclusi
articoli su come l'intelligenza artificiale sta rivoluzionando vari settori economici.
* AI News: è un portale di notizie specializzato sull'intelligenza artificiale, che fornisce
notizie, approfondimenti e analisi sul mondo dell'AI.
* Tgcom24: fornisce notizie in tempo reale e approfondimenti sull'intelligenza artificiale.
*  Quotidiano.net: fornisce notizie e approfondimenti sull'intelligenza artificiale, inclusi
articoli esclusivi e interviste con esperti del settore.

È importante notare che le notizie sull'intelligenza artificiale sono in costante evoluzione,
quindi è importante consultare fonti aggiornate per rimanere informati sugli ultimi sviluppi
nel settore.

============================================================
  Domanda: Cos'è il machine learning e quali sono i suoi principali paradigmi?
============================================================

[AGENTE] Analizzo la domanda...
[AGENTE] Risponde con conoscenza interna (nessun tool necessario)

RISPOSTA (conoscenza interna):

Il machine learning è un campo dell'intelligenza artificiale che si concentra sullo sviluppo
di algoritmi e modelli in grado di apprendere da dati e migliorare le loro prestazioni su
compiti specifici senza essere esplicitamente programmato. I principali paradigmi del machine
learning includono:

1. **Apprendimento supervisionato** (Supervised Learning): in questo approccio, il modello
viene addestrato su dati etichettati, ovvero dati per cui è nota la risposta corretta.
L'obiettivo è di apprendere una funzione che mappa input in output in modo da poter fare
previsioni accurate su nuovi, dati non visti.
2. **Apprendimento non supervisionato** (Unsupervised Learning): in questo caso, il modello
viene addestrato su dati non etichettati e cerca di scoprire pattern, relazioni o strutture
nascoste all'interno dei dati.
3. **Apprendimento per rinforzo** (Reinforcement Learning): qui, l'agente apprende prendendo
decisioni in un ambiente e ricevendo una ricompensa o una penalità in base alle azioni
intraprese. L'obiettivo è di apprendere una politica che massimizzi la ricompensa cumulativa
nel lungo termine.

Questi paradigmi costituiscono le fondamenta del machine learning e vengono applicati in una
vasta gamma di campi, dalla visione artificiale al riconoscimento vocale, dalle reti neurali
profonde alla robotics.
```

```
Tutte le risposte salvate in 'risposte_agente.txt'
```

Rispetto alla versione del codice illustrato nei precedenti capitoli, in questo è stata inserita la feature della creazione di un file con il risultato delle risposte fornite dal programma (codice su sfondo grigio).

8.6 Riepilogo dei pattern studiati

In questo capitolo abbiamo visto tre pattern fondamentali per costruire sistemi agentici con Python:

- ➔ **Pipeline sequenziale**: gli agenti si passano l'output in catena. Semplice da implementare, adatto quando ogni passo dipende strettamente dal precedente. Tempo totale: somma dei tempi di ogni agente.
- ➔ **Agenti paralleli con message bus**: gli agenti girano in thread separati e comunicano attraverso una bacheca condivisa. `threading.Event` coordina le dipendenze. Tempo totale: determinato dal cammino critico, non dalla somma.
- ➔ **Tool calling (agentic loop)**: un singolo agente decide autonomamente quale strumento esterno invocare, lo esegue, e integra il risultato nella risposta finale. È la base dei sistemi AI moderni come i chatbot con accesso a internet.

Il concetto trasversale a tutti e tre i pattern è quello di astrazione: la funzione `chiama_llm` non cambia mai; cambiano solo il system_prompt, la struttura del flusso e gli strumenti disponibili. Questo principio, separare la logica di orchestrazione dai dettagli di implementazione, è ciò che rende i sistemi agentici scalabili e manutenibili.

NOTA: Ribadiamo che i provider LLM utilizzati in questo capitolo (*Groq*, *Google Gemini*, *Ollama*) sono intercambiabili: la logica degli agenti rimane identica indipendentemente dal provider scelto. Cambia solo la funzione `chiama_llm`, che adatta la chiamata al formato specifico dell'API. Questo è un esempio pratico del principio di sostituzione applicato alla programmazione con LLM.

9. Le interfacce grafiche

9.1 La libreria tkinter

Fino a questo punto i nostri programmi hanno comunicato con l'utente attraverso il terminale: con print stampiamo messaggi, con input riceviamo dati. Questo approccio, per quanto utile in fase di apprendimento, è ben lontano da come siamo abituati a interagire con un programma nella vita di tutti i giorni: finestre, pulsanti, caselle di testo, menu. Per costruire questo tipo di interfacce (dette GUI, Graphical User Interface) Python mette a disposizione una libreria chiamata tkinter.

Le origini di tkinter risalgono a Tcl/Tk, un toolkit grafico sviluppato da John Ousterhout intorno al 1988. Tkinter (contrazione di "Tk interface") nasce a metà degli anni '90 come "ponte" tra Python e Tk, e viene scelto per la sua semplicità e per la sua capacità di realizzare interfacce grafiche portabili su più sistemi operativi. Con il tempo tkinter è diventato parte della libreria standard di Python: questo significa che, a differenza di altre librerie grafiche, non è necessario installare nulla per iniziare a utilizzarla, basta importarla come un qualunque altro modulo. Proprio per la sua semplicità è considerata la libreria ideale per chi si affaccia per la prima volta alla programmazione delle interfacce grafiche, pur non rinunciando alla flessibilità necessaria per costruire applicazioni più complesse e personalizzate.

Nei paragrafi che seguono costruiremo, un pezzo alla volta, le conoscenze necessarie per creare una semplice interfaccia grafica: partiremo dalla finestra vuota, la arricchiremo di widget, impareremo a posizionarli con ordine, a farli reagire agli eventi generati dall'utente e infine scopriremo il widget Canvas, che ci permetterà di disegnare.

9.2 Finestre, frame e widget

9.2.1 Creare la prima finestra

Ogni applicazione tkinter nasce da un'unica istruzione: la creazione di un oggetto finestra. Scriviamo il seguente programma:

```
import tkinter as tk

# Creare una finestra
root = tk.Tk()
root.title("La mia GUI")

# Inizializza il ciclo di eventi
root.mainloop()
```

Lanciamo il programma: otteniamo una finestra vuota, con la sola barra del titolo "La mia

GUI" e i pulsanti standard di minimizzazione, ridimensionamento e chiusura.



Finestra vuota creata `con tk.Tk()`

Dall'analisi del codice e dell'output osserviamo che:

- `import tkinter as tk` importa la libreria e le assegna un *alias*, così come già avevamo fatto per altri moduli;
- `tk.Tk()` crea la finestra principale dell'applicazione, quella che in gergo viene chiamata *root* (radice), perché sarà il contenitore di partenza per tutti gli altri widget;
- `root.title(...)` permette di impostare il testo mostrato nella barra del titolo;
- `root.mainloop()` è l'istruzione più importante e, allo stesso tempo, meno intuitiva: avvia il ciclo di eventi (*event loop*) dell'applicazione, cioè tiene la finestra aperta e "in ascolto" di ciò che l'utente fa (click, digitazioni, ridimensionamenti) finché non viene chiusa. Se dimenticassimo questa istruzione, il programma terminerebbe subito, senza mostrare nulla a video.

9.2.2 Cosa sono i widget

I *widget* sono gli elementi che compongono l'interfaccia utente: *pulsanti, etichette, caselle di testo*, e così via. Ogni volta che vogliamo aggiungere un elemento visibile alla nostra finestra dobbiamo:

1. istanziare l'oggetto *widget*, specificando come primo parametro il contenitore (la *finestra* o il *frame*) in cui deve comparire;
2. renderlo visibile con uno dei metodi di posizionamento (di questo ci occuperemo nel paragrafo 9.3).

Vediamo alcuni tra i *widget* più comuni.

L'*etichetta* (`Label`)

```
label = tk.Label(root, text="Benvenuti a tkinter!")
label.pack()
```

`tk.Label(root, text="Testo")` crea un'etichetta contenente il testo specificato; `label.pack()` la rende visibile nella finestra, disponendola secondo le regole che vedremo tra poco.

Il *pulsante* (`Button`)

```
button = tk.Button(root, text="Cliccami")
button.pack()
```

`tk.Button(root, text="Cliccami")` crea un pulsante contenente il testo specificato;
`button.pack()` la rende visibile nella finestra.

La *casella di controllo* (`Checkbutton`)

```
check = tk.Checkbutton(root, text="checkButton")
check.pack()
```

`tk.Checkbutton(root, text="checkButton")` crea una casella di controllo denominata con il testo specificato;
`Checkbutton.pack()` la rende visibile nella finestra.

Il *pulsante di opzione* (`Radiobutton`)

```
radio1 = tk.Radiobutton(root, text="Opzione 1", value="Opzione1")
radio2 = tk.Radiobutton(root, text="Opzione 2", value="Opzione2")
radio1.pack()
radio2.pack()
```

`tk.Radiobutton(root, text="Opzione 1", value="Opzione1")` crea un pulsante di opzione denominato con il testo specificato e che assuma il valore specificato;
`radio.pack()` la rende visibile nella finestra.

I `radiobutton` vengono generalmente utilizzati in gruppo, per permettere all'utente di scegliere una sola opzione tra tante. La gestione completa di un gruppo di `radiobutton`, che richiede l'uso di una variabile condivisa tra i vari pulsanti, è un argomento che approfondiremo più avanti.

La *casella di testo* (`Entry`)

```
entry = tk.Entry(root)
entry.insert(0, "Inserisci del testo qui")
```

```
entry.pack()
```

`tk.Entry(root)` crea una *casella di testo* su una sola riga;
il metodo `.insert(posizione, testo)` permette di inserire un contenuto preimpostato, a partire dalla posizione indicata (0, in questo caso, cioè dall'inizio).
`entry.pack()` la rende visibile nella finestra.

L'*area di testo* (`Text`)

```
textArea = tk.Text(root, width=20, height=5)
textArea.pack()
```

`tk.Text(root, width=20, height=5)` crea un'*area di testo* di 5 righe e 20 caratteri;
`textarea.pack()` la rende visibile nella finestra.
A differenza di `Entry`, il *widget* `Text` consente di scrivere e visualizzare più righe di testo; i parametri `width` e `height` ne definiscono la larghezza e l'altezza, misurate per convenzione in caratteri e righe.

La *lista di voci* (`Listbox`)

```
Lb = tk.Listbox(root)
Lb.insert(1, "Opzione 1")
Lb.insert(2, "Opzione 2")
Lb.insert(3, "Opzione 3")
Lb.pack()
```

`tk.tk.Listbox(root)` crea una *lista di voci*;
il metodo `.insert()`, come nel caso della *casella di testo*, viene utilizzato per popolare la lista, indicando la posizione e il testo della voce da inserire.
`Lb.pack()` la rende visibile nella finestra.

La *casella combinata* (`Combobox`)

```
from tkinter import ttk

combo = ttk.Combobox(root, values=["Opzione 1", "Opzione 2",
"Opzione 3"])
combo.set("Seleziona un'opzione")
combo.pack()
```

Da notare che questo widget non appartiene al modulo tkinter "standard", ma al sottomodulo `ttk` (`Themed Tkinter`)[45], che fornisce *widget* dallo stile più moderno e coerente su diverse piattaforme rispetto a quelli tradizionali; per questo motivo va importato separatamente.
`tk.tk.Combobox(root, values=["Opzione 1", "Opzione 2", "Opzione 3"])` crea una *casella combinata* con 3 voci;

[45] https://docs.python.org/3/library/tkinter.ttk.html

il metodo `.set()` permette di visualizzare un suggerimento per l'utente.
`combo.pack()` la rende visibile nella finestra.

Mettendo insieme tutti i widget visti finora, uno sopra l'altro nella stessa finestra, otterremmo qualcosa di simile a questo:



Finestra con etichetta, pulsante, checkbutton, radiobutton, entry, textarea, listbox e combobox

9.2.3 Il frame: un contenitore di widget

Man mano che un'interfaccia cresce, il numero di widget aumenta e diventa difficile mantenere ordine tra di essi. Il frame (`tk.Frame`) risolve questo problema, si tratta di un'area rettangolare che funziona da contenitore, proprio come la finestra root, ma che può essere posizionata, spostata e organizzata all'interno della finestra stessa. Questa caratteristica permette di raggruppare logicamente i widget e di applicare a un intero gruppo uno stile o un layout comune.

```
frame = tk.Frame(root)
```

```
frame.pack()
```

Attenzione: se eseguiamo questo codice così com'è, non otteniamo alcuna finestra visibile a video. Questo capita perché un *frame*, da solo, è invisibile: esso acquista significato soltanto quando lo riempiamo con altri widget.

```
frame = tk.Frame(root)
frame.pack()

# Creare un'etichetta all'interno del frame
label = tk.Label(frame, text="Sono dentro al frame!")
label.pack()
```

Da notare la differenza rispetto agli esempi precedenti: il primo parametro passato a `tk.Label` non è più *root*, ma *frame*. È proprio questo parametro a stabilire "dentro quale contenitore" un widget deve comparire, e possiamo annidare frame dentro altri frame quante volte vogliamo, per costruire strutture via via più complesse.



Etichetta visualizzata all'interno di un frame

Nota Bene: per eseguire il codice devi inserire il codice per creare una finestra come indicato al 9.2.1.

9.2.4 Le finestre di dialogo

Oltre ai *widget* "permanenti", che restano visibili nella finestra, tkinter mette a disposizione delle finestre di dialogo (*dialog box*): piccole finestre temporanee che appaiono per comunicare un messaggio all'utente o per porgli una domanda. Sono contenute nel sottomodulo `messagebox`.

Tra le funzioni più comuni troviamo:

- `showinfo`, `showwarning`, `showerror`: mostrano rispettivamente un messaggio informativo, di avvertimento o di errore, con un solo pulsante OK;
- `askyesno`, `askokcancel`, `askretrycancel`: pongono una domanda all'utente, offrendo due possibili risposte (Sì/No, OK/Annulla, Riprova/Annulla).

Esaminiamo il seguente codice:

```
from tkinter import messagebox

def show_message():
    messagebox.showinfo("Messaggio", "Questo è un messaggio di
esempio.")

button = tk.Button(root, text="Mostra Messaggio", command=
show_message)
button.pack()
```

Qui incontriamo per la prima volta il parametro `command`, che collega la pressione del pulsante all'esecuzione della funzione `show_message`. Torneremo su questo meccanismo, in maniera più approfondita, nel paragrafo dedicato alla gestione degli eventi.

Lanciando il programma viene visualizzata la seguente finestra:



Finestra principale con il pulsante "Mostra Messaggio"

Cliccando sul pulsante "Mostra Messaggio", si apre una seconda finestra con l'icona di informazione, il messaggio impostato e il pulsante OK per chiuderla.



Finestra di dialogo informativa aperta dopo il click

9.3 Posizionare i widget

Sapere creare un *widget* non basta, dobbiamo anche indicare a tkinter dove e come

disporlo all'interno del suo contenitore. A questo scopo la libreria mette a disposizione tre diversi metodi, detti *geometry manager*:

- `.pack()`,
- `.grid()` e
- `.place()`.

Questi tre metodi non vanno mai mescolati all'interno dello stesso contenitore (root o *frame*), pena comportamenti imprevedibili dell'interfaccia.

9.3.1 Il metodo `.pack()`

Il metodo `.pack()` è il più semplice tra i tre e, come abbiamo già visto negli esempi precedenti, dispone i *widget* in sequenza, uno dopo l'altro. Se non specifichiamo alcun parametro, i *widget* vengono impilati in verticale, dall'alto verso il basso, nell'ordine in cui vengono "*pacchettizzati*".

```
button1 = tk.Button(frame, text="Primo")
button2 = tk.Button(frame, text="Secondo")

button1.pack()
button2.pack()
```

Per modificare l'allineamento predefinito possiamo usare il parametro `side`, che accetta i valori `left`, `right`, `top` (il default) e `bottom` dall'ovvio significato.

Lanciamo il seguente codice:

```
button1 = tk.Button(frame, text="Sinistra")
button2 = tk.Button(frame, text="Destra")

button1.pack(side="left")
button2.pack(side="right")
```

In questo caso i due pulsanti vengono disposti uno a sinistra e l'altro a destra del frame, anziché uno sopra l'altro.



Due pulsanti allineati orizzontalmente con `side="left"` e `side="right"`

9.3.2 Il metodo `.grid()`

Il metodo `.grid()` organizza i *widget* secondo una griglia composta da righe (`row`) e colonne (`column`), esattamente come le celle di una tabella.

Consideriamo le seguenti righe di codice:

```
label1 = tk.Label(frame, text="Riga 1, Colonna 1")
label2 = tk.Label(frame, text="Riga 1, Colonna 2")
label3 = tk.Label(frame, text="Riga 2, Colonna 2")

label1.grid(row=0, column=0)
label2.grid(row=0, column=1)
label3.grid(row=1, column=1)
```

Dall'analisi del codice osserviamo che la numerazione di righe e colonne parte da 0, così come già avevamo incontrato per gli indici delle stringhe (capitolo 7.1 Volume 1). Aggiungendo altre etichette con coordinate diverse possiamo costruire tabelle di qualsiasi dimensione, semplicemente indicando in quale cella (`row`, `column`) ciascun *widget* deve comparire.



Etichette disposte secondo righe e colonne con il metodo `.grid()`

9.3.3 Il metodo `.place()`

Il terzo metodo, `.place()`, è quello che offre il controllo più preciso in quanto permette di posizionare un *widget* specificando le coordinate esatte, in pixel, rispetto all'angolo in alto a sinistra del contenitore.

```
lbl2 = tk.Label(frame, text="x=50, y=50")
lbl2.place(x=50, y=50)
```

A differenza di `.pack()` e `.grid()`, che riorganizzano automaticamente il layout quando cambia la dimensione della finestra, un widget posizionato con `.place()` rimane fisso alle coordinate indicate, anche se la finestra viene ridimensionata o spostata sullo schermo. Proprio per questo motivo `.place()` va utilizzato con cautela: è comodo per posizionamenti puntuali, ma rende più difficile realizzare interfacce che si adattino bene a finestre di dimensioni diverse.



Etichetta posizionata alle coordinate esatte x=50, y=50 con `.place()`

9.3.4 Riempimento ed espansione: `fill` ed `expand`

Quando lo spazio disponibile nel contenitore è maggiore di quanto serva ai *widget*, possiamo scegliere se lasciarlo vuoto oppure farlo occupare dai widget stessi. A questo scopo, sia `.pack()` sia `.grid()` accettano i parametri `fill` ed `expand`:

- `fill` indica in quale direzione un *widget* deve espandersi per riempire lo spazio a disposizione (`"x"` orizzontalmente, `"y"` verticalmente, `"both"` in entrambe le direzioni);
- `expand`, un valore booleano, indica se il *widget* debba occupare, insieme agli altri, tutto lo spazio libero eventualmente disponibile nel contenitore.

```
frame = tk.Frame(root)
frame.pack(fill="both", expand=True)

btn1 = tk.Button(frame, text="Sinistra")
btn2 = tk.Button(frame, text="Destra")

btn1.pack(side="left", fill="y", expand=True)
btn2.pack(side="right", fill="x", expand=True)
```

Attenzione: perché questi parametri abbiano effetto è necessario che anche il contenitore (nel nostro caso il frame) sia stato "*impacchettato*" indicando `fill` ed `expand`; in caso contrario i *widget* al suo interno non avrebbero comunque spazio extra da riempire.



I due pulsanti si espandono per riempire tutto lo spazio disponibile

9.3.5 Ridimensionare i widget

Oltre a decidere dove posizionare un *widget*, possiamo anche stabilirne la dimensione, tramite i parametri `width` e `height`. Questo può avvenire in due momenti diversi:

1) al momento della creazione del widget:

```
button1 = tk.Button(frame, text="Button1", width=20, height=5)
button1.pack()
```

2) in un secondo momento, richiamando il metodo `.config()` sul *widget* già creato:

```
button2 = tk.Button(frame, text="Button2")
button2.pack()
button2.config(width=10, height=10)
```

Il metodo `.config()` è molto utile in generale: ci permette di modificare, in qualsiasi momento del programma, una o più proprietà di un *widget* già esistente (colore, testo, dimensione, e così via), senza doverlo ricreare da capo.
Da notare, in questo caso, che non è necessario indicare che il frame debba espandersi: al contrario di quanto visto con `fill` ed `expand`, qui è la dimensione del *widget* stesso a essere fissata esplicitamente.



Due pulsanti ridimensionati tramite i parametri width e height

9.4 La gestione degli eventi

9.4.1 Perché servono gli eventi

Fino a questo momento abbiamo imparato a costruire interfacce statiche: *finestre*, *frame* e *widget* vengono disegnati a video, ma non reagiscono in alcun modo a ciò che l'utente fa. Un'interfaccia grafica, però, diventa davvero utile solo quando è in grado di rispondere alle azioni dell'utente: ad esempio un click del mouse, la pressione di un tasto, lo spostamento del puntatore. Il paradigma di programmazione che si occupa di gestire questo tipo di interazioni si chiama *programmazione guidata dagli eventi* (*event-driven programming*).

In questo modello, a ogni interazione dell'utente viene associato un evento: un avvenimento asincrono, cioè che può manifestarsi in qualsiasi momento, senza rispettare un ordine o un tempo prestabilito (a differenza, per esempio, delle istruzioni di un ciclo for, che vengono eseguite una dopo l'altra in sequenza, secondo un ordine ben definito).

Il processo si articola in due fasi distinte:

- il riconoscimento dell'evento, cioè il momento in cui il sistema si accorge che l'utente ha compiuto un'azione. Questo compito è affidato al sistema che gestisce gli eventi (nel nostro caso, il ciclo avviato da `root.mainloop()`);
- la gestione dell'evento, cioè la scelta di cosa fare in risposta a quell'azione. Questo compito è affidato al programmatore, che deve scrivere una funzione, detta *gestore di eventi* (*event handler*), da associare all'evento stesso.

Esistono moltissimi tipi di evento, tra i più comuni troviamo:

- `<ButtonPress-1>` / `<ButtonRelease-1>`: pressione o rilascio del tasto sinistro del mouse (il tasto centrale corrisponde al 2, il tasto destro al 3);
- `<B1-Motion>`: movimento del mouse con il tasto sinistro premuto;

- ➔ `<Double-Button-1>`: doppio clic con il tasto sinistro (per il triplo clic il prefisso è Triple);
- ➔ `<Enter>` / `<Leave>`: il puntatore del mouse entra o esce dall'area visibile di un *widget*;
- ➔ `<KeyPress>` / `<KeyRelease>`: pressione o rilascio di un tasto qualsiasi della tastiera;
- ➔ `<KeyPress-nometasto>` / `<KeyRelease-nometasto>`: pressione o rilascio di un tasto specifico, per esempio `Return`, `Escape`, `BackSpace`, `Delete`, `Home`, `End`, `Tab`, `F1`, `F2`, ...

9.4.2 Il parametro `command`

Il modo più semplice per gestire un evento lo abbiamo già incontrato senza saperlo: il parametro `command` della classe `Button`, utilizzato nel paragrafo 9.2.4 per collegare un pulsante a una finestra di dialogo.

```
def onButtonClick():
    print("Hai cliccato il pulsante!")

button = tk.Button(root, text="Cliccami", command=onButtonClick)
button.pack()
```

Dall'analisi del codice osserviamo che:

- ➔ `onButtonClick()` è una funzione *user-defined*, scritta dal programmatore, che stabilisce cosa fare quando l'utente clicca sul pulsante;
- ➔ il parametro `command` non contiene una chiamata alla funzione (attenzione a non scrivere `command=onButtonClick()`, con le parentesi), ma il riferimento alla funzione stessa, sarà tkinter a occuparsi di richiamarla al momento opportuno, cioè quando si verifica l'evento.

Lanciando il programma e cliccando ripetutamente sul pulsante, vediamo comparire sul terminale una riga "Hai cliccato il pulsante!" per ogni click effettuato.



Finestra con il pulsante "Cliccami"

```
terminale
$ python3 slide9.py
Hai cliccato il pulsante!
Hai cliccato il pulsante!
```

Terminale con i messaggi stampati a ogni click

Il parametro `command` è molto semplice da utilizzare ma è limitato in quanto:

- ➔ è disponibile solo per alcuni widget (come `Button`, `Checkbutton`, `Radiobutton`)
- ➔ gestisce un solo tipo di evento, quello "naturale" per il widget in questione (il click, nel caso del pulsante).

Per gestire eventi diversi, o per associare un evento a un qualsiasi *widget* (finestra compresa), dobbiamo ricorrere a un meccanismo più generale: il metodo `.bind()`.

9.4.3 Il metodo .bind()

Il metodo `.bind()` permette di "legare" (*bind*, in inglese) un qualsiasi tipo di evento a una funzione di gestione, applicandolo direttamente al *widget* interessato.

Consideriamo il seguente codice:

```
def onKeyPress(event):
    print(f"Hai premuto il tasto {event.keysym}")

root.bind("<KeyPress>", onKeyPress)
```

Dall'analisi del codice osserviamo che:

- ➔ il metodo `.bind()` viene richiamato dal widget che deve "ascoltare" l'evento (in questo caso `root`, cioè l'intera finestra);
- ➔ il primo parametro è una stringa che identifica il tipo di evento da gestire, tra quelli visti nella tabella del paragrafo precedente;
- ➔ il secondo parametro è la funzione di gestione, che deve obbligatoriamente accettare un parametro, di consuetudine chiamato `event`: si tratta di un oggetto che tkinter costruisce automaticamente e che contiene tutte le informazioni relative all'evento verificatosi. `event` è una classe di Tkinter[46]
- ➔ l'attributo `event.keysym` restituisce il nome simbolico del tasto premuto.

Lanciamo il programma, cliccando su un tasto qualsiasi della tastiera, viene stampato sul terminale il nome del tasto premuto.

[46] https://docs.python.org/3/library/tkinter.html#tkinter.Event



Finestra vuota in ascolto degli eventi da tastiera

```
terminale
$ python3 slide13.py
Hai premuto il tasto q
Hai premuto il tasto 7
```

Terminale con il nome dei tasti premuti dall'utente

Lo stesso meccanismo può essere applicato agli eventi del mouse. Riprendiamo, ad esempio, il caso del *click*, e lo andiamo a gestire con `.bind()` anziché con `command`:

```
def onWidgetClick(event):
    x, y = event.x, event.y
    print(f"Hai cliccato in posizione ({x}, {y})")

widget.bind("<Button-1>", onWidgetClick)
```

In questo caso l'oggetto `event` contiene, tra le altre informazioni, gli attributi `x` e `y`, cioè le coordinate del punto in cui si è verificato il *click*, relative al widget su cui è avvenuto.

9.4.4 Rimuovere un evento: `.unbind()`

Così come è possibile associare un gestore a un evento, è altrettanto possibile rimuoverlo, quando non serve più, tramite il metodo `.unbind()`:

```
    root.unbind("<KeyPress>", onKeyPress)
```

Da quel momento in poi, il widget smetterà di reagire a quel particolare tipo di evento.

9.5 Canvas: disegnare oggetti grafici

9.5.1 Cos'è il Canvas

Tra tutti i *widget* di tkinter, il `Canvas` merita un discorso a parte. I *widget* che abbiamo

incontrato (`Label`, `Button`, `Entry`...) sono elementi "preconfezionati", pronti all'uso, `Canvas` è invece un'area rettangolare completamente vuota, destinata a essere riempita con figure, linee, testo e persino altri `widget`, disegnati "a mano" dal programmatore. È lo strumento ideale ogni volta che abbiamo bisogno di realizzare grafica personalizzata come diagrammi, piccoli giochi, visualizzazioni di dati, e così via.

Analizziamo questo codice:

```
import tkinter as tk

root = tk.Tk()
root.title("Slides Canvas")

canvas = tk.Canvas(root, width=400, height=300)
canvas.pack()

root.mainloop()
```

Da notare che `tk.Canvas` si comporta, dal punto di vista della sua creazione e del suo posizionamento, come un qualsiasi altro widget: viene istanziato passando come primo parametro il contenitore (`root`) e reso visibile con uno dei metodi di posizionamento che già conosciamo, `.pack()` in questo caso. I parametri `width` e `height` ne definiscono, questa volta in pixel, la larghezza e l'altezza.
Se lanciamo il programma così com'è, otteniamo una finestra con un'area grigia, vuota, il nostro `Canvas` è pronto, ma ancora privo di contenuto.

9.5.2 Disegnare linee

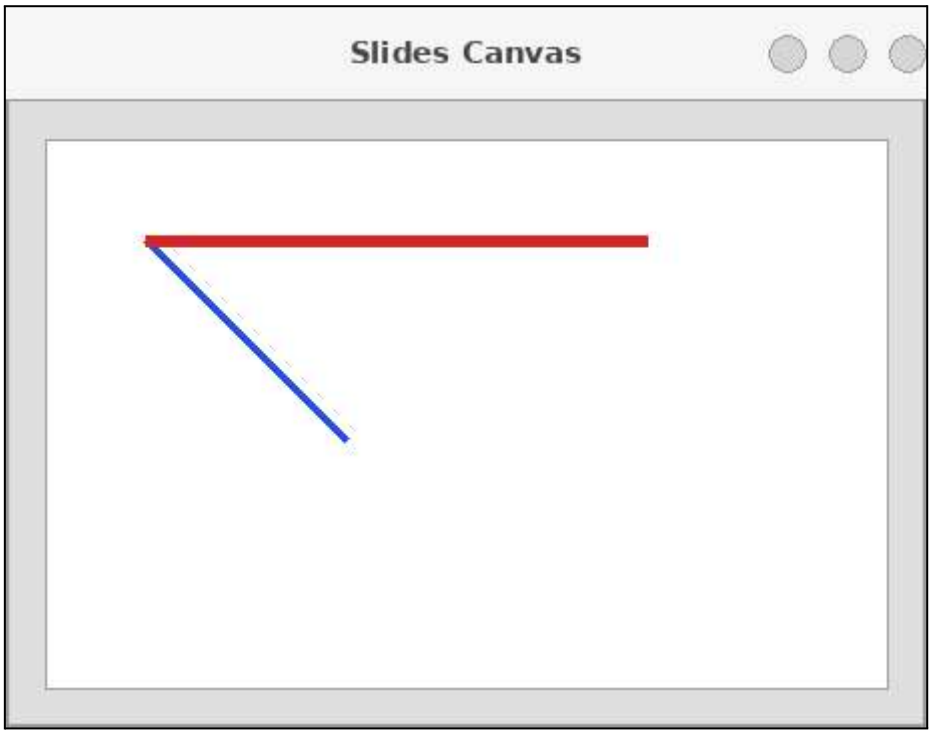
Per disegnare una linea su Canvas si utilizza il metodo `.create_line()`, specificando le coordinate *(x1, y1)* e *(x2, y2)* dei due punti estremi del segmento.

```
line = canvas.create_line(50, 50, 150, 150, fill="blue", width=2)
canvas.create_line(50, 50, 300, 50, fill="red", width=5)
```

I parametri `fill` e `width`, che ritroveremo anche per gli altri metodi di disegno, permettono rispettivamente di impostare il colore e lo spessore del tratto.

Osserviamo un dettaglio importante nel sistema di coordinate di `Canvas`: a differenza del piano cartesiano l'origine *(0, 0)* si trova nell'angolo in alto a sinistra dell'area di disegno, e l'asse delle y cresce verso il basso, anziché verso l'alto. È un dettaglio a cui prestare attenzione ogni volta che calcoliamo delle coordinate.

Da notare, inoltre, che il metodo `.create_line()` restituisce un valore, nell'esempio assegnato alla variabile `line`, si tratta di un identificatore univoco dell'oggetto appena disegnato, che ci servirà per poter apportare modifiche o cancellazioni.



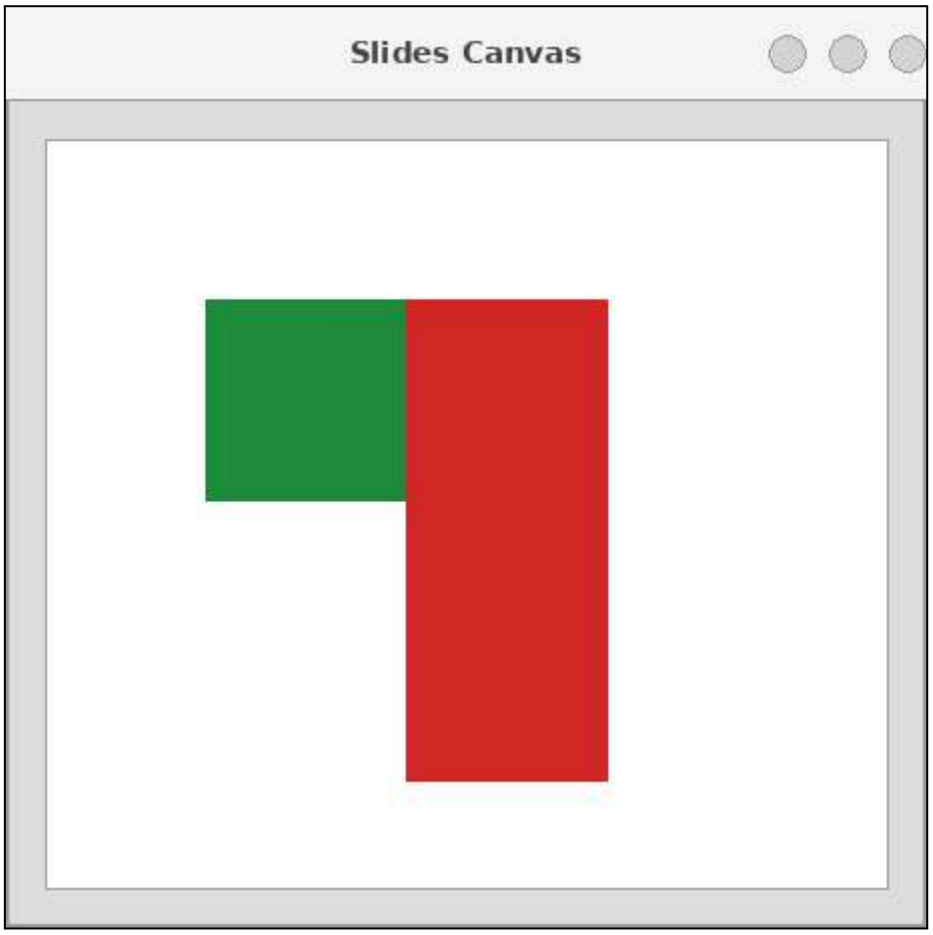
Due linee disegnate sul Canvas con `.create_line()`

9.5.3 Disegnare rettangoli e ovali

In maniera analoga possiamo disegnare rettangoli, con il metodo `.create_rectangle()`, specificando le coordinate di due vertici opposti:

```
rectangle = canvas.create_rectangle(100, 100, 250, 200, fill=
"green")
canvas.create_rectangle(200, 100, 300, 400, fill="red")
```

Lanciando questo codice notiamo un ulteriore dettaglio interessante: se le figure disegnate si sovrappongono, l'ultima a essere disegnata compare "sopra" alle precedenti. Provando a invertire l'ordine delle due istruzioni `.create_rectangle()`, infatti, cambia anche quale dei due rettangoli risulta visibile nella zona di sovrapposizione: la logica è la medesima con cui, dipingendo su un foglio, l'ultimo colore steso copre quelli sottostanti.

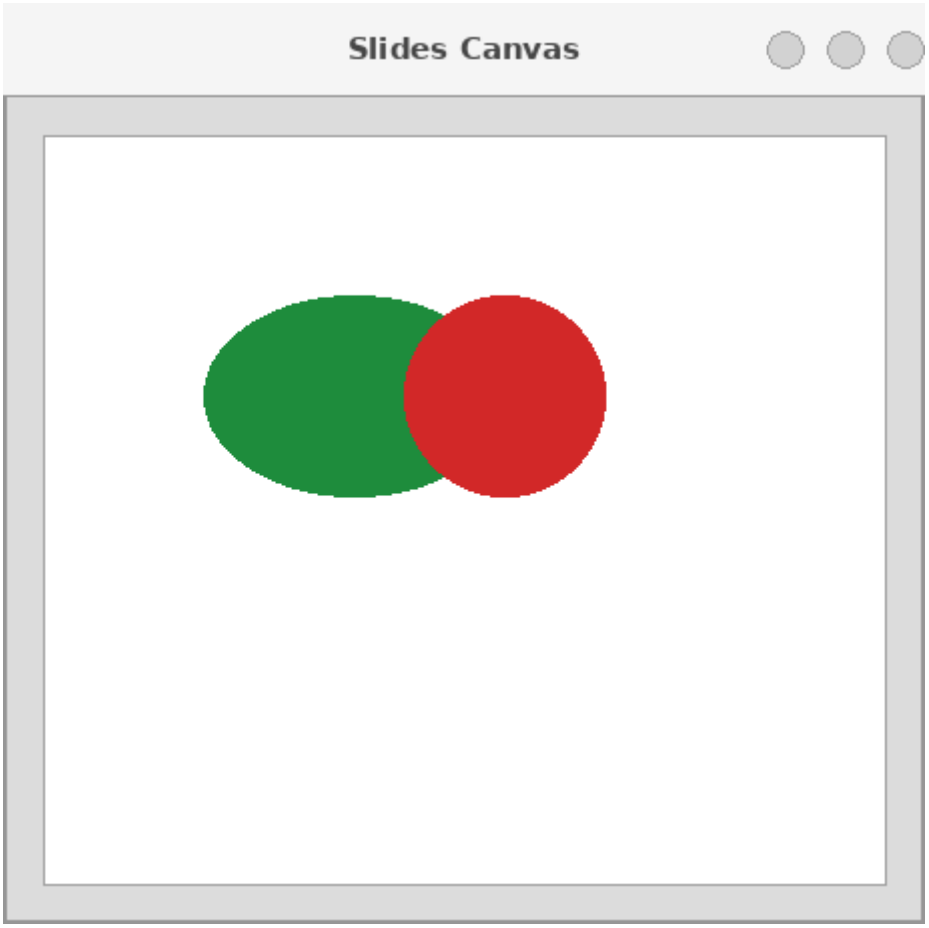


Due rettangoli sovrapposti: quello disegnato per ultimo compare sopra

Per disegnare ovali si utilizza invece il metodo `.create_oval()`, che funziona in maniera del tutto simile a `.create_rectangle()`: le coordinate specificate sono i vertici del rettangolo all'interno del quale l'ovale risulta iscritto.

```
canvas.create_oval(100, 100, 250, 200, fill="green")
canvas.create_oval(200, 100, 300, 400, fill="red")
```

Da questa osservazione deriva una conseguenza utile per disegnare una circonferenza, e non una generica ellisse, è sufficiente indicare le coordinate di un rettangolo che sia in realtà un quadrato, cioè con base e altezza uguali.



Un ovale verde e un cerchio rosso, disegnati con .create_oval()

9.5.4 Inserire del testo

Il metodo `.create_text()` permette di scrivere del testo sul `Canvas`, a partire da una posizione *(x, y)*:

```
text = canvas.create_text(200, 50, text="Hello, Canvas!", font=
("Arial", 12))
canvas.create_text(200, 150, text="Testo in ROSSO", font=("Arial",
12), fill="red")
```

Il parametro `font` accetta una *tupla* composta dal nome del carattere e dalla sua dimensione, mentre `fill`, come già visto per le altre figure, ne definisce il colore.



Testo nero e testo rosso scritti sul Canvas con `.create_text()`

9.5.5 Modificare ed eliminare gli oggetti

Ogni oggetto disegnato sul `Canvas`, una linea, un rettangolo, un ovale, del testo, può essere in seguito spostato o rimosso, a patto di averne conservato l'identificatore (l'oggetto) restituito al momento della creazione, come abbiamo fatto ad esempio con la variabile line.

```
# Spostare un oggetto sul Canvas
canvas.move(oggetto1, 50, 0)

# Eliminare un oggetto dal Canvas
canvas.delete(oggetto2)
```

Il metodo `.move(oggetto, dx, dy)` sposta l'oggetto indicato di `dx` pixel in orizzontale e `dy` pixel in verticale, rispetto alla posizione attuale (non alle coordinate assolute); il metodo .delete(oggetto) lo rimuove definitivamente dalla scena. Entrambi i metodi accettano come argomento l'identificatore dell'oggetto, indipendentemente dal fatto che si tratti di una linea, di un ovale o di qualsiasi altra figura.

9.5.6 Interagire con gli eventi sul Canvas

Le nozioni di gestione degli eventi introdotte nel paragrafo 9.4 trovano nel Canvas una delle loro applicazioni più naturali: possiamo, ad esempio, disegnare una figura ogni volta che l'utente clicca all'interno dell'area di disegno.

```
def onCanvasClick(event):
    x, y = event.x, event.y
    canvas.create_oval(x-5, y-5, x+5, y+5, fill="black")

canvas.bind("<Button-1>", onCanvasClick)
```

Dall'analisi del codice osserviamo che:

- la funzione `onCanvasClick(event)` legge le coordinate del click dagli attributi `event.x` e `event.y`;
- a partire da queste coordinate viene disegnato un piccolo ovale, di fatto un cerchietto avendo base e altezza uguali, centrato nel punto cliccato;
- il `.bind()` viene applicato al `Canvas` stesso, e non alla finestra `root`, così da intercettare esclusivamente i *click* effettuati all'interno dell'area di disegno.

Mettendo insieme quanto visto finora possiamo costruire un piccolo programma completo:

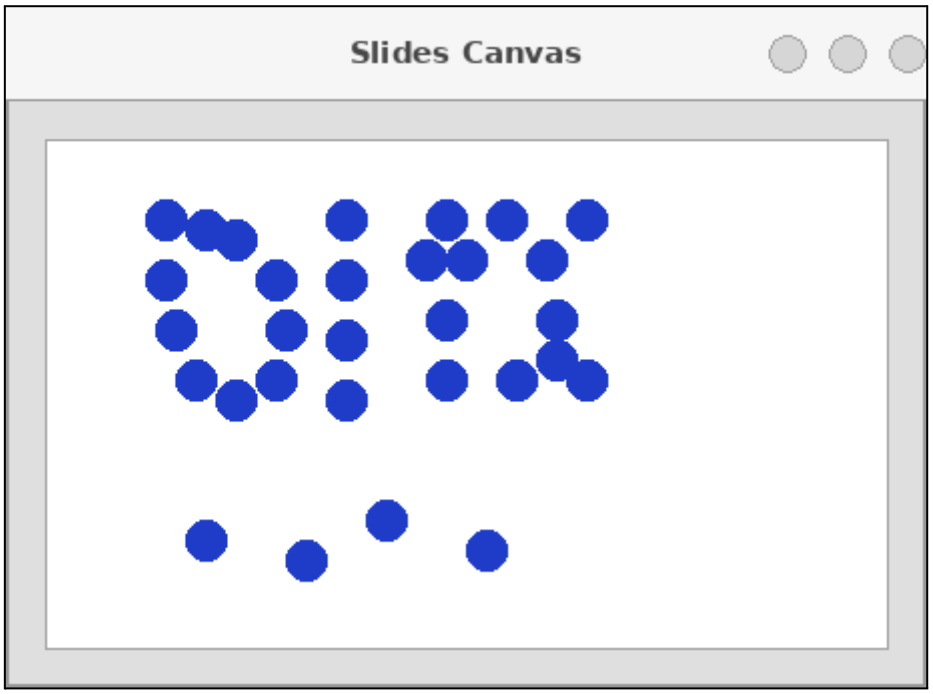
```
import tkinter as tk

def drawCircle(event):
    x, y = event.x, event.y
    canvas.create_oval(x - 10, y - 10, x + 10, y + 10,
fill="blue")

root = tk.Tk()
root.title("Semplice app Canvas")
canvas = tk.Canvas(root, width=400, height=300)
canvas.pack()

canvas.bind("<Button-1>", drawCircle)
root.mainloop()
```

Lanciando il programma e cliccando ripetutamente in punti diversi del Canvas, possiamo disegnare tanti piccoli cerchi blu quanti sono i click effettuati, arrivando a comporre a piacimento anche disegni, scritte o composizioni più elaborate.



Cerchi blu disegnati sul Canvas a ogni click del mouse

Come esercizio, il lettore provi a modificare il programma in modo che:

- il colore dei cerchi cambi in base al tasto del mouse utilizzato (per esempio nero per il tasto sinistro, rosso per il tasto destro, associando due diversi gestori con `.bind()`);
- la dimensione dei cerchi dipenda da una variabile impostata all'inizio del programma, anziché essere fissata direttamente nel codice della funzione;
- venga aggiunto un pulsante "Cancella tutto" che, tramite `command`, richiami il metodo `canvas.delete("all")` per svuotare completamente il `Canvas`.

9.6 Creare e gestire i menu

9.6.1 La barra dei menu

Quasi tutte le applicazioni desktop che utilizziamo quotidianamente possiedono una barra dei menu: quella striscia orizzontale, posta subito sotto la barra del titolo, che raccoglie in voci come *File, Modifica, ?* le funzionalità principali del programma.
Anche tkinter mette a disposizione un widget dedicato a questo scopo, chiamato *Menu*, con cui possiamo costruire sia la barra dei menu di una finestra sia i cosiddetti *menu contestuali*, cioè quelli che si aprono cliccando con il tasto destro del mouse.

A differenza dei *widget* che abbiamo incontrato finora, un oggetto `Menu` non viene mai reso visibile con `.pack()`, `.grid()` o `.place()`, per comparire come barra dei menu di una finestra, viene invece assegnato alla finestra stessa tramite il metodo `.config()`.

```
import tkinter as tk
```

```
root = tk.Tk()
root.title("La mia GUI")

# Creare la barra dei menu
menubar = tk.Menu(root)
root.config(menu=menubar)

root.mainloop()
```

Lanciamo il programma otteniamo una finestra vuota. Pur avendo creato l'oggetto menu, la finestra non presenta ancora nessuna voce visibile, perché non abbiamo aggiunto alcun contenuto alla barra.

Dall'analisi del codice osserviamo che:

- `tk.Menu(root)` crea l'oggetto menu, associandolo alla finestra root;
- `root.config(menu=menubar)` è l'istruzione che rende effettivamente visibile la barra dei menu, agganciandola alla finestra; da questo momento in poi `root` possiede una barra dei menu, seppure ancora vuota.

9.6.2 Aggiungere voci con `.add_cascade()` e `.add_command()`

Per popolare la barra dei menu dobbiamo creare, per ciascuna voce principale (*File, Modifica,* ...), un secondo oggetto `Menu`, che rappresenta il sottomenu a tendina a essa associato, e collegarlo alla barra con il metodo `.add_cascade()`.

```
file_menu = tk.Menu(menubar, tearoff=0)
menubar.add_cascade(label="File", menu=file_menu)

help_menu = tk.Menu(menubar, tearoff=0)
menubar.add_cascade(label="?", menu=help_menu)
```

Lanciando il codice otteniamo in output la seguente finestra:



Barra dei menu con le voci File e ?

Dall'analisi del codice osserviamo che:

- `tk.Menu(menubar, tearoff=0)` crea il sottomenu a tendina, questa volta figlio della barra dei menu e non della finestra;
- il parametro `tearoff=0` disattiva una caratteristica storica di Tk, ereditata da Tcl/Tk: senza di essa, la prima riga di ogni tendina sarebbe una linea tratteggiata che permette di "staccare" il menu trasformandolo in una piccola finestra indipendente. Si tratta di un comportamento ormai desueto, che nella quasi totalità dei casi conviene disattivare;
- `.add_cascade(label=..., menu=...)` collega il sottomenu alla barra principale, associandogli il testo che comparirà come voce cliccabile.

Una volta creati i sottomenu possiamo popolarli sia con

- `.add_command()`, il quale si comporta in maniera del tutto analoga al parametro `command` già visto per i pulsanti,
- `.add_separator()`, che inserisce una semplice linea divisoria tra le voci.

Analizziamo le segunti righe di codice:

```
def nuovo_file():
    print("Hai scelto Nuovo")

file_menu.add_command(label="Nuovo", command=nuovo_file)
file_menu.add_separator()
file_menu.add_command(label="Esci", command=root.quit)
```

Assemblando tutte le righe di codice viste sino a ora e lanciandole otteniamo a video la seguente finestra:



Il menu File aperto, con le voci Nuovo, un separatore ed Esci

Cliccando sulla voce "File" della barra, si apre la tendina con le voci configurate. Cliccando su "Nuovo" viene eseguita la funzione `nuovo_file()`, mentre cliccando su "Esci" viene richiamato `root.quit()`, che termina il ciclo di eventi e chiude l'applicazione.

9.6.3 Scorciatoie da tastiera

Molti programmi permettono di richiamare le voci di menu più utilizzate anche tramite una combinazione di tasti. Questa combinazione viene mostrata accanto alla voce stessa (per esempio Ctrl+N per "Nuovo file").
In tkinter questo si ottiene in due passaggi distinti, ed è importante non confonderli.

Esaminiamo le seguenti righe di codice:

```
file_menu.add_command(label="Nuovo", command=nuovo_file,
accelerator="Ctrl+N")
root.bind_all("<Control-n>", lambda event: nuovo_file())
```

Dall'analisi del codice osserviamo che:

- il parametro `accelerator` si limita a scrivere la scorciatoia accanto alla voce di menu, come semplice indicazione testuale per l'utente, da solo non attiva alcun comportamento;
- per far sì che la combinazione di tasti funzioni realmente, dobbiamo associarla esplicitamente a un evento con il metodo `.bind_all()`, che si comporta come il `.bind()` già incontrato nel paragrafo 9.4, con la differenza che la scorciatoia viene riconosciuta in qualsiasi punto in cui l'applicazione abbia il focus, e non solo su un singolo *widget*;
- utilizziamo qui una *funzione lambda*, cioè una funzione anonima, in quanto `.bind_all()` richiede una funzione che accetti un parametro `event`, mentre la

funzione `nuovo_file()` non ne ha bisogno: la lambda fa da "adattatore" fra le due.

9.6.4 I menu contestuali (*popup menu*)

Oltre alla barra dei menu, tkinter permette di realizzare *menu contestuali*, cioè piccoli menu a comparsa che si aprono, generalmente, in corrispondenza del clic con il tasto destro del mouse, mostrando solo le azioni pertinenti al punto in cui si trova il cursore.
La costruzione delle voci avviene esattamente come già visto, cambia soltanto il modo in cui il menu viene mostrato a video.

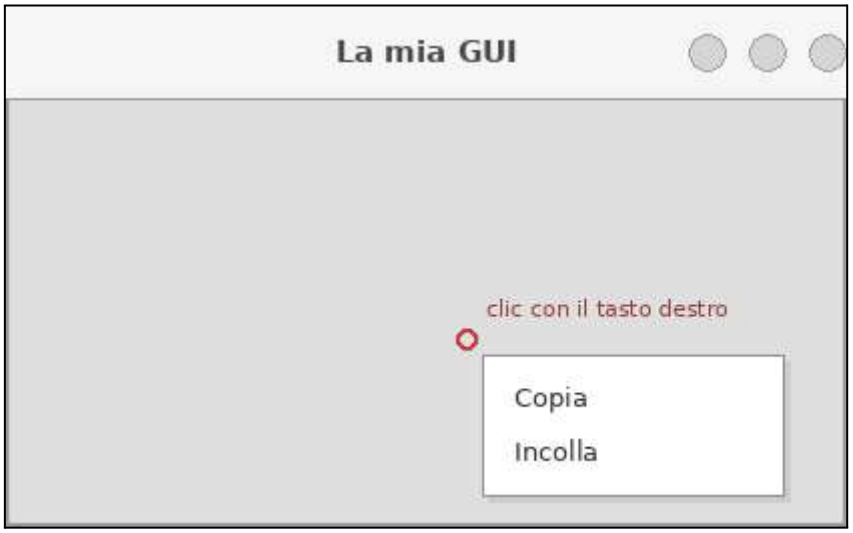
Esaminiamo le seguenti righe di codice (da integrare nell'applicazione completa):

```
popup = tk.Menu(root, tearoff=0)
popup.add_command(label="Copia")
popup.add_command(label="Incolla")

def show_popup(event):
    popup.tk_popup(event.x_root, event.y_root)

root.bind("<Button-3>", show_popup)
```

Se lanciamo il codice completo otterremo la seguente finestra a video:



Menu contestuale aperto in corrispondenza del clic con il tasto destro

Dall'analisi del codice osserviamo che:

- `<Button-3>` è l'evento associato alla pressione del tasto destro del mouse, coerentemente con quanto visto nella tabella degli eventi del paragrafo 9.4.1;
- il metodo `.tk_popup(x, y)` mostra il menu contestuale, posizionandolo alle coordinate indicate; a differenza degli attributi `event.x` e `event.y`, che come sappiamo sono relativi al *widget* su cui si è verificato l'evento, qui dobbiamo utilizzare `event.x_root` e `event.y_root`, cioè le coordinate assolute rispetto

allo schermo, poiché il menu contestuale è una finestra indipendente e non un disegno all'interno del widget.

9.6.5 Un esempio completo

Mettendo insieme quanto visto in questo paragrafo possiamo costruire un piccolo editor di testo, dotato di una barra dei menu con le voci "Nuovo" (anche tramite scorciatoia da tastiera) ed "Esci", oltre a un menu di aiuto che mostra una finestra informativa.

Ecco il codice completo dell'app:

```
import tkinter as tk
from tkinter import messagebox

def nuovo_file():
    testo.delete("1.0", tk.END)

def mostra_info():
    messagebox.showinfo("Informazioni",
        "Piccolo editor didattico realizzato con tkinter.")

root = tk.Tk()
root.title("Editor di testo")

testo = tk.Text(root, width=50, height=15)
testo.pack()

menubar = tk.Menu(root)
root.config(menu=menubar)

file_menu = tk.Menu(menubar, tearoff=0)
menubar.add_cascade(label="File", menu=file_menu)
file_menu.add_command(label="Nuovo", command=nuovo_file,
accelerator="Ctrl+N")
file_menu.add_separator()
file_menu.add_command(label="Esci", command=root.quit)

help_menu = tk.Menu(menubar, tearoff=0)
menubar.add_cascade(label="?", menu=help_menu)
help_menu.add_command(label="Informazioni", command=mostra_info)

root.bind_all("<Control-n>", lambda event: nuovo_file())

root.mainloop()
```

Se lanciamo il programma otteniamo a video la seguente schermata:



Il piccolo editor di testo completo di barra dei menu

Dall'analisi del codice osserviamo che:

- `testo.delete("1.0", tk.END)` svuota il *widget* . `"1.0"` indica la prima riga, primo carattere (il formato utilizzato da `Text` per individuare una posizione è `riga.colonna`, con la numerazione delle righe che, a differenza di quanto avevamo osservato per liste e stringhe, parte da 1 e non da 0), mentre `tk.END` è una costante di tkinter che rappresenta la fine del contenuto del *widget*.

Come esercizio, il lettore provi a:

- aggiungere al menu File una voce "Apri..." che, tramite il sottomodulo `tkinter.filedialog` e la sua funzione `askopenfilename()`, permetta di scegliere un file esistente dal proprio computer;
- aggiungere un secondo livello di sottomenu (un cascade dentro un altro cascade), per esempio una voce "Recenti" all'interno del menu File, che elenchi alcuni nomi degli ultimi files aperti ;
- collegare le voci "Copia" e "Incolla" del menu contestuale alle rispettive funzionalità del *widget* `Text`, richiamando i metodi `testo.event_generate("<<Copy>>")` e `testo.event_generate("<<Paste>>")`.

9.7. Un tocco di modernità: la libreria customtkinter

9.7.1 Perché customtkinter

I *widget* di tkinter che abbiamo utilizzato in questo capitolo hanno un pregio innegabile, la semplicità, ma anche un difetto altrettanto evidente: il loro aspetto grafico è rimasto

pressoché invariato negli ultimi vent'anni, e risulta ormai lontano dagli standard estetici a cui siamo abituati (bordi arrotondati, temi scuri, transizioni morbide al passaggio del mouse).

Per questo motivo, nel corso del tempo, la comunità Python ha sviluppato diverse librerie "di terze parti" costruite sopra tkinter, con l'obiettivo di modernizzarne l'aspetto senza stravolgerne il funzionamento di base.
Tra queste, una delle più diffuse è *customtkinter*[47].

Tutto ciò che abbiamo imparato in questo capitolo, dalla creazione della finestra alla gestione degli eventi, passando per i metodi di posizionamento .pack(), `.grid()` e `.place()`, resta valido: customtkinter non sostituisce tkinter, ma lo estende, offrendo una serie di *widget* moderni, riconoscibili dal prefisso `CTk`, che possiamo utilizzare al posto di quelli classici.

9.7.2 Installazione

A differenza di tkinter, che come sappiamo fa parte della libreria standard di Python, *customtkinter* è una libreria esterna e va installata separatamente, utilizzando il gestore di pacchetti pip, direttamente da terminale:

```
pip install customtkinter
```

Una volta completata l'installazione, la libreria è pronta per essere importata nei nostri programmi, esattamente come qualunque altro modulo.

9.7.3 La prima finestra moderna

Scriviamo il seguente programma:

```
import customtkinter as ctk

# Impostiamo aspetto e tema colore prima di creare la finestra
ctk.set_appearance_mode("dark")  # "light", "dark" oppure "system"
ctk.set_default_color_theme("blue") # "blue", "green", "dark-blue"

root = ctk.CTk()
root.title("La mia GUI moderna")
root.geometry("440x280")

root.mainloop()
```

Lanciamo il programma: otteniamo una finestra dallo sfondo scuro, ancora vuota, ma già

[47] https://customtkinter.tomschimansky.com/

visivamente ben diversa dalle finestre grigie viste finora.



Finestra vuota in stile customtkinter, con tema scuro attivo

Dall'analisi del codice osserviamo che:

- `ctk.CTk()` sostituisce `tk.Tk()` nella creazione della finestra principale: si tratta, di fatto, di una versione "potenziata" della classe Tk, da cui eredita gran parte del comportamento (compresi `title()` e `mainloop()`);
- `root.geometry("440x280")` è un metodo che, sebbene appartenga in realtà già a tkinter e non sia esclusivo di *customtkinter*, non avevamo ancora incontrato. Permette di impostare larghezza e altezza iniziali della finestra, espresse in pixel e separate dalla lettera "`x`";
- `ctk.set_appearance_mode(...)` e `ctk.set_default_color_theme(...)` non sono metodi di un widget, ma funzioni globali della libreria, per questo motivo vanno richiamate prima di creare la finestra e i *widget*, in modo che questi ultimi vengano disegnati fin da subito con la palette di colori scelta.

9.7.4 I widget moderni

Molti dei widget che abbiamo incontrato nel paragrafo 9.2 hanno un corrispondente in *customtkinter*, riconoscibile dal prefisso `CTk`: `CTkLabel`, `CTkButton`, `CTkEntry`, `CTkCheckBox`, e così via. Quasi sempre possono essere posizionati esattamente come già sappiamo fare, con `.pack()`, `.grid()` o `.place()`.

Esaminiamo il seguente codice:

```
label = ctk.CTkLabel(root, text="Benvenuti a customtkinter!",
                     font=ctk.CTkFont(size=16, weight="bold"))
label.pack(pady=(20, 10))

bottone = ctk.CTkButton(root, text="Cliccami")
bottone.pack(pady=10)
```

```
entry = ctk.CTkEntry(root, placeholder_text="Inserisci il tuo
nome")
entry.pack(pady=10)

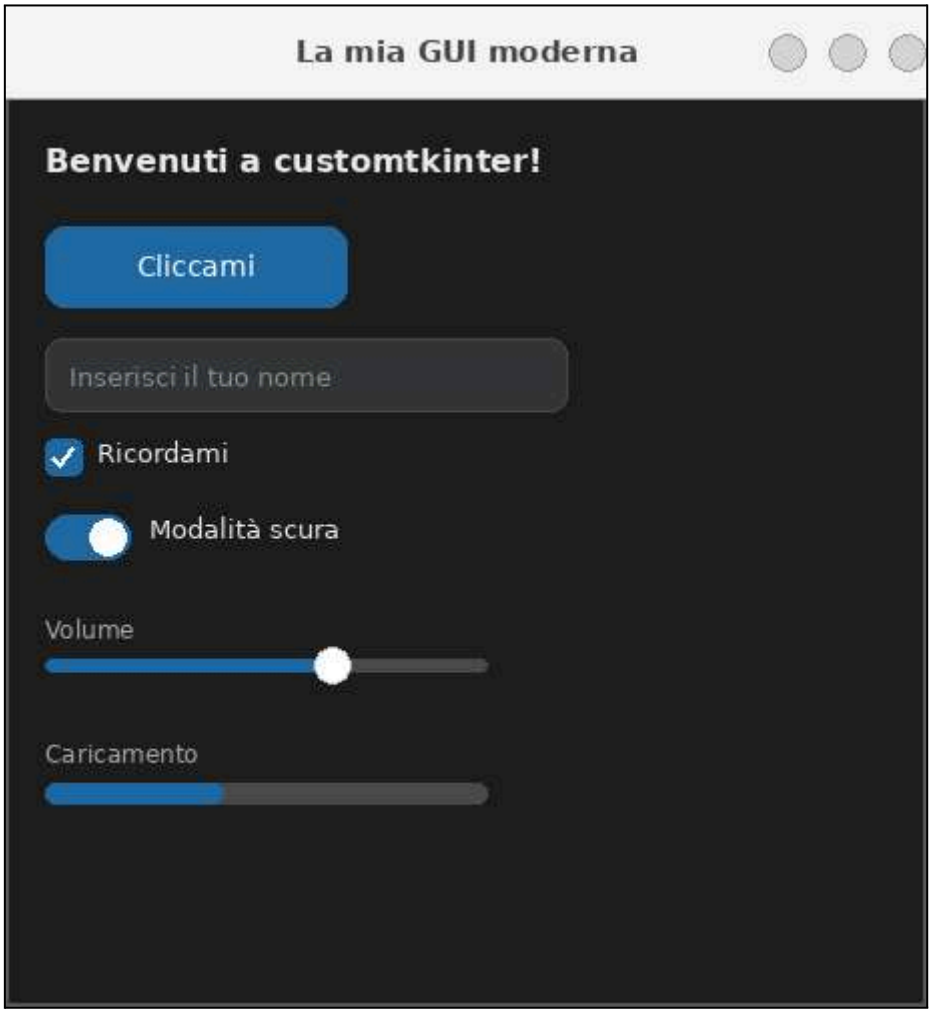
check = ctk.CTkCheckBox(root, text="Ricordami")
check.pack(pady=10)

switch = ctk.CTkSwitch(root, text="Modalità scura")
switch.pack(pady=10)

slider = ctk.CTkSlider(root, from_=0, to=100)
slider.pack(pady=10)

progress = ctk.CTkProgressBar(root)
progress.set(0.4)
progress.pack(pady=10)
```

Lanciando il programma otterremo a video la seguente finestra:



Alcuni dei widget moderni offerti da customtkinter: pulsante, casella di testo, checkbox, switch, slider e barra

Dall'analisi del codice osserviamo diverse novità rispetto ai *widget* classici:

- `pady=(20, 10)`, a differenza del semplice `pady=10` (esiste anche su tkinter) accetta anche una tupla di due valori, per indicare rispettivamente lo spazio da lasciare sopra e sotto il widget: qui vogliamo, per esempio, un margine superiore più ampio di quello inferiore;
- `ctk.CTkFont(size=16, weight="bold")` è una classe di supporto che permette di costruire font in maniera più leggibile rispetto alla tupla `("Arial", 16, "bold")` utilizzata con i *widget* di tkinter;
- il parametro `placeholder_text` di `CTkEntry` mostra un testo suggerito, visualizzato in grigio chiaro, che scompare automaticamente non appena l'utente inizia a scrivere. Questo è un netto miglioramento rispetto all'escamotage che avevamo utilizzato nel paragrafo 9.2.2 con `.insert(0, "Inserisci del testo qui")` sul *widget* `Entry` di tkinter, che invece lasciava un testo fisso da cancellare manualmente;
- `CTkSwitch` e `CTkSlider` sono due widget che non hanno un vero e proprio corrispondente tra quelli di tkinter, il primo è un interruttore a scorrimento che restituisce 1 o 0 tramite il metodo `.get()` a seconda della posizione, il secondo un cursore trascinabile che restituisce, sempre con `.get()`, un valore compreso tra `from_` e `to`;
- `CTkProgressBar` mostra una barra di avanzamento, la cui percentuale di riempimento si imposta con il metodo `.set(valore)`, dove valore è un numero decimale compreso tra 0.0 (vuota) e 1.0 (piena).

9.7.5 Personalizzare l'aspetto dei widget

Oltre al tema globale impostato con `set_appearance_mode()` e `set_default_color_theme()`, ogni singolo *widget* può essere personalizzato individualmente, sovrascrivendo i colori e le dimensioni previste dal tema di default.

Analizziamo il seguente codice:

```
bottone_personalizzato = ctk.CTkButton(
    root,
    text="Conferma",
    corner_radius=20,
    fg_color="#2E8B57",
    hover_color="#256B45",
    text_color="white",
    font=ctk.CTkFont(size=14, weight="bold")
)
bottone_personalizzato.pack(pady=20)
```

Dall'analisi di questo codice osserviamo che:

- `corner_radius` definisce, in pixel, il raggio di arrotondamento degli angoli: il valore 0 restituisce un *widget* dagli angoli squadrati, come quelli classici, mentre valori più alti lo rendono via via più arrotondato;

- ➔ `fg_color` imposta il colore principale del *widget* (qui specificato, come talvolta accade anche in altri contesti legati alla grafica, tramite un codice esadecimale, anziché con un nome come "green");
- ➔ `hover_color` imposta il colore che il *widget* assume quando il puntatore del mouse vi transita sopra. Si tratta di un effetto che, con i *widget* kdi tkinter, avremmo dovuto realizzare via codice, collegando gli eventi `<Enter>` e `<Leave>` visti nel paragrafo 9.4.1 a una funzione che modificasse il colore con `.config()`;
- ➔ `text_color`, infine, imposta il colore del testo visualizzato sul *widget*.

9.7.6 Un esempio completo: una finestra di login

Mettiamo insieme quanto visto in questo capitolo per realizzare una piccola finestra di accesso (login), composta da un riquadro arrotondato contenente titolo, due campi di inserimento, uno switch e un pulsante.

Ecco il codice:

```
import customtkinter as ctk

ctk.set_appearance_mode("dark")
ctk.set_default_color_theme("blue")

def esegui_login():
    utente = entry_utente.get()
    print(f"Accesso richiesto per l'utente: {utente}")

root = ctk.CTk()
root.title("Login")
root.geometry("360x460")

frame = ctk.CTkFrame(root, corner_radius=15)
frame.pack(padx=30, pady=30, fill="both", expand=True)

titolo = ctk.CTkLabel(frame, text="Bentornato!",
                      font=ctk.CTkFont(size=22, weight="bold"))
titolo.pack(pady=(30, 20))

entry_utente = ctk.CTkEntry(frame, placeholder_text="Nome utente")
entry_utente.pack(pady=10, padx=30, fill="x")

entry_password = ctk.CTkEntry(frame, placeholder_text= "Password",
show="*")
entry_password.pack(pady=10, padx=30, fill="x")

ricordami = ctk.CTkSwitch(frame, text="Ricordami")
ricordami.pack(pady=10)

bottone_login = ctk.CTkButton(frame, text="Accedi", command=
```

```
esegui_login)
bottone_login.pack(pady=20, padx=30, fill="x")

root.mainloop()
```

Lanciando il programma otteniamo a video la seguente finestra:



La finestra di login completa, con riquadro arrotondato, campi di inserimento, switch e pulsante

Dall'analisi del codice osserviamo che:

- `CTkFrame`, come il Frame di tkinter, è un contenitore, ma può a sua volta avere un corner_radius, comportandosi visivamente come una "scheda" (card) sulla quale disponiamo gli altri widget;
- il parametro `show="*"` di `CTkEntry` funziona esattamente come per il widget Entry classico, mascherando i caratteri digitati. E' utile ricordare che si tratta di una funzionalità già presente in tkinter, e non di una novità di *customtkinter*;
- `fill="x"`, applicato a *entry* e *pulsante*, li fa espandere in larghezza fino a occupare tutto lo spazio orizzontale disponibile nel frame, secondo la stessa logica vista nel paragrafo 9.3.4;
- la funzione `esegui_login()`, collegata al pulsante tramite il parametro `command`, legge il contenuto del campo utente con `.get()`, un metodo che avevamo già incontrato, con lo stesso nome, anche per lo *switch* e per lo *slider*.

Come esercizio, il lettore provi a:

- ➔ aggiungere un secondo `CTkSwitch` che permetta di passare dinamicamente tra tema chiaro e scuro, richiamando `ctk.set_appearance_mode("light")` oppure `ctk.set_appearance_mode("dark")` dentro una funzione collegata allo switch stesso;
- ➔ mostrare un messaggio di errore, tramite una `CTkLabel` inizialmente vuota il cui testo viene aggiornato con il metodo `.configure(text="...")`, se l'utente prova ad accedere lasciando vuoto il campo del nome utente;
- ➔ riprendere l'editor di testo realizzato nel paragrafo 9.6.5 e sostituire i widget di tkinter (`tk.Text`, `tk.Menu`, `tk.Button`) con i corrispondenti widget di *customtkinter*, verificando che la logica del programma, comandi ed eventi compresi, non debba essere modificata.

APPENDICE

1. Installazione Python su GNU/Linux

Python è preinstallato sulla maggior parte delle distribuzioni Linux ed è disponibile come pacchetto su tutte le altre. [48]

Tuttavia, ci sono alcune funzionalità che potresti voler utilizzare che non sono disponibili nel pacchetto della tua distribuzione. Puoi facilmente compilare l'ultima versione di Python dal sorgente.

1.1 Installazione da pacchetto

Supponiamo di utilizzare una distribuzione GNU/Linux molto diffusa, p.e. Ubuntu.
La distribuzione Ubuntu prevede l'installazione della versione 3.9 del python (alla data di stesura del presente testo è presente la versione 10.x ee è in beta la versione 11.x del linguaggio).

E' utile eseguire l'aggiornamento del sistema. Inanzitutto si esegue l'aggiornamento degli indici dei vari repositories:

```
$ sudo apt update
```

Eseguire l'aggiornamento del sistema:

```
$ sudo apt -y upgrade
```

Il flag `-y` confermerà che stiamo accettando l'installazione di tutti gli elementi, ma a seconda della tua versione di Linux, potresti dover confermare ulteriori richieste come aggiornamenti e upgrade del tuo sistema.

Installiamo ora il pacchetto dell'interprete python:

```
$ sudo apt install python3
```

Una volta completato il processo, se non si sono verificati errori, dovremmo avere la versione python 3 installata nel sistema. Per verificarlo:

```
$ python3 -V
```

[48] https://docs.python.org/3.9/using/unix.html#on-linux

Riceverai un output nella finestra del terminale che ti farà conoscere il numero di versione. Sebbene questo numero possa variare, l'output sarà simile a questo:

```
$ Python 3.9.7
```

In questo caso la versione che il sistema operativo installa è la 3.9.x (7 è l'ultima minor release disponibile nei repositories ufficiali alla data di stesura di questa guida)

1.2 Installazione da sorgenti

L'installazione dai sorgenti è un po' più complessa ed è possibile incontrare con più facilità conflitti con le librerie installate sul sitema.

Innanzitutto devi scaricare i files sorgenti dal repositories ufficiale:

https://www.python.org/downloads/source/

A questo link troverai le varie release, scegli tra le release indicate come *stable*.

Seleziono i sorgenti di Python 3.9.7 compressi con l'utility gzip, quindi eseguo il comando:

```
$ wget -c https://www.python.org/ftp/python/3.9.7/Python-3.9.7.tgz
```

Decomprimo il file scaricato:

```
$ tar xzf Python-3.9.7.tgz
```

Entro nella cartella creata:

```
$ cd Python-3.9.7
```

Ora posso lanciare i comandi di configurazione dei sorgenti per la preparazione al build dell'eseguibile python:

```
 $ ./configure --enable-optimizations
```

il flag `--enable-optimizations` non è necessario, indica se si vogliono ottimizzare alcuni parametri di compilazione.

Successivamente si compila:

```
 $ make
```

Se non si sono presentati errori si puo' effettuare l'installazione degli eseguibili. Per effettuare l'installazione è necessario entrare come amministratore del sistema:

```
 # make install
```

Il prompt # indica che si è entrati come utente *root*.

Le opzioni di configurazione e le avvertenze per specifiche piattaforme Unix sono ampiamente documentate nel file README.rst nella radice dell'albero dei sorgenti Python.

Avvertimento: `make install` può sovrascrivere o mascherare il python3 eventualmente installato di default nella vostra distribuzione. Quindi è consigliato di effettuare invece di `make altinstall` poiché installa solo in `exec_prefix/bin/pythonversion`[49]

1.2.1 Percorsi e file relativi all'installazione

Questi sono soggetti a differenze a seconda delle convenzioni di installazione locali; prefix (`${prefix}`) eexec_prefix (`${exec_prefix}`) dipendono dall'installazione e devono essere interpretati come per il software GNU; potrebbero essere uguali.

Ad esempio, sulla maggior parte dei sistemi Linux, l'impostazione predefinita per entrambi è `/usr`.

File/cartella	Significato
`exec_prefix/bin/python3`	Posizione consigliata dell'interprete.
`prefix/lib/pythonversion`, `exec_prefix/lib/pythonversion`	Posizioni consigliate delle directory contenenti i moduli standard.
`prefix/include/pythonversion`, `exec_prefix/include/pythonversion`	Percorsi consigliati delle directory contenenti i file include necessari per lo sviluppo di estensioni Python e l'incorporamento dell'interprete.

1.4 Installazione librerie

Di seguito i comandi da digitare nella console per installare le librerie

1.4.1. Installazione di PIP (Python Package Index)

pip che è un'applicazione che ci aiuterà nell'installazione dei pacchetti python. Quando affronteremo il capitolo degli ambienti virtuali vedremo come utilizzarlo.

Solitamente pip è installato di default. Per verificare che sia installato:

```
$ python3 -m pip --version
```

Se il comando restituisce un errore allora pip non è installato, quindi procedere con il

[49] https://docs.python.org/3.9/using/unix.html#on-linux

seguente comando:

```
$ python3 -m pip install --upgrade pip
```

Se incontraste dei problemi potete installare il pacchetto specifico per la vostra distribuzione Linux. Per esempio per la distribuzione Ubuntu il comando è:

```
$ sudo apt install python3-pip
```

1.4.1.1 Nota su PIP da Python 3.11 (PEP 668)[50]

Python 3.11 implementa il nuovo PEP 668, *Marcatura degli ambienti di base Python come "gestito esternamente"*. Usare `pip` regolarmente su Debian (e derivate), porterà a ricevere l'errore di *externally-managed-environment*. Cioè il seguente:

```
error: externally-managed-environment
× This environment is externally managed
╰─> To install Python packages system-wide, try apt install
    python3-xyz, where xyz is the package you are trying to
    install.

    If you wish to install a non-Debian-packaged Python package,
        create a virtual environment using python3 -m venv
path/to/venv.
      Then use path/to/venv/bin/python and path/to/venv/bin/pip.
Make
    sure you have python3-full installed.

        If you wish to install a non-Debian packaged Python
application,
    it may be easiest to use pipx install xyz, which will manage a
        virtual environment for you. Make sure you have pipx
installed.

           See /usr/share/doc/python3.11/README.venv for more
information.
note: If you believe this is a mistake, please contact your Python
installation or OS distribution provider. You can override this,
at the risk of breaking your Python installation or OS, by passing
--break-system-packages.
hint: See PEP 668 for the detailed specification.
```

In poche parole con questa PEP, gli strumenti Python ora possono distinguere tra i pacchetti che sono stati installati dall'utente con uno strumento come `pip` e quelli

[50] https://veronneau.org/python-311-pip-and-breaking-system-packages.html

installati utilizzando il gestore di pacchetti della distribuzione, come `apt`.

Questo porta ad avere una migliore gestione dei pacchetti e una loro più robusta gestione separando di fatto i pacchetti utente da quelli di sistema.

Se sei abituato a utilizzare pip per installare sia pacchetti di sistema che pacchetti utente allora potrai aggirare le policy più severe di Python 3.11 con due metodi:

1. Utilizzare il parametro a linea di comando di pip `--break-system-packages` che indica in buona sostanza di "rompere" i pacchetti di sistema
2. Oppure ottenere lo stesso scopo del punto precedente settando la variabile di ambiente `PIP_BREAK_SYSTEM_PACKAGES` a 1.

La migliore soluzione è quella di costringersi a utilizzare gli ambienti virtuali python in cui installare tutti i pacchetti di cui abbiamo bisogno senza andare a “sporcare” l’ambiente python di sistema. Vedi il capitolo 1.5 per sapere come installare un ambiente virtuale python.

1.4.2 Numpy

```
$ pip install numpy
```

1.4.3 Pillow

```
$ pip install pillow
```

1.4.4 Matplotlib

```
$ pip install matplotlib
```

Il comando andrà a installare la libreria e eventuali dipendenze non installate (come p.e. numpy)

1.4.5 Pandas

```
$ pip install pandas
```

Il comando andrà a installare la libreria e eventuali dipendenze non installate.

1.5. Installazione di un ambiente virtuale

Questo capitolo è preso interamente da:”Ambienti virtuali e pacchetti” dal sito ufficiale di Python [51] (tranne il paragrafo 1.5.2)

[51] https://docs.python.org/3/tutorial/venv.html

1.5.1 Introduzione

Le applicazioni Python utilizzeranno spesso pacchetti e moduli che non fanno parte della libreria standard. Le applicazioni a volte richiedono una versione specifica di una libreria, perché l'applicazione potrebbe richiedere la correzione di un bug particolare o l'applicazione potrebbe essere scritta utilizzando una versione obsoleta dell'interfaccia della libreria.

Ciò significa che potrebbe non essere possibile per un'installazione Python soddisfare i requisiti di ogni applicazione. Se l'applicazione A richiede la versione 1.0 di un particolare modulo ma l'applicazione B richiede la versione 2.0, i requisiti sono in conflitto e l'installazione della versione 1.0 o 2.0 lascerà un'applicazione non in grado di funzionare.
La soluzione a questo problema è creare un ambiente virtuale , un albero di directory autonomo che contiene un'installazione Python per una particolare versione di Python, oltre a una serie di pacchetti aggiuntivi.

Diverse applicazioni possono quindi utilizzare diversi ambienti virtuali. Per risolvere l'esempio precedente di requisiti in conflitto, l'applicazione A può disporre del proprio ambiente virtuale con la versione 1.0 installata mentre l'applicazione B dispone di un altro ambiente virtuale con la versione 2.0. Se l'applicazione B richiede l'aggiornamento di una libreria alla versione 3.0, ciò non influirà sull'ambiente dell'applicazione A.

1.5.2 Installazione

Installiamo la versione di venv compatibile con la versione di python installata sulla nostra macchina. In questo caso io ho la versione 3.9.

```
$ sudo apt install python3.9-venv
```

1.5.3 La creazione di ambienti virtuali

Il modulo utilizzato per creare e gestire gli ambienti virtuali si chiama `venv`. `venv` di solito installerà la versione più recente di Python che hai a disposizione. Se hai più versioni di Python sul tuo sistema, puoi selezionare una versione specifica di Python eseguendo python3 o qualsiasi versione tu voglia.

Per creare un ambiente virtuale, scegli una directory in cui posizionarlo ed esegui il modulo venv come script con il percorso della directory:

```
$ python3 -m venv intropython-env
```

Questo creerà la directory `intropython-dev` se non esiste e creerà anche directory al suo interno contenenti una copia dell'interprete Python e vari file di supporto.
Un percorso di directory comune per un ambiente virtuale è `.venv`. Questo nome mantiene la directory tipicamente nascosta nella tua shell e previene conflitti con variabili

di ambiente utilizzati da alcuni programmi.

Dopo aver creato un ambiente virtuale devi attivarlo.

Su Windows, esegui:

```
c:> intropython-env\Scripts\activate.bat
```

Su Unix o MacOS, esegui:

```
$ source intropython-env/bin/activate
```

(Questo script è stato scritto per la shell *bash*. Se si utilizzano le shell *csh* o *fish*, è necessario utilizzare gli script alternativi *activate.csh* e *activate.fish*).

L'attivazione dell'ambiente virtuale cambierà il prompt della shell per mostrare quale ambiente virtuale stai utilizzando e modificherà l'ambiente in modo che l'esecuzione di python ottenga quella particolare versione e installazione di Python.
Per esempio:

```
$ source ~/intropython-env/bin/activate
(intropython-env) $ python
Python 3.9.7 (default, Sep 10 2021, 14:59:46)
  ...
>>> import sys
>>> sys.path
['',            '/usr/lib/python39.zip',            ...            ,
'/home/massimo/intropython-env/lib/python3.9/site-packages']
>>>
```

Per uscire dall'ambiente virtuale devi eseguire:

```
(intropython-env) $ deactivate
```

1.5.4. Gestire i pacchetti con pip (Python Package Index)

Puoi installare, aggiornare e rimuovere pacchetti usando un programma chiamato pip . Per impostazione predefinita pip verranno installati i pacchetti dall'indice dei pacchetti Python, < https://pypi.org >. Puoi sfogliare questo indice andando su di esso nel tuo browser web.

pip dispone di una serie di sottocomandi: "`install`", "`uninstall`", "`freeze`", ecc. (Consultare la guida all'installazione dei moduli Python per la documentazione completa di pip.)

Puoi installare l'ultima versione di un pacchetto specificando il nome di un pacchetto:

```
(intropython-env) $ python -m pip install novas
Collecting novas
  Downloading novas-3.1.1.3.tar.gz (136kB)
Installing collected packages: novas
  Running setup.py install for novas
Successfully installed novas-3.1.1.3
```

Puoi anche installare una versione specifica di un pacchetto fornendo il nome del pacchetto seguito da `==` seguito dal numero di versione:

```
(tutorial-env) $ python -m pip install requests==2.6.0
Collecting requests==2.6.0
  Using cached requests-2.6.0-py2.py3-none-any.whl
Installing collected packages: requests
Successfully installed requests-2.6.0
```

Se esegui nuovamente questo comando, pip noterai che la versione richiesta è già installata e non farà nulla. Puoi fornire un numero di versione diverso per ottenere quella versione, oppure puoi eseguire per aggiornare il pacchetto all'ultima versione:

`pip install --upgrade`

```
(intropython-env) $ python -m pip install --upgrade requests
Collecting requests
Installing collected packages: requests
  Found existing installation: requests 2.6.0
    Uninstalling requests-2.6.0:
      Successfully uninstalled requests-2.6.0
Successfully installed requests-2.7.0
```

`pip uninstall` seguito da uno o più nomi di pacchetti rimuoverà i pacchetti dall'ambiente virtuale.

`pip show` visualizzerà informazioni su un particolare pacchetto:

```
(tutorial-env) $ pip show requests
---
Metadata-Version: 2.0
Name: requests
Version: 2.7.0
Summary: Python HTTP for Humans.
Home-page: http://python-requests.org
Author: Kenneth Reitz
Author-email: me@kennethreitz.com
License: Apache 2.0
Location:
/Users/akuchling/envs/tutorial-env/lib/python3.4/site-packages
```

```
Requires:
```

`pip list` visualizzerà tutti i pacchetti installati nell'ambiente virtuale:

```
(tutorial-env) $ pip list
novas (3.1.1.3)
numpy (1.9.2)
pip (7.0.3)
requests (2.7.0)
setuptools (16.0)
```

`pip freeze` produrrà un elenco simile dei pacchetti installati, ma l'output utilizza il formato previsto. Una convenzione comune è quella di inserire questo elenco in un file:

`pip install requirements.txt`

```
(tutorial-env) $ pip freeze > requirements.txt
(tutorial-env) $ cat requirements.txt
novas==3.1.1.3
numpy==1.9.2
requests==2.7.0
```

Il file `requirements.txt` può quindi essere vincolato al controllo della versione e spedito come parte di un'applicazione. Gli utenti possono quindi installare tutti i pacchetti necessari con : `install -r`

```
(tutorial-env) $ python -m pip install -r requirements.txt
Collecting novas==3.1.1.3 (from -r requirements.txt (line 1))
  ...
Collecting numpy==1.9.2 (from -r requirements.txt (line 2))
  ...
Collecting requests==2.7.0 (from -r requirements.txt (line 3))
  ...
Installing collected packages: novas, numpy, requests
  Running setup.py install for novas
Successfully installed novas-3.1.1.3 numpy-1.9.2 requests-2.7.0
```

`pip` ha molte più opzioni. Consulta la guida all'installazione dei moduli Python per la documentazione completa per pip. Quando hai scritto un pacchetto e vuoi renderlo disponibile nell'indice dei pacchetti Python, consulta la guida alla distribuzione dei moduli Python .

2. Installazione Python su Microsoft Windows

A differenza della maggior parte dei sistemi e servizi Unix, Windows non include un'installazione di Python supportata dal sistema. Per rendere disponibile Python, il team di CPython ha compilato per molti anni programmi di installazione di Windows (pacchetti MSI) con ogni versione . Questi programmi di installazione hanno principalmente lo scopo di aggiungere un'installazione per utente di Python, con l'interprete principale e la libreria utilizzati da un singolo utente. Il programma di installazione è anche in grado di eseguire l'installazione per tutti gli utenti di una singola macchina ed è disponibile un file ZIP separato per le distribuzioni locali dell'applicazione.[52]

Si possono utilizzare diversi metodi e pacchetti per l'installazione. Esiste anche un pacchetto con funzionalità ridotte presente sullo store di Micrososft.

Noi indichiamo il pacchetto completo che è possibile installare anche senza essere utente amministratore del sistema.

2.1 Installazione da pacchetto

Installeremo la versione 3.9.8 del python.
L'indirizzo web è il seguente:
https://www.python.org/ftp/python/3.9.8/python-3.9.8-amd64.exe

Una volta scaricato il programma di installazione si dovrà lanciare. Appare la seguente finestra:

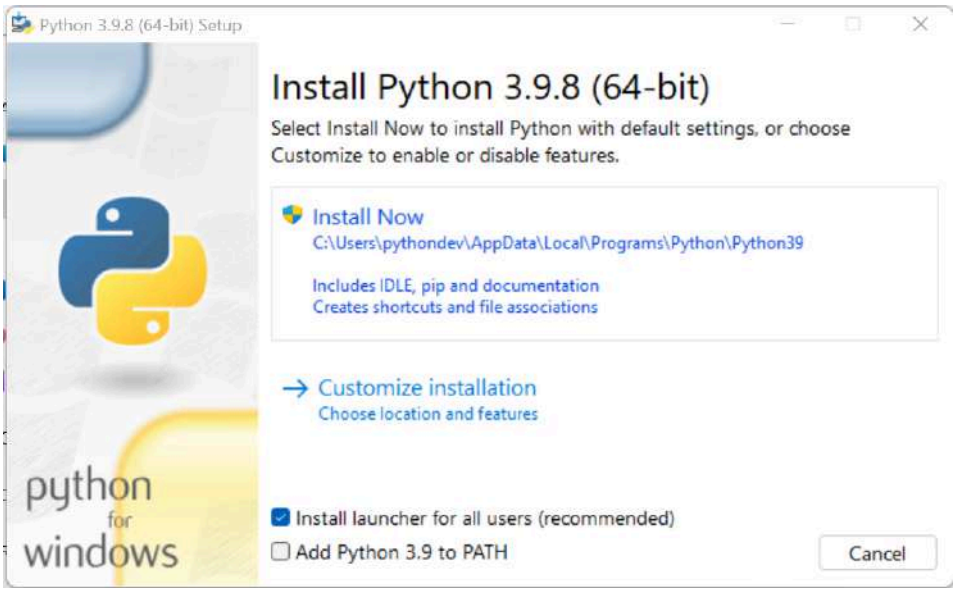


Figura A.2.1

Se non si è amministratori di Windows è necessario deselezionare il primo checkbox per impedire l'installazione per tutti gli utenti.
E' utile selezionare il checkbox per aggiungere il percorso di Python alla nostra PATH.
Infine cliccare su:"Install Now" per cominciare l'installazione oppure :"Customize

[52] https://docs.python.org/3.9/using/windows.html

Installation” per selezionare ulteriori parametri di installazione.

Se si opta per l'installazione personalizzata appare la seguente finestra:

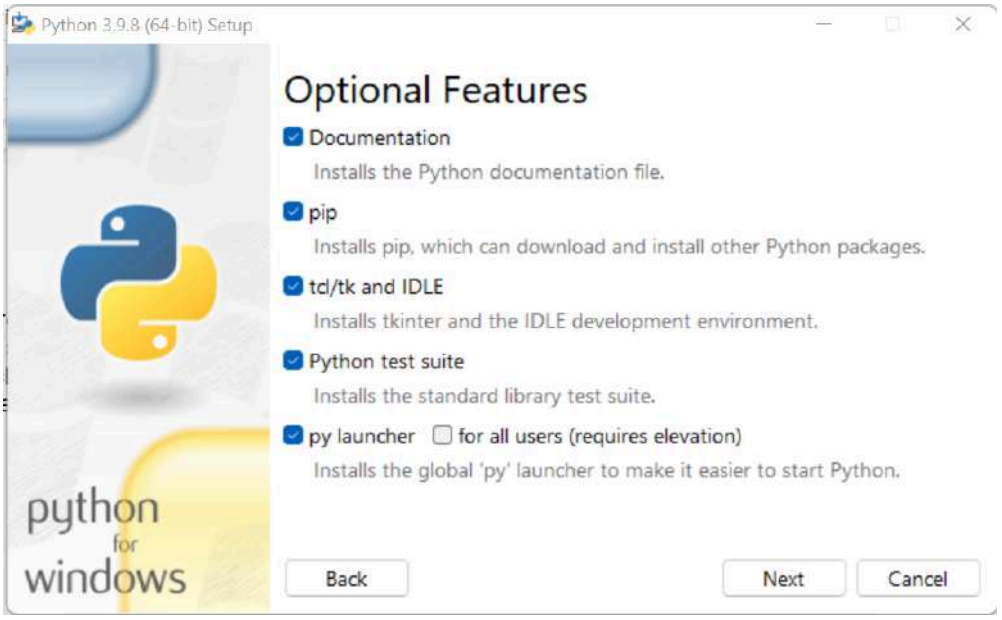


Figura A.2.2

Ora dovrete selezionare le voci opzionali che volete installare. È sconsigliabile non installare pip (che è il comando che ci permetterà di installare ulteriori librerie) e IDLE che è la console che andremo a utilizzare.

Al successivo:”Next” apparirà la seguente finestra:

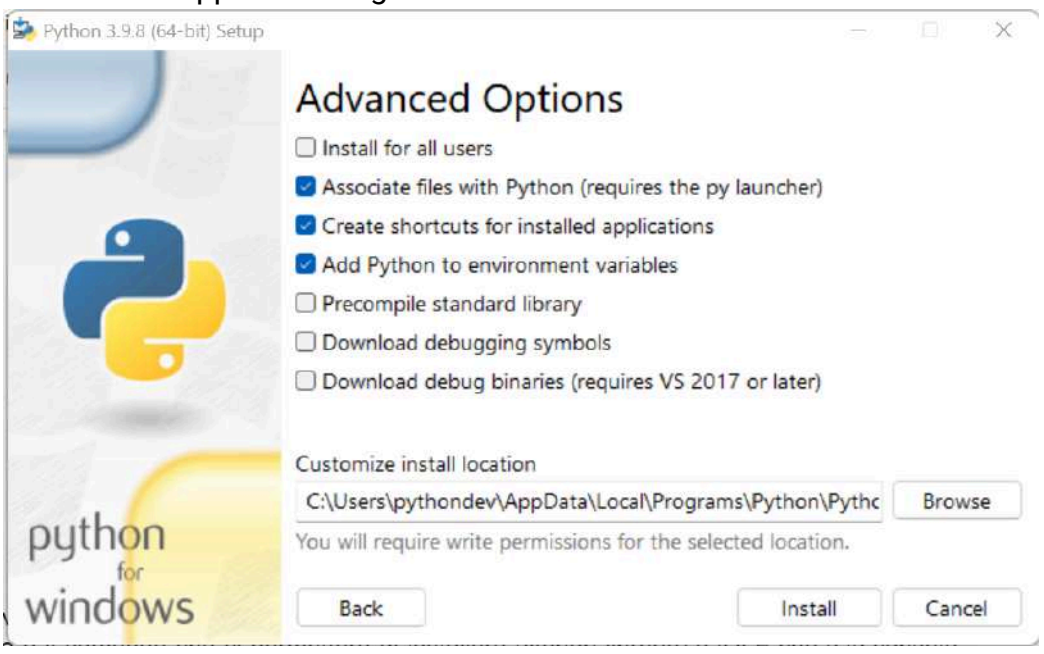


Figura 2.A.3

Cliccando su:”Install” si procede finalmente all’installazione.

Dopo qualche tempo (a seconda della potenza del calcolatore su cui si sta effettuando l’installazione) appare la schermata di fine installazione:

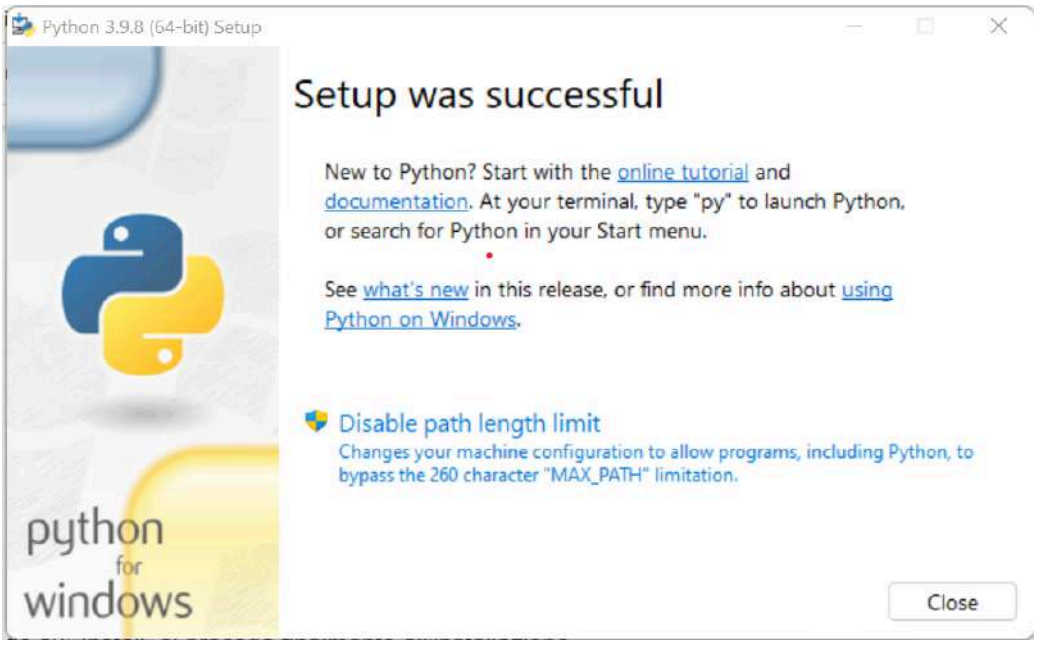


Figura A.2.4

La voce:"Disable path length limit" non si rende necessaria per i nostri scopi. Per maggiori delucidazioni si rimanda alla documentazione ufficiale dell'installazione su sistemi Microsoft Windows[53]

2.2 Lanciare l'interprete python

L'installazione ha configurato nel sistema due principali eseguibili:

- ➔ Un interprete testuale che gira su una console come la command (cmd) che si chiama PYTHON
- ➔ Un interprete che apre una finestra grafica che si chiama IDLE

L'interfaccia del primo è la seguente:

[53] https://docs.python.org/3.9/using/windows.html#removing-the-max-path-limitation

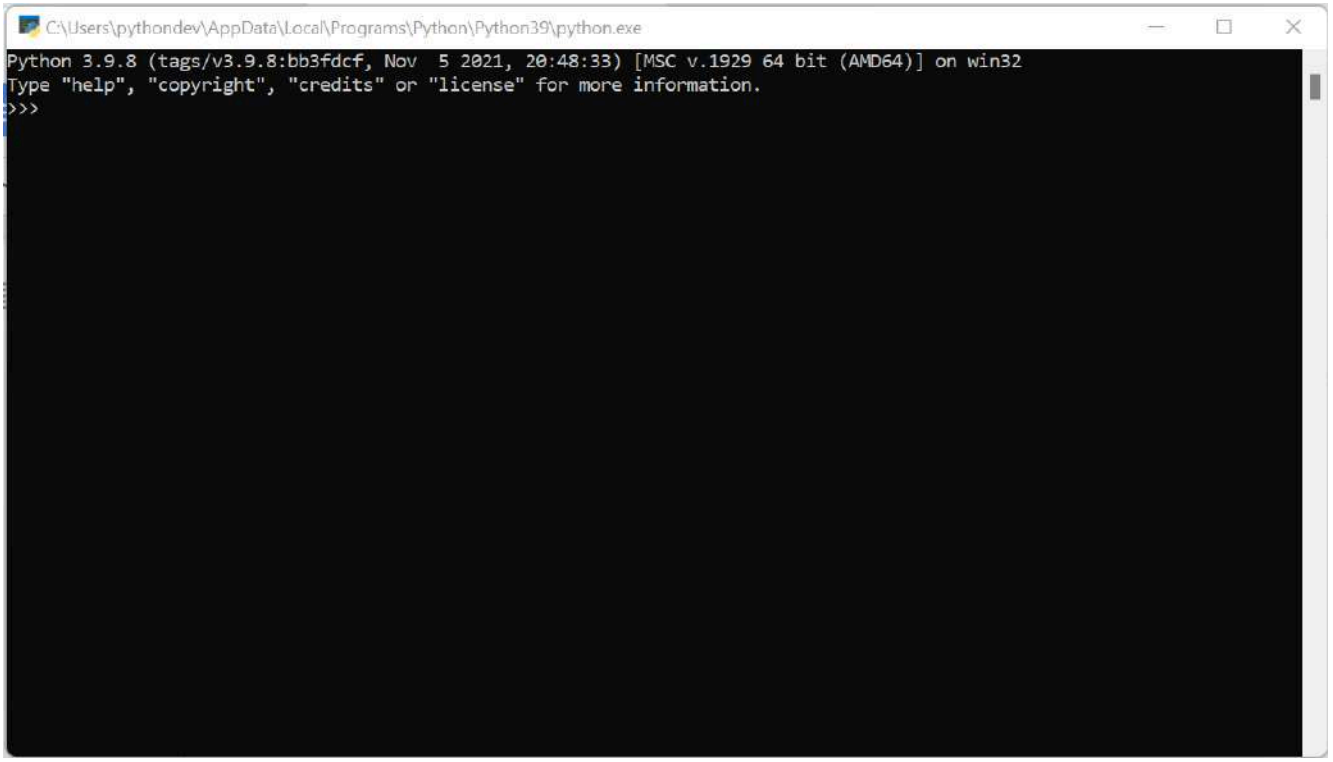


Figura A.2.5

L’interfaccia del secondo è la seguente:

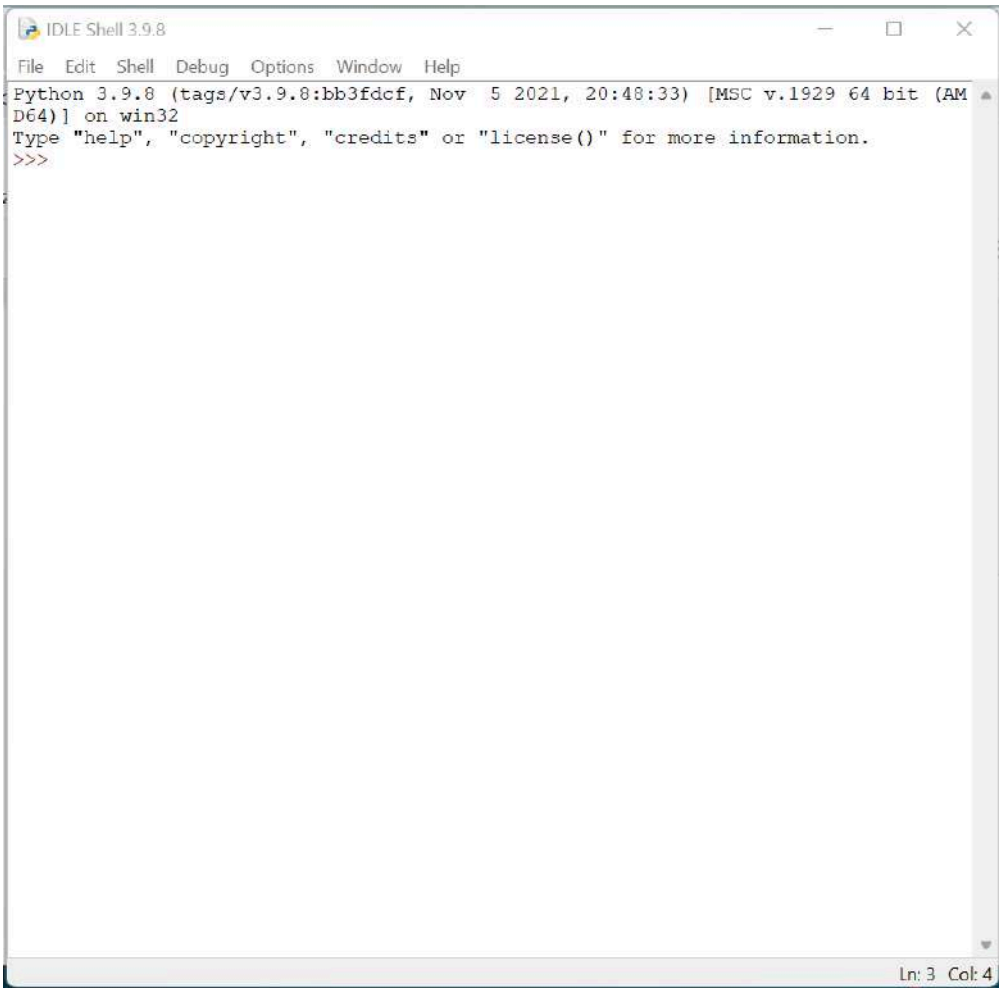


Figura A.2.6

2.3 Installazione librerie

Di seguito i comandi da digitare nella console di windows (cmd) per installare le librerie

2.3.1. Installazione di PIP (Python Package Index)

pip che è un'applicazione che ci aiuterà nell'installazione dei pacchetti python. Nell'installazione di python lo abbiamo selezionato per richiederne l'esplicita installazione.

Per verificaare l'installazione possiamo provare i seguenti comandi:

```
C:\Users\pythondev>python -m pip –version
```

oppure:

```
C:\Users\pythondev>pip --version
```

Se i comandi restituiscono degli errori allora pip non è installato, quindi procedere andando nel gestore di app di Windows, ricercare l'app Python- 3.9.8 e poi selezionare la voce di modifica. In questo modo apparirà la finestra di Figura 2.A.3 che vi peremtterà di

selezionare la voce PIP per l'installazione.

Un'altra modalità è simile a quella utilizzata con GNU/Linux e cioè utilizzando un comando dalla console:

```
C:\Users\pythondev>python -m pip install --upgrade pip
```

2.3.2 Numpy

```
C:\Users\pythondev>pip install numpy
```

2.3.3 Pillow

```
C:\Users\pythondev>pip install pillow
```

2.3.4 Matplotlib

```
C:\Users\pythondev>pip install matplotlib
```

Il comando andrà a installare la libreria e eventuali dipendenze non installate (come p.e. numpy)

2.3.5 Pandas

```
C:\Users\pythondev>pip install pandas
```

Il comando andrà a installare la libreria e eventuali dipendenze non installate.

3. Digressione s.o. GNU/Linux

3.1 La console GNU/Linux

La console Linux è una console di sistema interna al kernel Linux . Una console di sistema è il dispositivo che riceve tutti i messaggi e gli avvisi del kernel e che consente l'accesso in modalità utente singolo). La console Linux fornisce un modo per il kernel e altri processi di inviare output di testo all'utente e di ricevere input di testo dall'utente. L'utente in genere inserisce il testo con la tastiera di un computer e legge il testo di output sul monitor di un computer.[54]

Quindi attraverso la console l'utente ha la possibilità di interagire con il kernel del sistema operativo impartendo comandi e ricevendo risposte come stringhe che appaiono a video all'interno della console stessa.

In questo testo andremo a lanciare i comandi e i nostri programmi utilizzando solo la console.

3.1.1 Come aprire una console GNU/Linux?

Consideriamo due casi:

1. Il sistema GNU/Linux è installato direttamente sulla macchina su cui abbiamo accesso fisico
2. Il sistema GNU/Linux è installato su una macchina collegata in rete alla macchina a cui abbiamo accesso fisico

Nel primo caso si accede al sistema GNU/Linux utilizzando le opportune credenziali. Una volta entrati è necessario lanciare l'applicativo che implementa la console, solitamente è un terminale.

Nel secondo caso è necessario utilizzare un applicativo che permetta la connessione criptata (solitamente su SSL) in quanto le connessioni telnet non sono più installate per questioni di sicurezza.

3.1.1.1 Ambiente Microsoft Winodws

In ambiente Microsoft Windows è utile scaricare l'applicazione putty.exe che è un client SSH e permette la connessione criptata a un server GNU/Linux. Si può scaricare dal sito: https://www.putty.org/ Una volta lanciato si apre una finestra che richiede principalmente tre parametri:

- ➔ l'indirizzo ip del server (o il nome a dominio)
- ➔ il protocollo che si vuole utilizzare, nel nostro caso SSL

Si apre una finestra simile a questa:

[54] https://en.wikipedia.org/wiki/Linux_console

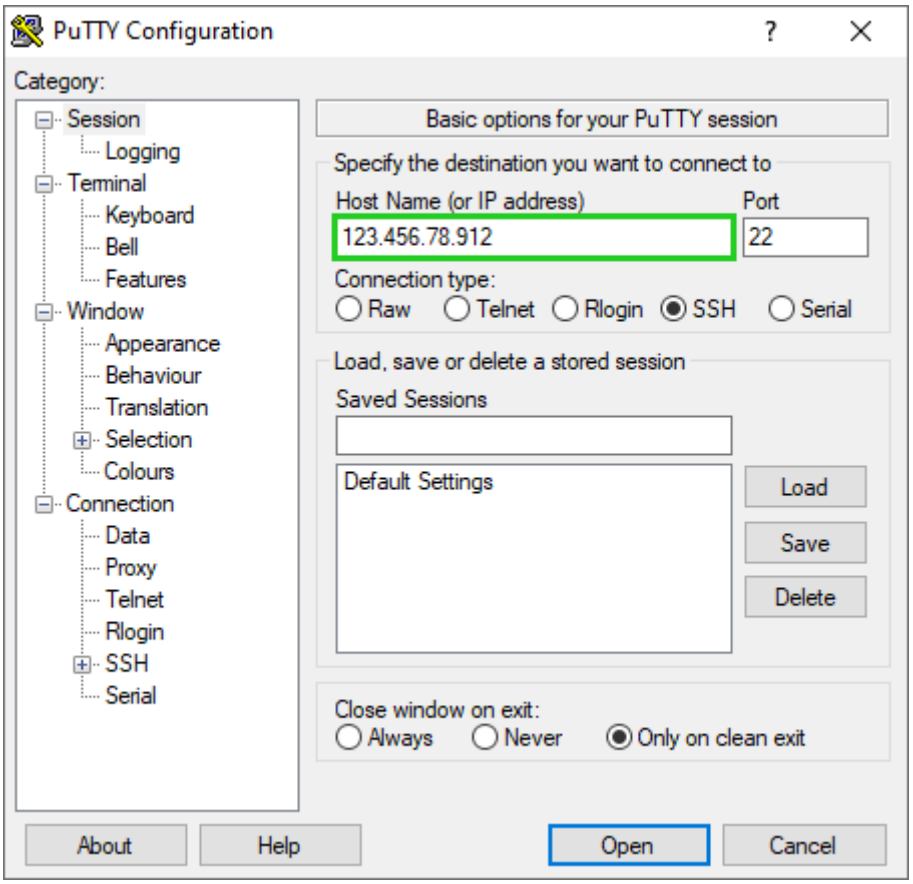


Figura 2.1

Una volta creata la connessione alla macchina server, il sistema richiederà le credenziali di accesso.

3.1.1.2 Ambiente GNU/Linux e UNIX like

In ambiente GNU/Linux è necessario utilizzare un qualsiasi client SSH per potersi collegare attraverso una connessione criptata.
Analogamente a quanto visto in ambiente Microsoft Windows, si dovranno inserire due parametri:

- ➔ l'indirizzo ip del server (o il nome a dominio)
- ➔ il nome dell'utente

Supponiamo di utilizzare l'applicazione ssh, in questo caso si scriverà al prompt della console del sistema client: `ssh nomeserver -l nomeutente`

Una volta autenticati si ha l'accesso alla console Linux con la possibilità di inserire i comandi.

3.1.1.3 Anatomia del prompt

I comandi si inseriscono subito dopo il prompt del s.o., questo si presenta nel seguente modo:

user1@linuxMachine:~/cartella$

dove:

user1	è il login con cui ci si è autenticati,
linuxMachine	è la macchina che ospita Linux,
~/cartella	è la directory in cui ci si trova,
$	indica la posizione in cui si possono digitare i comandi nella shell aperta con la console.

Per uscire dalla console è necessario digitare exit seguito da ritorno carrello o premere la combinazione di tasti [control+D].

3.2 I comandi in linea forniti dalla shell

La shell è un'interprete che legge ed esegue i comandi impartiti dagli utenti.
E' importante sapere che, a differenza del dos, la shell linux è case sensitive, cioè fa differenza digitare comandi con lettere maiuscole o minuscole.
Segue un elenco di comandi con relativo esempio di utilizzo :

man: è l'help in linea

```
user1@linuxMachine:~$ man nomecomando
```

3.2.1 Comandi per la gestione del filesystem

cd: cambia la directory

```
user1@linuxMachine:~$ cd nomedirectory
```

pwd: indica il nome della directory corrente

```
user1@linuxMachine:~$ pwd
```

ls: crea la lista dei files presenti in una directory

```
user1@linuxMachine:~$ ls
```

rm: cancella un file o una directory

```
user1@linuxMachine:~$ rm nomefile
```

cp: copia uno o più files

```
user1@linuxMachine:~$ cp nomefile nomedir/.
```

mv: sposta uno o più files o directory

```
user1@linuxMachine:~$ mv nomefile1 nomefile2
```

mkdir: crea una directory

```
user1@linuxMachine:~$ mkdir nomedir
```

3.2.2 Comandi per la manipolazione dei files

cat: visualizza il contenuto di un file

```
user1@linuxMachine:~$ cat nomefile
```

more: visualizza il contenuto di un file paginandolo

```
user1@linuxMachine:~$ more nomefile
```

less: consente di navigare all'interno di un file

```
user1@linuxMachine:~$ less nomefile
```

sort: ordina le righe di un file

```
user1@linuxMachine:~$ sort nomefile
```

3.2.3 Comandi per la gestione degli utenti e dei processi

who: stampa l'elenco di tutti gli utenti loggati in quel momento nel sistema

```
user1@linuxMachine:~$ who
```

whoami: Stampa a video il proprio nome utente

```
user1@linuxMachine:~$ whoami
```

id: stampa a video il proprio identificativo utente (UID) e l'identificativo di gruppo (GID)

```
user1@linuxMachine:~$ id
```

ps: visualizza i processi in esecuzione appartenenti all'utente

```
user1@linuxMachine:~$ ps
```

top: visualizza tutti i processi in corso

```
user1@linuxMachine:~$ top
```

kill: termina un processo in esecuzione dato l'identificativo di processo (PID)

```
user1@linuxMachine:~$ kill –p PID -9
```

3.2.4 Comandi per la gestione dei permessi sui files

chmod: consente di modificare i diritti di uso e apertura dei file

```
user1@linuxMachine:~$ chmod u+x nomefile
```

chown: cambia il proprietario del file

```
user1@linuxMachine:~$ chown nomeutente nomefile
```

chgrp: cambia il gruppo di utenti in cui è inserito il file

```
user1@linuxMachine:~$ chgrp nomegruppo nomefile
```

3.2.4.1 Digressione sui permessi sui files

Linux è un s.o. multiutente, cioè permette a più utenti di coesistere e di utilizzare le risorse da lui messe a disposizione. Una di queste risorse è il sistema di file (filesystem).
Potete vedere i file come degli oggetti che possono contenere le informazioni più disparate, p.e. un file di testo contiene dello scritto (`divinacommedia.txt`), un file di musica contiene la codifica di suoni (`yellowsubmarine.wav`), un file grafico contiene disegni (`gioconda.bmp`) e così via.
Potrebbe capitare che l'utente `user1` non voglia far vedere i propri file all'utente `user2`, come procedere? Ad ogni file creato il filesystem di Linux associa dei permessi che nel momento in cui il file viene richiesto vengono verificati, se la verifica dà esito positivo allora al richiedente si dà accesso al file altrimenti il richiedente riceve una risposta negativa.
I permessi possibili sono principalmente tre: autorizzazione all'esecuzione , alla lettura e alla scrittura. Questi permessi possono essere combinati come indicato nella seguente tabella:

Numero ottale	***Permesso***	***Descrizione***
0	---	*Nessuna autorizzazione*
1	--x	Esecuzione
2	-w-	Scrittura
3	-wx	Scrittura,Esecuzione
4	r--	Lettura
5	r-x	Lettura,Esecuzione

6	rw-	Lettura,Scrittura
7	rwx	Lettura,Scrittura,Esecuzione

Il *7* indica quindi che tutte le autorizzazioni sono permesse, mentre 0 indica che non è possibile eseguire alcuna operazione sul file. Con *4* il file è disponibile solo per la lettura ma non può essere modificato o eseguito.
Complichiamo un po' la situazione. E' possibile raggruppare gli utenti Linux in gruppi, p.e.: il gruppo degli amministratori di sistema, il gruppo degli sviluppatori o il gruppo degli studenti di una certa classe. Possiamo, allora, ipotizzare che l'utente 1 del gruppo A voglia condividere in lettura i suoi files con tutti gli utenti del gruppo e voglia impedire l'accesso a tutti gli altri. Per far questo i permessi vengono suddivisi in tre categorie: permessi riferiti all'utente, permessi riferiti al gruppo di appartenenza dell'utente e quelli riferiti a tutti gli altri.
p.e. *lavoro.txt rw-r----* oppure *640* indica che lavoro.txt può essere letto e modificato dall'utente, gli appartenenti al gruppo possono leggerlo e tutti gli altri non hanno alcuna autorizzazione.
programma rwx--x--x indica che l'utente può leggere,modificare ed eseguire il file, gli appartenenti al gruppo e tutti gli altri possono solo eseguire il file.
Esistono file speciali come le directory che vengono indicate con una *d* prima dei permessi.
Esistono delle autorizzazioni speciali che non vengono in questa introduzione affrontati.

3.3 I comandi di ridirezione dell'output e i comandi di pipe

3.3.1 Ridirezione dell'output

Solitamente i comandi visualizzano il loro output a video, p.e.

```
user1@linuxMachine:~$ cat elenco.txt
```

visualizza le righe del file elenco.txt sulla finestra della console Linux. Lo standard output di ogni comando è il video. Ma se noi volessimo stampare l'output del comando sort su un file, in maniera tale da ottenere il file ordinato? Si esegue questa operazione utilizzando i comandi di ridirezione dell'output. Ne esistono due:

- `>`
- `>>`

Vediamo subito un'esempio. Si abbiano due files primo.txt e secondo.txt il cui contenuto è:

```
primo.txt:          secondo.txt:
bravo               paperino
cattivo             pluto
bello               pippo
brutto
```

eseguiamo il seguente comando:

```
user1@linuxMachine:~$ sort primo.txt > secondo.txt
```

L'output del sort viene reindirizzato sul file secondo.txt, se questo non esiste viene creato, altrimenti, viene sovrascritto. Se digitiamo:

```
user1@linuxMachine:~$ cat secondo.txt
```

otteniamo la stampa a video di:

```
secondo.txt:
bello
bravo
brutto
cattivo
```

cioè il contenuto del file primo.txt è stato ordinato e il contenuto del file secondo.txt è andato perduto.

Ora proviamo il seguente comando:

```
user1@linuxMachine:~$ sort primo.txt >> secondo.txt
```

Stampiamo a video, mediante il comando cat, il contenuto del file secondo.txt:

```
secondo.txt:
     bello
     bravo
     brutto
     cattivo
     bello
     bravo
     brutto
     cattivo
```

Notiamo che il contenuto ordinato di primo.txt è stato accodato al contenuto del file secondo.txt. Quindi il comando di ridirezione >> non è distruttivo come il comando >, se il file esiste la ridirezione dello standard output viene accodata al contenuto esistente. Se il file non esiste viene creato ex-novo.

3.3.2 Comandi di pipe

E'possibile connettere lo standard output di un programma con l'input di un altro, cioè l'uscita di un programma invece di comparire a video viene letta come ingresso dal secondo programma. Questo modo di lavorare si avvale dell"uso dell'operatore pipe "|". Una linea di comando contenente la pipe si chiama pipeline.

Analizziamo un semplice esempio: vogliamo esaminare il contenuto di una directory che contiene un centinaio di files. Se lanciamo semplicemente il comando ls non riusciamo a vedere i primi files dell'elenco perché scorrono molto velocemente a video. Una soluzione è quella di usare il comando more che, come abbiamo visto, permette la paginazione di una lista molto lunga.
Come fare? Proviamo a digitare il seguente comando dalla console:

```
user1@linuxMachine:~$ ls | more
```

Abbiamo così concatenato il comando ls con il comando more, l'output di ls viene mandato in input al comando more che stampa sullo standard output il contenuto della directory paginato.

Abbiamo la possibilità di concatenare diversi comandi tra di loro:

```
comando1 | comando2 | comando3| ... | comandoN
```

L'uso della pipe dà la possibilità di agganciare semplici programmi fra loro per produrre un output complesso. Le pipe stigmatizzano la filosofia della metodologia bottom-up, cioè si raggruppano componenti e funzionalità sino ad arrivare alla sintesi dell'intero progetto (programma). In questo modo costruiamo semplici programmi ma assolutamente ben strutturati e funzionali, in maniera tale che concatenandoli tra di loro otteniamo un output ben più complesso degli output dei programmi presi singolarmente. In questa metodologia si racchiude buona parte della filosofia Unix.

3.4 Editare testi dalla console

In ambiente Linux sono due gli editor di testo più diffusi e importanti: EMACS e VI

- ➔ EMACS (www.gnu.org/software/emacs/) è l'acronimo di Editor MACroS (la prima versione di emacs era un'estensione di un editor chiamato TECO). E' stato scritto da Richard Stallman, il fondatore della Free Software Foundation (FSF www.fsf.org) inventore della licenza GPL (Gnu General Public License).
- ➔ VIM (www.vim.org) è l'acronimo di VI Improved (VI è un editor usato sui sistemi unix, anche in questo caso abbiamo un editor che parte sulla base di un altro più vecchio).

Noi utilizzeremo VIM (d'ora in poi vi)

3.4.1 Breve guida ai comandi di VI

Per editare un file già esistente o per crearne uno nuovo si digita dalla console Linux:

```
user1@linuxMachine:~$ vi nomefile
```

dove *nomefile* è il nome del file che si vuol aprire o creare. In questo modo si apre l'editor in modalità full screen, cioè tutta la finestra della console ospita la schermata di vi.

VI permette di lavorare in diverse modalità, noi ne uilizzeremo principalmente le seguenti tre:

- ➔ Normal mode
- ➔ Insert mode
- ➔ Command mode

3.4.1.1 Normal mode

Appena entrati in ambiente VI ci si trova in normal mode. Per spostarsi si usano i tasti cursore oppure i tasti:

- ➔ **h** freccia sinistra
- ➔ **l** freccia destra
- ➔ **k** freccia su
- ➔ **j** freccia giù
- ➔ **x** cancella un carattere (10x cancella 10 caratteri)
- ➔ **dd** cancella la riga
- ➔ **u** esegue undo dell'ultimo comando dato
- ➔ **s** cancella carattere e passa alla modalità di inserimento (10s cancella 10 caratteri e passa a modalità inserimento)
- ➔ **yy** copia riga in buffer di memoria
- ➔ **p** esegue il paste del contenuto del buffer di memoria

3.4.1.2 Insert mode

Digitando il carattere ***i*** si entra nella modalità di inserimento, sull'ultima riga apparirà un indicatore di tale modo. Si esce dalla modalità inserimento premendo il tasto ESC. In questa modalità si ha la possibilità di digitare il testo in full screen.

3.4.1.3 Command mode

Digitando il carattere ***:*** si entra nella modalità comandi (la precedente modalità deve essere quella normale). Dopo *:* si possono inserire i seguenti comandi:

- ➔ ***:q*** è il comando di quit, chiude la sessione di lavoro e chiede se si vuole salvare il testo
- ➔ ***:q!*** quit senza salvare le modifiche effettuate sul testo
- ➔ ***:w*** salva il testo
- ➔ ***:wq*** salva ed esci
- ➔ ***:sp*** *nomefile* divide in due (splitta) la finestra di editor per ospitare un nuovo file di nome nomefile